

Machine Learning-Based Prediction of Blood Plasma Biomarker Levels Using Clinical and Cerebrospinal Fluid Data for Diagnosis and Prognosis of Alzheimer's Disease

Third Year Individual Project – Final Report

April 2024

Georgios Theofilakos

10669224

Supervisor:

Dr. Fumie Costen

Contents

Contents	2
Abstract.....	8
Declaration of originality.....	9
Intellectual property statement.....	10
Acknowledgements.....	11
1 Introduction	12
1.1 Background	12
1.2 Motivation.....	13
1.3 Aims and Objectives.....	13
2 Literature Review	14
2.1 Review of Recent Alzheimer's Disease Research.....	14
2.2 Recent Developments in Alzheimer's Disease Biomarkers	15
2.3 Existing Literature on Blood Plasma Biomarkers	16
2.4 Machine Learning for Biomarker Quantification	17
3 Theoretical Development	18
3.1 Introduction	18
3.2 Statistical Analysis.....	18
3.2.1 Measures of Central Tendency and Dispersion	18
3.2.2 Statistical Significance.....	19
3.2.3 Normality Tests	20
3.2.4 Correlation Analysis and Multicollinearity.....	21
3.3 Support Vector Regression	21
3.3.1 Theoretical Background	21
3.3.2 Hard Margin and Soft Margin	21

3.3.3	Dual Formulation and Solution	22
3.3.4	Kernel Trick	22
3.3.5	Common Kernel Functions.....	23
3.3.6	Selection of Support Vector Regression for Outlier-Rich, Small Datasets.....	23
3.4	Bayesian Optimization	23
3.4.1	Gaussian Processes	23
3.4.2	Acquisition Function	24
3.5	Evaluation Metrics and Cross-Validation	24
4	Design and Data Description.....	24
4.1	Introduction	24
4.2	Overview of Study Design	25
4.3	Data Source and Study Population	26
4.3.1	Plasma and CSF Data Collection.....	26
4.3.2	Clinical Data Collection, Selection and Basic Pre-Processing.....	27
4.3.3	Main Issues of the Plasma and CSF Datasets.....	27
4.4	Assessing Normality on Pre-Processed Data	28
4.5	Dataset Descriptive Statistics.....	29
4.6	Outliers.....	30
5	Implementation	30
5.1	Introduction	30
5.2	Biomarker Levels by Diagnosis Category	30
5.3	Exploration of Clinical Data and Biomarker Relationships & Selection of Dataset	31
5.4	Feature Selection and Preparation for Support Vector Regression Model.....	33
5.4.1	Multicollinearity Testing Using Spearman Correlation Matrix	33
5.4.2	Considerations on Normalizing Features.....	35
5.5	Model Selection	35

5.5.1	Outlier Augmentation	35
5.5.2	Feature Standardization.....	35
5.5.3	Hyperparameter Tuning Using Bayesian Optimization	36
5.6	Model Evaluation	36
5.6.1	Evaluation Process and Metrics	36
5.6.2	Residual Analysis	37
5.6.3	Evaluating Model Robustness by Utilizing a Second Dataset	37
5.6.4	Baseline Evaluation	37
5.6.5	SVR Models for the Remaining Biomarkers	38
6	Results and Discussion	38
6.1	Primary Biomarker Prediction Results	38
6.1.1	Model Selection	38
6.1.2	Assessing the Generalizability of the Model Across Different Assay Systems	40
6.1.3	Residual Analysis	40
6.1.4	Predicted Performance Overview.....	41
6.1.5	Comparison Against Benchmark and Standard Deviation Test Evaluation	42
6.1.6	Failed Attempts at Improving the Model.....	42
6.2	Evaluation Summary for All Biomarkers	42
6.3	Limitations.....	44
7	Conclusions and Future Work.....	45
7.1	Conclusions	45
7.2	Future work.....	46
	References.....	47
	Appendices.....	52
A.	Supplemental Literature Review.....	52
A.1	Positron Emission Tomography (PET) Imaging Biomarker	52

A.2	Cerebrospinal Fluid (CSF) Biomarkers	52
A.2.1	Amyloid-beta ($A\beta$) Peptides	52
A.2.2	Tau Proteins	53
A.2.3	Neurofilament Light Chain (NfL)	53
A.2.4	CSF Biomarker Collection	53
A.2.5	Challenges for CSF Biomarkers	53
A.3	Methods of Quantification for CSF and Plasma Biomarkers	54
A.4	Technological Platforms for Biomarker Detection	54
A.4.1	Comparative analysis of biomarker main quantification methods	55
A.4.2	Supplemental Table for Technological Platforms for Biomarker Detection.....	56
A.5	Supplemental Material for Theoretical Development.....	56
A.5.1	Descriptive Statistics	56
A.5.2	Kolmogorov-Smirnov Test for Normality	57
A.5.3	Correlation	57
A.5.4	Support Vector Regression Hard and Soft Margins	58
A.5.5	Evaluation Metrics	59
A.6	Supplemental Material for the Dataset	59
A.7	Supplemental Material for $A\beta_{42}$ Analysis	61
A.7.1	Descriptive Statistics Across Non-AD and AD Groups	61
A.7.2	Nominal Variables against $A\beta_{42}$	62
A.8	Evaluation metrics for $A\beta_{42}$	64
B.	Analysis of $A\beta_{40}$ Blood Plasma Biomarker	66
B.1	Normality and Descriptive Statistics	66
B.2	Exploration of Clinical Data and Biomarker Relationships	67
B.3	Correlation Analysis	69
B.4	Results, Evaluation and Discussion	70

B.4.1	Combination Selection and Hyper-Parameters	70
B.4.2	Residual Analysis and Predictive Performance	72
B.4.3	Benchmark and Standard Deviation Evaluation	74
C.	Analysis of A β 42/40 Blood Plasma Biomarker	75
C.1	Normality and Descriptive Statistics	75
C.2	Exploration of Clinical Data and Biomarker Relationships	76
C.3	Correlation Analysis	78
C.4	Results, Evaluation and Discussion	79
C.4.1	Combination Selection and Hyper-Parameters	79
C.4.2	Residual Analysis and Predictive Performance	82
C.4.3	Benchmark and Standard Deviation Evaluation	83
D.	Analysis of P-tau181 Blood Plasma Biomarker	84
D.1	Normality and Descriptive Statistics	84
D.2	Exploration of Clinical Data and Biomarker Relationships	85
D.3	Correlation Analysis	88
D.4	Results, Evaluation and Discussion	89
D.4.1	Combination Selection and Hyper-Parameters	89
D.4.2	Residual Analysis and Predictive Performance	92
D.4.3	Benchmark and Standard Deviation Evaluation	93
E.	Analysis of NfL Blood Plasma Biomarker	94
E.1	Normality and Descriptive Statistics	94
E.2	Exploration of Clinical Data and Biomarker Relationships	95
E.3	Correlation Analysis	97
E.4	Results, Evaluation and Discussion	98
E.4.1	Combination Selection and Hyper-Parameters	98
E.4.2	Residual Analysis and Predictive Performance	101

E.4.3	Benchmark Evaluation	102
F.	Dissertation Requirements	103
F.1	Project Outline	103
F.1.1	Background	103
F.1.2	Motivation.....	103
F.1.3	Aims and Objectives.....	104
F.1.4	Project Plan	105
F.2	Risk Assessment	106
F.3	Risk Register	107
F.4	Code Environment	107
F.5	Code for Primary Biomarker A β 42	111
F.5.1	Initial Preprocessing of All ABETA Datasets	111
F.5.2	Graphical Representation of Blood Plasma AD VS NON-AD for A β 42.....	126
F.5.3	Correlation Analysis for A β 42	131
F.5.4	Machine Learning Pipeline and Evaluation for A β 42.....	133
F.6	Code for the Machine Learning Pipeline and Evaluation of A β 40.....	144
F.7	Code for the Machine Learning Pipeline and Evaluation of A β 42/40	152
F.8	Code for the Machine Learning Pipeline and Evaluation of P-tau181.....	160
F.9	Code for the Machine Learning Pipeline and Evaluation of NfL.....	168

Word count: 10,646

Abstract

As of 2021, the prevalence of dementia worldwide surpasses 50 million individuals with projections indicating a threefold increase to 152 million by 2050 due to increasing aging trends. Alzheimer's disease (AD) is the most frequent neurodegenerative disease and the most common type of dementia, affecting at least 27 million people worldwide. Facing the escalating challenge of AD, this study is the first to propose a machine learning approach to predict blood plasma biomarker levels for AD diagnosis using clinical and CSF data, addressing the urgent need for accessible and cost-effective early detection methods.

Expanding on this foundation, the primary machine learning model developed in this study focuses on predicting the A β 42 peptide concentration in blood by employing Support Vector Regression (SVR), tuned utilizing Bayesian optimization. This approach resulted in a statistically significant 19% reduction in Mean Absolute Error (MAE) compared to the benchmark, with enhanced precision of estimating plasma A β 42 to within 62% of the standard deviation. The robustness of this model was then assessed on a second dataset, where it effectively maintained accuracy despite variations in assay methodologies, confirming its generalizability.

Additional SVR models were developed targeting peptides such as A β 40, A β 42/40, P-tau181, and NfL in blood plasma. The A β 42/40 ratio model exhibited superior predictive capability, aligning closely with clinical indicators of Alzheimer's disease. However, the models for A β 40, P-tau181, and NfL faced higher variability and errors, compounded by notable data outliers. Nonetheless, all SVR models surpassed the benchmark by an improvement of at least 16% in MAE, and all but the A β 42/40 model, maintained precision within 70% of the standard deviation, indicating robust precision and reliability in predictions across a population with varying blood plasma profiles but limited demographic diversity.

The findings confirm the feasibility of predicting blood plasma biomarker levels, using machine learning, and set the stage for broader diagnostic accessibility. By democratizing access to AD biomarker diagnostics, this method has the potential to make a substantial impact on global healthcare equity, significantly contributing to the worldwide effort to combat Alzheimer's disease.

Declaration of originality

I hereby confirm that this dissertation is my own original work unless referenced clearly to the contrary, and that no portion of the work referred to in the dissertation has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Intellectual property statement

- i. The author of this thesis (including any appendices and/or schedules to this thesis) owns certain copyright or related rights in it (the “Copyright”) and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.
- ii. Copies of this thesis, either in full or in extracts and whether in hard or electronic copy, may be made **only** in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has from time to time. This page must form part of any such copies made.
- iii. The ownership of certain Copyright, patents, designs, trademarks and other intellectual property (the “Intellectual Property”) and any reproductions of copyright works in the thesis, for example graphs and tables (“Reproductions”), which may be described in this thesis, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.
- iv. Further information on the conditions under which disclosure, publication and commercialisation of this thesis, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy (see <http://documents.manchester.ac.uk/DocuInfo.aspx?DocID=24420>), in any relevant Dissertation restriction declarations deposited in the University Library, and The University Library’s regulations (see <http://www.library.manchester.ac.uk/about/regulations/files/Library-regulations.pdf>).

Acknowledgements

I am grateful to Professor Fumie Costen, whose expertise and guidance were invaluable throughout this research. Her encouragement and support made this journey not just possible, but deeply rewarding.

Most importantly, I owe everything to my parents. Each day, I am reminded of the lengths you have gone to, all the sacrifices you made, to ensure I could pursue my education and dreams. This dissertation is my gift to you, symbolizing my gratitude for all the hardships you had to endure for me to become who I am today. Thank you for everything.

1 Introduction

1.1 Background

Dementia encompasses a group of symptoms related to an ongoing decline in brain functioning. It impairs various aspects of a person's life, including memory, thinking, behaviour, and communication [1]. As of 2021, the prevalence of dementia worldwide surpasses 50 million individuals with projections indicating a threefold increase to 152 million by 2050 due to increasing aging trends [2]. In the United Kingdom (UK) in 2019 there were close to 900,000 people affected by dementia and this number is projected to increase to about 1,600,000 people by 2040 [3]. The impact of dementia on the UK's economy is substantial, in 2019 the total costs of dementia reached almost £35 billion and are projected to increase by 172% to reach close to 95 billion by 2040 [3].

Alzheimer's disease (AD) is the most frequent neurodegenerative disease [4] and the most common type of dementia, affecting at least 27 million people worldwide [5]. AD is believed to occur from the accumulation of amyloid-beta ($A\beta$) in the brain. According to the National Institute on Aging and the Alzheimer's Association (NIA-AA), the progression of AD follows three stages: Preclinical, Mild Cognitive Impairment (MCI) and Dementia [6]. The early detection of MCI in individuals who will eventually develop AD is crucial for the application of disease-modifying therapies (DMTs) [7]. DMTs have the potential to slow the progression of the disease. Additionally, early detection of AD is crucial for patient management and treatment planning. Therefore, early detection and intervention remain key strategies in the fight against AD [7].

The current diagnostic methods, such as neuroimaging are expensive, invasive, and not widely available. Cerebrospinal fluid (CSF) Biomarkers have emerged as a cheaper tool for the early detection of AD and have shown clinical utility [8]. CSF biomarker testing is included in current dementia diagnosis guidelines, including those from England's National Institute for Health and Care Excellence (NICE) [8]. However, CSF is not available for all patients and there are concerns regarding lumbar puncture due to its perceived more invasive nature [8]. In recent years blood plasma biomarkers have emerged as a new tool for AD prediction as they are less invasive, cheaper, and potentially more accessible than CSF biomarkers. However because blood

biomarkers have only recently become available, the evidence on their clinical validity and performance to screen memory clinic patients as an effective method is not established yet [9].

1.2 Motivation

Predicting AD from a simple blood test will help in early treatment and prevention for a lot of people in the future. It will allow people who could not afford it the ability to be relieved from suffering. However, the reality of widespread adaptation of this technique is more complicated. Biological variability, influenced by age, genetics, and lifestyle, introduces additional complexity in biomarker interpretation, requiring extensive validation across diverse populations [10]. Analytical variabilities in plasma biomarkers, including differences in sample handling and assay methodologies, further hinder the standardization and reproducibility of results [11]. Finally, the analysis of blood plasma to find AD biomarkers is not a simple process, specific technological platforms are needed which currently are both few and expensive. As a result, most clinics especially in underprivileged areas will be the last to enjoy this technology, even though these areas will be the ones that need it most [12].

Consequently, there is a need for advancements in methodologies that can simplify and economize the AD prediction process using blood plasma biomarkers. Integrating machine learning models for the prediction of blood plasma concentrations offers a promising solution to overcome these obstacles.

1.3 Aims and Objectives

This dissertation aims to develop a machine-learning algorithm that can estimate blood biomarker levels from clinical and CSF biomarker concentrations. Achieving this goal holds significant importance for three main reasons. Initially, by using clinical and CSF data to predict blood biomarker levels, clinics could more efficiently identify candidates for research studies or pharmaceutical trials. This could expedite the development of blood tests for Alzheimer's disease that are as reliable as CSF analyses or PET scans. Additionally, if the model is calibrated with data from a specific assay, it could serve as a benchmark for new technologies, allowing them to quickly validate their accuracy against the model's predictions. Lastly, such algorithms could empower clinics with existing CSF samples to estimate their patients' blood plasma levels, facilitating early diagnostic efforts even before the adoption of the actual blood testing technology. Thus,

narrowing the disparity in access to advanced diagnostic methods and advancing global healthcare equity.

The intermediate objectives for achieving the aim are the following.

1. Conduct a comprehensive review of the literature to identify existing machine learning methodologies in blood biomarker prediction from CSF concentrations, highlight one or more significant research gaps and understand Alzheimer's disease.
2. Acquire a dataset with plasma, CSF biomarkers, and clinical data. Preprocess the data to normalize values, detect and manage outliers, and remove extraneous information. Employ data visualization and analyse relationships between features and the target to ensure comprehensive understanding and preparation for predictive modelling.
3. Perform a feature selection analysis to identify the top 5 most predictive features linking clinical data and CSF biomarkers to the target blood biomarkers, using statistical correlation.
4. Implement a hyperparameter tuning strategy, to optimize the model parameters.
5. Develop a machine learning model to predict at least one plasma biomarker's levels, aiming for a prediction precision to be at minimum 70% within the plasma biomarker's standard deviation as validated on a separate validation set.
6. Conduct a residual analysis of the model's results to ensure no systematic bias and that residuals approximate a normal distribution, adjusting the model as necessary to meet these criteria.
7. Validate the model's performance against an established benchmark, aiming for a performance metric that meets or exceeds this benchmark by at least 5%.

2 Literature Review

2.1 Review of Recent Alzheimer's Disease Research

Research has shown that the process a person can develop AD happens either outside the neural cell as amyloid plaques or inside the cell as neurofibrillary tangles [5] as seen in Figure 2.1. These abnormalities disrupt neuronal function and connectivity leading to a gradual decline in brain function [13]. In the majority of patients with Alzheimer's disease, the reason for the disease remains elusive with only 1-2% of cases having as an underlying cause deterministic genetic anomalies [14].

According to the National Institute of National Institute on Aging and the Alzheimer's Association (NIA-AA), the progression of AD follows three stages: Preclinical, Mild Cognitive Impairment (MCI) and Dementia [6]. During the preclinical phase, there are no signs or symptoms of AD, however, one or more biomarkers are present, indicating the onset of neuropathological changes associated with AD. During the MCI stage, there are noticeable challenges in "cognitive domains, including memory, executive function, attention, language, and visuospatial skills" [15] while there is typically minimal disruption to one's capacity for daily tasks and participation in social or occupational pursuits. When AD progresses to the dementia stage, it exhibits varying degrees of severity. For severe AD communication and voluntary functions are compromised and the disease often precipitates fatal complications.

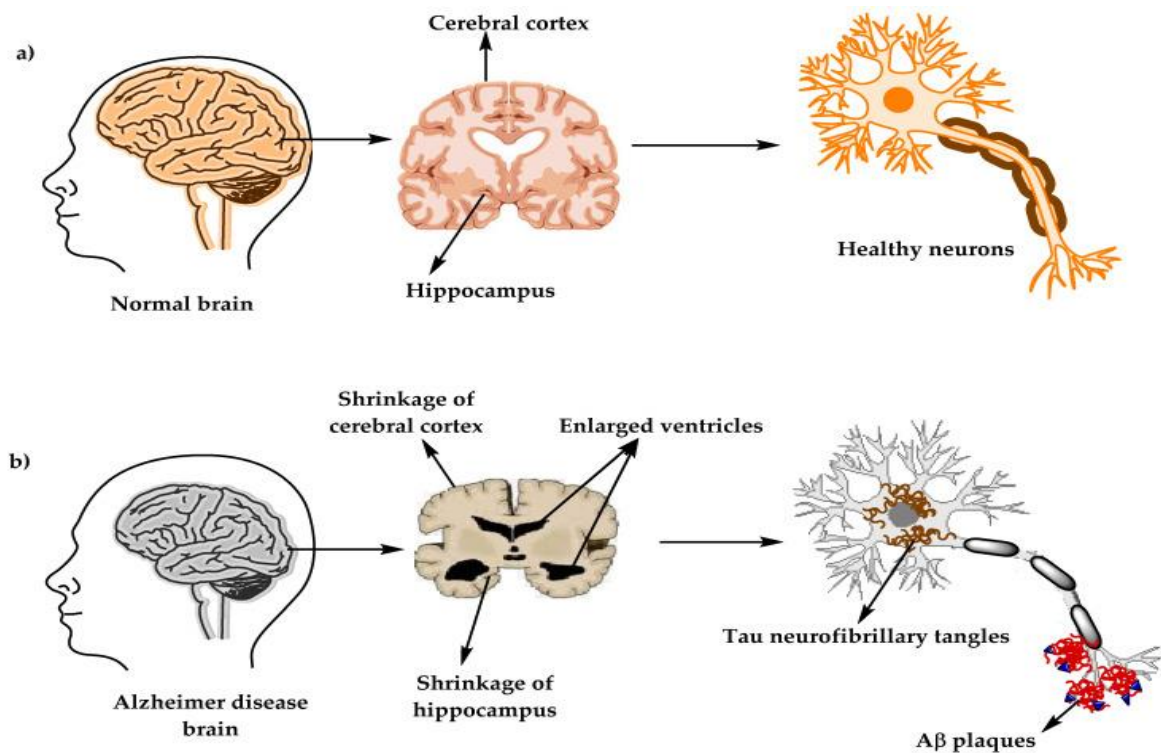


Figure 2.1: The anatomical configuration of brain tissue and neurons in (a) a normally functioning brain and (b) a brain affected by AD [14].

2.2 Recent Developments in Alzheimer's Disease Biomarkers

A fundamental objective in Alzheimer's disease research is to identify biological markers (biomarkers) that enable early diagnosis and intervention [16]. They serve as measurable cues which reflect bodily functions and conditions, and they are detected in various bodily substances and tissues. Biomarkers are critical as they provide insight into the early signs of pathological processes occurring in the brain related to AD, often before clinical symptoms manifest.

Biomarkers associated with AD are commonly identified within bodily fluids, including cerebrospinal fluid (CSF) and blood, or via neuroimaging methods (Positron Emission Tomography (PET)). These play a critical role in both the early diagnosis of AD and the ongoing monitoring of disease progression and treatment efficacy [16]. Early identification of the disease is essential for its effective management, providing an opportunity for prompt interventions that may decelerate the progression of AD [16]. More information about these biomarkers can be found in Appendices A.1 and A.2.

In 2018, the NIA-AA introduced a refined framework for classifying AD biomarkers, known as the Amyloid/Tau/Neurodegeneration (AT(N)) system [17]. This system categorizes biomarkers based on the presence of beta-amyloid plaques (A), tau protein deposits (T), and neurodegeneration or neuronal injury (N), providing a more nuanced understanding of the disease's pathology. Additionally, it states that while potentially not causal, these are the features that define AD “as a unique neurodegenerative disease among different disorders that can lead to dementia” [17]. This shift towards a biomarker-based classification of AD not only enhances understanding of the disease mechanism but also improves the stratification of patients for clinical trials, potentially leading to more targeted and effective therapies [17].

2.3 Existing Literature on Blood Plasma Biomarkers

In exploring biomarkers for AD recent research pivots towards blood plasma, which comprises 55% of blood's total volume [18]. This field of study aims to pinpoint specific molecules within the plasma indicative of AD's presence or its progression. Unlike the CSF biomarkers, which require an invasive lumbar puncture, plasma biomarkers can be accessed through a simple, non-invasive blood draw. This fundamental difference enhances the feasibility of regular monitoring but also positions plasma biomarkers as a more patient-friendly option in the ongoing battle against AD. A summary of this section can be seen in Figure 2.2. The key biomarkers for AD are the following:

Amyloid- β (A β) Peptides: The imbalance between the production and clearance of A β peptides, primarily A β 42, leads to amyloid plaque formation in the brain, a hallmark of AD. Research shows that plasma levels of A β 42, A β 40, and particularly the A β 42/40 ratio are lower in AD patients compared to cognitively unimpaired individuals, correlating with brain amyloid pathology as confirmed by amyloid- β positron emission tomography (A β -PET) [19] [20] [21].

Total Tau and Phosphorylated Tau (p-tau): Plasma tau proteins are crucial in indicating neurodegeneration and AD pathology. Elevated plasma p-tau levels are closely associated with AD diagnosis and have “significantly outperformed all biomarkers in differentiating AD dementia from CN and predicting disease progression [22].

Neurofilament Light Chain (NfL): NfL is a potent marker of neuroaxonal damage, with its levels correlating with the severity of neurodegeneration [58]. Plasma NfL levels are elevated in MCI and AD dementia stages compared to CU [23] and correlate strongly with CSF NfL levels [24].

Glial Fibrillary Acidic Protein (GFAP): GFAP is a protein found primarily in astrocytes, a type of glial cell in the brain and spinal cord. When the brain is damaged or when neurodegenerative diseases like AD are present, astrocytes become activated and increase in number, a process known as astrogliosis [25]. In individuals with AD, there's a noticeable rise in GFAP levels within their blood plasma [26]. This increase is linked to both the worsening of cognitive functions and the buildup of amyloid plaques in the brain [27], a connection that A β PET scans have verified [26].

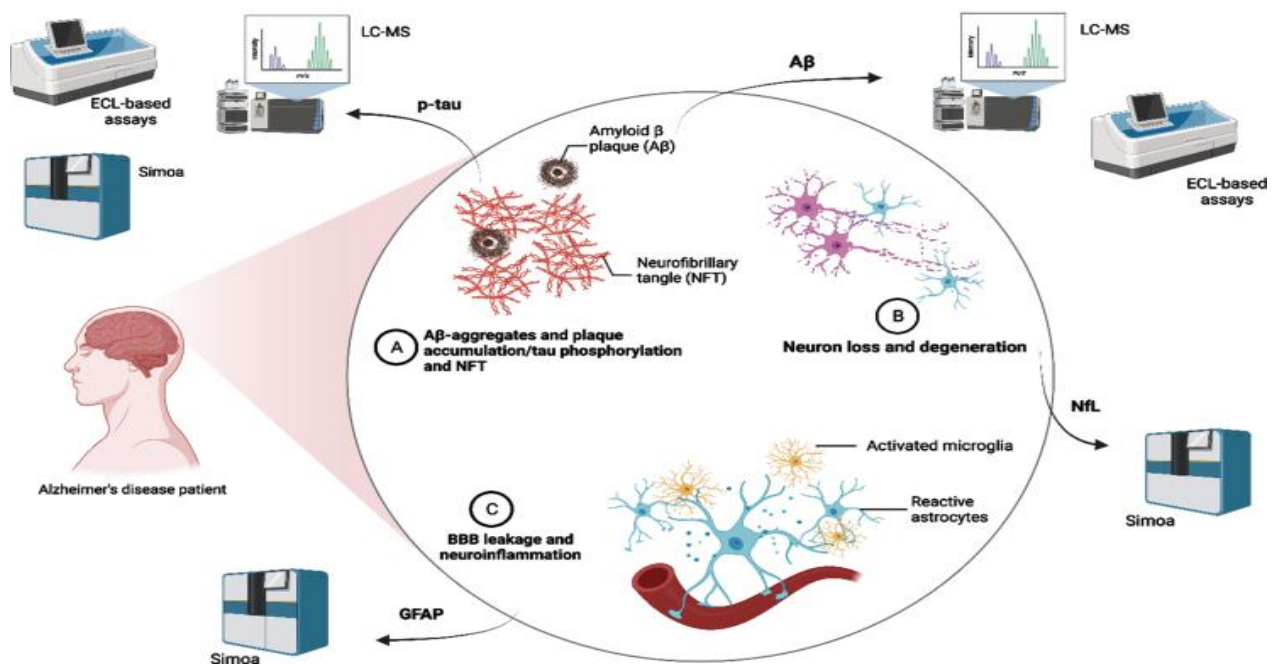


Figure 2.2: Overview of key neuropathological indicators, prominent biomarkers and their quantification methods for Alzheimer's Disease [20].

2.4 Machine Learning for Biomarker Quantification

Quantifying CSF and plasma biomarkers is crucial for the diagnosis, prognosis, and treatment monitoring of AD. The choice of quantification methods and platforms is dictated by factors

including the biomarker's nature, sensitivity requirements, sample volume, cost, and desired throughput. CSF, being in direct contact with the central nervous system, has higher concentrations of biomarkers such as A β , tau proteins, and NfL. The blood-brain barrier significantly restricts these molecules' transfer into systemic circulation, leading to notably lower concentrations of AD biomarkers in blood. Detailed discussions on quantification methods and platforms are presented in Appendices A.3, and A.4.

Despite the advancements in biomarker quantification, a notable research gap exists in the application of machine learning for this purpose. To date, there has been no documented research utilizing machine learning to quantify blood or CSF biomarker levels. Until now only direct methods have been employed for determining levels of CSF and blood biomarkers, overlooking the potential enhancements machine learning could offer in terms of accuracy, efficiency, and predictive capability. This observation highlights a significant opportunity for innovation and improvement in the field of biomarker analysis that will be researched in this dissertation.

3 Theoretical Development

3.1 Introduction

This chapter is dedicated to the tools employed to achieve the study's goals. It begins with explaining the statistical tools used that form the basis for most of the subsequent analyses. Then, it delves into the mathematical principles behind machine learning concepts utilized in the study, including Support Vector Regression, Bayesian Optimization, and various evaluation metrics.

3.2 Statistical Analysis

3.2.1 Measures of Central Tendency and Dispersion

Descriptive statistics summarize the dataset, guiding decisions in data preprocessing, feature engineering, and anomaly detection, which in turn influence the accuracy of predictive models.

Statistics describing the central tendency of a dataset are the mean median and mode. Measures of dispersion are the variance, standard deviation (SD) and interquartile range (IQR). In Appendix A.5.1 further details and given about the foundational statistic measures with their equations.

3.2.2 Statistical Significance

Statistical significance helps determine whether observed differences or relationships within data or between datasets are likely due to chance or reflect true effects. This ensures that inferences made from data analysis are robust and reliable. The metric behind statistical significance is the p-value which quantifies the probability of observing results as those in the data, assuming the null hypothesis is true. It is calculated based on the specific statistical test used as shown in Table 3.1, comparing the observed data against the distribution expected under the null hypothesis. A low p-value indicates that the observed data are unlikely under the null hypothesis, suggesting a statistically significant difference or relationship.

Table 3.1: Tests to determine statistical significance.

Test	Test Description	Formula Description	Statistic Formula
Student's t-test	Compares the means of two groups, providing insights into their statistical differences under the assumption of normal distribution	$\bar{x}_1$ and $\bar{x}_2$ are the sample means, s^2 is the pooled sample variance, and n_1 and n_2 are the sample sizes [28].	$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{\frac{s^2}{n_1} + \frac{s^2}{n_2}}} \quad (3.1)$
Mann-Whitney U test	Used when data do not follow a normal distribution, by comparing the ranks of two independent samples to assess their disparity.	R_1 is the sum of ranks for the group, and n_1 is the sample size. R_2 for the second group follows the same calculation [28].	$U = R_1 - \frac{n_1(n_1 + 1)}{2} \quad (3.2)$
Chi-square test	Used for categorical data, examines the independence between variables, helping to identify associations.	O is the observed and E is the expected frequency [28].	$\chi^2 = \sum \frac{(O - E)^2}{E} \quad (3.3)$

For all tests—Student's t-test, Mann-Whitney U test, and Chi-square test—the p-value is found by comparing the test statistic (t , U , or χ^2) to its respective distribution under the null hypothesis. This comparison yields the probability of observing a statistic as extreme as the one calculated [28]. While the t-test uses the t-distribution and degrees of freedom, the Mann-Whitney U test may use a conversion to a z-score for large samples, referencing the standard normal distribution the Chi-square test directly utilizes the chi-square distribution. The p-value typically found using statistical software, indicates the likelihood of the observed result under the null hypothesis [29]. In all cases, statistical software or tables typically facilitate the p-value determination, streamlining

the process of assessing statistical significance. In this study, the conventional p-value <0.05 will be used as the metric for determining if a relationship is statistically significant.

3.2.3 Normality Tests

To date, there are graphical and numerical ways to test for normality. This study will rely on using numerical methods for proving normality since they are more objective measures. For continuous data, the following tests that can determine normality include the Shapiro–Wilk test, Kolmogorov–Smirnov test (KS), skewness and kurtosis. Shapiro-Wilk is for small datasets of less than 50 samples and thus will not be used [30]. A significant KS statistic (resulting in a low p-value<0.05) indicates the data deviate from a normal distribution [31]. Using this test, it will be determined what is the best way to describe the dataset (using mean and std or median and IQR). Further discussion for the KS test can be found in Appendix A.5.2. Finally, skewness and kurtosis are more specific markers for a distribution’s asymmetry and tailedness and are discussed in Table 3.2.

Table 3.2: Skewness and Kurtosis Metrics for Assessing Distribution Asymmetry and Tailedness.

Metric Name	Metric Description	Formula	Formula Description
Skewness	Measures the asymmetry of a dataset's distribution around its mean.	Skewness $= \frac{N}{(N-1)(N-2)} \sum_{i=1}^N \left(\frac{x_i - \bar{x}}{s} \right)^3 \quad (3.4)$	For both N is the sample size, x_i are the individual observations, $\bar{x}$ is the sample mean, and s is the standard deviation [32].
Kurtosis	Assesses the "tailedness" of a dataset's distribution, reflecting the distribution's peakedness or flatness relative to a normal distribution.	Kurtosis $= \frac{N(N+1)}{(N-1)(N-2)(N-3)} \sum_{i=1}^N \left(\frac{x_i - \bar{x}}{s} \right)^4 - \frac{3(N-1)^2}{(N-2)(N-3)} \quad (3.5)$	

A skewness value of 0 shows a symmetric distribution, values greater than 0 denote a right-skewed distribution (tail extends towards more positive values), and values less than 0 signify a left-skewed distribution (tail extends towards more negative values). A kurtosis value greater than 3 designates a distribution that has heavier tails with a sharper peak (leptokurtic) compared to a normal distribution, a value less than 3 suggests a distribution with lighter tails and a flatter peak (platykurtic), and a value of 3 signifies a normal distribution (mesokurtic). Fisher’s kurtosis subtracts 3 from the metric and thus a normal distribution is considered at 0.

3.2.4 Correlation Analysis and Multicollinearity

Correlation measures will help in identifying the strength and direction of the linear and monotonic relationships between the features and the target variable assisting in exploring how each feature influences the target variable. The Pearson correlation coefficient (r) quantifies the linear association between two continuous variables. Spearman's rank correlation coefficient (ρ) examines the monotonic relationship between two variables by using their ranks rather than the actual data. Detailed explanation and their equations are given in Appendix A.5.3.

Multicollinearity refers to the scenario where two or more predictors in a regression model are highly correlated, leading to difficulty in distinguishing their individual effects on the target variable. A correlation matrix, created using Pearson or Spearman coefficients, serves as an effective tool to detect multicollinearity by revealing the strength and direction of relationships between all pairs of features. High correlation values indicate potential multicollinearity issues, guiding adjustments in feature selection to enhance model reliability.

3.3 Support Vector Regression

3.3.1 Theoretical Background

The foundation of SVR is built upon the principle of finding a hyperplane that best separates the data points in the feature space. In the context of machine learning the features refer to the independent variables. In regression, this hyperplane becomes a decision surface aiming to fit the maximum number of data points within a specified ϵ -tube while minimizing the model complexity. The hard margin approach, initially devised for classification, seeks a hyperplane with the maximum margin separating the two classes of data points. Mathematically, for a dataset $\{(x_i, y_i)\}_{i=1}^n$ where $x_i \in R^d$ is the feature vector having d dimensions and $y_i \in R^d$ is the target value. The aim is to find a function shown by (3.6) [33], that predicts the target value y_i for a given input x_i , with w being the weight vector, b the bias term, and $\langle ., . \rangle$ the dot product.

$$f(x) = \langle w, x \rangle + b \quad (3.6)$$

3.3.2 Hard Margin and Soft Margin

The hard margin SVR aims to find a hyperplane described by parameters w, b seeking the widest possible margin between the hyperplane and the nearest data points. Soft margin SVR modifies

this by allowing some deviations outside the ϵ -tube using slack variables ξ_i, ξ_i^* , and minimizes a different objective function to balance the regression function's flatness with penalty on these deviations. Equation (3.7) [33] where C is the regularization parameter, as shown below, encapsulates this optimization problem. Further derivation details are available in Appendix A.5.4.

$$\text{Minimize: } \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*), \text{ subject to } \begin{cases} y_i - \langle w, x_i \rangle - b < \epsilon + \xi_i \\ \langle w, x_i \rangle + b - y_i < \epsilon + \xi_i^* \\ \xi_i, \xi_i^* \geq 0 \end{cases} \quad (3.7)$$

3.3.3 Dual Formulation and Solution

To solve the optimization problem more efficiently and enable the application of kernel methods, the soft margin SVR problem is transformed into its dual form. By introducing Lagrange multipliers $\alpha_i, \alpha_i^*, \mu_i, \mu_i^*$ for each of the constraints, the Lagrangian for the soft margin SVR is:

$$\mathcal{L} = \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*) - \sum_{i=1}^n \alpha_i (y_i - \langle w, x_i \rangle - b - \epsilon - \xi_i) - \sum_{i=1}^n \alpha_i^* (\langle w, x_i \rangle + b - y_i - \epsilon - \xi_i^*) - \sum_{i=1}^n (\mu_i \xi_i + \mu_i^* \xi_i^*) \quad (3.8)$$

The dual problem is obtained by differentiating $\mathcal{L}$ with respect to w, b, ξ_i, ξ_i^* setting these derivatives to zero and solving for the Lagrange multipliers [33]. Then by substituting them in (3.6) objective function can be rewritten as follows:

$$f(x) = (\alpha_i - \alpha_i^*) \langle x_i, x \rangle + b \quad (3.9)$$

The solution to this dual problem reveals the complexity of the function does not depend on the dimensions of the $\mathcal{X}$ space but rather on the number of support vectors.

3.3.4 Kernel Trick

The kernel trick enables SVR to model non-linear relationship by allowing the implicit mapping of input data into high-dimensional feature spaces without the need for explicit computation of the coordinates in that space. It achieves this by replacing dot products in the original feature space with a kernel function, $K(x, x')$ which implicitly performs this mapping. Kernel functions, $K(x, x') = \langle \Phi(x), \Phi(x') \rangle$, compute the inner product of vectors in a transformed feature space Φ without direct calculation in that space. The choice of K determines the nature of the feature space and by extension the ability of SVR to capture non-linear patterns. Coming back to the dual formulation of the SVR, the dual problem and decision function, rely on dot products $\langle x_i, x \rangle$ will instead use $K(x, x)$. This leads to a decision function for predictions seen in (3.10) [33], which operates in the high-dimensional feature space implicitly.

$$f(x) = (\alpha_i - \alpha_i^*)K(x_i, x) + b \quad (3.10)$$

3.3.5 Common Kernel Functions

Linear Kernel: $K(x, x') = \langle x, x' \rangle$ This kernel does not map to a higher-dimensional space and is equivalent to performing linear SVR [34].

Polynomial Kernel: $K(x, x') = (\gamma \langle x, x' \rangle + r)^d$ where γ, r, d are parameters defining the kernel's shape. This kernel maps inputs into a polynomial feature space [34].

Radial Basis Function (RBF) Kernel: $K(x, x') = \exp(-\gamma \|x - x'\|^2)$ where $\gamma > 0$ controls the kernel's width. The RBF kernel maps inputs into an infinite-dimensional space, facilitating complex non-linear mappings [34].

3.3.6 Selection of Support Vector Regression for Outlier-Rich, Small Datasets

The SVR algorithm is suitable for small datasets with high variance and the presence of outliers due to its inherent properties. SVR is robust against outliers, as it uses an epsilon-insensitive loss function, which ignores errors within a certain threshold, thereby reducing the influence of outlier points [33]. Additionally, SVR is effective even with limited data because it focuses on the support vectors rather than the entire dataset, making it less prone to overfitting [35].

3.4 Bayesian Optimization

The parameters of all the kernels described in the previous section are called hyperparameters and will guide the SVR's predictions, thus they need to be optimized. Optimizing a machine learning model's hyperparameters is called hyperparameter tuning. Bayesian optimization is an approach for finding the optimal parameters (hyperparameters) of a function that is expensive to evaluate. This method relied on three key concepts: a probabilistic model, an acquisition function, and an iterative optimization process.

3.4.1 Gaussian Processes

Gaussian Processes (GPs) are a set of arbitrary variables, any finite number of which have a shared Gaussian distribution. In the context of Bayesian optimization GPs are the surrogate models that are used to model the unknown objective function model the unknown objective function $f(x)$ (e.g., validation accuracy in machine learning context) as a function of hyperparameters [36]. It's defined by two main components, the mean function which represents the average prediction of

the function at any point and the covariance function which describes how points in the input space are correlated with each other. The GP model provides a probabilistic forecast of the objective function, allowing for the estimation of both the expected value and the degree of uncertainty at previously unexplored points in the hyperparameter space.

3.4.2 Acquisition Function

The next hyperparameter configuration to evaluate is selected based on an acquisition function that leverages the GP's predictive mean and variance to balance the exploration (sampling from regions with high uncertainty) with exploitation (sampling from regions where the model predicts high performance). Several types of acquisition functions exist, each with its own strategy for this balance [36]. The most common is the Expected Improvement (EI) which focuses on areas that might offer improvement over the current best result.

3.5 Evaluation Metrics and Cross-Validation

For regression models, key metrics that measure accuracy and performance are detailed in Appendix A.5.5. Cross-validation ensures the model's robustness by training and testing on different data subsets. K-fold cross-validation involves dividing the data into k parts, using each for testing while the others are used for training, to validate effectiveness on unseen data.

4 Design and Data Description

4.1 Introduction

The NIA-AA AT(N) framework provides a biological definition of AD that serves as the foundation for this study to determine the plasma counterparts of CSF. This study will analyse the Amyloid (A) biomarkers, specifically $A\beta_{42}$, $A\beta_{40}$, and the $A\beta_{42}/A\beta_{40}$ ratio. Additionally, it will examine P-tau for the Tau (T) component and NfL for the Neurodegeneration (N) component, thereby covering all essential aspects. Given length constraints, the primary focus will be on the $A\beta_{42}$ as two datasets are available, allowing for more comprehensive analysis and validation of results. Detailed exploratory data analysis of the remaining biomarkers is available in Appendices B, C, D, and E. In terms of the features used in the model, the study will focus exclusively on easily accessible clinical data so that the research can be more widely applicable and replicable in a variety of healthcare settings, without the need for specialized equipment or expertise.

4.2 Overview of Study Design

Figure 4.1 illustrates the workflow for this study. Initially, a dataset was collected and pre-processed, then data exploration was performed involving visualization and statistical analysis to understand the patterns and relationships within the target plasma biomarker.

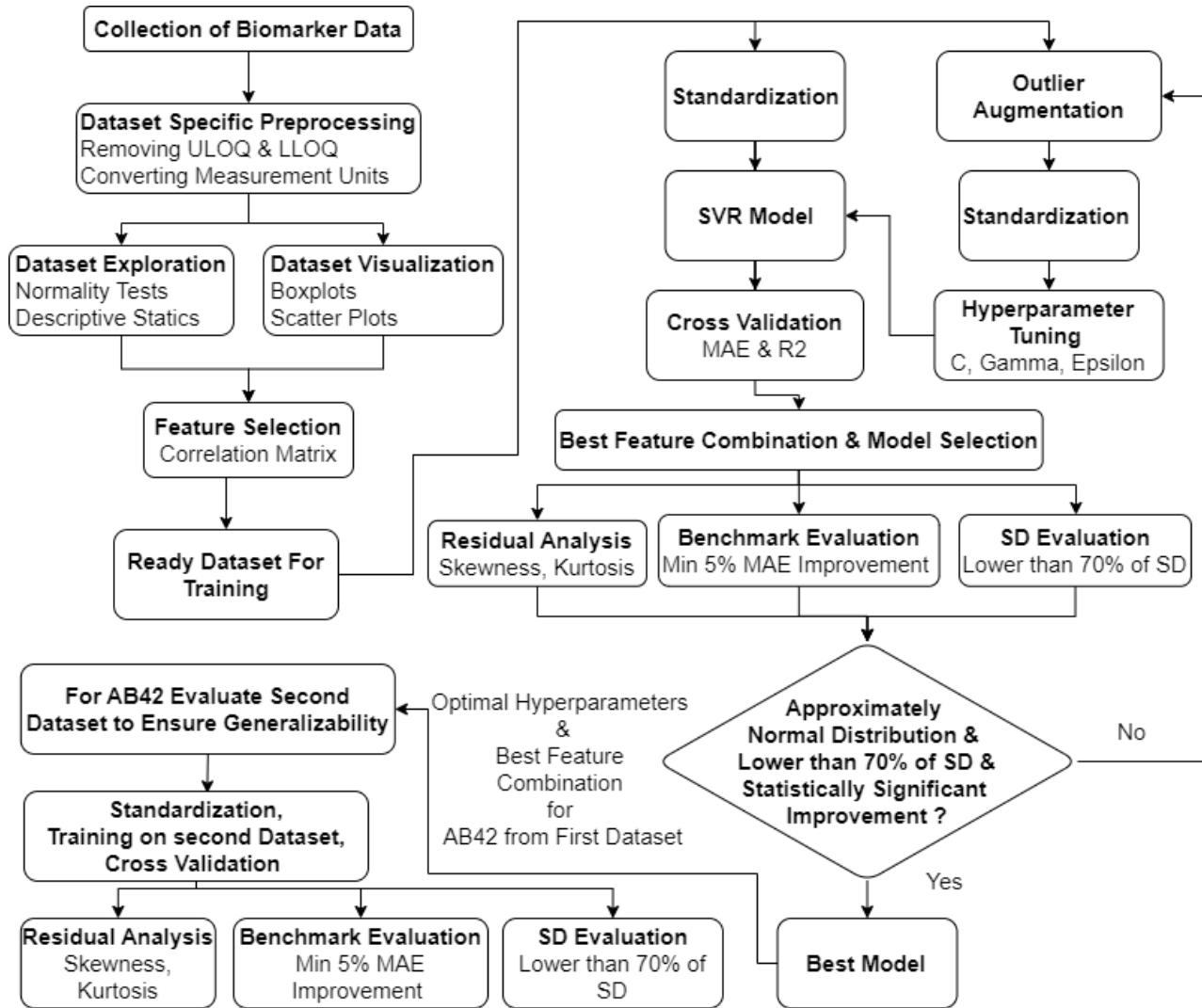


Figure 4.1: Flowchart of Methodology Process

Multicollinearity was evaluated using a correlation matrix, followed by feature selection. The dataset was then prepared for an SVR model, split into two: one for Bayesian optimization tuning (augmented with outliers and scaled) to ensure quick convergence and prevent overfitting during hyperparameter tuning, and another for training and cross-validation using a 10-fold method.

Post cross-validation, the feature set yielding the lowest MAE (with R^2 as a secondary criterion) was selected. This set underwent residual analysis to check for model bias and was benchmarked to objectively measure performance. If the model showed no bias and significantly improved MAE

(p-value < 0.05) compared to the benchmark, it was chosen as the best predictor for the plasma biomarker. Otherwise, the hyperparameter tuning process was revisited. At the end the final model was also tested on its precision by computing MAE/SD. The optimal hyperparameters for A β 42 were applied to train an SVR model, tested on a distinct dataset to verify its generalizability.

4.3 Data Source and Study Population

The dataset utilized in this study was sourced from the Alzheimer's Disease Neuroimaging Initiative (ADNI), encompassing data across its 3 phases: ADNI 1, ADNI-GO - ADNI 2, and ADNI 3. ADNI has systematically enrolled over 2,000 participants since its inception, aimed at investigating the progression of Alzheimer's disease through the analysis of clinical, imaging, genomic, and biomarker data [37]. Each participant in every dataset is uniquely identified by their RID (Research ID). Participants often have multiple assessments within a year, and across different ADNI phases leading to repeated RIDs. The VISCODE2 column in each dataset distinguishes each visit with month-specific codes, differentiating baseline assessments ('bl') from subsequent visits. To address data independence issues crucial for machine learning models, only the baseline assessments will be used, thus maintaining data integrity by preventing covariance. However, this results in the elimination of over half the dataset.

4.3.1 Plasma and CSF Data Collection

CSF A β biomarkers, were obtained from the FUJIREBIOABETA.csv (FUJI) [38] and UPENNBIOBK_MASTER_FINAL.csv (UPEN) [39] files. The UPEN file also provided data on CSF P-tau181, while levels of NfL in the CSF were sourced from BLENNOWCSFNFL.csv (Blennow_csf) [40].

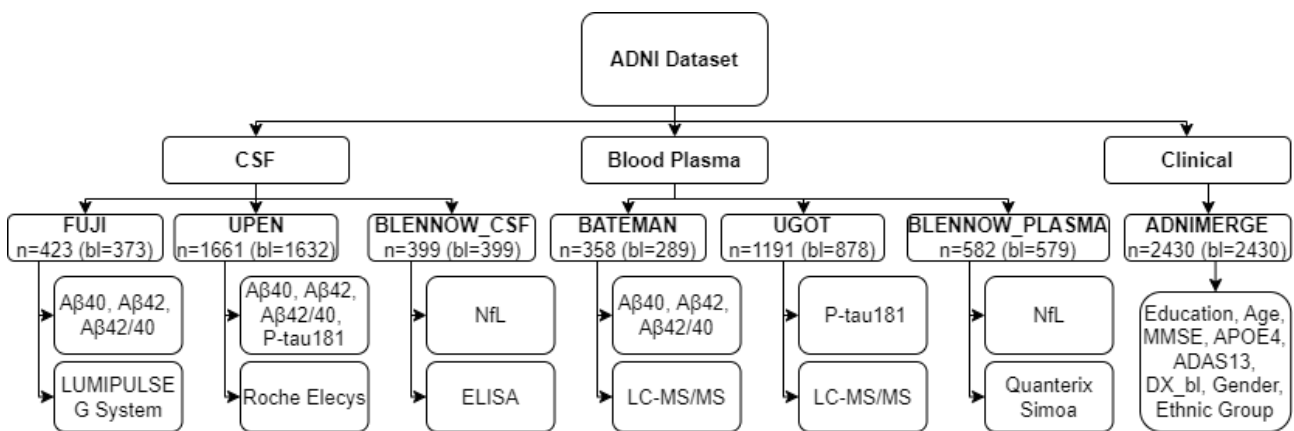


Figure 4.2: Summary of data, including plasma and CSF biomarkers, their quantification methods, and the number of participants in total (n) and at baseline (bl) along with the clinical data.

For plasma biomarkers, amyloid-beta peptide levels were derived from batemanlab_20221118.csv (Bateman) [41], plasma P-tau181 levels from UGOTPTAU181.csv (UGOT) [42], and plasma NfL measurements from ADNI_BLENNOWPLASMANFL.csv (Blennow_plasma) [40]. Detailed information regarding each file is presented in Figure 4.2.

4.3.2 Clinical Data Collection, Selection and Basic Pre-Processing

Clinical data were extracted from the "ADNIMERGE.csv" file, which consolidates key clinical information from all patients participating in ADNI into a single file. From this dataset, numerous variables were chosen to serve as features in the machine learning model. These included the years of education (Education), Age, the Mini-Mental State Examination (MMSE) score, the existence of Apolipoprotein E ϵ 4 allele (APOE4), the Alzheimer's Disease Assessment Scale-Cognitive Subscale 13 at baseline score (ADAS13), the gender, the ethnic group and finally the diagnosis at baseline (DX_bl) as can be seen from Figure 4.2. A thorough analysis and explanation of each variable is available in the appendix A.6.

4.3.3 Main Issues of the Plasma and CSF Datasets

UPEN Batches: The UPEN dataset, initiated in 2007 at the University of Pennsylvania, utilized the INNO-BIA AlzBio3 immunoassay, collecting eight sample batches through 2016 across various ADNI phases [43]. Due to the variability in kit measurements impacting data reliability, in 2017, ahead of ADNI3, a switch was made to the Roche Elecsys automated immunoassay, known for its precision, accuracy, and consistent reagent performance [39]. All previous samples were re-quantified using this method, creating a new batch (UPENBIOMK9) used for analyses post-2016 to ensure higher data quality and address issues with earlier immunoassay tests. In this thesis data from batch 9 onwards were exclusively utilized, thereby ensuring that the analysis is based on the most accurate and reliable quantification method available, addressing previous concerns associated with RUO immunoassay tests.

Upper and Lower Limits of Quantification (ULOQ, LLOQ): These limits show the highest/lowest concentrations of a substance that can be accurately quantified with a given analytical method [44]. Most of the datasets had some participants that exceeded these, and thus were removed.

Repeated RID values: Despite the dataset comprising solely baseline visits, 30 RID values were duplicated. For these repeated RID values, the biomarker values were averaged.

Bateman units: In the Bateman study for A β plasma the units were given in ng/mL and were converted to pg/mL for easier comparison with other studies.

4.4 Assessing Normality on Pre-Processed Data

The Kolmogorov-Smirnov (KS) test along with skewness and Fisher's kurtosis in Table 4.1 are utilized to assess the distribution of continuous and ordinal data, guiding the choice of statistical descriptions and correlations in later analysis.

Table 4.1: Kolmogorov-Smirnov test for normality on continuous and ordinal variables.

Variable	UPEN			FUJI		
	Skewness	Kurtosis	KS p-value	Skewness	Kurtosis	KS p-value
Age	-0.008	-0.189	0.917	<0.001	-0.450	0.968
Education	-0.230	-0.830	<0.001	-0.436	-0.457	0.002
MMSE	-1.278	1.268	<0.001	-1.228	0.944	<0.001
ADAS13	0.743	0.288	0.033	0.851	0.493	0.139
CSF ABETA42	0.425	-0.738	0.025	1.051	0.580	0.012
Plasma ABETA42	0.365	1.221	0.182	-0.126	1.110	0.716

Education, MMSE, and CSF A β 42 exhibit significant deviations from normal distributions across both datasets, as evidenced by their low KS p-values (Education in UPEN: <0.001, FUJI: 0.002; MMSE in both: <0.001; CSF ABETA42 in FUJI: KS p-value: 0.012). MMSE shows pronounced left skewness (UPEN: -1.278, FUJI: -1.228), indicating a distribution leaning towards lower scores, while CSF ABETA42 in FUJI displays notable right skewness, pointing to a leptokurtic distribution where data are more peaked and tails are heavier, suggesting a concentrated range of values with more extreme outliers in biomarker levels. ADAS13, despite its skewness (UPEN: 0.743, FUJI: 0.851) and kurtosis indicating a mild deviation, has KS p-values (UPEN: 0.033, FUJI: 0.139) that do not support a normal distribution assumption, especially in UPEN. After a visual inspection and given the larger size of the UPEN dataset, ADAS13 will be treated as not following a normal distribution.

Conversely, Age in both datasets approximates a normal distribution with minimal skewness and kurtosis, reflected in a high KS p-value (UPEN: 0.917, FUJI: 0.968), indicating a symmetric and moderately tailed distribution of ages. Finally, Plasma ABETA42 in FUJI and UPEN demonstrates a

tendency towards normal distribution, with FUJI showing slight left skewness (-0.126) and UPEN a mild right skewness (0.365). Despite the observed skewness and kurtosis differences, the KS p-values (FUJI: 0.716, UPEN: 0.182) suggest these distributions are relatively normal. However, the difference between datasets highlights the large variance for this variable.

4.5 Dataset Descriptive Statistics

Considering the normality tests, data adhering to a normal distribution (Age, Plasma A β 42) were presented as means $\pm$ standard deviations. In contrast, non-normal data were described using medians and IQR including the 25th and 75th percentiles. P-values were not computed for differences between the datasets because they use overlapping patient samples from the same source (ADNI). Therefore, comparisons between these datasets should be avoided due to their lack of independence. An depth comparison of AD versus Non-AD can be found in Appendix A.7.

Table 4.2: Descriptive statistics of the UPEN and FUJI datasets.

Variable	UPEN (n=211)	FUJI (n=127)
Age, years (mean, SD)	71.95 $\pm$ 7.33	71.42 $\pm$ 7.34
Education, years (median, IQR)	16.00 (15.00 - 18.00)	16.00 (14.00 - 18.00)
MMSE (median, IQR)	29.00 (27.00 - 30.00)	28.00 (26.00 - 29.50)
ADAS13 (median, IQR)	13.67 (9.00 - 19.00)	14.00 (8.00 - 19.50)
CSF A β 42, pg/mL (median, IQR)	915.70 (662.45 - 1178.50)	712.00 (531.00 - 1083.00)
Plasma A β 42, pg/mL (mean, SD)	32.99 $\pm$ 5.55	33.07 $\pm$ 5.57
E4+ (%)	46.92	48.82
Gender (M%)	55.92	54.33
Ethnic Group (White%)	90.52	92.91

Table 4.2 reveals a notable limitation in the diversity of the datasets, as they predominantly consist of individuals from a single ethnic group. This lack of diversity may limit the generalizability of findings to more ethnically diverse populations. The ethnic group feature will be omitted from further analysis as its inclusion will not provide any meaningful value due to most participants in both cohorts being white. Additionally, another notable limitation is the considerable variance observed from the standard deviation in CSF and plasma levels in both datasets which indicates a wider spread in the biomarker levels that will probably affect the SVR model's performance.

4.6 Outliers

In this study, outliers that represent extreme values will be retained, as it is assumed they constitute natural measurements rather than inaccuracies. Only outliers resulting from measurement errors, such as exceeding the ULOQ and LLOQ, have been excluded.

5 Implementation

5.1 Introduction

Data analysis is the most important and time-consuming part of the implementation of a machine learning model. Currently, for the Plasma A β 42 peptide there are two available datasets (UPEN and FUJI), this section will compare them, to unveil their differences but also the visualize the patterns of their variables. Using this information the best dataset will be selected for the training of the primary SVR and the second will be used to test the robustness of the first model. Additionally, this process along with a correlation matrix will investigate the correlation between various features and Plasma A β 42 and identify the most predictive features to be selected as inputs for the final model. Finally, a description for evaluating the performance of the best SVR model will be implemented.

5.2 Biomarker Levels by Diagnosis Category

To highlight variations within the datasets box plots were utilized. The whiskers in the boxplot show values that extend within 1.5 times the IQR, values outside those are considered outliers. This approach provides a clear visual representation of how biomarker levels differ across AD progression stages, offering insights into which dataset offers the best diagnostic utility of CSF and plasma samples. The P-values were calculated using the Mann-Whitney U-test for CSF A β 42 and the Student t-test for Plasma A β 42.

In Figure 5.1, CSF A β 42 can distinguish between all groups with p-values near zero across all categories, indicating significant differences. Additionally, higher concentrations of A β 42 are observed for the cognitive normal patients declining with AD progression. Conversely, for plasma A β 42 there is no statistically significant distinction between the groups across both datasets, with the FUJI dataset having higher p values because of the smaller size. Notably, it can be observed both datasets exhibit much variance, especially FUJI and they both contain outliers.

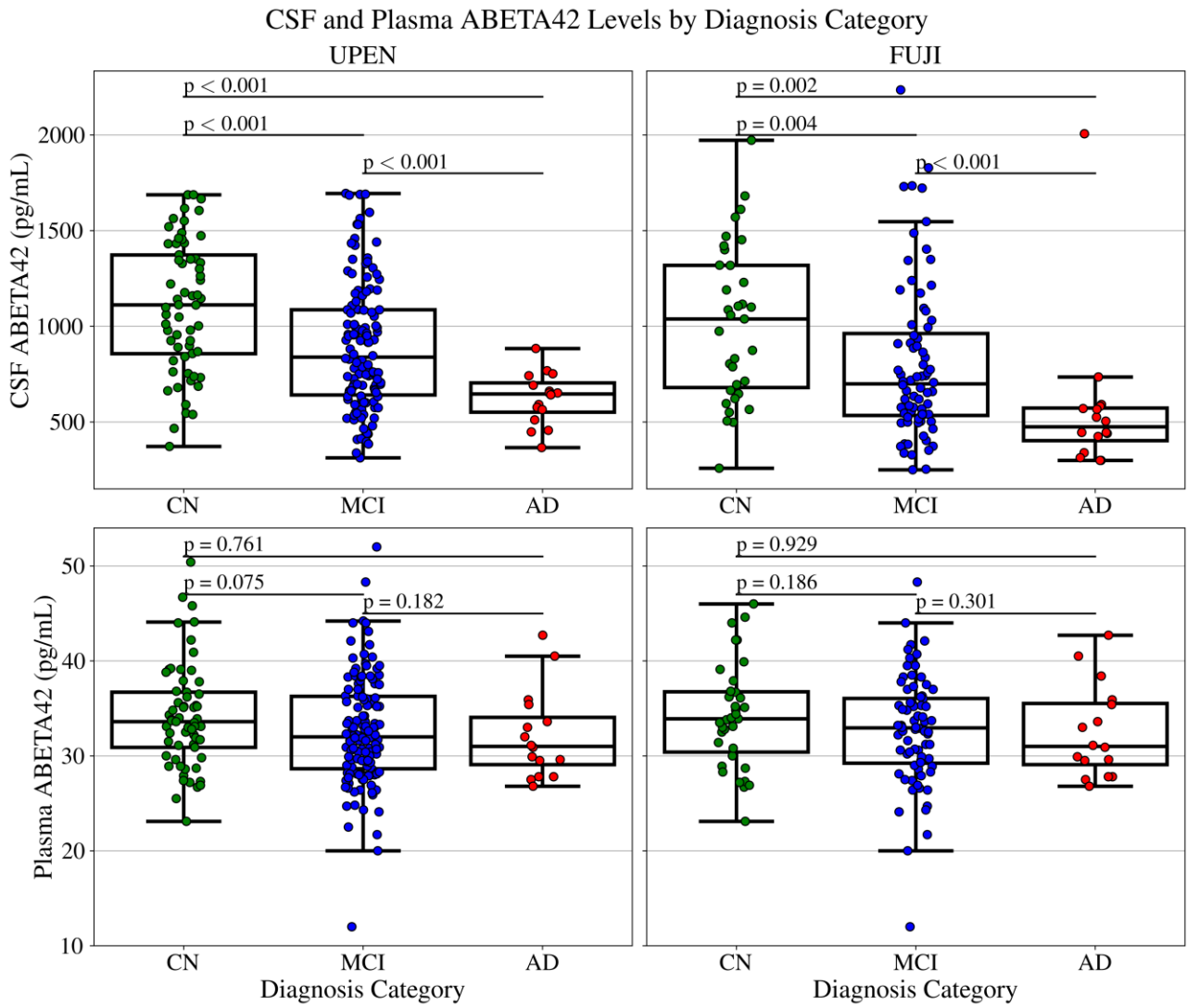


Figure 5.1: Box Plots of CSF and plasma by diagnosis category across UPEN and FUJI datasets.

5.3 Exploration of Clinical Data and Biomarker Relationships & Selection of Dataset

This section aims to reveal the relationships between plasma A β 42 and the continuous and ordinal clinical data as well as the biomarker's interactions with AD progression. This information will facilitate the final dataset for training to be selected. The objective is to visualise biomarker trends and inconsistencies within the datasets and across different stages of AD. The graphs use a consistent y-axis, simplifying comparisons and enhancing understanding of AD stage effects on the biomarker. The upcoming sections will delve deeper into the analysis of correlations.

Figure 5.2 indicates that only ADAS13 exhibits a weak, yet statistically significant, correlation with plasma A β 42, while education appears uncorrelated. A notable finding is the ability to distinguish patients across different AD stages despite the lack of strong correlations between MMSE, ADAS13, and plasma A β 42, accompanied by a decrease in plasma levels as AD progresses.

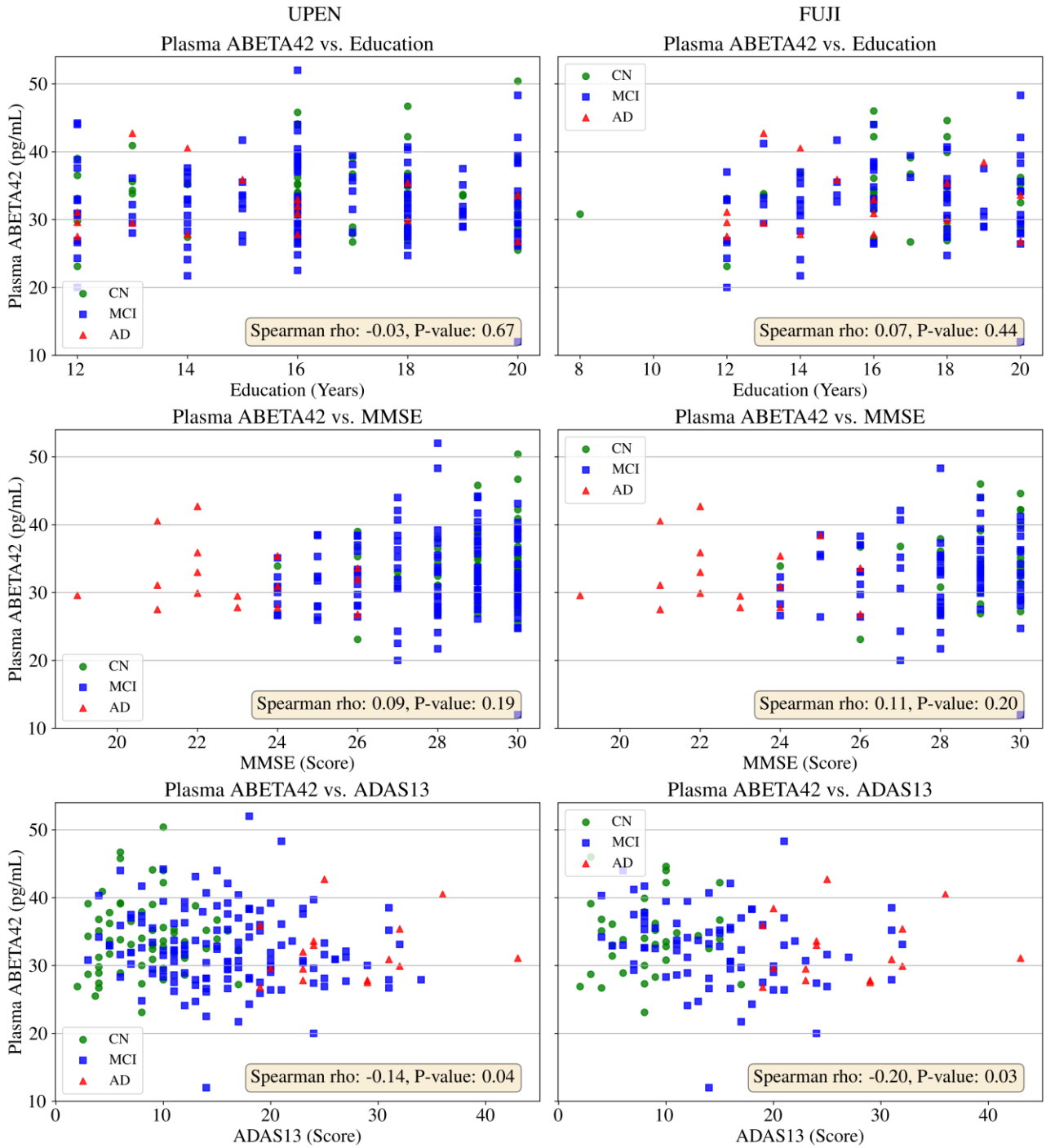


Figure 5.2: Plasma A β 42 versus ordinal categorical variables across UPEN and FUJI datasets.

In Figure 5.3, a regression line is introduced to show the correlation between age, CSF A β 42, and plasma A β 42 levels, aiming to demonstrate how fluctuations in age and CSF A β 42 are associated with changes in plasma biomarker levels. Despite the skewed distribution of CSF A β 42, which typically advises against using Pearson correlation for analysis, the regression lines were included to provide a visual approximation of these relationships. In both datasets, it is observed that both age and CSF A β 42 exhibit a moderate, statistically significant correlation with plasma A β 42,

showing a linear increase in tandem. A distinct separation between AD stages is apparent in CSF A β 42 levels with AD patients presenting lower values and MCI showing an increase and CN individuals having higher levels in both plasma and CSF A β 42.

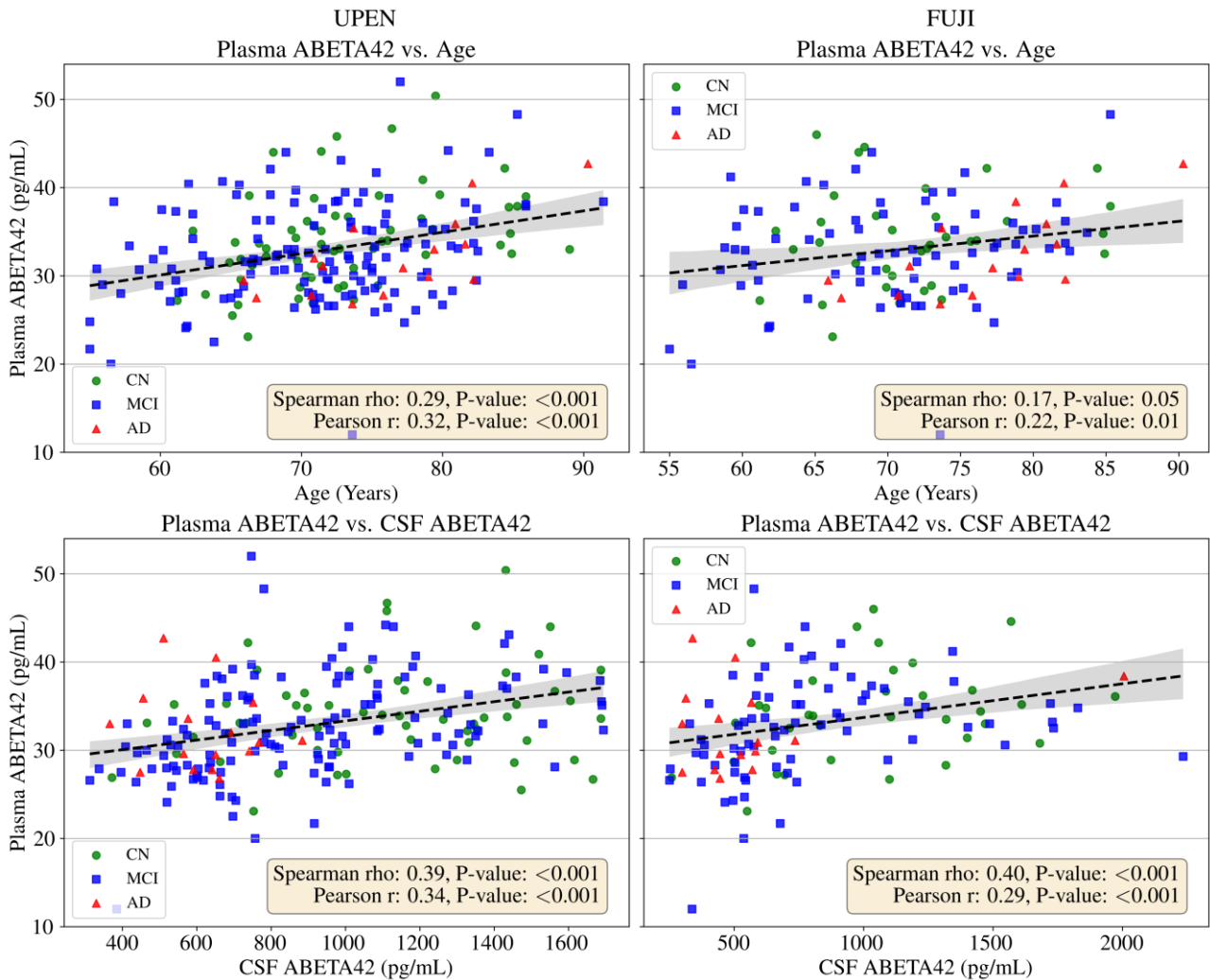


Figure 5.3: Plasma AB42 versus continuous variables across UPEN and FUJI datasets.

Overall, high variance is observed across all figures, making prediction difficult for machine learning models. Since UPEN has more data and exhibits clearer relationships between the features and the target variable it will be selected as the core dataset the training will be performed on.

5.4 Feature Selection and Preparation for Support Vector Regression Model

5.4.1 Multicollinearity Testing Using Spearman Correlation Matrix

The Spearman's correlation matrix shown in Figure 5.4 will be used to assess the relationship

between variables because it does not assume a normal distribution or linear relationship between the variables and is more robust against outliers.

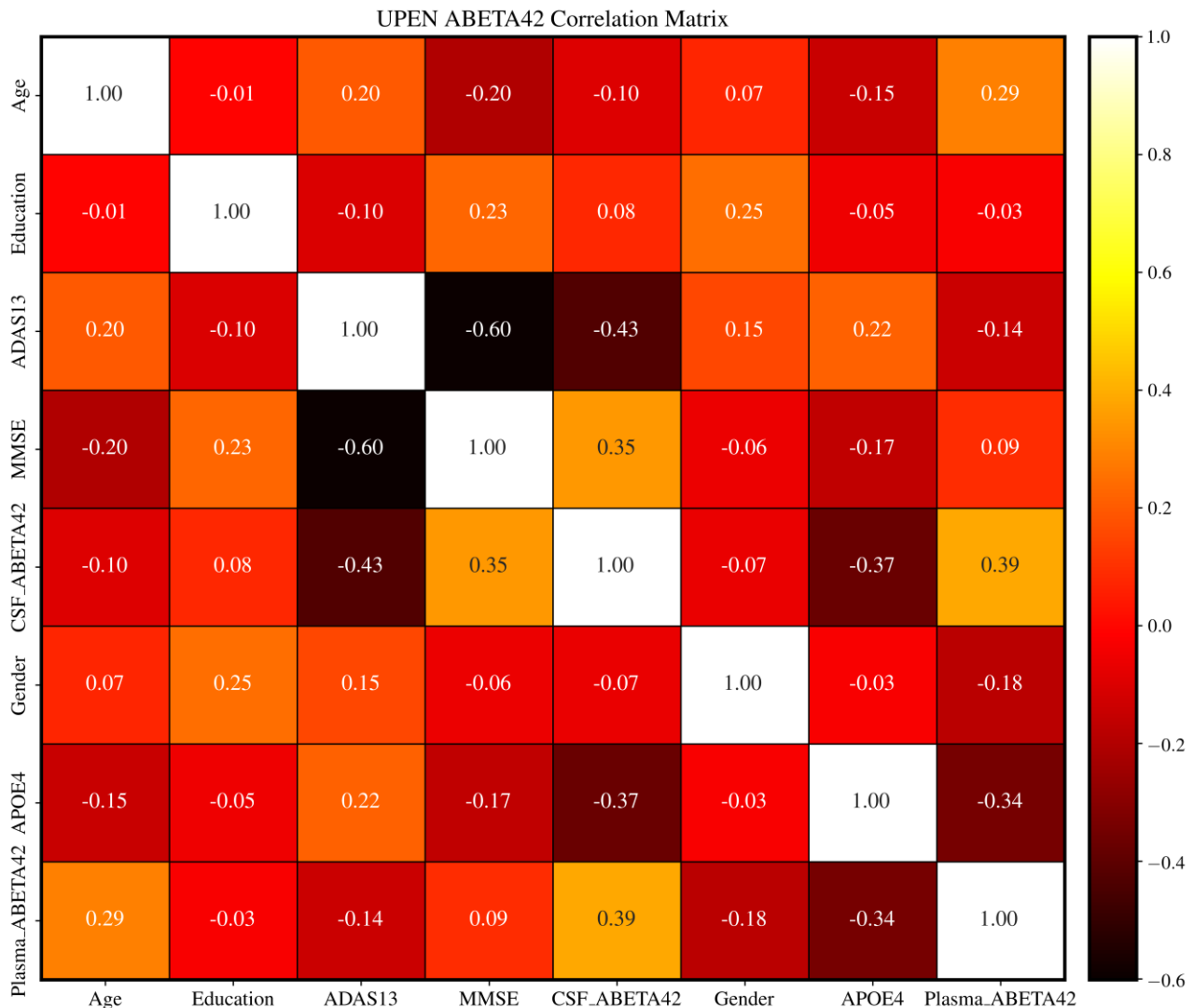


Figure 5.4: Spearman correlation matrix for UPEN A β 42 dataset.

Regarding multicollinearity among features, as expected there is a strong correlation between MMSE and ADAS13, as both serve as indicators of AD in clinical settings. Additionally, concerning correlations emerge between CSF A β 42 and APOE4, and between ADAS13 and MMSE. This stems from their shared association with AD, as these features aid in distinguishing AD from non-AD and categorizing within AD groups. These correlations present a challenge since each feature is vital to the model. Consequently, despite ADAS13's slightly stronger correlation with the target, its higher correlation with other features, compared to MMSE, necessitates its removal.

Thus, the final set of independent variables (features) that will be used to predict plasma A β 42 using the UPEN dataset, are age, education, APOE4, MMSE, CSF A β 42 and gender.

5.4.2 Considerations on Normalizing Features

Preliminary assessments in Section 4.4 revealed that A β 42 levels across the UPEN dataset did not conform to a normal distribution, a characteristic often observed in clinical biomarker data due to underlying biological variability and disease heterogeneity. Attempts to normalize the distribution of CSF A β 42, including logarithmic and Box-Cox transformations, were considered. However, these transformations altered the inherent non-linear relationship between CSF A β 42 levels and the plasma A β 42 potentially obscuring biologically meaningful associations.

Given the importance of preserving the original distribution's characteristics to maintain the integrity of the biomarker's relationship with the target variable, CSF A β 42 was analysed in its skewed state. This decision allows for a more accurate reflection of the biological reality represented by the data, ensuring that the non-linear dynamics are not compromised.

5.5 Model Selection

5.5.1 Outlier Augmentation

In the preprocessing phase, the presence of outliers was identified, and to address them a unique approach was implemented. Instead of removing them, they were identified before the optimization stage and adding these outliers five more times to the dataset. This serves as a regularization parameter within the Bayesian optimization process, ensuring that hyperparameter tuning does not overly conform to the specific characteristics of the dataset, thereby promoting better generalization. Outlier detection is done by calculating z-scores for the target variable and observations displaying z-scores exceeding an absolute value of 3 — a conventional threshold indicating deviations from the mean by more than three standard deviations — identified as outliers. These outliers are then replicated in the dataset, increasing their count fivefold, only for the Bayesian optimization phase.

5.5.2 Feature Standardization

Following the outlier augmentation process, the dataset undergoes standardization to ensure a consistent scale across all features. This preprocessing step is crucial for models sensitive to the scale of input data, such as Support Vector Regression, by mitigating potential biases that could arise from the disparate ranges of feature values. By standardizing the features, the model's optimization algorithms can converge more efficiently, enhancing the overall predictive

performance and stability of the model across diverse datasets. This step is applied to both the Bayesian optimization set and the original dataset made for the evaluation.

5.5.3 Hyperparameter Tuning Using Bayesian Optimization

After feature standardization, Bayesian optimization is utilized for hyperparameter tuning of the SVR model. This method constructs a probabilistic model to efficiently identify hyperparameter combinations that are likely to improve model performance, specifically aiming to minimize the MAE. Bayesian optimization uses information from previous evaluations to make informed decisions on which hyperparameters to explore next. This approach ensures an effective balance between exploring new hyperparameter spaces and exploiting known configurations that offer promising results. The choice of MAE as the performance metric during this optimization process is due to its interpretability and robustness to outliers. The search space given to the optimizer was constrained, not allowing very large values of C or very small values of epsilon and gamma that will increase the accuracy but make the model susceptible to overfitting.

5.6 Model Evaluation

5.6.1 Evaluation Process and Metrics

Given the dataset's noisiness and presence of outliers, MAE is chosen as the primary performance metric due to its robustness against outliers, unlike Root Mean Squared Error (RMSE) and R-squared (R^2), which are more susceptible to their influence.

Aiming for an MAE that is about 60% to 70% of the SD will be considered the goal for this biomarker. This range acknowledges the variability of the data but sets a goal that the model's predictions are, on average, closer to the true values than a significant portion of the data's natural variability. Given the SD of 5.55 for A β 42 in UPEN dataset, an accurate MAE target would thus be between 3.33 – 3.89 pg/mL.

To bolster the reliability and generalizability of the analysis, 10-fold cross-validation is used ensuring thorough evaluation and minimization of overfitting by training and validating the model across multiple data subsets. The cross-validation will be implemented using Scikit-learn's cross-validate function incorporated within a pipeline that standardizes the training set for each fold and configures the SVR model with the optimal hyperparameters identified via Bayesian optimization.

The approach outlined permits an indirect evaluation of the contribution of individual features and their combinations to the SVR model. By plotting the average MAE for every feature combination, the impact on model performance can be visualized. Additionally, R^2 will be utilized as a secondary metric to identify which feature combinations most accurately fit the data and explain its variance. Based on these analyses, the optimal set of features, alongside their respective SVR configurations, will be selected for the best prediction of each plasma biomarker.

5.6.2 Residual Analysis

Before selecting a model, its performance is assessed to identify systematic biases or specific weaknesses through residual analysis. Utilizing Sci-kit-learn's `cross_val_predict` with the cross-validation setup, predictions for each target value are made. The residuals—differences between actual and predicted values—are then scrutinized. A histogram with Kernel Density Estimation (KDE) from the Seaborn library will visualize the error distribution. A well-behaved model would ideally show a random scatter of residuals, suggesting no unaccounted patterns or relationships. Furthermore, the overlay of KDE should reveal a normal distribution shape, reinforcing the absence of skewness or heavy tails that would indicate model underperformance in specific segments. Such outcomes would affirm the model's reliability and its capability to generalize well. If this does not hold then the whole evaluation process will start again making the necessary changes until any observed bias is eliminated.

5.6.3 Evaluating Model Robustness by Utilizing a Second Dataset

Assay-specific differences, such as the varying sensitivities and specificities between the LUMIPULSE G system used in the FUJI dataset and the Roche Elecsys used in the UPENN dataset, can introduce variability that affects the model's performance. To investigate the impact of these differences on model generalizability, the optimization parameters and feature combinations identified from the UPENN dataset were applied to train a model on the FUJI dataset. This step serves to explore the extent to which the model can maintain consistent predictive quality despite the extra variability from distinct assay methodologies.

5.6.4 Baseline Evaluation

The ultimate assessment of the model's performance will hinge on its comparison with a baseline predictor. Given the absence of prior studies on biomarker prediction, there isn't a universally

accepted 'good' evaluation metric specific to this domain. Therefore, a regressor model will serve as the benchmark for the final evaluation. Success will be defined by the model's ability to achieve a statistically significant improvement by at least 5% in MAE over this regressor model. Such an improvement would validate the model's effectiveness and its utility in the context of biomarker prediction. In this instance, a naive regressor, which predicts outcomes based on the mean of the target variable, will be utilized as the benchmark. Given that SVR models exhibit a degree of robustness to outliers and often yield predictions that gravitate towards the target variable's mean, it becomes particularly informative to contrast their performance against a model whose predictions are fundamentally anchored in calculating the mean. This comparison aims to illuminate the SVR model's relative advantage in handling complex data patterns beyond mere central tendency estimation.

5.6.5 SVR Models for the Remaining Biomarkers

The process detailed previously will be used to quantify other plasma biomarkers. A β 40, A β 42/40, PTAU, and NFL are analysed in appendices B, C, D, and E, respectively, with findings summarized in the results section.

6 Results and Discussion

6.1 Primary Biomarker Prediction Results

This section presents the optimized results for Plasma A β 42, achieved through an iterative evaluation process until the evaluation metrics and residual analysis were fully optimized. These results are the culmination of numerous trials detailed in Section 6.1.6.

6.1.1 Model Selection

Figure 6.1 presents the optimally tuned models across various feature combinations, specifically focusing on minimizing the MAE. Using this and taking into consideration the R^2 scores as outlined in Appendix A.8, the best feature set was selected.

The feature set comprising age, APOE4 allele presence, CSF A β 42 levels, and gender emerges as the most effective combination, achieving the best MAE of 3.46 pg/mL — which is within 62% of the SD (5.55 pg/mL) passing the threshold of 70% target — and the best R^2 value (Appendix A.8), indicating approximately 28% of the variance in the target variable is explained by this model.

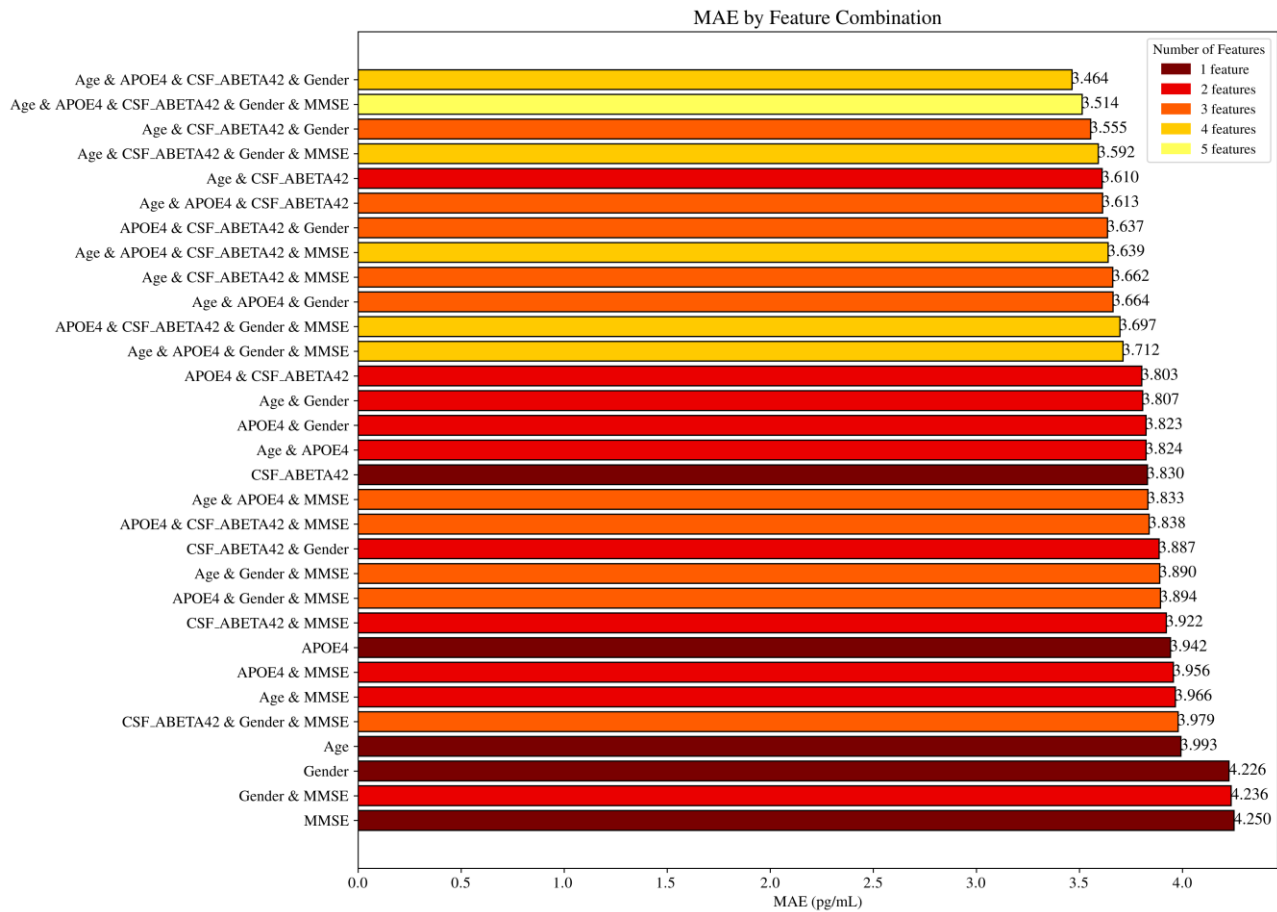


Figure 6.1: Evaluation of A β 42 models by feature combination using MAE.

This is closely followed by the combination of Age, APOE4, CSF A β 42, Gender and MMSE which ranks second for both MAE performance (3.51 pg/mL) and for R^2 (26%), getting worse in MAE by close to 1% and a decline in R^2 by 2%. Thus, suggests that while the model remains robust, the inclusion of the MMSE score probably due to multicollinearity lowers both the precision and explanatory power of the predictions.

For the optimal SVR model, the hyperparameters selected through Bayesian optimization show a balanced approach to model training. A regularization parameter C of 62.344 suggests a high penalty on errors, effectively focusing on achieving high accuracy but with the risk of overfitting. The ϵ value being quite low at 0.021, enforces a tight margin for errors, emphasizing precision in the model's predictions. Notably, the γ parameter at 0.007 is small allowing for a wider and smoother decision boundary, reducing the influence of noise from the training data, and focusing on the model's generalizability. The use of the RBF kernel further aids in dealing with non-linear patterns enhancing the model's ability to delineate complex relationships within the data.

6.1.2 Assessing the Generalizability of the Model Across Different Assay Systems

Testing the model on the FUJI dataset, which employs the LUMIPULSE G system, provided key insights into its generalizability. Despite the differing assay methodologies between UPENN's Roche Elecsys and FUJI's LUMIPULSE G, the model demonstrated a robust ability to maintain accuracy, achieving a MAE of 3.69 pg/mL. This result is not only an 18% improvement over the benchmark of 4.52 pg/mL but also indicates a significant reduction in error of 66% relative to the dataset's standard deviation of 5.57 pg/mL. Thus, the model's predictive quality is resilient to the variability introduced by different assay types. However, due to the small sample size in the FUJI dataset, these promising results were not statistically significant requiring further verification with a larger cohort to ensure they are reflective of the model's true generalizability.

6.1.3 Residual Analysis

Figure 6.2 shows the distribution of residuals from the best SVR model's predictions compared to actual values overlayed with KDA to visualize the distribution's form more clearly.

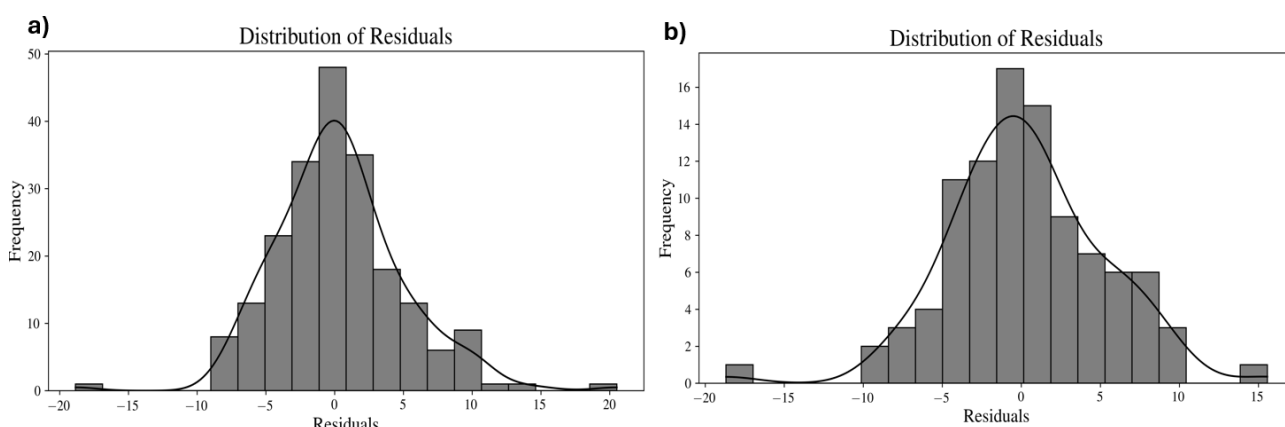


Figure 6.2: Distribution of Residuals for Aβ42 a) Optimally tuned SVR model on UPEN dataset, b) SVR model on FUJI dataset.

Figure 6.2a presents the residuals of the primary model with a bell-shaped distribution, skewed left, indicating a slight systematic underestimation of values. Skewness at 0.449 reflects this asymmetry toward lower predictions, revealing a consistent but mild bias. A Fisher's kurtosis of 2.19 suggests the distribution has heavier tails than a normal distribution, indicative of a leptokurtic tendency. This means that when the model's predictions are off, they tend to be significantly higher or lower than the actual values, albeit such errors are not the norm. This characteristic suggests that while the model's predictions are generally accurate when it errs, it's more likely to do so with a significant margin. This could be attributed to the model's response to

outliers or inherent variability in the data, which results in occasional significant deviations from the actual values when errors occur [45]. Figure 6.2b for the FUJI model shows a similar distribution but more platykurtic with high tails showing similar characteristics to Figure 10a. Thus, the model retains its predictive quality, managing to generalize well but still indicating room for improvement in handling dataset-specific variances.

6.1.4 Predicted Performance Overview

The residual analysis in Section 6.1.3, which shows a leptokurtic distribution for the both models with a mild systematic underestimation, directly informs the predictive performance variances detailed in this section. While the models demonstrate high precision around plasma A β 42 values within one SD of the median (28-38 pg/mL) with most predictions aligning closely with or exactly on the ideal fit line —underscored by a notably narrow 95% confidence interval (CI) — the accuracy of the model decreases for plasma values that deviate from this central range in both Figures 6.3a, b.

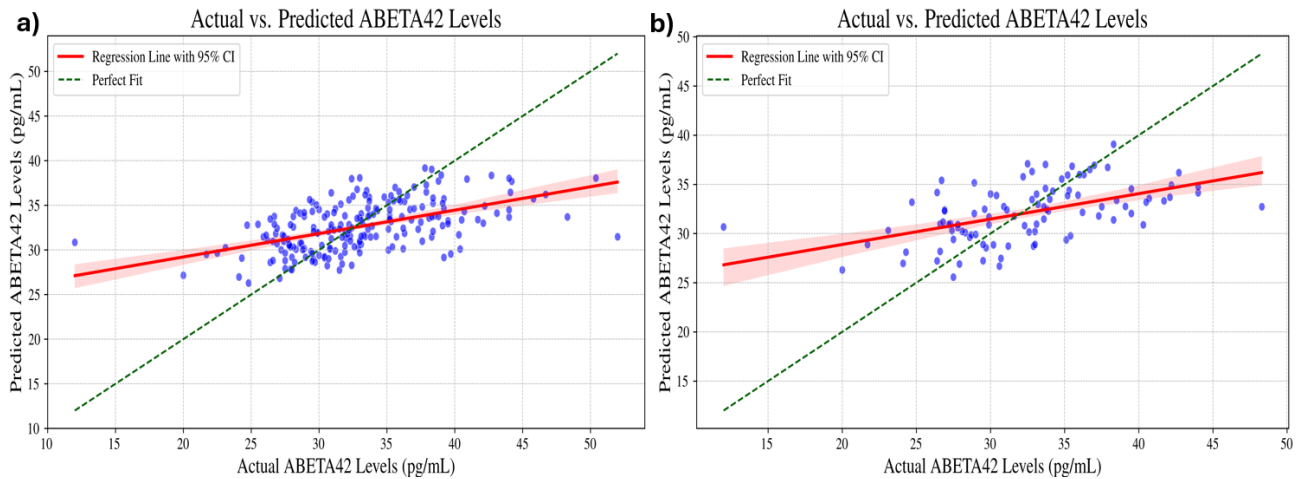


Figure 6.3: Scatter Plots of Estimated vs. Actual A β 42 Levels; (a) Performance on UPENN Dataset, (b) Performance on FUJI Dataset.

This decrease in predictive reliability for values outside this range may be linked to the dataset's noise as it was previously discussed, because of the considerable variance in plasma levels, which evidently impacts model performance. These outcomes were mirrored in Figure 6.3b, showcasing that the difference in CSF assay did not significantly impact the model's decisions.

6.1.5 Comparison Against Benchmark and Standard Deviation Test Evaluation

For the primary model using the UPEN dataset, under identical cross-validation parameters, the baseline naïve regressor benchmark model reported a MAE of 4.29 pg/mL and a R^2 score of -0.03%. Subsequent analysis utilizing the Mann-Whitney U test to compare predictions from the SVR model against those from the baseline model yielded a p-value of 0.046. This result signifies a statistically significant difference in their predictive performances. Thus, in predicting plasma A β 42 concentrations, the optimal SVR model demonstrated a noteworthy improvement, reducing the primary evaluation metric, MAE, by 19% and increasing the R^2 score by 27% compared to the baseline. In terms of the SD, the best model's MAE (3.46 pg/mL) was 62% of the SD (5.55 pg/mL) implying that the model was substantially more consistent than the random fluctuations inherent in the dataset. Consequently, the model achieved all the dissertation's objectives.

6.1.6 Failed Attempts at Improving the Model

The model consistently underpredicts for values above 38 and overpredicts below 28. Various strategies were implemented to address these prediction biases in the SVR model. Feature engineering was applied, doubling the number of features where the target variable exceeded these thresholds, but failed to enhance the performance. Complex polynomial features up to the third degree were created to reduce overfitting during optimization and training; however, this approach led to increased overfitting. A polynomial kernel initially showed promise with a lower MAE, but further analysis indicated it was overfitting, primarily predicting around the mean without effectively learning from the data. Different evaluation metrics, including RMSE, were also tested, but the presence of outliers confirmed MAE as the most robust metric for this model, despite these efforts not yielding better outcomes.

6.2 Evaluation Summary for All Biomarkers

In the same way, the previous section selected and analysed the best SVR model for plasma A β 42 all the other biomarkers were analysed in Appendices B, C, D, and E. In this evaluation, MAPE was also added to give perspective on the accuracy of the results, but MAE still is the primary evaluation marker. Before analysing the results from Table 6.1 it is important to note that the moderate R^2 values observed across all biomarkers, are common in medical research. This is particularly true in fields dealing with complex, multifactorial diseases such as neurodegenerative

disorders where the etiology is diverse and influenced by various genetic, environmental, and lifestyle factors [46]. In such scenarios, conventionally lower R^2 values are considered significant.

Table 6.1: Summary of performance metrics for all blood plasma biomarker models.

Blood Biomarker	R^2 (%)	MAPE (%)	MAE (pg/mL)	Benchmark MAE (pg/mL)	MAE Difference to Benchmark (%)	Benchmark P-value	MAE/SD (%)
A β 42	27.700	10.800	3.464	4.285	+19.160	0.046	62.342
A β 40	15.800	10.800	25.108	30.627	+18.020	0.941	61.169
A β 42/40	33.500	5.930	0.007	0.010	+30.000	0.059	74.000
P-tau181	8.600	29.900	6.027	7.203	+16.327	<0.001	42.654
NfL	26.100	31.400	12.370	16.524	+25.139	<0.001	45.730

The plasma A β 42/40 model exhibits a high R^2 (33.5%) and a low MAPE (5.93%), these suggest that it can effectively explain a portion of the variance in the target variable and does so with predictions that closely align with the true data. This can be due to a combination of the lower variance, and fewer outliers within the dataset or because plasma A β 42/40 might be closely linked to specific pathophysiological processes that manifest consistently across the studied population, which is possible since it has the best diagnostic capacity among all plasma biomarkers for AD [47] [48]. Thus, providing a consistent pattern that the SVR model can learn from. However, the MAE/SD ratio slightly exceeds the ideal threshold of 70%, indicating that while the model predictions are generally close to the true values, the size of the errors relative to the data's variability is higher than desired. Variability in this ratio could stem from biological diversity among subjects, measurement inaccuracies, or other unmodeled factors affecting the biomarkers. It also managed a massive 30% improvement to the benchmarks MAE which did not reach statistical significance due to the dataset's size.

In contrast, the P-tau181 model exhibits a low R^2 (8.6%) and a massive MAPE (29.9%), indicating its limited capability to capture and predict variance in clinical settings it is intended to track. This high error rate is likely exacerbated by the pronounced variance and frequent outliers in the data, which suggest that P-tau181 levels may be influenced by a wide range of factors or conditions not fully captured by the current model. The presence of these outliers and the variability could be indicative of complex biological processes affecting P-tau181 levels that are not consistently

represented across the population. Notably, the model's MAE/SD ratio of 42.65% is significantly below the 70% threshold, which, while generally positive, could also imply that the model, although conservative in error magnitude relative to data variability, struggles with accuracy in outlier detection and handling. Finally, it achieved a statistically significant improvement compared to the benchmark's MAE of 16%.

The A β 40 model, with an R^2 of 15.8%, demonstrates a limited ability to capture the variance in plasma A β 40 levels, yet it still outperforms the benchmark with a lower MAE by 18%. This suggests some efficacy in understanding core patterns despite a moderate MAPE of 10.8%. However, the MAE/SD ratio of 61.169% indicates that error magnitudes remain a significant fraction of data variability, suggesting a need for model refinement, possibly by improving feature selection or model complexity to better address A β 40's biological nuances.

The NFL model shows a reasonable R^2 of 26.1% but struggles with a high MAPE of 31.4%, reflecting substantial prediction errors likely exacerbated by high variance and frequent outliers. Despite these challenges, it achieves a lower MAE compared to the benchmark by 25% indicating some predictive accuracy improvements. The MAE/SD ratio at 45.730% suggests that while the model is broadly effective.

6.3 Limitations

While the study shows promise, its findings need further validation due to several limitations. The accuracy and representativeness of the data are crucial concerns, potentially affecting the validity of the developed machine learning models. Because some of the data are skewed or have heavy tails, the standard deviation might not capture the central tendency well. This misalignment means the MAE/SD ratio might either understate or overstate the model's effectiveness. For skewed distributions, other measures like the median absolute deviation (MAD) would be more appropriate.

The assumption that outliers are primarily due to genuine biological variations could ignore potential technical errors like measurement inaccuracies or data entry mistakes, which could mislead the learning algorithm and produce unreliable predictions. Furthermore, the study overlooks factors like medication interactions or other pathologies that may affect biomarker levels, potentially resulting in oversimplified models that fail to accurately represent biomarker

dynamics. This limitation diminishes the models' practical utility. The participant pool's limited demographic diversity, primarily white, also restricts the generalizability of the findings. These issues highlight the necessity for more comprehensive datasets to enhance the generalizability and reliability of future research outcomes.

The study's inability to precisely identify the sources of variance and outliers—whether from measurement errors or biological factors—compromises the robustness of the results. The reliance on a single assay technique for analysing CSF and plasma samples also limits the exploration of biomarkers and degradation pathways. These technical and methodological limitations raise questions about the results' applicability to broader populations, especially since the testing was restricted to a specific group.

7 Conclusions and Future Work

7.1 Conclusions

This dissertation has successfully developed the first machine learning model to predict blood biomarker levels using clinical and CSF data, representing a substantial advancement in Alzheimer's disease diagnostics. The study met all intermediate objectives despite facing challenges such as significant variance in blood plasma levels, the presence of outliers, and limited ethnic diversity in the dataset which restricted the models' generalizability. By employing methodologies such as feature selection, hyperparameter tuning, and outlier augmentation, all the SVR models outperformed their respective benchmarks in terms of MAE, indicating foundational effectiveness.

For the primary biomarker A β 42, the model improved mean absolute error (MAE) by 19% over the benchmark, maintaining precision within 62% of the standard deviation and an average absolute error of 10.8% relative to actual values. The model demonstrated robust generalizability as it was tested on a second dataset, achieving, an 18% improvement over the benchmark, and a 66% reduction in error relative to the standard deviation, though further validation with a larger sample is necessary.

The appendix details the predictive performance for other biomarkers (A β 40, A β 42/40, P-tau181, NfL), showing varied results. The A β 42/40 model showed the most promising results with a high R^2 and a low MAPE, suggesting strong predictive power and a reliable alignment with clinical

manifestations of Alzheimer's disease, though its MAE/SD ratio exceeded the ideal threshold, indicating room for improvement in error management. Conversely, the models for A β 40, P-tau181, and NfL encountered challenges with higher variability and error rates, exacerbated by significant data outliers. Each model's MAE/SD ratio, particularly the lower ratios observed in the P-tau181 and NfL models, reflects their relative efficiency in dealing with data variability, albeit with room for precision enhancement.

A limitation was the study's inability to thoroughly understand the reasons behind data variance and outliers, affecting the models' adjustment and prediction capabilities related to these issues.

This study lays the foundation for further research into quantifying AD biomarkers using machine learning. The findings confirm the feasibility of this approach and set the stage for broader diagnostic accessibility. By democratizing access to AD biomarker diagnostics, this method has the potential to make a substantial impact on global healthcare equity, significantly contributing to the worldwide effort to combat Alzheimer's disease.

7.2 Future work

Improving the SVR model's accuracy by eliminating the bias for values outside the conventional range is difficult due to the large variance of the datasets and the presence of outliers. Given the critical nature of outliers within the dataset, which represent valid biological variations, their exclusion from the analysis is also not tenable. However, there are still ways to improve the model's accuracy without overfitting. A unique method to solve this problem would be integrating feature engineering with ensemble methods to handle the dataset's inherent outliers. Specifically, the isolation forest algorithm can be used to generate an outlier score for each data point, which can then be included as an additional feature in the dataset. This method allows the SVR model to inherently adjust its predictions based on the likelihood of a data point being an outlier, effectively making it more robust in handling extreme values without the need to exclude them. Additionally, techniques such as Random Forest and Gradient Boosting are known for their robustness against data skewness and do not rely on the assumption of normality in error terms. These algorithms, by virtue of their structure, can handle outliers and non-linear relationships more effectively, making them suitable candidates for future exploration.

References

- [1] L. Robinson, E. Tang, and J.-P. Taylor, "Dementia: timely diagnosis and early intervention," *BMJ*, vol. 350, p. h3029, Jun. 2015, doi: 10.1136/bmj.h3029.
- [2] A. P. Porsteinsson, R. S. Isaacson, S. Knox, M. N. Sabbagh, and I. Rubino, "Diagnosis of Early Alzheimer's Disease: Clinical Practice in 2021," *J. Prev. Alzheimers Dis.*, vol. 8, no. 3, pp. 371–386, Jul. 2021, doi: 10.14283/jpad.2021.23.
- [3] R. Wittenberg, B. Hu, L. Barraza-Araiza, and A. Rehill, "Care Policy and Evaluation Centre, London School of Economics and Political Science".
- [4] M. G. Erkinen, M.-O. Kim, and M. D. Geschwind, "Clinical Neurology and Epidemiology of the Major Neurodegenerative Diseases," *Cold Spring Harb. Perspect. Biol.*, vol. 10, no. 4, p. a033118, Apr. 2018, doi: 10.1101/cshperspect.a033118.
- [5] M. V. F. Silva, C. de M. G. Loures, L. C. V. Alves, L. C. de Souza, K. B. G. Borges, and M. das G. Carvalho, "Alzheimer's disease: risk factors and potentially protective measures," *J. Biomed. Sci.*, vol. 26, no. 1, p. 33, May 2019, doi: 10.1186/s12929-019-0524-y.
- [6] C. R. Jack Jr. *et al.*, "Introduction to the recommendations from the National Institute on Aging-Alzheimer's Association workgroups on diagnostic guidelines for Alzheimer's disease," *Alzheimers Dement.*, vol. 7, no. 3, pp. 257–262, 2011, doi: 10.1016/j.jalz.2011.03.004.
- [7] A. Abbott, "Conquering Alzheimer's: a look at the therapies of the future," *Nature*, vol. 616, no. 7955, pp. 26–28, Apr. 2023, doi: 10.1038/d41586-023-00954-w.
- [8] J. Hazan, M. Wing, K. Y. Liu, S. Reeves, and R. Howard, "Clinical utility of cerebrospinal fluid biomarkers in the evaluation of cognitive impairment: a systematic review and meta-analysis," *J. Neurol. Neurosurg. Psychiatry*, vol. 94, no. 2, pp. 113–120, Feb. 2023, doi: 10.1136/jnnp-2022-329530.
- [9] D. Altomare *et al.*, "Plasma biomarkers for Alzheimer's disease: a field-test in a memory clinic," *J. Neurol. Neurosurg. Psychiatry*, vol. 94, no. 6, pp. 420–427, Jun. 2023, doi: 10.1136/jnnp-2022-330619.
- [10] C. E. Teunissen *et al.*, "Blood-based biomarkers for Alzheimer's disease: towards clinical implementation," *Lancet Neurol.*, vol. 21, no. 1, pp. 66–77, Jan. 2022, doi: 10.1016/S1474-4422(21)00361-6.
- [11] T. O. Klyucherev *et al.*, "Advances in the development of new biomarkers for Alzheimer's disease," *Transl. Neurodegener.*, vol. 11, no. 1, p. 25, Apr. 2022, doi: 10.1186/s40035-022-00296-z.
- [12] R. S. Wilson, P. A. Scherr, G. Hoganson, J. L. Bienias, D. A. Evans, and D. A. Bennett, "Early Life Socioeconomic Status and Late Life Risk of Alzheimer's Disease," *Neuroepidemiology*, vol. 25, no. 1, pp. 8–14, Jun. 2005, doi: 10.1159/000085307.
- [13] C. Tackenberg, L. Kulic, and R. M. Nitsch, "Familial Alzheimer's disease mutations at position 22 of the amyloid β -peptide sequence differentially affect synaptic loss, tau phosphorylation and neuronal cell death in an ex vivo system," *PLOS ONE*, vol. 15, no. 9, p. e0239584, Sep. 2020, doi: 10.1371/journal.pone.0239584.
- [14] Z. Breijyeh and R. Karaman, "Comprehensive Review on Alzheimer's Disease: Causes and Treatment," *Molecules*, vol. 25, no. 24, p. 5789, Dec. 2020, doi: 10.3390/molecules25245789.
- [15] M. S. Albert *et al.*, "The diagnosis of mild cognitive impairment due to Alzheimer's disease: Recommendations from the National Institute on Aging-Alzheimer's Association workgroups on diagnostic guidelines for Alzheimer's disease," *Alzheimers Dement. J. Alzheimers Assoc.*, vol. 7, no. 3, pp. 270–279, May 2011, doi: 10.1016/j.jalz.2011.03.008.

- [16] B. Dubois, C. A. F. von Arnim, N. Burnie, S. Bozeat, and J. Cummings, "Biomarkers in Alzheimer's disease: role in early and differential diagnosis and recognition of atypical variants," *Alzheimers Res. Ther.*, vol. 15, no. 1, p. 175, Oct. 2023, doi: 10.1186/s13195-023-01314-6.
- [17] C. R. Jack *et al.*, "NIA-AA Research Framework: Toward a biological definition of Alzheimer's disease," *Alzheimers Dement.*, vol. 14, no. 4, pp. 535–562, Apr. 2018, doi: 10.1016/j.jalz.2018.02.018.
- [18] T. Yuyen, "Composition of Blood," in *Transfusion Practice in Clinical Neurosciences*, H. Prabhakar, M. S Tandon, I. Kapoor, and C. Mahajan, Eds., Singapore: Springer Nature, 2022, pp. 249–255. doi: 10.1007/978-981-19-0954-2_24.
- [19] S. Janelidze *et al.*, "Plasma β -amyloid in Alzheimer's disease and vascular disease," *Sci. Rep.*, vol. 6, p. 26801, May 2016, doi: 10.1038/srep26801.
- [20] M. V. Pais, O. V. Forlenza, and B. S. Diniz, "Plasma Biomarkers of Alzheimer's Disease: A Review of Available Assays, Recent Developments, and Implications for Clinical Practice," *J. Alzheimers Dis. Rep.*, vol. 7, no. 1, pp. 355–380, doi: 10.3233/ADR-230029.
- [21] A. Koyama, O. I. Okereke, T. Yang, D. Blacker, D. J. Selkoe, and F. Grodstein, "Plasma amyloid- β as a predictor of dementia and cognitive decline: a systematic review and meta-analysis," *Arch. Neurol.*, vol. 69, no. 7, pp. 824–831, Jul. 2012, doi: 10.1001/archneurol.2011.1841.
- [22] J. Simrén *et al.*, "The diagnostic and prognostic capabilities of plasma biomarkers in Alzheimer's disease," *Alzheimers Dement.*, vol. 17, no. 7, pp. 1145–1156, Jul. 2021, doi: 10.1002/alz.12283.
- [23] O. Preische *et al.*, "Serum neurofilament dynamics predicts neurodegeneration and clinical progression in presymptomatic Alzheimer's disease," *Nat. Med.*, vol. 25, no. 2, pp. 277–283, Feb. 2019, doi: 10.1038/s41591-018-0304-3.
- [24] O. Hansson *et al.*, "Blood-based NfL," *Neurology*, vol. 88, no. 10, pp. 930–937, Mar. 2017, doi: 10.1212/WNL.0000000000003680.
- [25] S.-A. Ahadiat and Z. Hosseinian, "Astrocytes' innate role in neurodegenerative disorders," *Bull. Natl. Res. Cent.*, vol. 47, no. 1, p. 105, Jul. 2023, doi: 10.1186/s42269-023-01083-0.
- [26] "Diagnostic and prognostic plasma biomarkers for preclinical Alzheimer's disease - Chatterjee - 2022 - Alzheimer's & Dementia - Wiley Online Library." Accessed: Mar. 05, 2024. [Online]. Available: <https://alz-journals.onlinelibrary.wiley.com/doi/10.1002/alz.12447>
- [27] S. Mayo, J. Benito-León, C. Peña-Bautista, M. Baquero, and C. Cháfer-Pericás, "Recent Evidence in Epigenomics and Proteomics Biomarkers for Early and Minimally Invasive Diagnosis of Alzheimer's and Parkinson's Diseases," *Curr. Neuropharmacol.*, vol. 19, no. 8, pp. 1273–1303, Aug. 2021, doi: 10.2174/1570159X19666201223154009.
- [28] T. Hailemeskel Abebe, "The Derivation and Choice of Appropriate Test Statistic (Z, t, F and Chi-Square Test) in Research Methodology," *Math. Lett.*, vol. 5, no. 3, p. 33, 2019, doi: 10.11648/j.ml.20190503.11.
- [29] J. Frost, *Hypothesis Testing: An Intuitive Guide for Making Data Driven Decisions*. Statistics by Jim Publishing, 2020.
- [30] P. Mishra, C. M. Pandey, U. Singh, A. Gupta, C. Sahu, and A. Keshri, "Descriptive Statistics and Normality Tests for Statistical Data," *Ann. Card. Anaesth.*, vol. 22, no. 1, pp. 67–72, 2019, doi: 10.4103/aca.ACA_157_18.
- [31] "Kolmogorov–Smirnov Test," in *The Concise Encyclopedia of Statistics*, New York, NY: Springer, 2008, pp. 283–287. doi: 10.1007/978-0-387-32833-1_214.

- [32] M. K. Cain, Z. Zhang, and K.-H. Yuan, "Univariate and multivariate skewness and kurtosis for measuring nonnormality: Prevalence, influence and estimation," *Behav. Res. Methods*, vol. 49, no. 5, pp. 1716–1735, Oct. 2017, doi: 10.3758/s13428-016-0814-1.
- [33] A. J. Smola and B. Schölkopf, "A tutorial on support vector regression," *Stat. Comput.*, vol. 14, no. 3, pp. 199–222, Aug. 2004, doi: 10.1023/B:STCO.0000035301.49549.88.
- [34] D. Virmani and H. Pandey, "Comparative Analysis on Effect of Different SVM Kernel Functions for Classification," in *International Conference on Innovative Computing and Communications*, D. Gupta, A. Khanna, A. E. Hassanien, S. Anand, and A. Jaiswal, Eds., Singapore: Springer Nature, 2023, pp. 657–670. doi: 10.1007/978-981-19-3679-1_56.
- [35] H. Drucker, C. J. C. Burges, L. Kaufman, A. Smola, and V. Vapnik, "Support Vector Regression Machines," in *Advances in Neural Information Processing Systems*, MIT Press, 1996. Accessed: Apr. 13, 2024. [Online]. Available: https://papers.nips.cc/paper_files/paper/1996/hash/d38901788c533e8286cb6400b40b386d-Abstract.html
- [36] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. De Freitas, "Taking the Human Out of the Loop: A Review of Bayesian Optimization," *Proc. IEEE*, vol. 104, no. 1, pp. 148–175, Jan. 2016, doi: 10.1109/JPROC.2015.2494218.
- [37] M. W. Weiner *et al.*, "The Alzheimer's Disease Neuroimaging Initiative 3: Continued innovation for clinical trial improvement," *Alzheimers Dement.*, vol. 13, no. 5, pp. 561–571, 2017, doi: 10.1016/j.jalz.2016.10.006.
- [38] R. N. Esquivel *et al.*, "Clinical validation of the Lumipulse G β -amyloid ratio (1-42/1-40) in a subset of ADNI CSF samples," *Alzheimers Dement.*, vol. 17, no. S5, p. e055657, 2021, doi: 10.1002/alz.055657.
- [39] L. M. Shaw *et al.*, "P2-143: Method Comparison of AB(1-42) Measured in Human Cerebrospinal Fluid Samples by Liquid Chromatography-Tandem Mass Spectrometry, the Inno-Bia ALZBIO3 Assay, and the Elecsys® B-Amyloid(1-42) Assay," *Alzheimers Dement.*, vol. 12, no. 7S_Part_13, pp. P668–P668, 2016, doi: 10.1016/j.jalz.2016.06.1513.
- [40] N. Mattsson, U. Andreasson, H. Zetterberg, and K. Blennow, "Association of Plasma Neurofilament Light With Neurodegeneration in Patients With Alzheimer Disease," *JAMA Neurol.*, vol. 74, no. 5, pp. 557–566, May 2017, doi: 10.1001/jamaneurol.2016.6117.
- [41] C. Udeh-Momoh *et al.*, "Blood Derived Amyloid Biomarkers for Alzheimer's Disease Prevention," *J. Prev. Alzheimers Dis.*, vol. 9, no. 1, pp. 12–21, Jan. 2022, doi: 10.14283/jpad.2021.70.
- [42] T. K. Karikari *et al.*, "Blood phosphorylated tau 181 as a biomarker for Alzheimer's disease: a diagnostic performance and prediction modelling study using data from four prospective cohorts," *Lancet Neurol.*, vol. 19, no. 5, pp. 422–433, May 2020, doi: 10.1016/S1474-4422(20)30071-5.
- [43] J.-H. Kang *et al.*, "The Alzheimer's Disease Neuroimaging Initiative 2 Biomarker Core: A review of progress and plans," *Alzheimers Dement. J. Alzheimers Assoc.*, vol. 11, no. 7, pp. 772–791, Jul. 2015, doi: 10.1016/j.jalz.2015.05.003.
- [44] D. Tsikas, "A proposal for comparing methods of quantitative analysis of endogenous compounds in biological systems by using the relative lower limit of quantification (rLLOQ)," *J. Chromatogr. B*, vol. 877, no. 23, pp. 2244–2251, Aug. 2009, doi: 10.1016/j.jchromb.2009.02.029.
- [45] L. Wilkinson, "Visualizing Big Data Outliers Through Distributed Aggregation," *IEEE Trans. Vis. Comput. Graph.*, vol. 24, no. 1, pp. 256–266, Jan. 2018, doi: 10.1109/TVCG.2017.2744685.

- [46] S. Nakagawa and H. Schielzeth, "A general and simple method for obtaining R² from generalized linear mixed-effects models," *Methods Ecol. Evol.*, vol. 4, no. 2, pp. 133–142, 2013, doi: 10.1111/j.2041-210x.2012.00261.x.
- [47] V. Pérez-Grijalba *et al.*, "Plasma A β 42/40 Ratio Detects Early Stages of Alzheimer's Disease and Correlates with CSF and Neuroimaging Biomarkers in the AB255 Study," *J. Prev. Alzheimers Dis.*, vol. 6, no. 1, pp. 34–41, 2019, doi: 10.14283/jpad.2018.41.
- [48] H.-W. Klafki, O. Wirths, O. Jahn, B. Morgado, H. Esselmann, and J. Wiltfang, "Blood plasma biomarkers for Alzheimer's disease: A β 1–42/1–40 vs. A β X–42/X–40," *Clin. Chem. Lab. Med. CCLM*, vol. 62, no. 2, pp. e56–e57, Jan. 2024, doi: 10.1515/cclm-2023-0990.
- [49] M. Amft *et al.*, "The cerebrospinal fluid biomarker ratio A β 42/40 identifies amyloid positron emission tomography positivity better than A β 42 alone in a heterogeneous memory clinic cohort," *Alzheimers Res. Ther.*, vol. 14, no. 1, p. 60, Apr. 2022, doi: 10.1186/s13195-022-01003-w.
- [50] Y. Zhang, K. Sun, Y. Liu, Z. Ou, and D. Shen, "Vector Quantized Multi-modal Guidance for Alzheimer's Disease Diagnosis Based on Feature Imputation," in *Machine Learning in Medical Imaging*, X. Cao, X. Xu, I. Rekik, Z. Cui, and X. Ouyang, Eds., in Lecture Notes in Computer Science. Cham: Springer Nature Switzerland, 2024, pp. 403–412. doi: 10.1007/978-3-031-45673-2_40.
- [51] H. Hampel, J. Cummings, K. Blennow, P. Gao, C. R. Jack, and A. Vergallo, "Developing the ATX(N) classification for use across the Alzheimer disease continuum," *Nat. Rev. Neurol.*, vol. 17, no. 9, pp. 580–589, Sep. 2021, doi: 10.1038/s41582-021-00520-w.
- [52] J. Hardy and D. J. Selkoe, "The Amyloid Hypothesis of Alzheimer's Disease: Progress and Problems on the Road to Therapeutics," *Science*, vol. 297, no. 5580, pp. 353–356, Jul. 2002, doi: 10.1126/science.1072994.
- [53] S. Janelidze *et al.*, "CSF A β 42/A β 40 and A β 42/A β 38 ratios: better diagnostic markers of Alzheimer disease," *Ann. Clin. Transl. Neurol.*, vol. 3, no. 3, pp. 154–165, Jan. 2016, doi: 10.1002/acn3.274.
- [54] O. Hansson, S. Lehmann, M. Otto, H. Zetterberg, and P. Lewczuk, "Advantages and disadvantages of the use of the CSF Amyloid β (A β) 42/40 ratio in the diagnosis of Alzheimer's Disease," *Alzheimers Res. Ther.*, vol. 11, no. 1, p. 34, Apr. 2019, doi: 10.1186/s13195-019-0485-0.
- [55] J. C. Lee, S. J. Kim, S. Hong, and Y. Kim, "Diagnosis of Alzheimer's disease utilizing amyloid and tau as fluid biomarkers," *Exp. Mol. Med.*, vol. 51, no. 5, pp. 1–10, May 2019, doi: 10.1038/s12276-019-0250-2.
- [56] C. Wattmo, K. Blennow, and O. Hansson, "Cerebro-spinal fluid biomarker levels: phosphorylated tau (T) and total tau (N) as markers for rate of progression in Alzheimer's disease," *BMC Neurol.*, vol. 20, no. 1, p. 10, Jan. 2020, doi: 10.1186/s12883-019-1591-0.
- [57] J. A. Pillai, A. Bonner-Jackson, L. M. Bekris, J. Safar, J. Bena, and J. B. Leverenz, "Highly Elevated Cerebrospinal Fluid Total Tau Level Reflects Higher Likelihood of Non-Amnesic Subtype of Alzheimer's Disease," *J. Alzheimers Dis. JAD*, vol. 70, no. 4, pp. 1051–1058, 2019, doi: 10.3233/JAD-190519.
- [58] L. Gaetani, K. Blennow, P. Calabresi, M. D. Filippa, L. Parnetti, and H. Zetterberg, "Neurofilament light chain as a biomarker in neurological disorders," *J. Neurol. Neurosurg. Psychiatry*, vol. 90, no. 8, pp. 870–881, Aug. 2019, doi: 10.1136/jnnp-2018-320106.
- [59] M. Dulewicz, A. Kulczyńska-Przybik, P. Mroczko, J. Kornhuber, P. Lewczuk, and B. Mroczko, "Biomarkers for the Diagnosis of Alzheimer's Disease in Clinical Practice: The Role of CSF

- Biomarkers during the Evolution of Diagnostic Criteria," *Int. J. Mol. Sci.*, vol. 23, no. 15, p. 8598, Aug. 2022, doi: 10.3390/ijms23158598.
- [60] S. Lehmann *et al.*, "Cerebrospinal fluid A beta 1–40 peptides increase in Alzheimer's disease and are highly correlated with phospho-tau in control individuals," *Alzheimers Res. Ther.*, vol. 12, no. 1, p. 123, Oct. 2020, doi: 10.1186/s13195-020-00696-1.
- [61] P. J. Visser *et al.*, "Cerebrospinal fluid tau levels are associated with abnormal neuronal plasticity markers in Alzheimer's disease," *Mol. Neurodegener.*, vol. 17, no. 1, p. 27, Mar. 2022, doi: 10.1186/s13024-022-00521-3.
- [62] C. M. Watson *et al.*, "Quantitative Mass Spectrometry Analysis of Cerebrospinal Fluid Protein Biomarkers in Alzheimer's Disease," *Sci. Data*, vol. 10, no. 1, p. 261, May 2023, doi: 10.1038/s41597-023-02158-3.
- [63] D. J. Reen, "Enzyme-Linked Immunosorbent Assay (ELISA)," in *Basic Protein and Peptide Protocols*, J. M. Walker, Ed., in *Methods in Molecular Biology*TM, Totowa, NJ: Humana Press, 1994, pp. 461–466. doi: 10.1385/0-89603-268-X:461.
- [64] L. Agnello *et al.*, "Diagnostic accuracy of cerebrospinal fluid biomarkers measured by chemiluminescent enzyme immunoassay for Alzheimer disease diagnosis," *Scand. J. Clin. Lab. Invest.*, vol. 80, no. 4, pp. 313–317, Jul. 2020, doi: 10.1080/00365513.2020.1740939.
- [65] W. Zhao, H.-Y. Chen, and J.-J. Xu, "Electrogenerated chemiluminescence detection of single entities," *Chem. Sci.*, vol. 12, no. 16, pp. 5720–5736, Apr. 2021, doi: 10.1039/D0SC07085H.
- [66] L. Álvarez-Sánchez, C. Peña-Bautista, M. Baquero, and C. Cháfer-Pericás, "Novel Ultrasensitive Detection Technologies for the Identification of Early and Minimally Invasive Alzheimer's Disease Blood Biomarkers," *J. Alzheimers Dis. JAD*, vol. 86, no. 3, pp. 1337–1369, 2022, doi: 10.3233/JAD-215093.
- [67] R. Hendricks *et al.*, "Establishment of neurofilament light chain Simoa assay in cerebrospinal fluid and blood," *Bioanalysis*, vol. 11, no. 15, pp. 1405–1418, Aug. 2019, doi: 10.4155/bio-2019-0163.
- [68] H. Jang *et al.*, "Performance of the plasma A β 42/A β 40 ratio, measured with a novel HPLC-MS/MS method, as a biomarker of amyloid PET status in a DPUK-KOREAN cohort," *Alzheimers Res. Ther.*, vol. 13, no. 1, p. 179, Oct. 2021, doi: 10.1186/s13195-021-00911-7.
- [69] J. Powers Carson, "A Simpler and Sensitive Mass Spectrometry Method for Quantitation of Plasma Amyloid Peptides?," *J. Appl. Lab. Med.*, vol. 6, no. 4, pp. 816–819, Jul. 2021, doi: 10.1093/jalm/jfab019.
- [70] L.-S. Wang *et al.*, "Comparison of xMAP and ELISA assays for detecting CSF biomarkers of Alzheimer's Disease," *J. Alzheimers Dis. JAD*, vol. 31, no. 2, pp. 439–445, 2012, doi: 10.3233/JAD-2012-120082.
- [71] M. G. Larson, "Descriptive Statistics and Graphical Displays," *Circulation*, vol. 114, no. 1, pp. 76–81, Jul. 2006, doi: 10.1161/CIRCULATIONAHA.105.584474.
- [72] C. Xiao, J. Ye, R. M. Esteves, and C. Rong, "Using Spearman's correlation coefficients for exploratory data analysis on big dataset," *Concurr. Comput. Pract. Exp.*, vol. 28, no. 14, pp. 3866–3878, 2016, doi: 10.1002/cpe.3745.
- [73] A. V. Tatachar, "Comparative Assessment of Regression Models Based On Model Evaluation Metrics," vol. 08, no. 09, 2021.
- [74] M. W. Weiner *et al.*, "The Alzheimer's Disease Neuroimaging Initiative: A review of papers published since its inception," *Alzheimers Dement.*, vol. 8, no. 1S, pp. S1–S68, 2012, doi: 10.1016/j.jalz.2011.09.172.

- [75] V. Giedraitis *et al.*, “The normal equilibrium between CSF and plasma amyloid beta levels is disrupted in Alzheimer’s disease,” *Neurosci. Lett.*, vol. 427, no. 3, pp. 127–131, Nov. 2007, doi: 10.1016/j.neulet.2007.09.023.
- [76] V. Planche *et al.*, “Structural progression of Alzheimer’s disease over decades: the MRI staging scheme,” *Brain Commun.*, vol. 4, no. 3, p. fcac109, Jun. 2022, doi: 10.1093/braincomms/fcac109.
- [77] A. A. Tahami Monfared, M. J. Byrnes, L. A. White, and Q. Zhang, “Alzheimer’s Disease: Epidemiology and Clinical Progression,” *Neurol. Ther.*, vol. 11, no. 2, pp. 553–569, Jun. 2022, doi: 10.1007/s40120-022-00338-8.

Appendices

A. Supplemental Literature Review

A.1 Positron Emission Tomography (PET) Imaging Biomarker

While the primary focus of this thesis is CSF and plasma biomarkers, it is imperative to acknowledge the complementary role of PET imaging in the broader biomarker landscape. PET facilitates the direct in vivo observation of pathological brain changes, through amyloid and tau PET scans. Amyloid PET detects amyloid plaques, while tau PET identifies tau tangles. These imaging modalities provide biomarker information that complements the biochemical markers found in cerebrospinal fluid (CSF) and plasma, offering a more comprehensive understanding of the disease's progression [49].

However, the widespread application of PET imaging is limited by its high cost and the associated radiation exposure, which pose significant barriers to its routine use in clinical diagnostics and monitoring [50].

A.2 Cerebrospinal Fluid (CSF) Biomarkers

CSF encapsulates the brain and spinal cord, presenting a window into the biochemical changes within the brain. Biomarkers found in CSF facilitate the detection of both A β and tau peptides and their variations are identifiable prior to the alterations observed through neuroimaging biomarkers [51]. A summary of CSF biomarkers can be found on Table A.1.

A.2.1 Amyloid-beta (A β) Peptides

The imbalance between the production and clearance of A β peptides, primarily A β 42, leads to amyloid plaque formation in the brain, a hallmark of AD. The A β peptides are produced from the amyloid precursor protein (APP) through enzymatic actions by beta-secretase and gamma-

secretase enzymes occurring predominantly within neuronal cell membranes [52]. The concentration of CSF A β 42 and the A β 42/40 ratio decline, inversely correlating with the extent of amyloid plaque accumulation serve as sensitive markers for AD [53]. CSF A β 42/A β 40 ratio significantly decreases in AD patients compared to non-AD patients, providing more reliable diagnostic information than A β 42 concentrations alone [53]. The inclusion of A β 40, despite not showing substantial changes in levels between AD and non-AD patients, has been found to increase the diagnostic accuracy of AD when used alongside other biomarkers, especially in distinguishing AD from other conditions like Parkinson's Disease Dementia (PDD) and Dementia with Lewy Bodies (DLB) [54] [55].

A.2.2 Tau Proteins

Tau proteins, specifically p-tau and t-tau, are integral to AD diagnosis. P-tau levels correlate with neurofibrillary tangle formation, while t-tau levels indicate neuronal damage [56]. However, t-tau is less specific to AD, showing fluctuations in various acute and chronic neurodegenerative disorders [57]. The integration of tau proteins with A β ratios enhances the diagnostic capability, effectively identifying AD pathology at early stages.

A.2.3 Neurofilament Light Chain (NfL)

The neurofilament light chain (NfL) in CSF has emerged as a potent marker of neuroaxonal damage, with its levels correlating with the severity of neurodegeneration [58]. In certain clinical settings NfL has superseded tTau as a biomarker for neurodegeneration, because of its superior sensitivity [51].

A.2.4 CSF Biomarker Collection

The acquisition of CSF is performed through lumbar puncture, which allows for the direct quantification of all CSF biomarkers.

A.2.5 Challenges for CSF Biomarkers

The integration of CSF biomarkers into clinical practice encounters significant hurdles, notably the invasive nature of lumbar puncture procedures, inconsistencies across assay methodologies, and the lack of universally accepted reference standards [59]. These challenges complicate the clinical

interpretation of biomarker data, impacting the reliability and comparability of results across different studies and laboratories.

Table A.1: Overview of Key CSF Biomarkers in Alzheimer's Disease.

CSF Biomarkers	Role in AD	Typical Change in AD	Biological Source	References
Aβ42	Reflects Aβ deposition	Decrease	APP	[54] [60]
Aβ40	Less specific to AD but used in ratios	Variable	APP	[60]
Aβ42/Aβ40	Improves specificity of Aβ measurements	Decrease	APP	[54]
t-Tau	Indicates neuronal damage	Increase	Neurons	[61] [56]
p-Tau181	Reflects tau pathology	Increase	Neurons	[56]
NFL	Indicates axonal damage	Increase	Neurons	[58]

A.3 Methods of Quantification for CSF and Plasma Biomarkers

While established methods like ELISA, mass spectrometry (MS), or immunoprecipitation offer sufficient sensitivity for CSF biomarker detection, plasma biomarker analysis necessitates highly sensitive technologies such as Single Molecule Array (Simoa), Electrochemiluminescence (ECL) based assays, Chemiluminescent Enzyme Immunoassay (CLEIA), and certain mass spectrometry (MS) methods like Liquid Chromatography-Mass Spectrometry (LC-MS) and Immunoprecipitation-Mass Spectrometry (IP-MS). For the MS methods the MS measures the mass-to-charge ratio of charged particles, allowing for precise quantification and structural characterization [62].

A.4 Technological Platforms for Biomarker Detection

Technological platforms advance beyond traditional quantification methods by offering automated, high-throughput analysis with superior sensitivity for CSF and plasma biomarkers which are essential for accurate diagnostics and research in neurodegenerative diseases. Table A.3 presents some of the main platforms that are utilized.

A.4.1 Comparative analysis of biomarker main quantification methods

Table A.2: Comparative analysis of biomarker main quantification methods.

Method	Principles	Key Features	References
ELISA	Antibody-antigen interaction leading to an enzymatic reaction that produces a measurable signal.	High specificity, widely available, cost-effective.	[63]
CLEIA	Binds to target biomarkers with specific antibodies linked to chemiluminescent markers.	high sensitivity, supports high-throughput analysis.	[64]
ECL	Antibody-antigen interaction with electrochemically active labels that emit light.	High sensitivity and specificity, suitable for low-abundance biomarkers, wide dynamic range.	[65]
Simoa	Digital ELISA technology in femtoliter-sized chambers to detect single protein molecules.	Ultra-high sensitivity, detecting low-abundance biomarkers.	[66] [67]
LC-MS	Separates mixtures with LC before MS analysis for compound identification and quantification.	High sensitivity and selectivity, broad range of application from small molecules to large biomolecules, capable of analysing complex mixtures.	[68]
IP-MS	Combines IP to enrich specific proteins from a complex mixture with MS for identification and quantification.	High specificity due to antibody usage	[69]
xMAP	Bead-based multiplexing with color-coded microspheres coated with specific capture antibodies.	High-throughput, Simultaneous analysis of multiple biomarkers.	[70]

A.4.2 Supplemental Table for Technological Platforms for Biomarker Detection

Table A.3: Overview of technological platforms for the quantification of CSF and plasma biomarkers.

Platform	Manufacturer	Method	Applications
Elecsys	Roche Diagnostics	ECL	A β peptides, tau proteins, GFAP, NfL (CSF/Plasma)
HD-X Analyzer	Quanterix	Simoa	A β peptides, tau proteins, GFAP, NfL (CSF/Plasma)
INNOBIA AlzBio3	Fujirebio	xMAP	A β peptides, tau proteins (CSF)
LUMIPULSE G Systems	Fujirebio	CLEIA	A β peptides, tau proteins (CSF)

A.5 Supplemental Material for Theoretical Development

A.5.1 Descriptive Statistics

Table A.4: Main Descriptive Statistics.

Measure	Description	Equation
Mean ($\bar{x}$)	Represents the average of all data points x_i for N samples. Is sensitive to outliers, providing insights into data symmetry and skewness [30].	$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i \quad (\text{A. 1})$
Variance (s^2)	Quantifies data spread around the mean highlighting data consistency or variability [30].	$s^2 = \frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2 \quad (\text{A. 2})$
Standard Deviation (s)	Is the square root of variance, it directly compares data dispersion and is essential for assessing data distribution [30].	$s = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2} \quad (\text{A. 3})$

The median, the middle value when data are ordered, serves as a robust central tendency measure unaffected by extreme values, thus offering a clearer view of the dataset's typical value.

Mode, the most frequent data point, highlights common patterns or values in the dataset, useful in identifying predominant trends or classifications.

Finally, the IQR, the difference between the 75th and 25th percentiles, measures the central spread of the dataset, mitigating the impact of outliers [71]. It offers a more nuanced view of dataset variability, crucial for detecting and handling outliers in data preprocessing [71].

A.5.2 Kolmogorov-Smirnov Test for Normality

The KS test for normality evaluates how well a dataset's distribution matches a normal distribution. It calculates the maximum difference, D , between the empirical cumulative distribution function (ECDF) of the dataset and the cumulative distribution function (CDF) of the normal distribution. The test statistic is given by (A.4) [31].

$$D = \max|F_n(x) - F(x)| \quad (\text{A. 4})$$

where $F_n(x)$ is the ECDF of the observed data and $F(x)$ is the CDF of the normal distribution expected under the null hypothesis.

A.5.3 Correlation

Pearson's correlation is calculated using (A.5) [72].

$$r = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2 \sum(y_i - \bar{y})^2}} \quad (\text{A. 5})$$

where x_i and y_i are the values of the two variables, and $\bar{x}$ and $\bar{y}$ are their respective means.

Pearson's r varies between -1 and 1, with 1 signifying a perfect positive linear correlation, -1 indicating a perfect negative linear correlation, and 0 representing the absence of any linear relationship. It assumes that both variables are normally distributed, the relationship between them is linear, and the data is homoscedastic, meaning the variance in the data is the same across all levels of a variable.

Spearman's correlation is calculated using (A.6) [72].

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)} \quad (\text{A. 6})$$

Where d_i is the difference between the ranks of corresponding values, and n is the number of observations. Spearman's ρ also ranges from -1 to 1, interpreting the direction and strength of the

monotonic relationship similarly to Pearson's r . It does not assume normal distribution of the variables or a linear relationship between them, making it suitable for ordinal data or non-linear relationships. However, it supposes that the association between the variables is monotonic.

A.5.4 Support Vector Regression Hard and Soft Margins

The hard margin SVR seeks a hyperplane with parameters w and b that minimizes the following objective function [33]:

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad (\text{A. 7})$$

This objective function is chosen to ensure that the margin, or the distance between the separating hyperplane and the nearest data points on either side, is maximized. Unlike SVM, SVR is introducing a constraint that all the predictions must lie within an ε -tube around the true regression line, except for a possible margin of error. This ε -insensitive loss function does not penalize errors that are within a distance ε from the true value but penalizes errors outside this margin linearly. The optimization problem is now given by [33]:

$$\text{Minimize: } \frac{1}{2} \|w\|^2, \text{ subject to } \begin{cases} y_i - \langle w, x_i \rangle - b < \varepsilon \\ \langle w, x_i \rangle + b - y_i < \varepsilon \end{cases} \quad (\text{A. 8})$$

This formulation represents a quadratic optimization problem with linear constraints, solvable using methods such as the Sequential Minimal Optimization (SMO) algorithm or by converting it into its dual form to exploit kernel methods, which is more common in soft margin SVR.

In soft margin SVR, the primary goal remains the same: to find a function $f(x) = \langle w, x \rangle + b$ that can predict the target value y_i for a given input x_i . However, the optimization objective and constraints are modified to introduce flexibility. The new objective function incorporates slack variables (ξ_i, ξ_i^*) for each data point, representing the degree of deviation allowed for points falling outside the ε -tube as shown in (A.9) [33]

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*) \quad (\text{A. 9})$$

The regularization parameter C determines the balance between the smoothness of the regression function and the extent to which deviations greater than ε are acceptable. The objective is now not only to minimize the norm of the weight vector (which corresponds to

maximizing the margin) but also to minimize the sum of the slack variables, thus penalizing deviations from the ϵ -insensitive zone.

The constraints are also updated to ensure that deviations are allowed but penalized, through the slack variables ξ_i, ξ_i^* which measure the degree of violation for points outside the ϵ -tube. Now the new optimization problem is described by (3.8).

A.5.5 Evaluation Metrics

Table A.5: Evaluation Metrics Overview

Evaluation Metric	Description	Formula	Formula Description
RMSE	Measures the average magnitude of the errors in the original units.	$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (\text{A. 10})$	n is the number of observations, y_i represents actual values, $\hat{y}_i$ denotes predicted values, and $\bar{y}$ is the average of actual values [73].
MAE	Measures the average magnitude of errors between predicted and actual values, without considering their direction.	$\text{MAE} = \frac{1}{n} \sum_{i=1}^n y_i - \hat{y}_i \quad (\text{A. 11})$	
R^2	Reflects the percentage of variability in the dependent variable that can be explained by the independent variables.	$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2} \quad (\text{A. 12})$	

Finally, some assumptions and applications for each needs to be noted. MAE assumes that all errors, regardless of direction, are equally important and does not penalize larger errors more than smaller ones. RMSE assumes that larger errors are exponentially more severe. R^2 assumes that every increase in an independent variable correlates with an increase or decrease in the dependent variable, but it does not assume the relationship is linear unless used with linear models [73].

A.6 Supplemental Material for the Dataset

The initial phase, ADNI 1, commenced with more than 800 participants, focusing on cognitive assessments, MRI, and blood biomarkers in older adults across various cognitive states [74].

Subsequent phases expanded the cohort and introduced more detailed analyses of fluid

biomarkers in cerebrospinal fluid (CSF) and plasma, specifically targeting the identification of markers predictive of disease progression. From the clinical data of this dataset the following were selected for analysis in this thesis.

Education: Integer. This feature represents the number of years of education. It's a numeric variable with an inherent order. Given its ordinal nature and the meaningful scale it will be treated as a continuous variable.

Age: Integer. Age variable quantifies the age of individuals and is treated as a continuous numerical variable.

MMSE (Mini-Mental State Examination at baseline): Integer from 0 to 30. The MMSE is a widely used 30-point questionnaire designed to assess cognitive impairment in both clinical and research environments. MMSE scores are ordinal and have a meaningful order and scale. Higher scores indicate better cognitive function thus it will be treated as a continuous variable.

APOE4 (Apolipoprotein E ϵ 4 allele): Either 0 or 1. The APOE4 gene variant is linked to a higher likelihood of developing Alzheimer's disease and is categorized based on the number of APOE ϵ 4 alleles a patient has: 0, 1, or 2. For simplification, the categories for 1 and 2 alleles have been combined into a single category, transforming APOE4 into a binary variable. This variable now indicates the presence (1) or absence (0) of at least one APOE ϵ 4 allele. As a result, APOE4 is treated as a nominal categorical variable, where the values 0 and 1 represent distinct categories without implying any order between them.

Gender: Either 0 or 1. The gender variable categorizes individuals based on their gender identity, represented as male or female thus it is a nominal categorical variable, where the categories do not imply any numerical order or hierarchy but simply distinguish between different groups. It can be used to explore differences, trends, or outcomes between gender groups.

ADAS13 (Alzheimer's Disease Assessment Scale-Cognitive Subscale 13 at baseline): Integer 0 to 85. The ADAS13 score is a tool used to evaluate the cognitive function of individuals, particularly in the context of AD. Higher scores indicating more severe cognitive impairment as such, ADAS13 is treated as a continuous numerical variable, allowing for the detailed measurement of cognitive function levels across a spectrum. This variable is used for analysing the progression of cognitive decline, and the correlation of cognitive impairment with other variables within the study.

DX_bl (Diagnosis at baseline): CN, MCI, AD. The DX_bl variable captures the initial diagnosis of patients during their baseline visit. During the first phase of ADNI (ADNI 1), diagnoses were categorized into Cognitive Normal (CN), Mild Cognitive Impairment (MCI), and Alzheimer's Disease (AD). In the subsequent phase, ADNI 2, the categorization of MCI was refined into two subcategories: Early MCI (EMCI) and Late MCI (LMCI). To maintain consistency across the studies, the EMCI and LMCI categories were consolidated into a single MCI category simplifying the classification scheme while preserving the essential diagnostic distinctions.

A.7 Supplemental Material for A β 42 Analysis

A.7.1 Descriptive Statistics Across Non-AD and AD Groups

Differences between groups were evaluated using the Student t-test for normally distributed data and the Mann-Whitney U test for skewed data. For categorical data, the Chi-square test was applied.

Table A.6: Dataset for Bateman Plasma A β 42 and UPEN CSF A β 42.

Variable	Non-AD (n = 196)	AD (n = 16)	P-value
Age, years (mean, SD)	71.53 $\pm$ 7.32	76.35 $\pm$ 6.47	0.011
Education, years (median, IQR)	16.0 (15-18)	15.5 (13-16.5)	<0.001*
MMSE (median, IQR)	29.0 (27-30)	22.5 (21.75-24)	<0.001*
ADAS13 (median, IQR)	12.00(8-18)	24.50(23-31)	<0.001*
CSF AB42, pg/mL (median, IQR)	949.9 (685.3-1201.5)	646.3 (551.05-704.3)	<0.001*
Plasma AB42, pg/mL (mean, SD)	33.06 $\pm$ 5.61	32.13 $\pm$ 4.60	0.516
E4+ (%)	43.88	87.5	<0.001**
Gender (M%)	57.14	43.75	0.467**
Race (White%)	90.31	93.75	0.391**

*Mann-Whitney U test, **Chi-square Test

Table A.7: Dataset for Bateman Plasma A β 42 and FUJI CSF A β 42.

Variable	Non-AD (n = 111)	AD (n = 16)	P-value
Age, years (mean, std)	70.64 $\pm$ 7.17	76.84 $\pm$ 6.32	0.001
Education, years (median, IQR)	17.0 (15-18)	15.5 (13-18)	0.132*
MMSE (median, IQR)	29.0 (28-30)	22.5 (22-24)	<0.001*
ADAS13 (median, IQR)	12 (8-17)	24.5(22-31)	<0.001*
CSF AB42, pg/mL (median, IQR)	747.0 (553-1102.5)	474.5 (402.75-573)	<0.001*
Plasma AB42, pg/mL (mean, std)	33.15 $\pm$ 5.68	32.53 $\pm$ 4.86	0.678
E4+ (%)	43.24	87.50	<0.001**
Gender (M%)	54.95	50.00	0.970**
Race (White%)	92.79	93.75	0.186**

*Mann-Whitney U test, **Chi-square Test

A.7.2 Nominal Variables against A β 42

To evaluate how categorical nominal variables correlate against A β 42 Point Biserial correlation needs to be used as it is the most appropriate measure for this application. Utilizing this measure APOE4 has a statistically significant moderate correlation with A β 42 in both datasets while Gender shows a weak statistically significant correlation in both datasets.

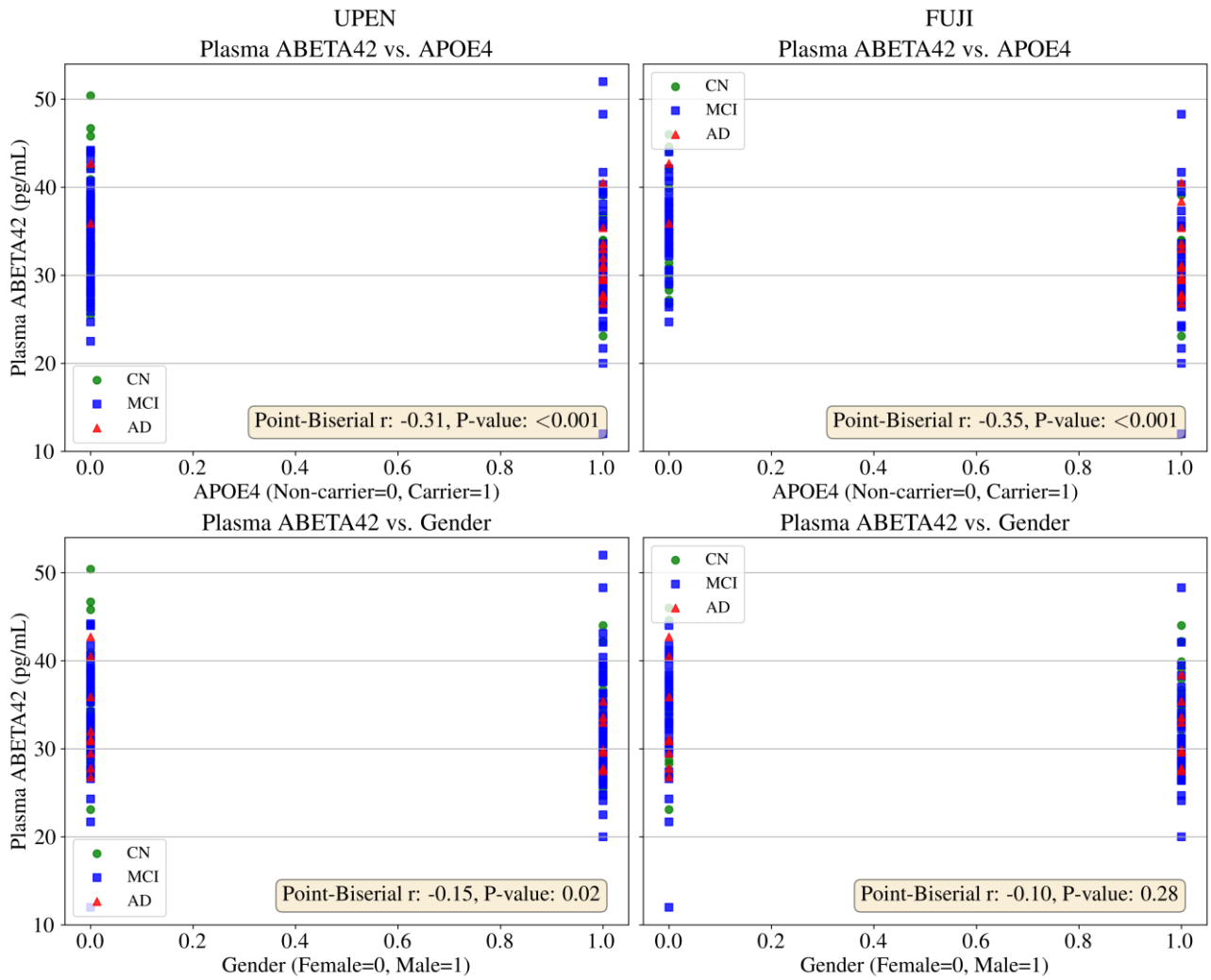


Figure A.1: Plasma AB42 versus categorical nominal features across UPEN and FUJI datasets.

A.8 Evaluation metrics for Aβ42

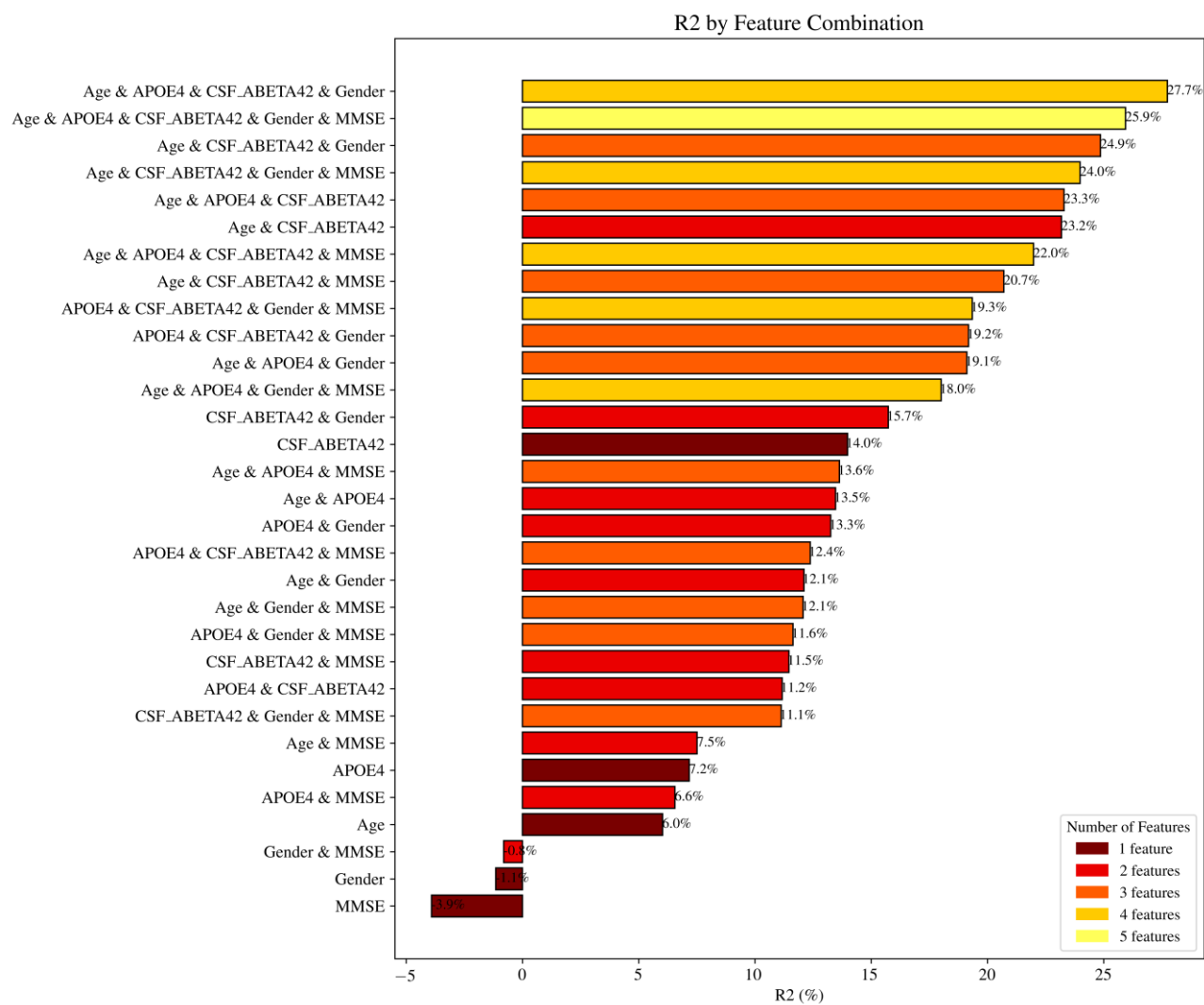


Figure A.2: Evaluation of Aβ42 models by feature combination using R^2 .

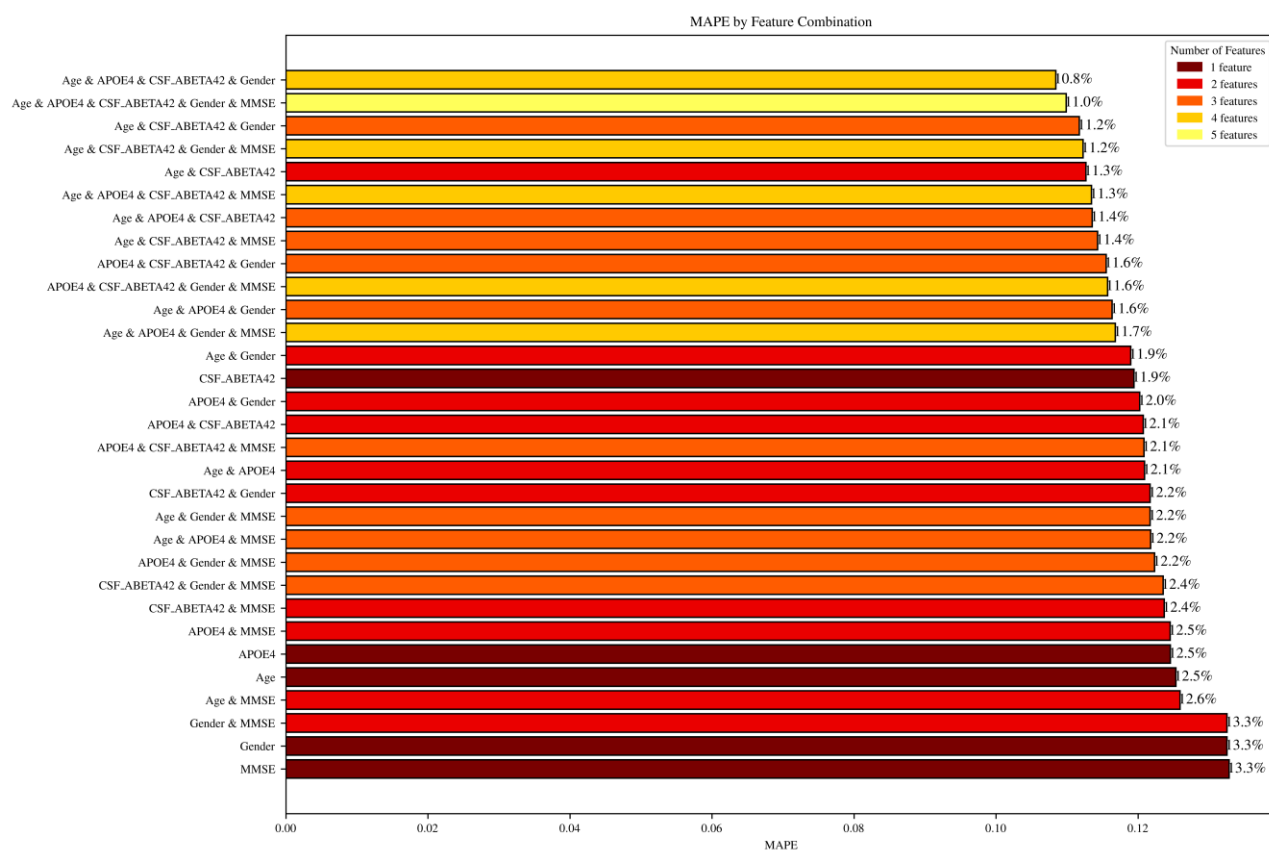


Figure A.3: Evaluation of A β 42 models by feature combination using MAPE.

B. Analysis of A β 40 Blood Plasma Biomarker

Only the FUJI dataset will be used since UPEN does not have enough A β 40 data that correspond to plasma data for BATEMAN

B.1 Normality and Descriptive Statistics

In the FUJI dataset, statistical analyses from Table B.1 reveal the dataset's distributional characteristics and normality deviations. Age and CSF A β 40 exhibit nearly symmetrical distributions as indicated by their skewness close to zero and their high KS p-values (>0.95), suggesting alignment with normal distribution. Education and Plasma AB40, however, demonstrate moderate negative skewness, though only education shows significant deviation from normality (KS p-value = 0.002). The Mini-Mental State Examination (MMSE) scores are significantly negatively skewed (skewness = -1.228) and deviate from normal distribution (KS p-value < 0.001), suggesting a potential left-tail dominance where lower scores are more frequent. Conversely, ADAS13 scores, while slightly positively skewed, do not significantly deviate from normality, supporting their use in further analyses without transformations. Notably, Plasma AB40 presents an unusually high kurtosis (4.247), indicating a leptokurtic distribution where data points are clustered around the mean more than a normal distribution, which warrants cautious interpretation due to potential outlier influence.

Table B.1: A β 40 Normality Tests.

Variable	FUJI		
	Skewness	Kurtosis	KS p-value
Age	<0.001	-0.450	0.968
Education	-0.436	-0.457	0.002
MMSE	-1.228	0.944	<0.001
ADAS13	0.851	0.493	0.139
CSF A β 40	0.618	1.030	0.972
Plasma A β 40	-0.491	4.247	0.50

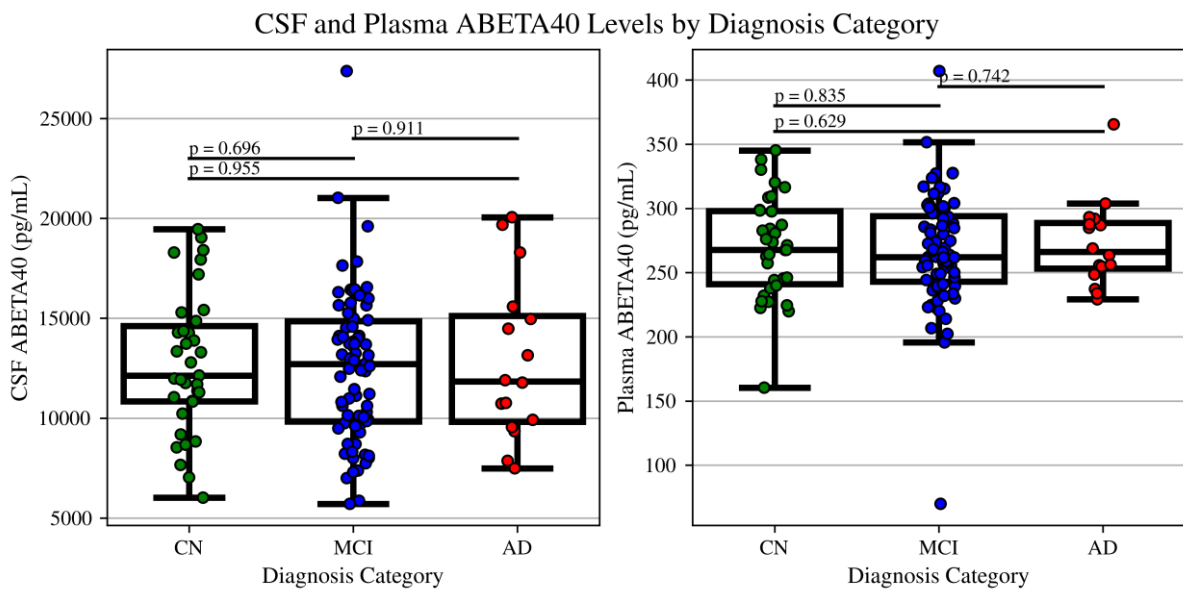
The dataset is the same to A β 42 the only difference is that the CSF is now normally distributed and can be seen in Table B.2. Thus, the analysis is discussed in the main part.

Table B.2: AB40 Descriptive Statistics.

Variable	FUJI (n=127)
Age, years (mean, SD)	71.42 ± 7.34
Education, years (median, IQR)	16.00 (14.00 – 18.00)
MMSE (median, IQR)	28.00 (26.00 - 29.50)
ADAS13 (median, IQR)	14.00 (8.00 - 19.50)
CSF A β 40, pg/mL (mean, SD)	12651.83 ± 3662.54
Plasma A β 40, pg/mL (mean, SD)	268.21 ± 41.05
E4+ (%)	48.82
Gender (M%)	54.33
Race (White%)	92.91

B.2 Exploration of Clinical Data and Biomarker Relationships

Figure B.1 shows the large variance of A β 40 even though there are few outliers the data are very dispersed and unable to differentiate between AD categories in every case both for CSF and Blood plasma.

**Figure B.1:** CSF and Plasma A β 40 Levels by diagnosis category.

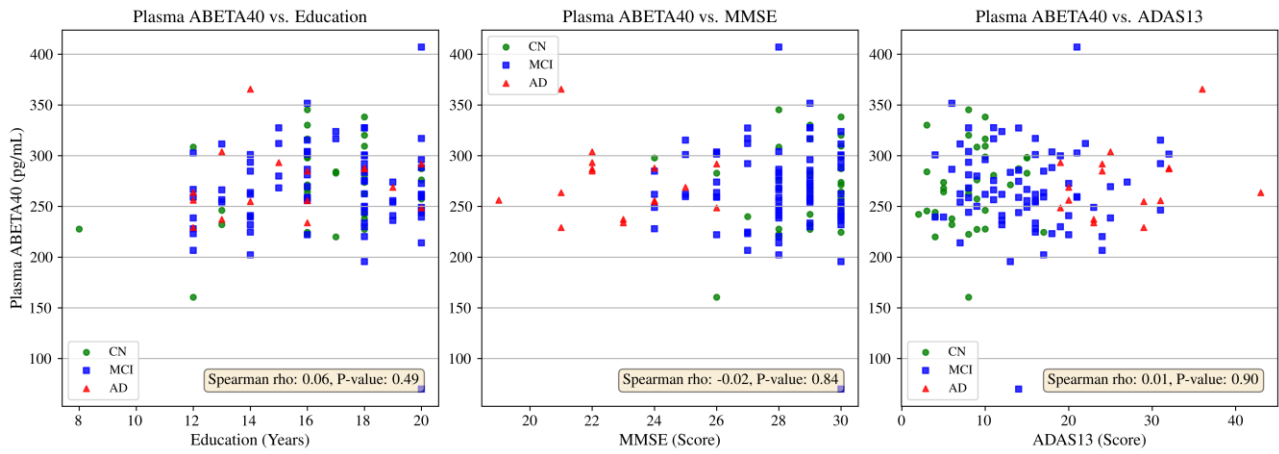


Figure B.2: Plasma A β 40 versus ordinal categorical variables.

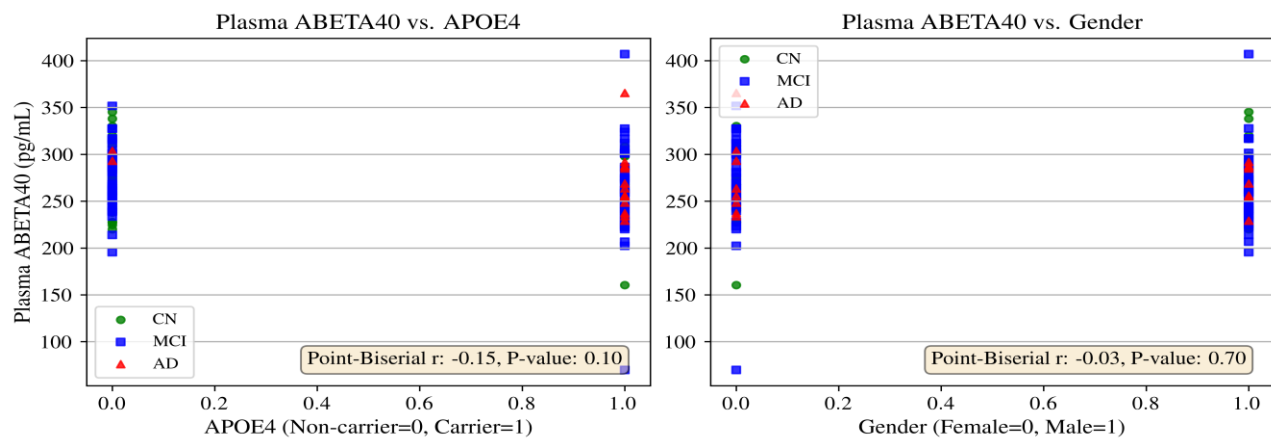


Figure B.3: Plasma AB40 versus categorical nominal features.

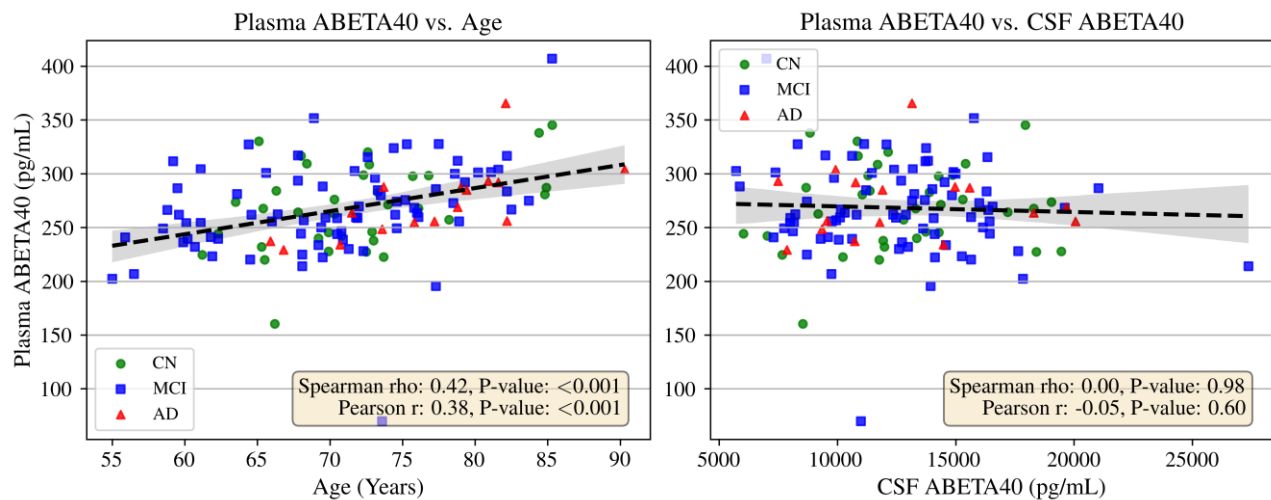


Figure B.4: Plasma A β 40 versus age and CSF A β 40 by diagnosis category.

For the relationship between A β 40 and the clinical variables, A β 40 has no relationship with any variable except age and APO4. Figure B.3 shows a weak correlation with APOE4 while no correlation with Gender. From Figure B.4 it is evident that A β 40 has a linear relationship with age,

and they are strongly correlated. Conversely, there is no correlation with CSF A β 40 which is very notable finding and is supported by previous research [75].

B.3 Correlation Analysis

From the Spearman correlation matrix in Figure B.5, only Age shows a moderate correlation with A β 40 with APOE4 having a very weak negative correlation. In this case since ADAS13 and MMSE have the least correlation with the most multicollinearity thus they will be removed. CSF A β 40 will be kept because even though it has no correlation it is a core part of the study and would be useful to see its predictive power.

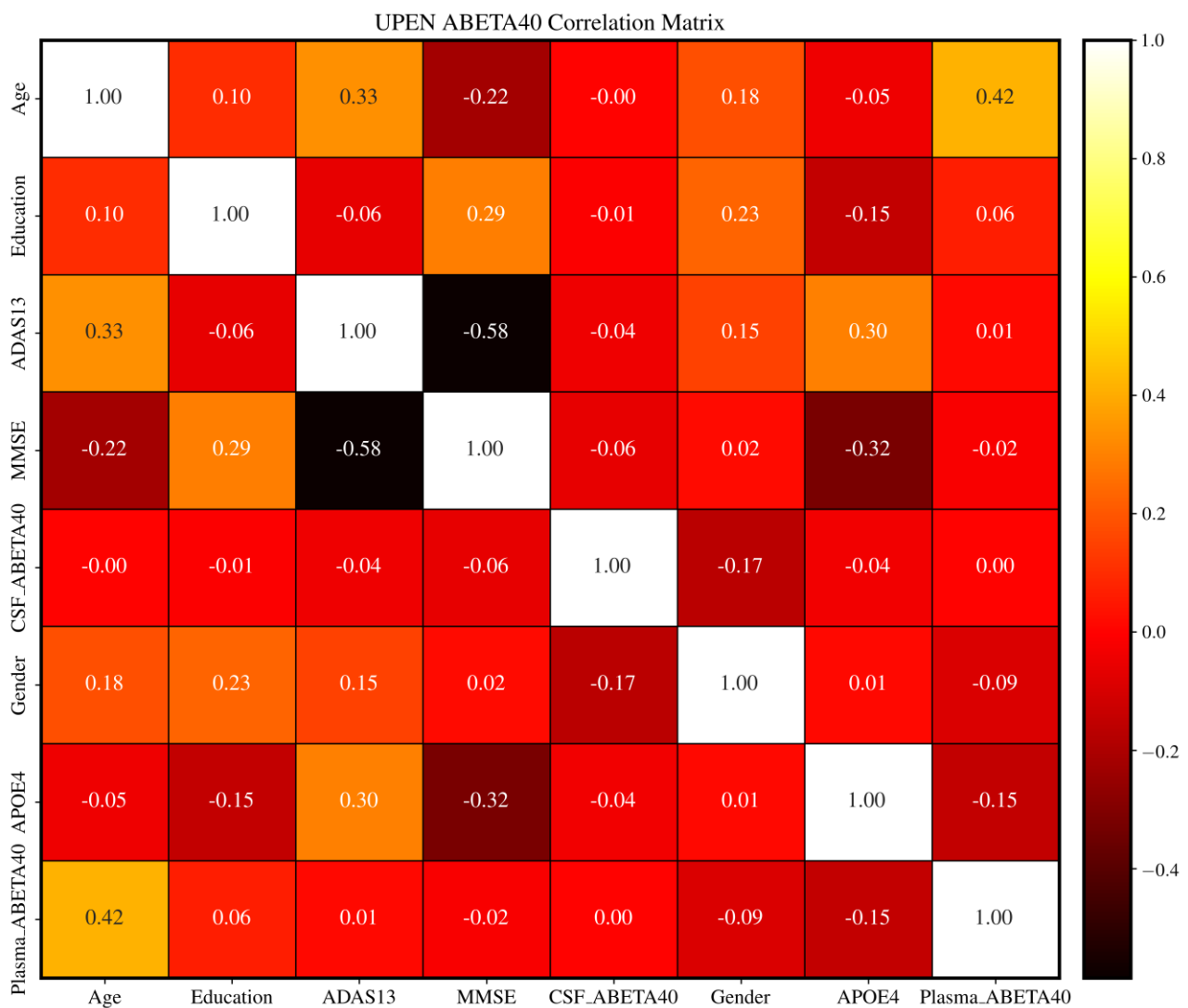


Figure B.5: Spearman correlation matrix for FUJI A β 40 dataset.

B.4 Results, Evaluation and Discussion

B.4.1 Combination Selection and Hyper-Parameters

The cross-validated evaluation of the model is shown in Figures B.6, B.7, B.8. Figure B.6 shows that the best MAE is 25.108 pg/mL from the combination of Age, Gender, and Education, this is also the best in terms of R^2 (15.8%), while having a MAPE of 10.8%. The optimal parameters for the regression model, determined through tuning, are as follows: The regularization parameter at 100.0 is relatively high, indicating a lower tolerance for errors across the model—this helps manage the high variance in the dataset by allowing the model to fit more closely to the training data. The epsilon value of approximately 0.0206 is quite small, which implies a narrow margin of tolerance where errors are considered acceptable, promoting accuracy in predictions. The gamma parameter at 0.0612, though moderate, suggests that each training example has a significant but not overly restrictive influence on the decision boundary, balancing the trade-off between bias and variance effectively.

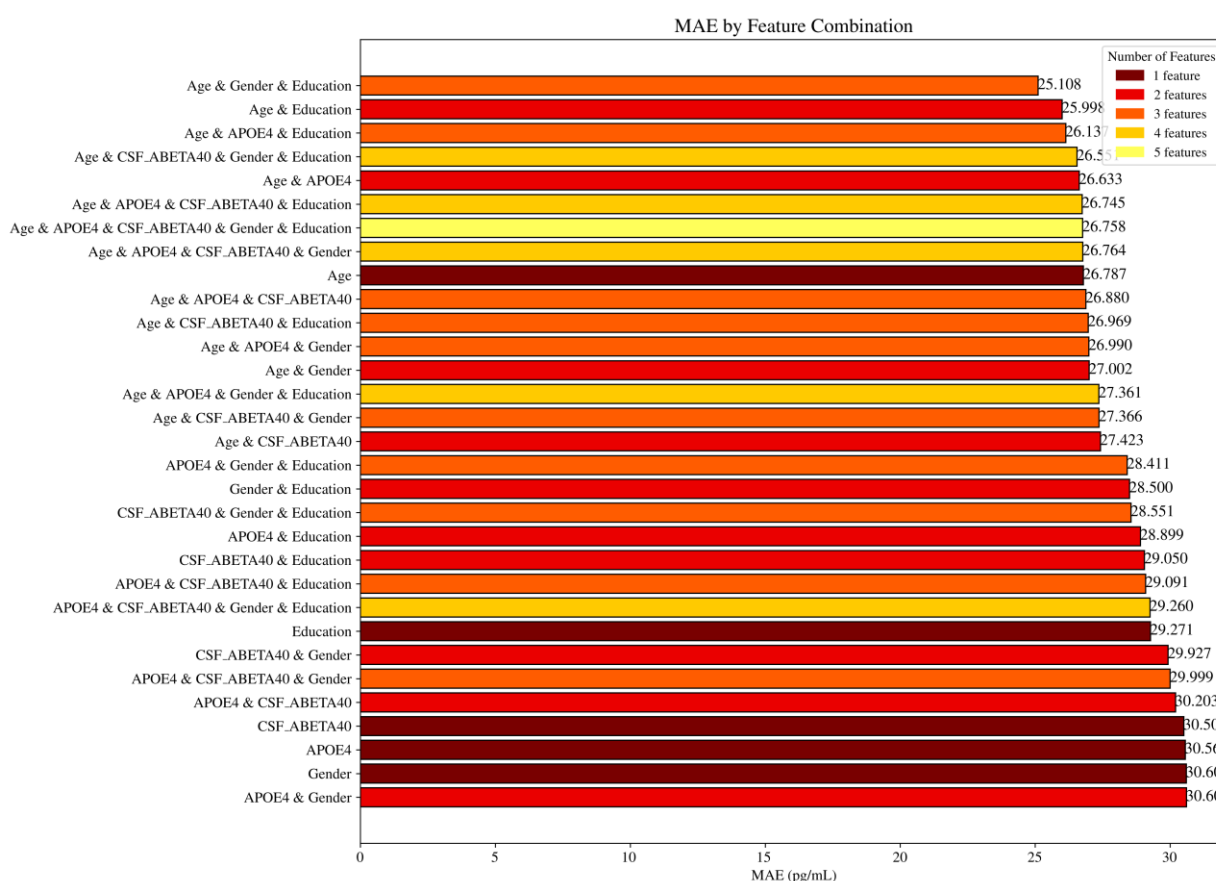


Figure B.6: Evaluation of A β 40 models by feature combination using MAE.

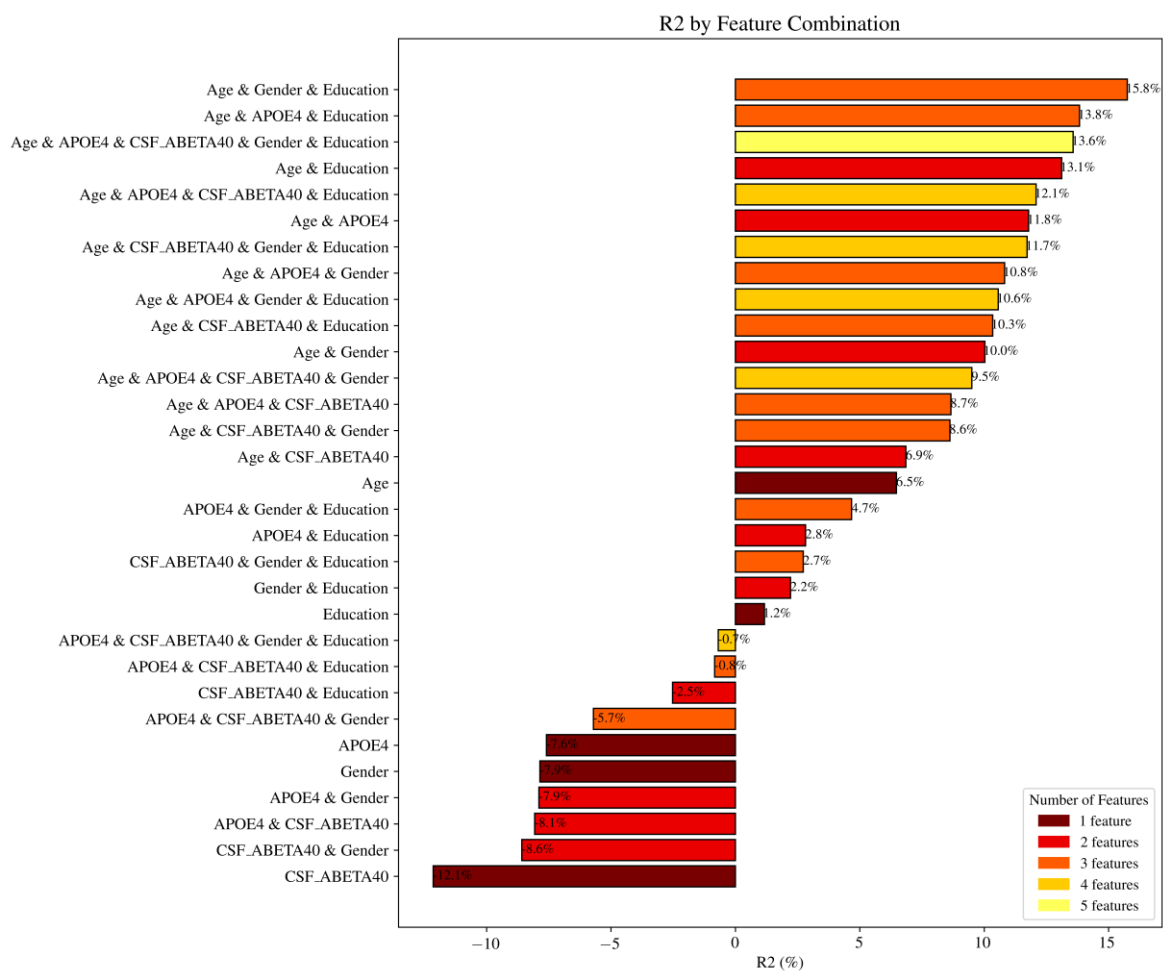


Figure B.7: Evaluation of A β 40 models by feature combination using R^2 .

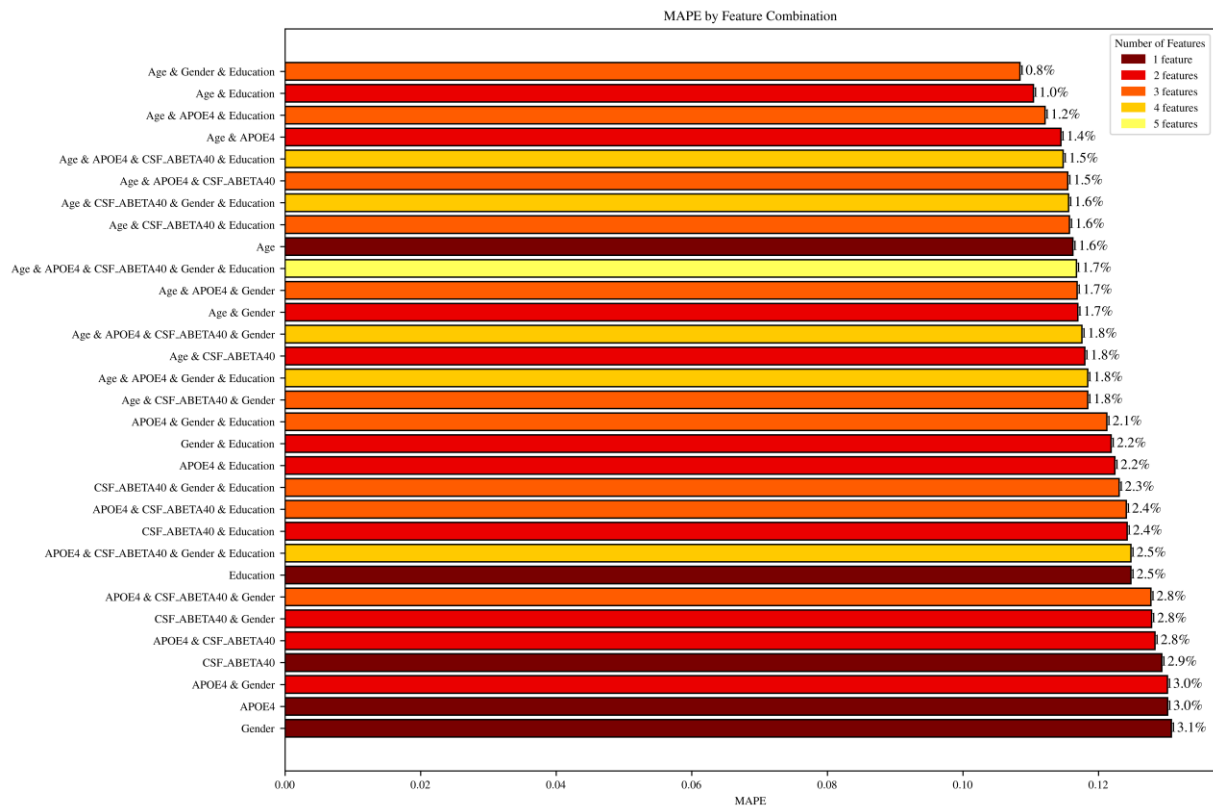


Figure B.8: Evaluation of A β 40 models by feature combination using MAPE.

B.4.2 Residual Analysis and Predictive Performance

The residual distribution in Figure B.9 approximates a normal distribution. The calculated skewness value of -1.074 suggests that the data has a tail on the left side. This implies the presence of outliers. The kurtosis value of 7.029 is significantly higher than 0, which is typical for a normal distribution, indicating a leptokurtic distribution. This suggests that the residuals have heavier tails and a sharper peak compared to a normal distribution, which can be indicative of outliers or extreme values in the data set. These make sense because indeed there were many outliers in the predictions since the dataset had a lot of variance with many outliers and not many predictive features.

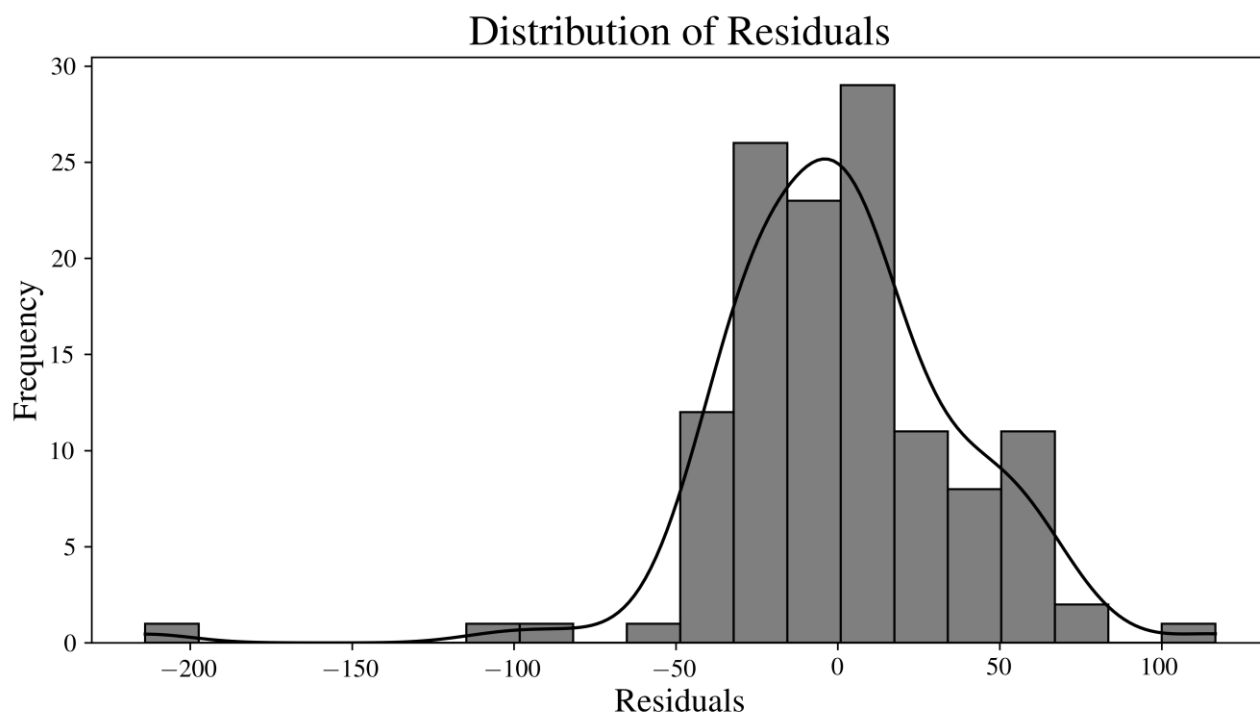


Figure B.9: Distribution of Residuals for A β 40.

Figure B.10 presents the predictions against the true values confirming the residual analysis. There is a clear bias in the data for smaller and higher values. However, the SVR algorithm cannot improve at predicting them.

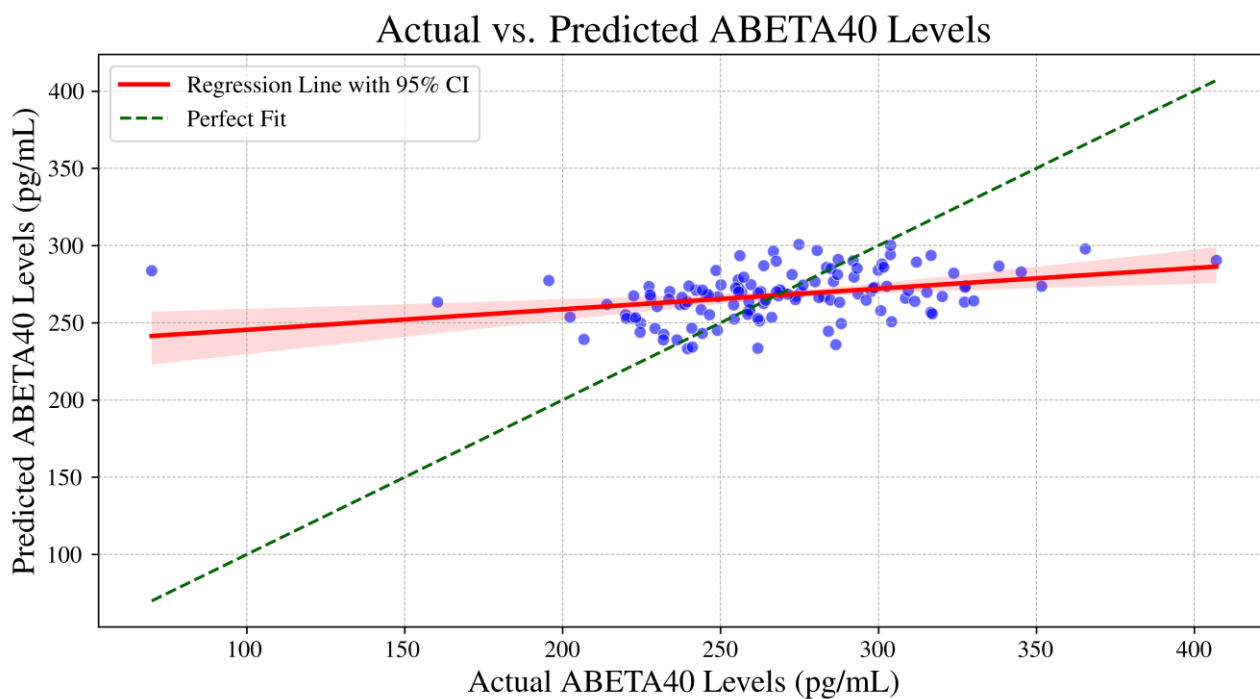


Figure B.10: Scatter Plots of Estimated vs. Actual A β 40 Levels.

B.4.3 Benchmark and Standard Deviation Evaluation

Using a naïve regressor as a benchmark as before, its MAE is 30.627, thus the current model even though it has biased results it exhibits an improvement compared to the benchmark by 18% for MAE and 15.8% for R^2 . However, these values got a very high p-value of 0.94 not making a statistically significant difference. Finally with a SD of 41.05 pg/mL, the model with MAE 25.11 pg/mL was on average 61% lower than the SD, thus surpassing the threshold of 70%.

The lack of statistical significance in the regression analysis, as indicated by a p-value over 0.90, can be attributed to several factors related to the dataset's characteristics. The high variance within the dataset suggests a broad dispersion of data points, which complicates the detection of a clear pattern. Outliers present in the data may also skew the results, disproportionately influencing the regression model and potentially leading to misleading conclusions. Additionally, the small size of the dataset limits the statistical power of the analysis, making it challenging to detect significant effects even if they exist. These elements together contribute to the observed non-significant result, highlighting the complexities of working with such data.

C. Analysis of A β 42/40 Blood Plasma Biomarker

Only the FUJI dataset will be used since UPEN does not have enough A β 42/40 data that correspond to plasma data.

C.1 Normality and Descriptive Statistics

All the dataset is the same as in Appendix B, except CSF and plasma A β 42/40. For those the normality test in Table C.1 shows CSF A β 42/40 and Plasma A β 42/40 ratios reveal differences in distribution characteristics. The CSF A β 42/40 ratio shows a skewness of 0.156, suggesting minimal asymmetry in distribution. Its kurtosis of -1.424, following Fisher's kurtosis adjustment, indicates a platykurtic distribution with fewer outliers and less peak than a normal distribution. Importantly, the KS p-value of 0.024 for CSF A β 42/40 suggests this distribution significantly deviates from normality. In contrast, the Plasma A β 42/40 ratio, with a skewness of 0.617, exhibits a mild rightward skew. Its kurtosis of 0.887 suggests a modestly leptokurtic distribution with slightly heavier tails. The higher KS p-value of 0.198 indicates that the deviation from normality is not statistically significant for Plasma A β 42/40.

Table C.1: A β 42/40 Normality Tests.

Variable	FUJI		
	Skewness	Kurtosis	KS p-value
Age	<0.001	-0.450	0.968
Education	-0.436	-0.457	0.002
MMSE	-1.228	0.944	<0.001
ADAS13	0.851	0.493	0.139
CSF A β 42/40	0.156	-1.424	0.024
Plasma A β 42/40	0.617	0.887	0.198

For the descriptive statistics of Table C.2, the dataset is the same to A β 42. Thus, the analysis is discussed in the main part.

Table C.2: A β 42/40 Descriptive Statistics.

Variable	FUJI (n=127)
Age, years (mean, SD)	71.42 $\pm$ 7.34
Education, years (median, IQR)	16.00 (14.00 – 18.00)
MMSE (median, IQR)	28.00 (26.00 - 29.50)
ADAS13 (median, IQR)	14.00 (8.00 - 19.50)
CSF A β 42/40, pg/mL (Median, IQR)	0.06 (0.04 - 0.09)
Plasma A β 42/40, pg/mL (mean, SD)	0.12 $\pm$ 0.01
E4+ (%)	48.82
Gender (M%)	54.33
Race (White%)	92.91

C.2 Exploration of Clinical Data and Biomarker Relationships

Similarly to all the previous datasets this also exhibits huge variance for CSF and blood plasma biomarkers along with a few outliers. However, unlike before it can distinguish between diagnosis categories in both biomarkers in every case except for CN and AD in plasma, and it is slightly over the threshold for CN and MCI too as is shown in Figure C.1.

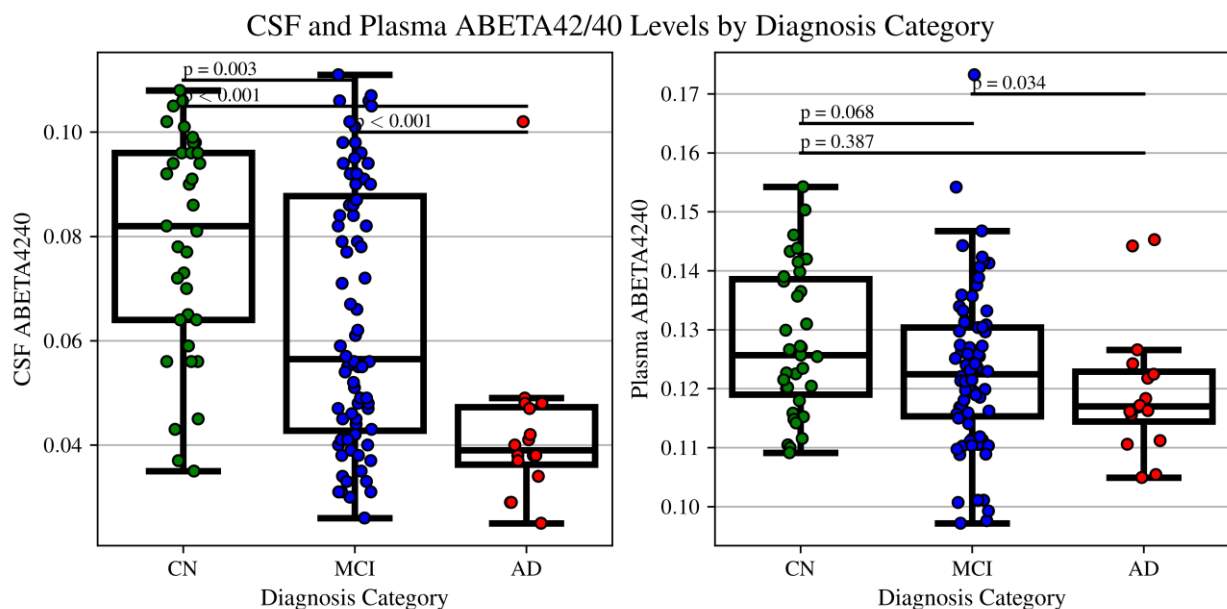


Figure C.1: CSF and Plasma A β 42/40 Levels by diagnosis category.

Figures C.2, C.3, C.4 show the relationship of plasma A β 42/40 with clinical and CSF data. Notably, A β 42/40 seems to be well correlated with most variables. ADAS13 shows a negative moderate spearman correlation, with a non-linear relationship. APOE4, also indicates a statistically significant moderate correlation with plasma A β 42/40. Then MMSE, Age, and Gender have a weak correlation with plasma A β 42/40, while education shows no correlation. In contrast, CSF A β 42/40 has substantial, statistically significant correlation with plasma A β 42/40 with a spearman coefficient of 0.63, with what seem like a linear relationship.

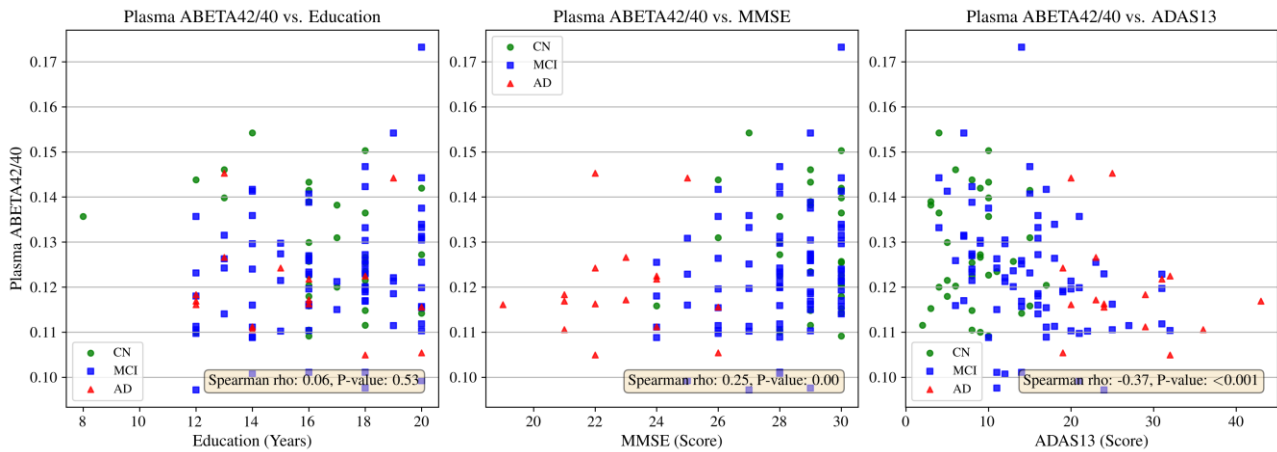


Figure C.2: Plasma A β 42/40 versus ordinal categorical variables.

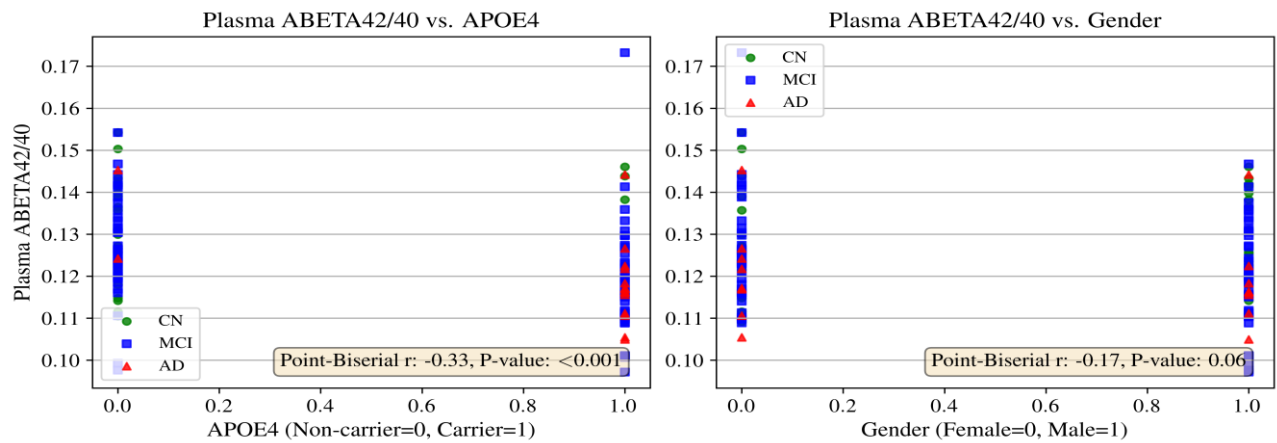


Figure C.3: Plasma A β 42/40 versus categorical nominal features.

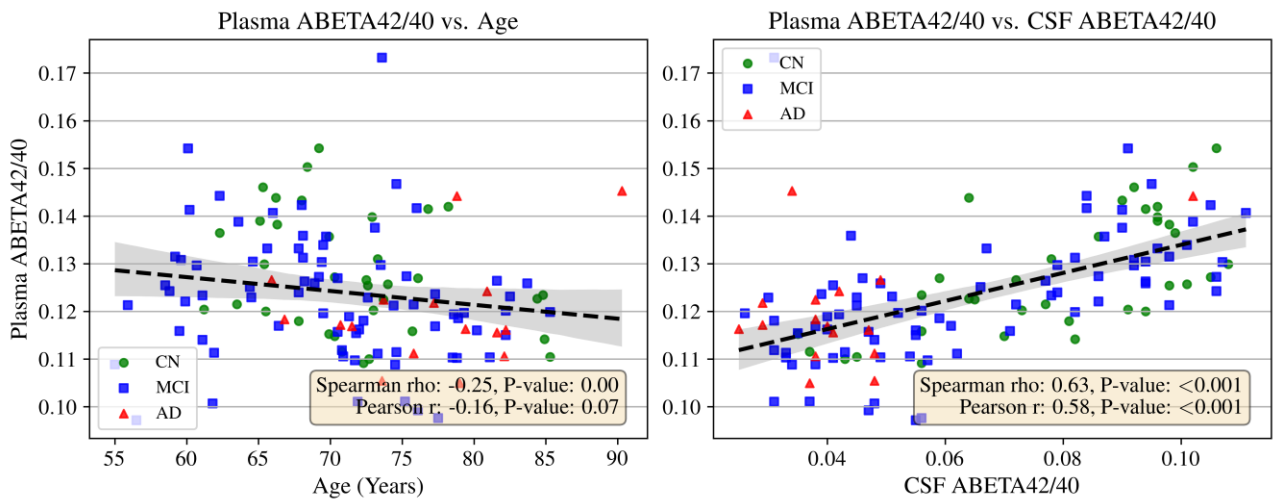


Figure C.4: Plasma A β 42/40 versus age and CSF A β 42/40 by diagnosis category.

C.3 Correlation Analysis

From the correlation matrix of Figure C.5, the selection of the features to go to the next stage will be selected. Even though Education and gender have a low correlation with plasma A β 42/40, they were very important features for plasma A β 40, thus they will not be eliminated. MMSE on the other hand has a relatively weak correlation with high multicollinearity thus it will be eliminated. Although the remaining features will have some multicollinearity between them, it is estimated that those with multicollinearity will be eliminated in the next phase because of their low scores.

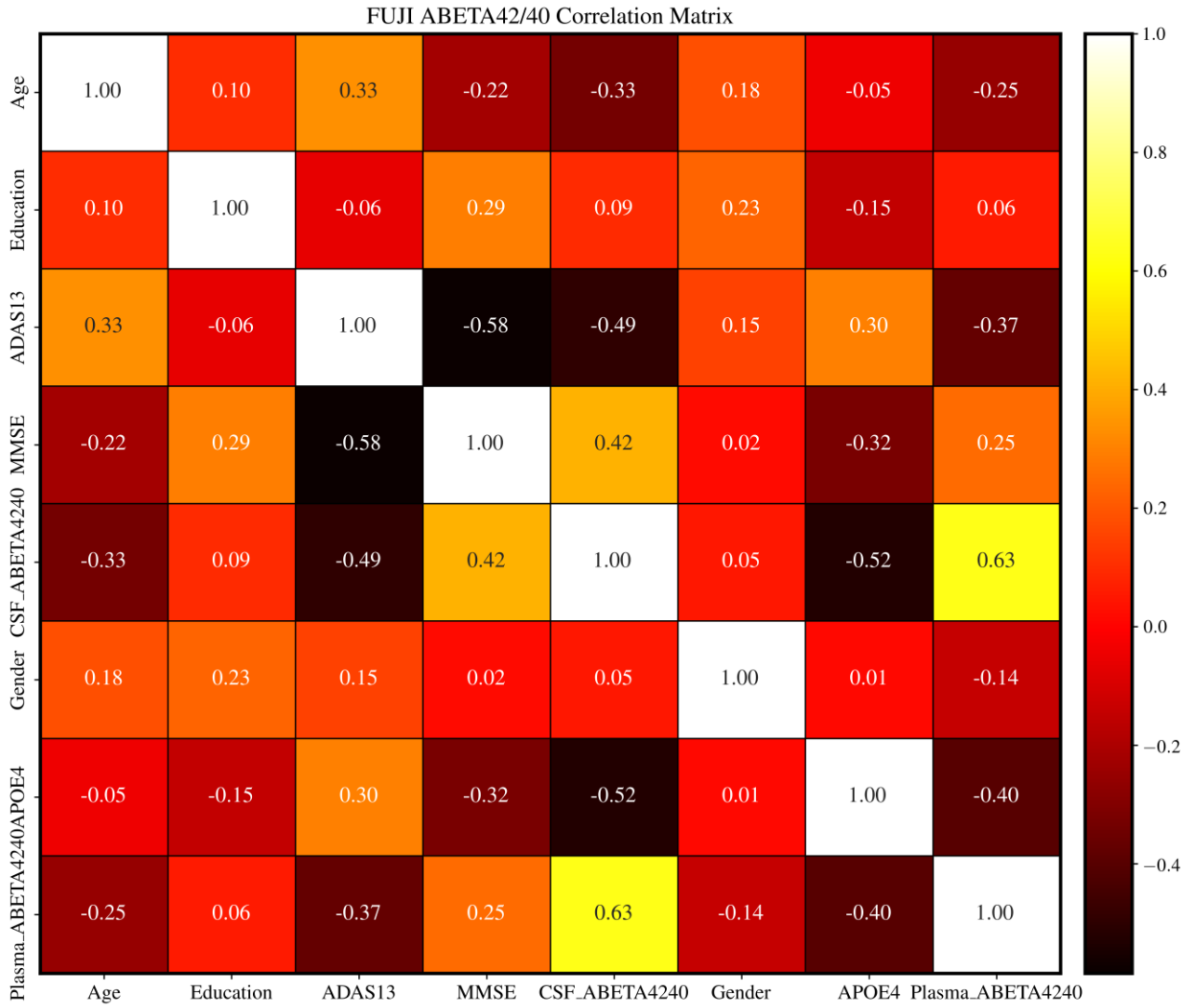


Figure C.5: Spearman correlation matrix for FUJI A β 42/40 dataset.

C.4 Results, Evaluation and Discussion

C.4.1 Combination Selection and Hyper-Parameters

The cross-validated evaluation of the model is shown in Figures C.6, C.7, C.8. Figure C.6 shows that the best MAE is 0.00074 from the combination of Age, APOE4, and CSF A β 42/40, while also having the best MAPE of 5.93%. The best combination in terms of R^2 (33.5%), is APOE4, CSF A β 42/40 and MMSE while the previous combination has the 3rd best R^2 (32.1%). Since the main criterion for choosing a model is accuracy, Age, APOE4, and CSF A β 42/40 will be chosen. The regularization parameter C set at 0.01 is notably low, suggesting a higher tolerance for errors in the model, this is used to prevent overfitting, probably because there is high noise. The epsilon value, at approximately 0.0036, is very small, indicating a very strict margin within which predictions are

considered accurate aiming to achieve precise predictions despite the potentially noisy data environment. The gamma parameter is set at a relatively high value of 0.807, which means that the model is sensitive to the training data, potentially leading to a model that captures more detailed data patterns but at the risk of overfitting.

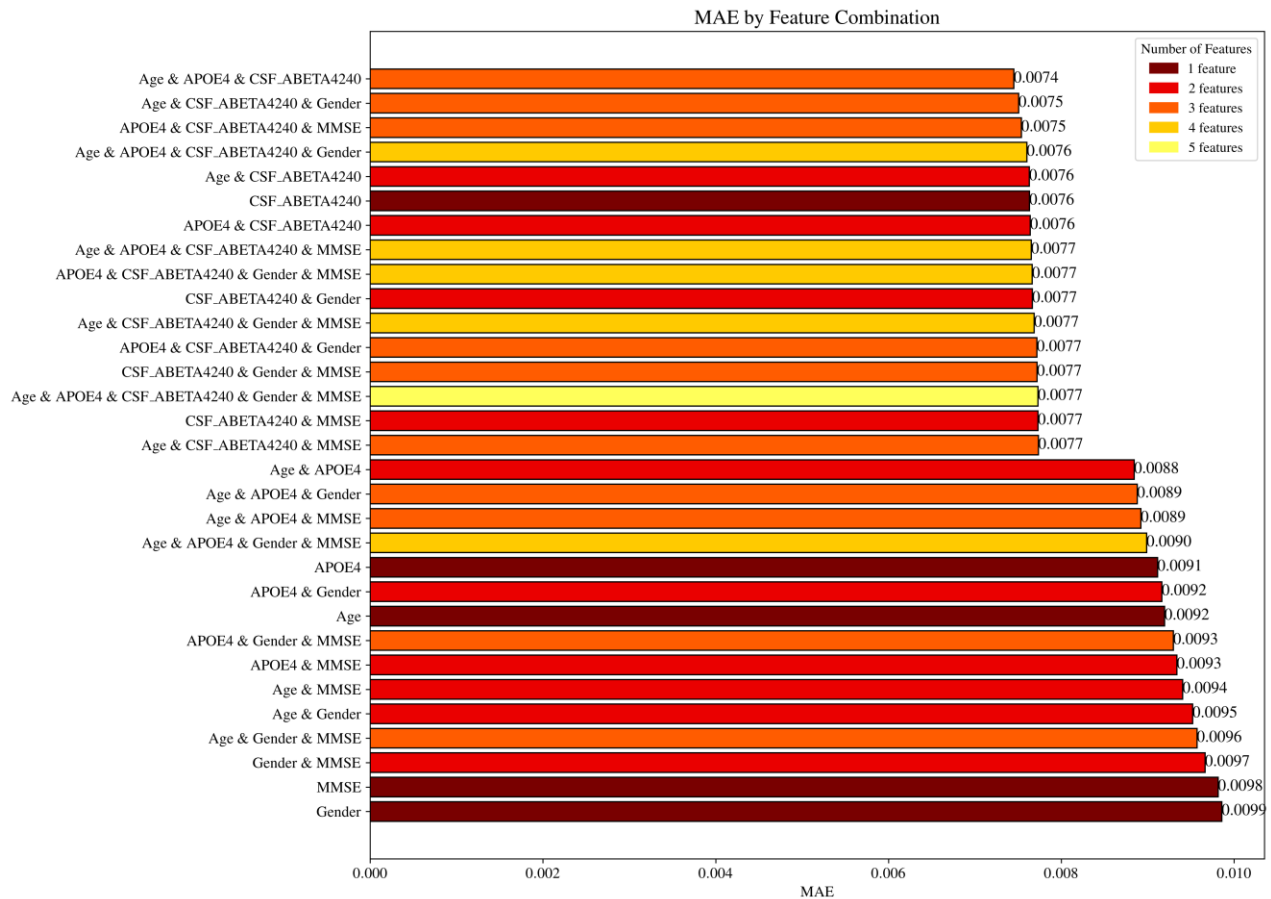


Figure C.6: Evaluation of A β 42/40 models by feature combination using MAE.

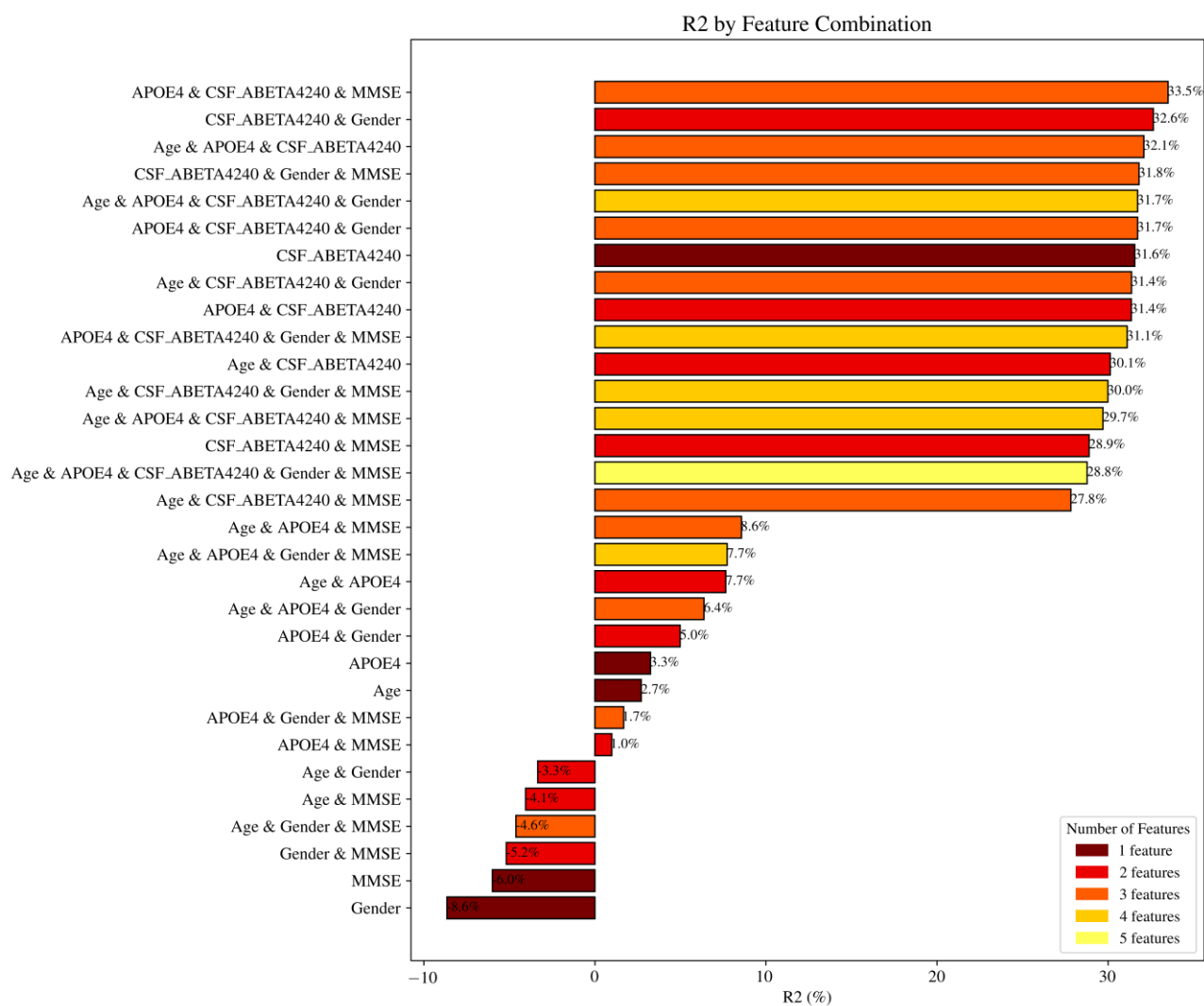


Figure C.7: Evaluation of A β 42/40 models by feature combination using R^2 .

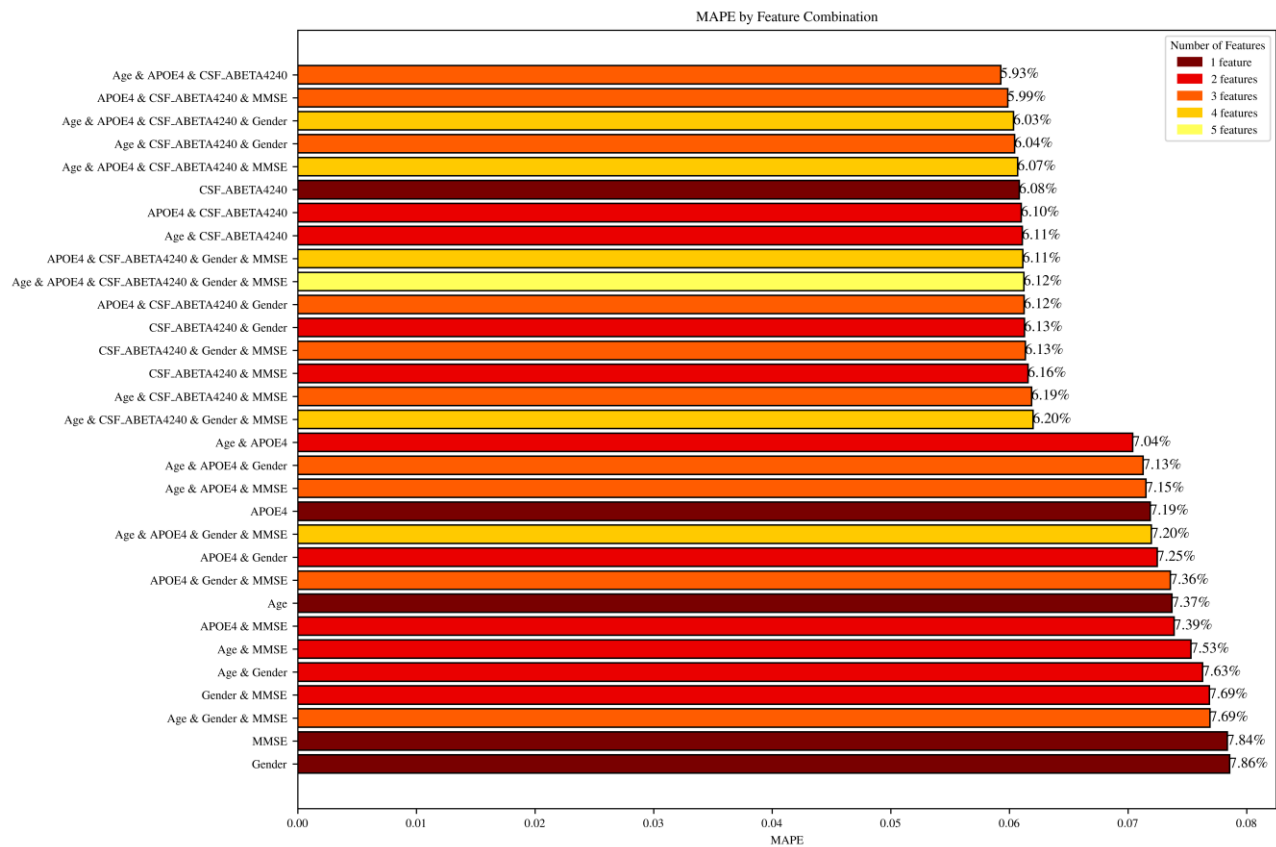


Figure C.8: Evaluation of A β 42/40 models by feature combination using MAPE.

C.4.2 Residual Analysis and Predictive Performance

The residual distribution in Figure C.9, shows an approximately normal distribution but with many outliers after 0.02. This is also shown in the skewness and kurtosis of the residuals. A skewness value of 1.487 suggests that the residuals are not symmetrically distributed but rather exhibit a rightward skew. The kurtosis value of 6.045, which is substantially higher than 0, indicates a leptokurtic distribution. This means the residuals are more peaked and have fatter tails than a normal distribution, suggesting that there are frequent extreme deviations from the mean, either too high or too low, which the model fails to capture effectively. This problem is caused by the variance and outliers in the data.

This pattern can also be observed in figure C.10, where most predictions are accurate close to the ideal best fit line and with low 95% confidence interval but for the higher values still the model underpredicted.

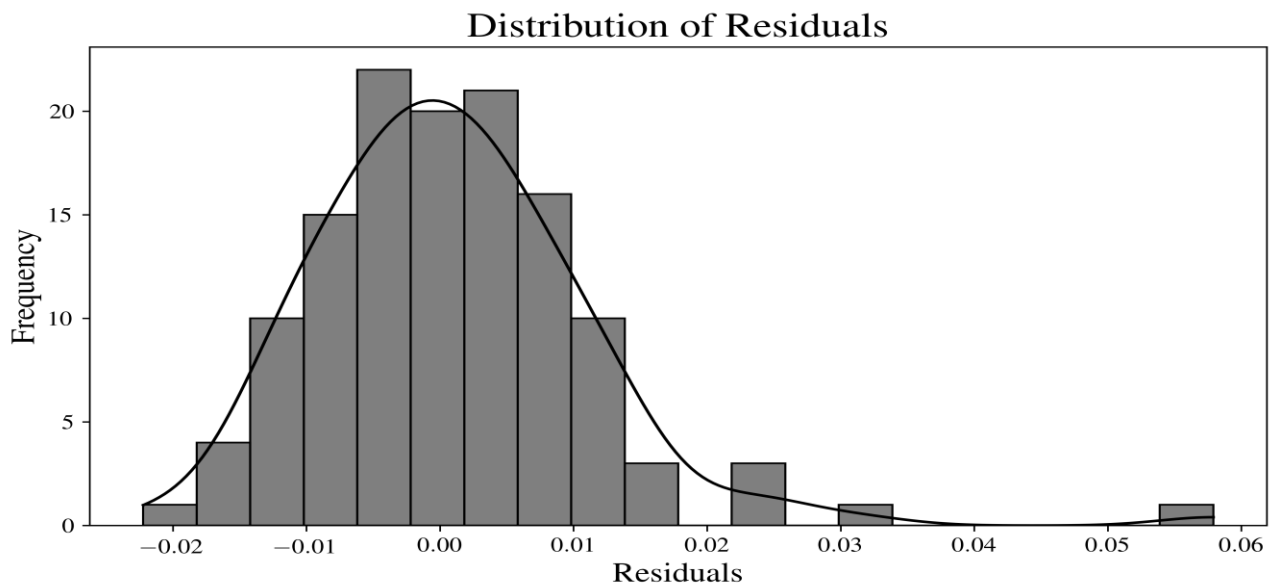


Figure C.9: Distribution of Residuals for Aβ42/40.

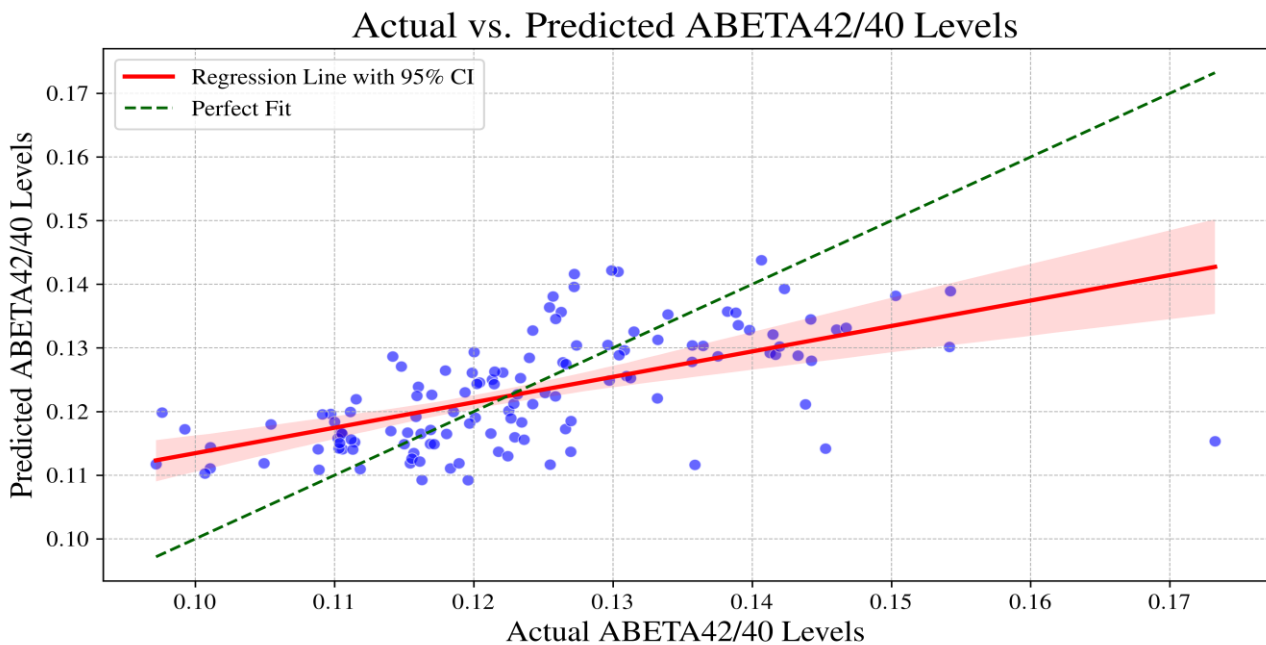


Figure C.10: Scatter Plots of Estimated vs. Actual Aβ42/40 Levels.

C.4.3 Benchmark and Standard Deviation Evaluation

Using a naïve regressor as a benchmark, its MAE is 0.01, thus the current model even though it has biased results it exhibits an improvement compared to the benchmark by 26% for MAE and 33.5% for R^2 . However, it does not have a statistically significant difference (p-value=0.059), most probably due to the small size as described before. In terms of SD (0.01) the best model with MAE 0.0074 was 74% lower than the SD not surpassing the threshold of 70%

D. Analysis of P-tau181 Blood Plasma Biomarker

D.1 Normality and Descriptive Statistics

The analysis of skewness, kurtosis, and KS test p-values in Table D.1 reveals distinct distribution traits. Age demonstrates almost perfect symmetry and slight flatness (kurtosis -0.450) with its normality largely confirmed by a KS p-value of 0.883. Education, MMSE, ADAS13, CSF P-tau181, and Plasma P-tau181 all show deviations from normality (KS p-values <0.001). Specifically, Education and Plasma P-tau181 exhibit leftward skewness, with Plasma P-tau181 also showing significant peakiness (kurtosis 4.247). In contrast, MMSE, ADAS13, and CSF P-tau181 are right-skewed, indicating a distribution with extended tails towards higher values.

Table D.1: P-tau181 Normality Tests

Variable	Skewness	Kurtosis	KS p-value
Age	<0.001	-0.450	0.883
Education	-0.436	-0.457	<0.001
MMSE	-1.228	0.944	<0.001
ADAS13	0.851	0.493	<0.001
CSF P-tau181	0.618	1.03	<0.001
Plasma P-tau181	-0.491	4.247	<0.001

The dataset in Table D.2 highlights a predominance of male participants (53.84%). Additionally, most participants are White (92.75%), suggesting limited racial diversity.

Table D.2: P-tau181 Descriptive Statistics

Variable	P-tau181 Dataset
	(n=676)
Age, years (mean, SD)	72.80 ± 7.18
Education, years (median, IQR)	16.00 (14.00 – 18.00)
MMSE (median, IQR)	28.00 (26.00 - 29.50)
ADAS13 (median, IQR)	14.00 (9.00 - 21.25)
CSF P-tau181, pg/mL (median, IQR)	24.5 (17.37 - 33.42)
Plasma P-tau181, pg/mL (median, IQR)	17.65 (12.86 - 23.75)
E4+ (%)	47.7
Gender (M%)	53.84
Race (White%)	92.75

D.2 Exploration of Clinical Data and Biomarker Relationships

As it has been observed from previous datasets, Figure D.1 demonstrates a huge variance within the dataset, but unlike before the dataset also has many extreme outliers, that cannot be removed. Thus, for this dataset no outlier augmentation will be performed. However due the datasets size, all the diagnosis categories can be distinguished in both plasma and CSF.

CSF and Plasma P-tau Levels by Diagnosis Category

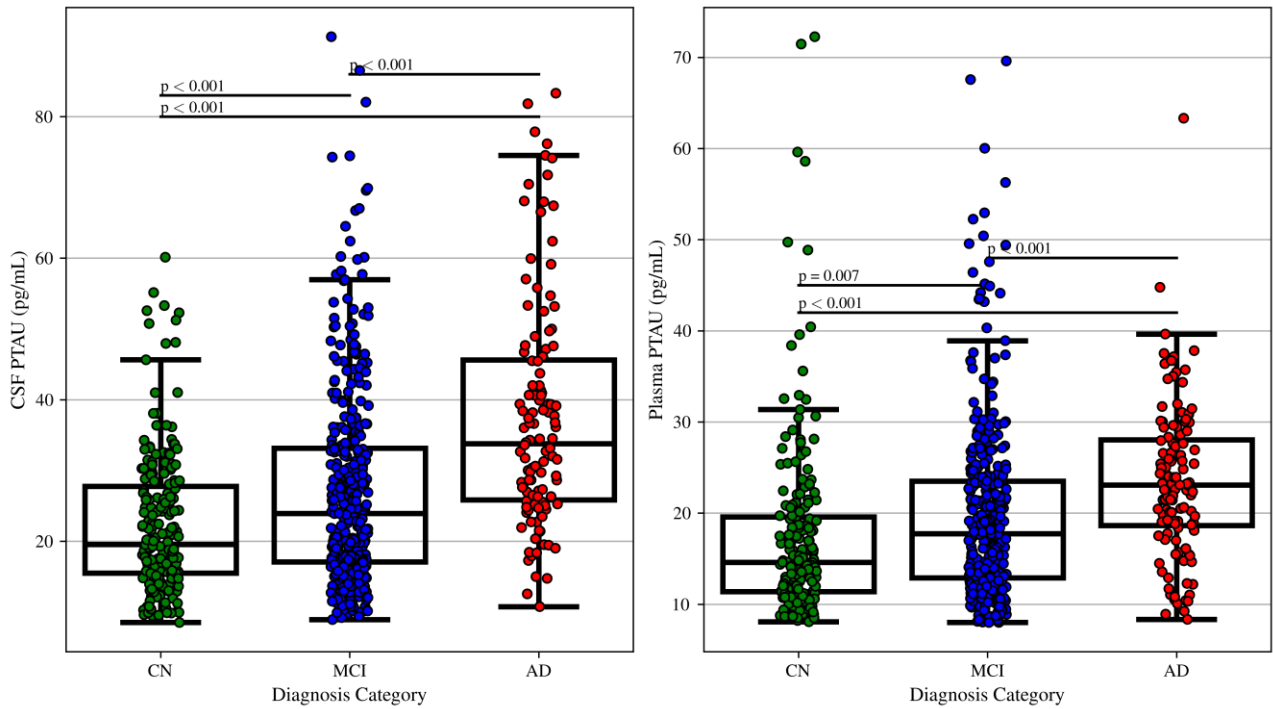


Figure D.1: CSF and Plasma P-tau181 Levels by diagnosis category.

Figure D.1 details no correlation or underlying pattern between education and p-tau. In contrast MMSE has a weak negative correlation with plasma p-tau and ADAS13 has a weak positive one which also seems to be linear. Figure D.2 shows a weak correlation with APO4 and no correlation with Gender. Finally, Figure D.4 details a weak correlation with age, and a strong nonlinear increasing correlation with CSF p-tau.

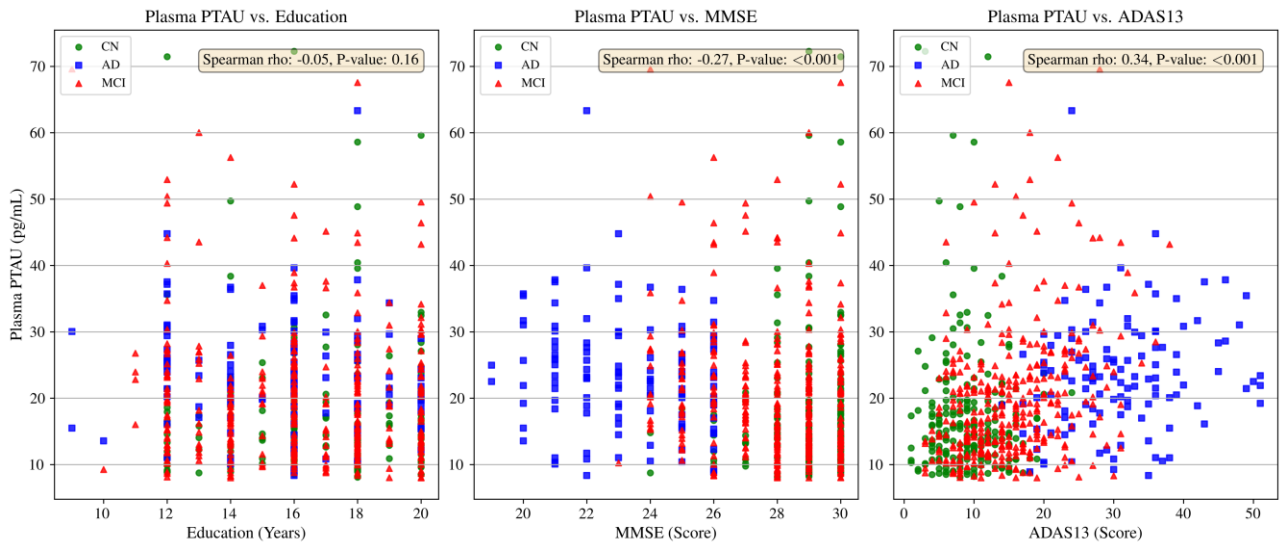


Figure D.2: Plasma P-tau versus ordinal categorical variables.

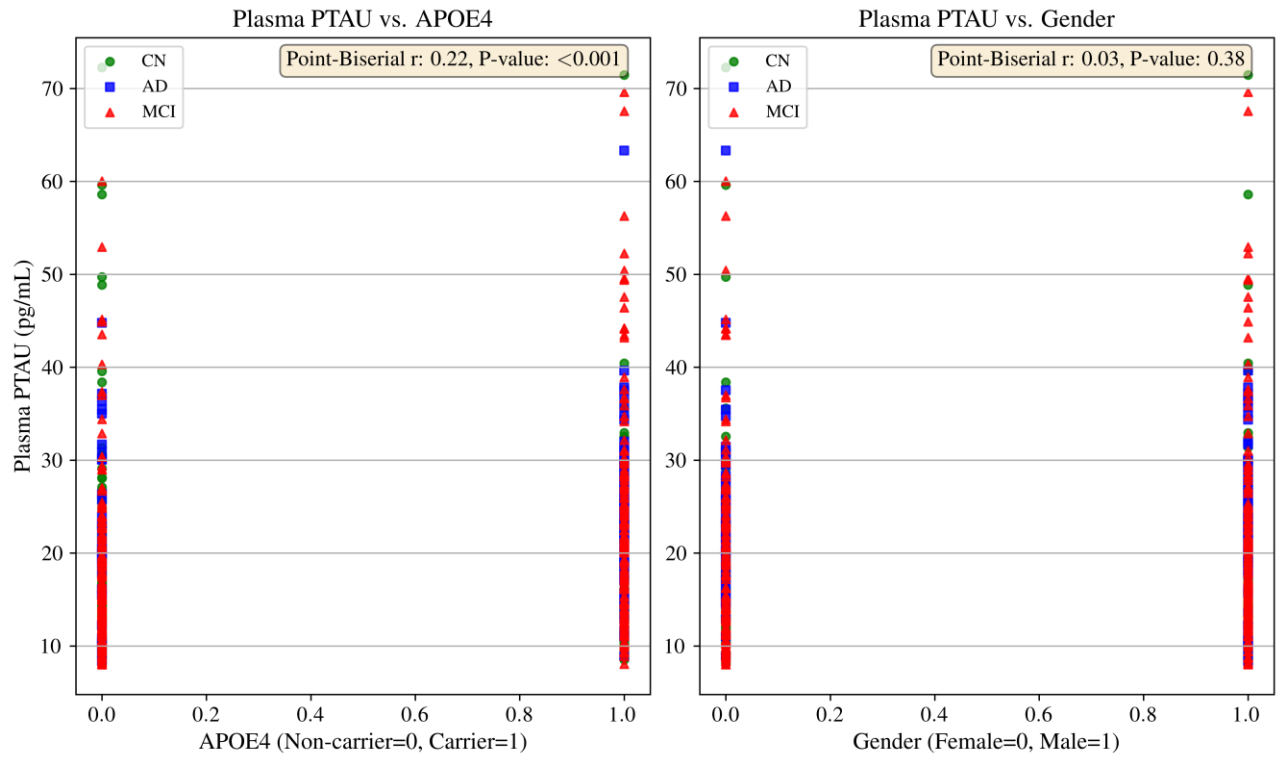


Figure D.3: Plasma P-tau versus categorical nominal features.

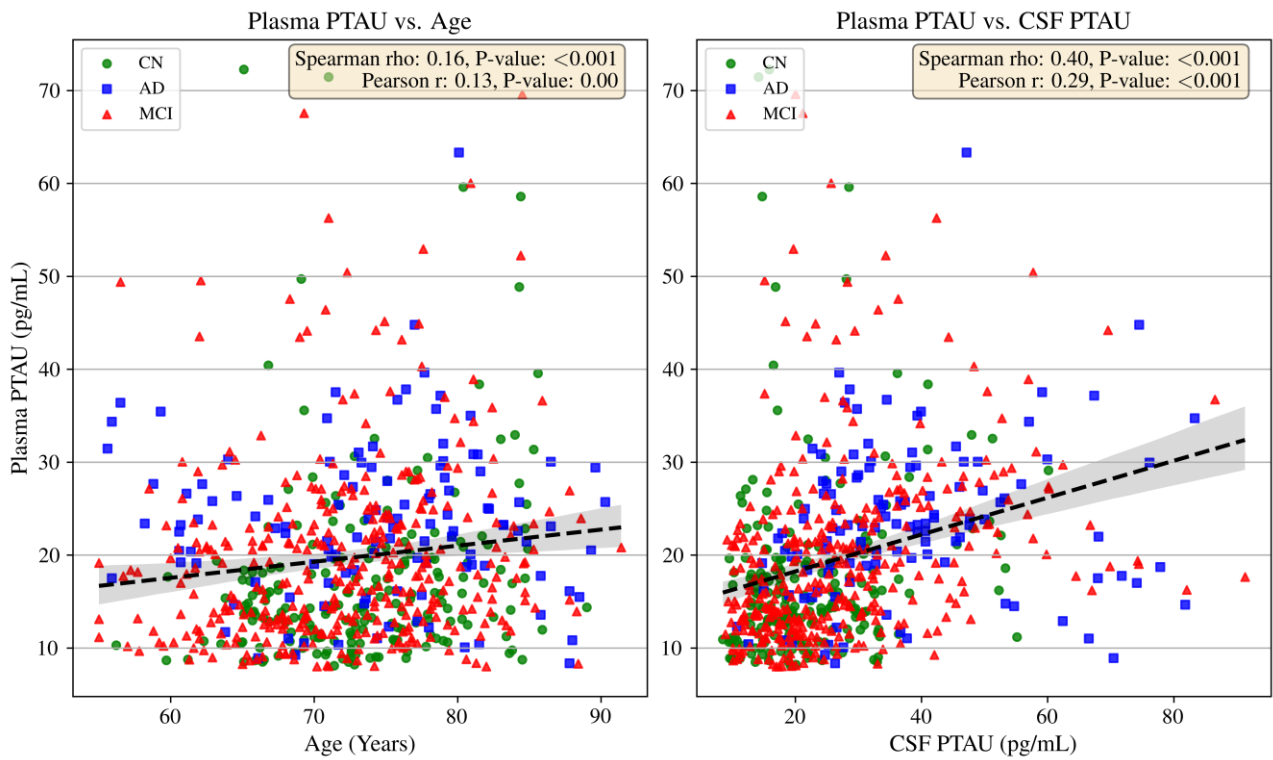


Figure D.4: Plasma P-tau versus age and CSF P-tau by diagnosis category.

D.3 Correlation Analysis

Figure D.5 provides a comprehensive overview of the correlations and multicollinearity within the dataset. It reveals that ADAS13 and CSF P-tau181 exhibit moderate correlations with plasma P-tau181, whereas MMSE, APOE4, and Age show weaker correlations. Education and gender display no significant correlations. To minimize multicollinearity issues, MMSE is selected over ADAS13 for inclusion in further analyses. Although MMSE has a lower correlation with plasma P-tau181, it is less intercorrelated with other variables, reducing potential redundancy. Education is also excluded from the analysis to streamline the variable set and focus on more impactful predictors.

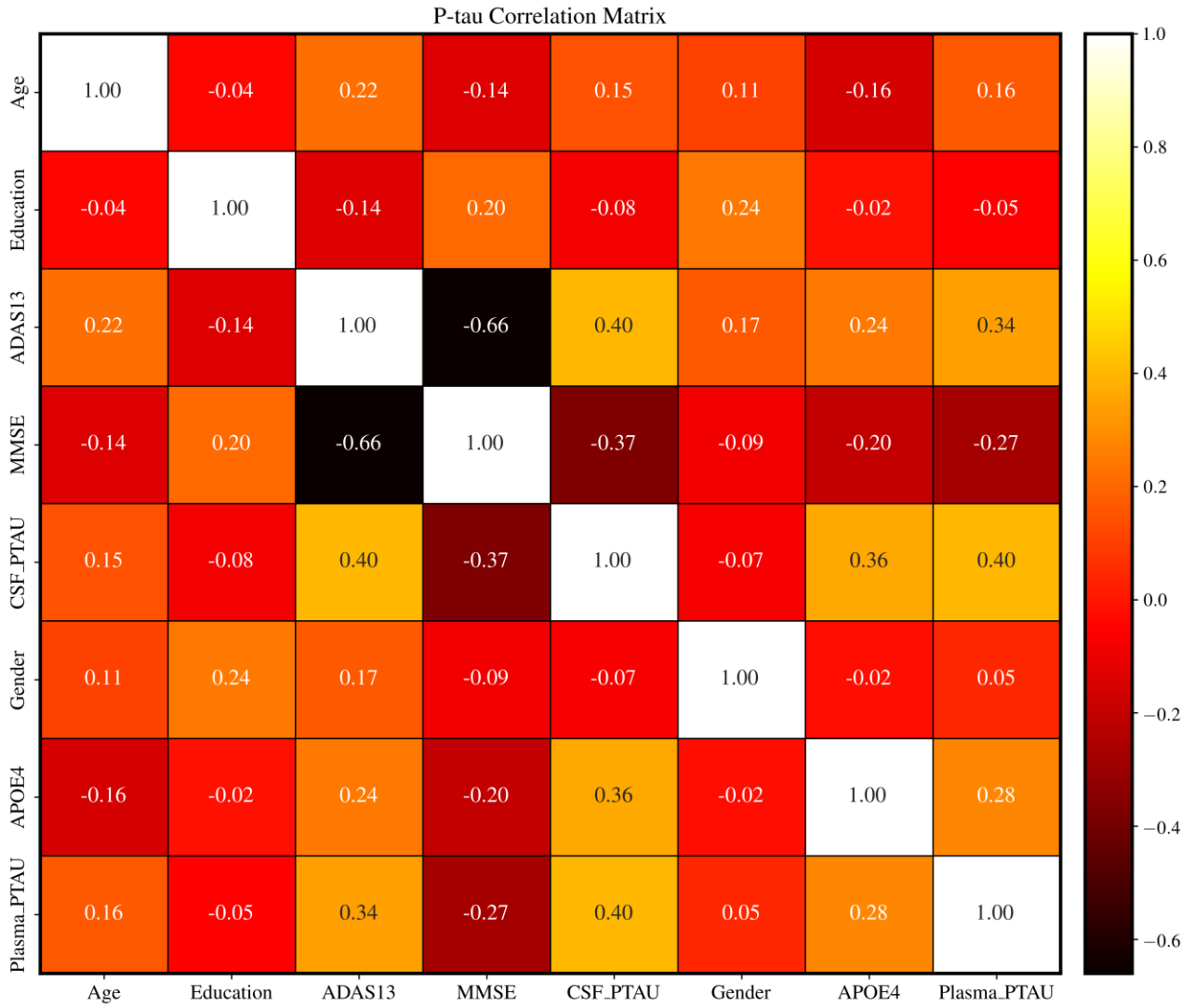


Figure D.5: Spearman correlation matrix for P-tau dataset.

D.4 Results, Evaluation and Discussion

D.4.1 Combination Selection and Hyper-Parameters

Utilizing the Figures D.6, D.7, D.8 the best combination for predicting plasma p-tau181 is Age, APOE4, CSF P-tau181 and MMSE with a MAE of 6.027 pg/mL and R^2 of 8.6%. The low R^2 was expected from the significant number of extreme outliers the dataset had. In terms of the hyper parameters the regularization parameter C at approximately 2.69 indicates a moderate level of penalty for errors, striking a balance between fitting the model closely to the training data and avoiding overfitting. The epsilon value set at 0.844 is relatively high, which implies a larger tolerance for prediction errors within the model potentially improving the robustness of the

model against noise and outliers. The gamma value at 0.0901, while not too high, still has an influence on the curvature of the decision boundary.

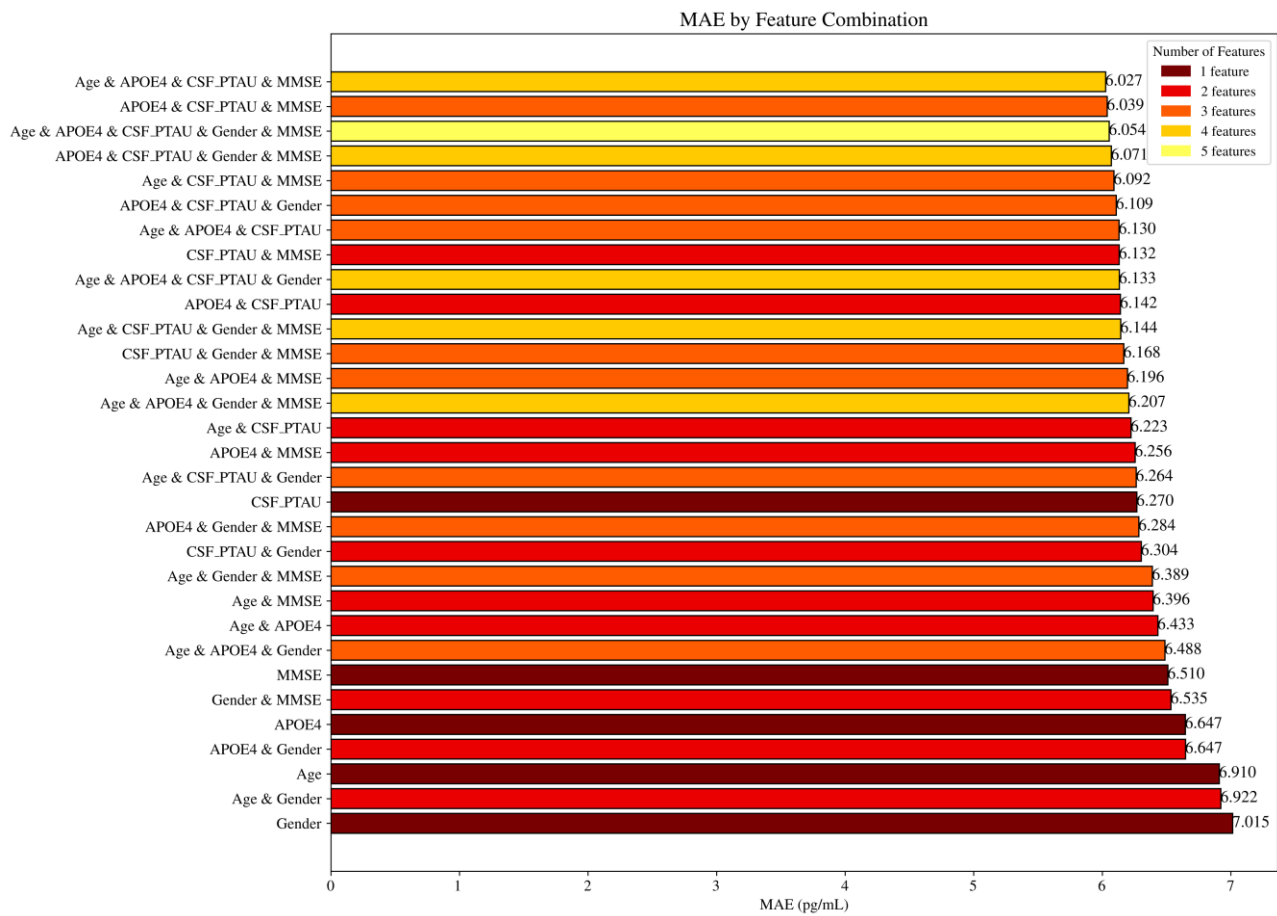


Figure D.6: Evaluation of P-tau models by feature combination using MAE.

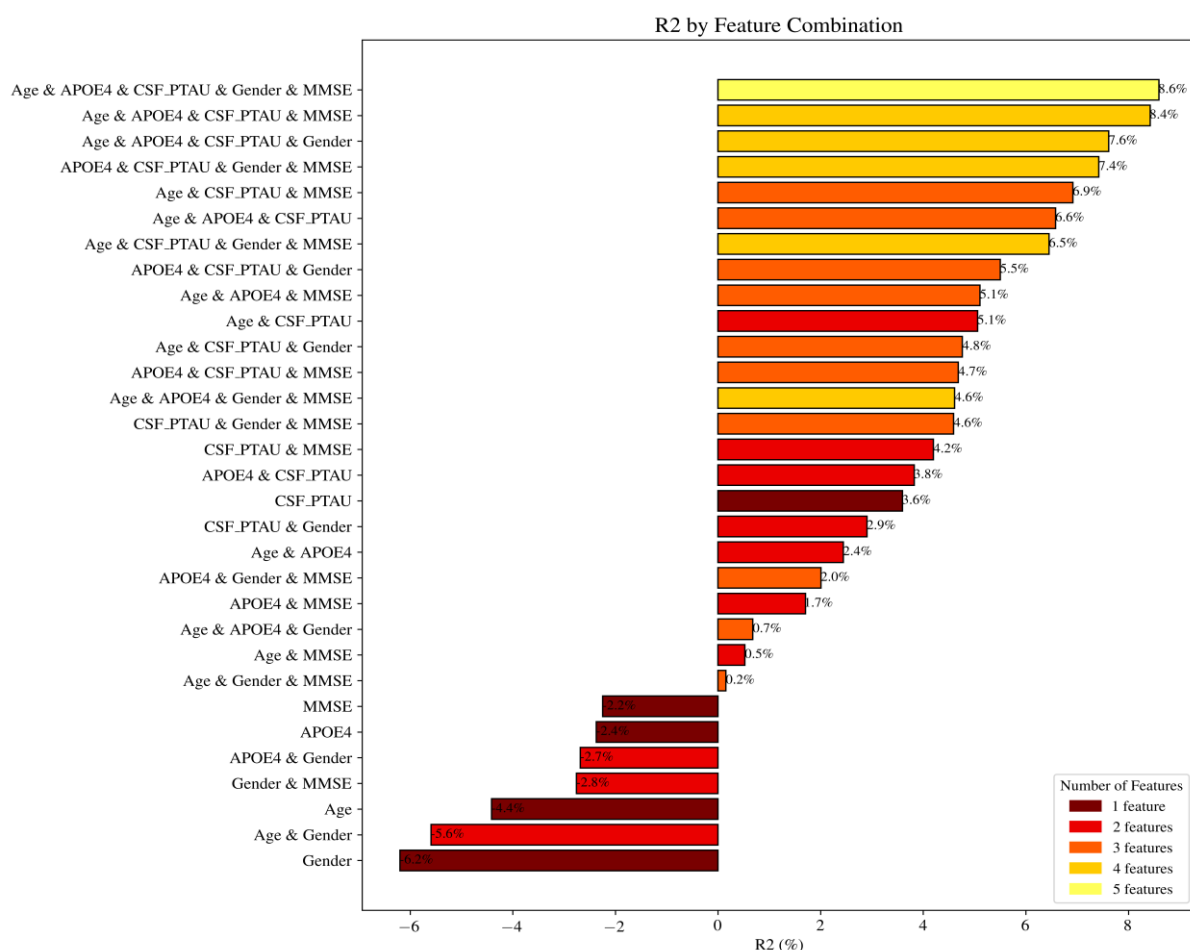


Figure D.7: Evaluation of P-tau models by feature combination using R^2 .

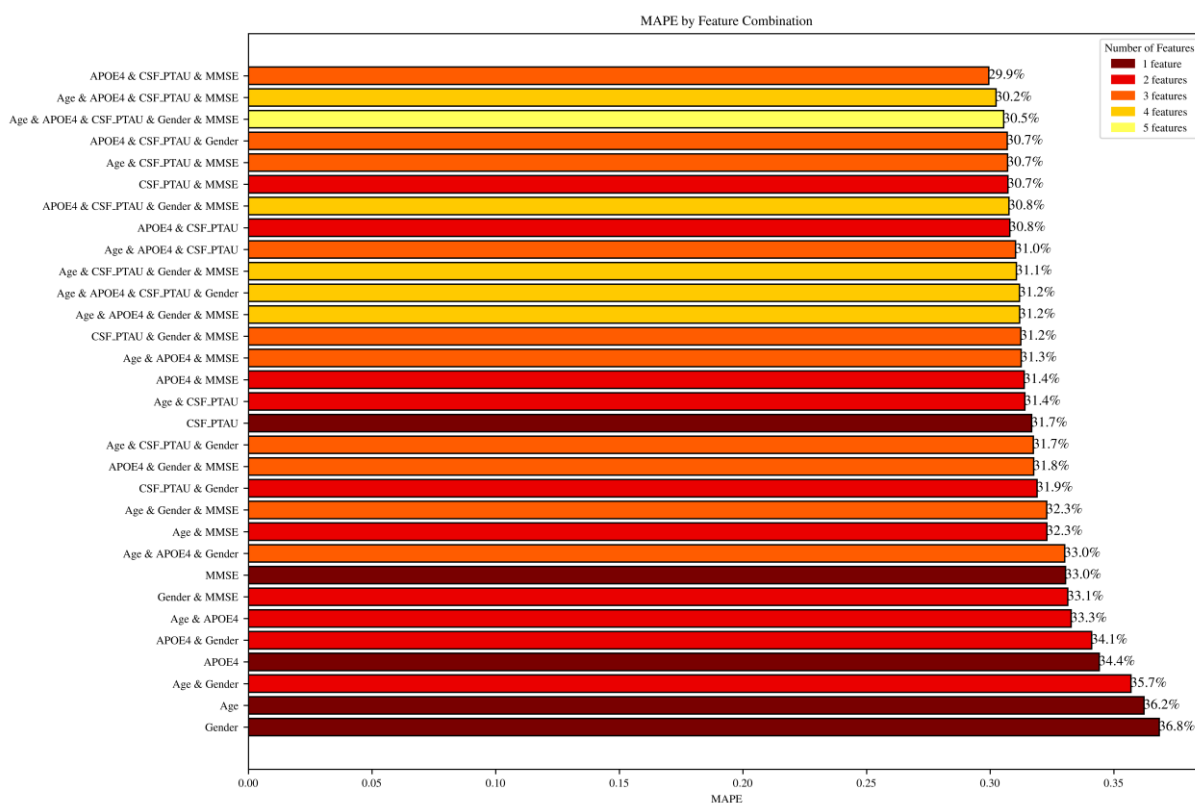


Figure D.8: Evaluation of P-tau models by feature combination using MAE.

D.4.2 Residual Analysis and Predictive Performance

Figure D.9 demonstrates an approximate normal distribution but with many outliers. The skewness of the residuals at 2.161 indicates a pronounced rightward skew, suggesting that the model tends to under-predict more significantly or more frequently. This asymmetric error distribution may highlight issues with model bias or an inadequate fit for higher dataset values. Additionally, a kurtosis value of 7.486, significantly above the normal distribution kurtosis of 0, points to a leptokurtic distribution. This implies that the residuals have heavier tails and a sharper peak, indicating the presence of extreme outliers in the errors.

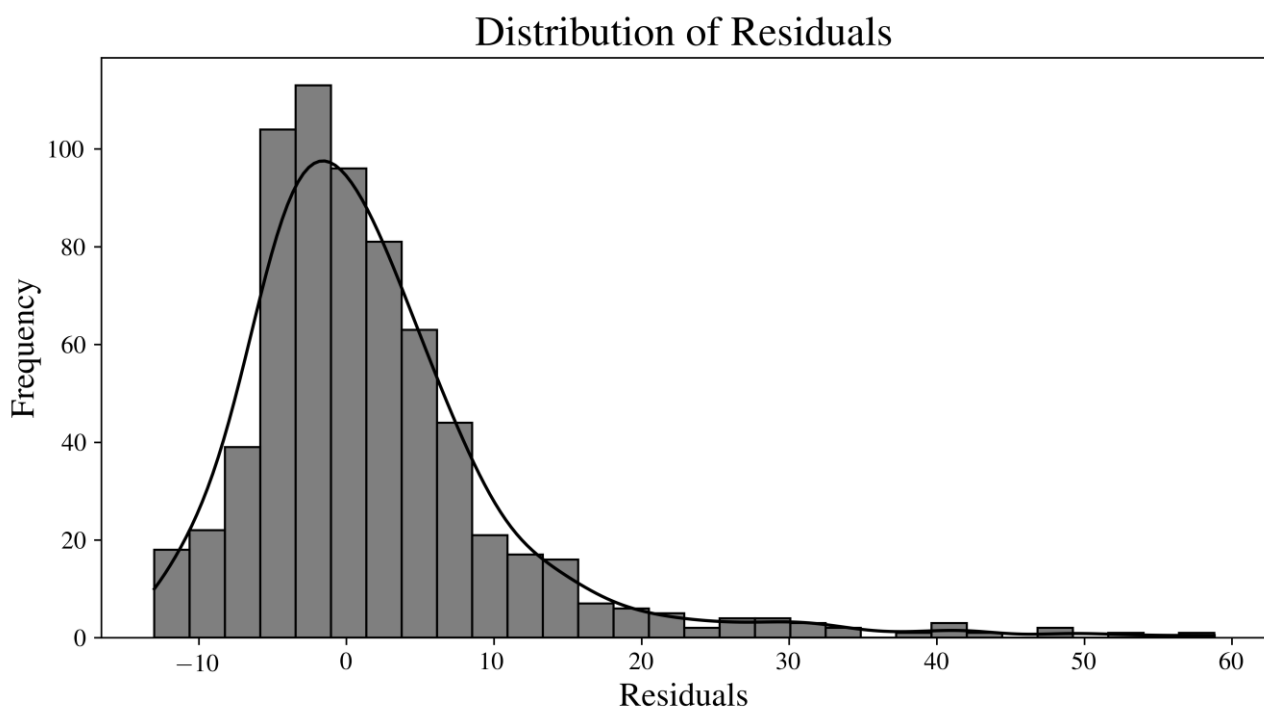


Figure D.9: Distribution of Residuals for P-tau.

These are pronounced in Figure D.10 where it is clear that the model was not able to predict the extreme actual values correctly that correspond to outliers.

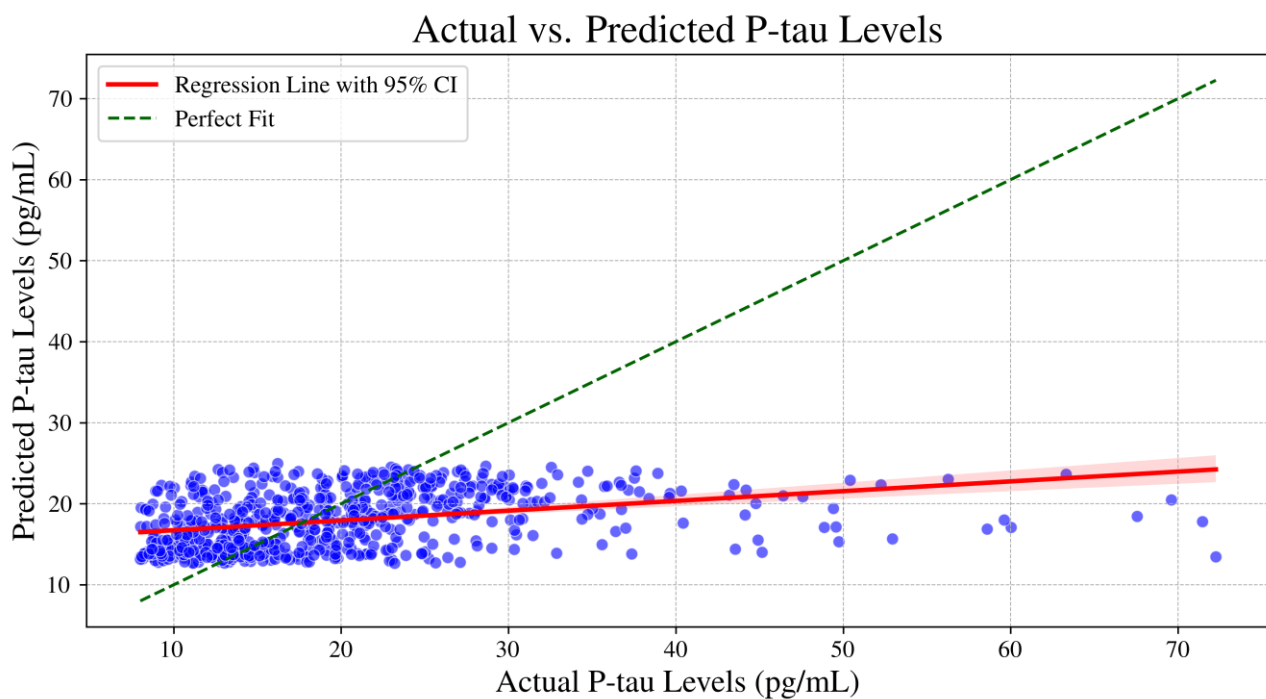


Figure D.10: Scatter Plots of Estimated vs. Actual P-tau Levels.

D.4.3 Benchmark and Standard Deviation Evaluation

Using a naïve regressor as a benchmark as before, its MAE is 7.20, thus the current model (MAE=6.027, $R^2 = 8.6\%$) even though it has biased results it exhibits an improvement compared to the benchmark by 16.3% for MAE and 8.5% for R^2 , this improvement is also statistically significant with $p\text{-value} < 0.001$. In terms of the SD (14.13 pg/mL) the average error of the model is 42.7% lower than the SD substantially surpassing the threshold of 70%.

E. Analysis of NfL Blood Plasma Biomarker

E.1 Normality and Descriptive Statistics

The analysis of skewness, kurtosis, and KS test p-values in Table E.1 reveals distinct distribution traits. Age and ADAS13 exhibit milder skewness and near-normal kurtosis, indicating their distributions are relatively symmetric with no significant deviations from normality, as supported by KS p-values of 0.168 and 0.223 respectively. In contrast, Education and MMSE show moderate left skewness with KS p-values indicating significant non-normality. Notably, CSF NfL and Plasma NfL are highly right-skewed with extreme kurtosis values, both markedly deviating from a normal distribution (KS p-values <0.001).

Table E.1: NfL Normality Tests.

Variable	Skewness	Kurtosis	KS p-value
Age	-0.397	0.080	0.168
Education	-0.549	0.182	<0.001
MMSE	-0.576	-0.464	<0.001
ADAS13	0.491	-0.027	0.223
CSF NfL	3.500	17.789	<0.001
Plasma NfL	3.387	16.842	<0.001

The dataset in Table E.2 highlights a predominance of male participants (60.57%). Additionally, most participants are White (92.75%), suggesting limited racial diversity. The SD of plasma is 27.05 pg/mL.

Table E.2: NfL Descriptive Statistics.

Variable	NfL Dataset (n=383)
Age, years (mean, SD)	74.75 ± 7.05
Education, years (median, IQR)	16.00 (14.00 – 18.00)
MMSE (median, IQR)	27.00 (25.00 - 29.00)
ADAS13 (mean, SD)	18.64 ± 9.08
CSF NfL, pg/mL (median, IQR)	1251.00 (957.50 – 1651.50)
Plasma NfL, pg/mL (median, IQR)	36.1 (25.80 - 47.55)
E4+ (%)	49.09
Gender (M%)	60.57
Race (White%)	95.04

E.2 Exploration of Clinical Data and Biomarker Relationships

As it has been observed from previous datasets, Figure E.1 demonstrates a huge variance within the dataset, but unlike before the dataset has the most extreme outliers until now, that cannot be removed. Thus, for this dataset no outlier augmentation will be performed. However due the datasets size, all the diagnosis categories except CN and AD on plasma can be distinguished in both plasma and CSF.

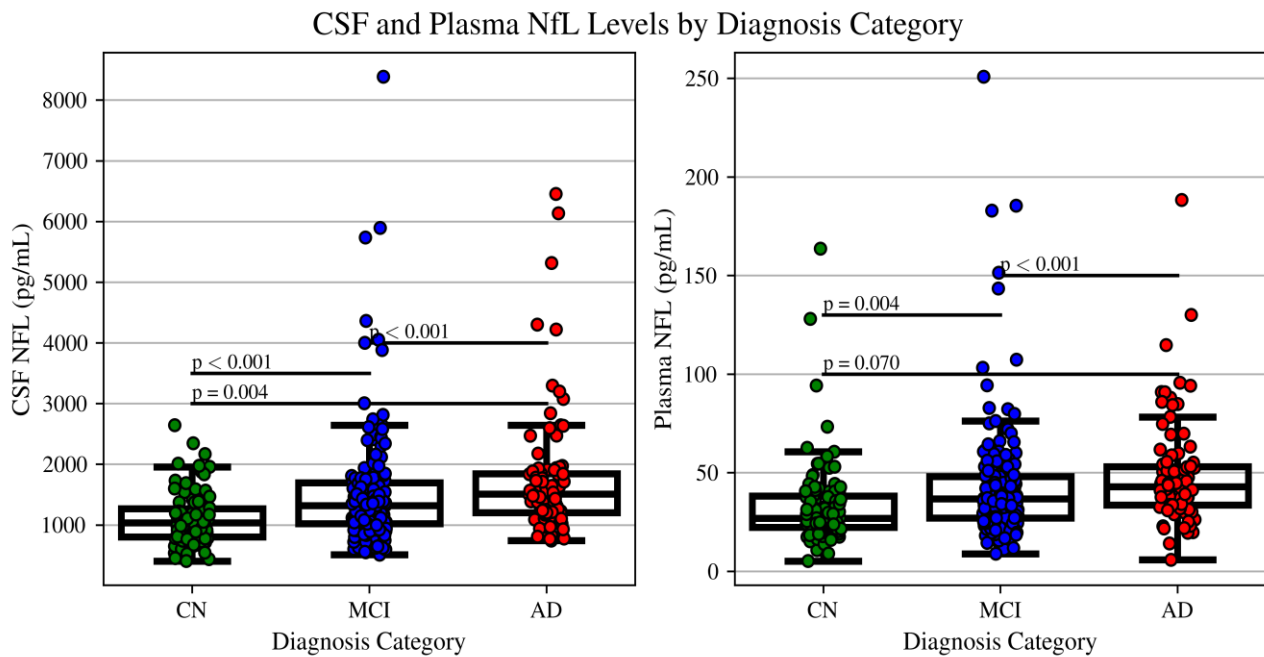


Figure E.1: CSF and Plasma NfL Levels by diagnosis category.

Figure E.2 demonstrates no correlation between plasma NfL and education as it had with all the other biomarkers. In contrast MMSE and ADAS13 show a moderate correlation with plasma NfL.

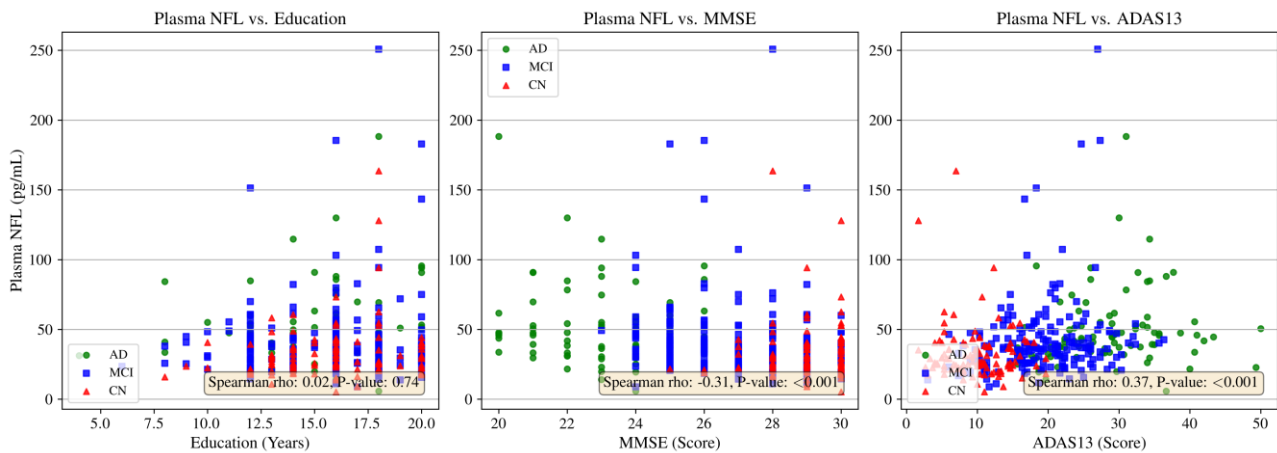


Figure E.2: Plasma NfL versus ordinal categorical variables.

Figure E.3 shows no correlation between APOE4, Gender with plasma NfL. Finally, Figure E.4 shows that Age and CSF NfL have a strong linear positive relationship with the plasma NfL

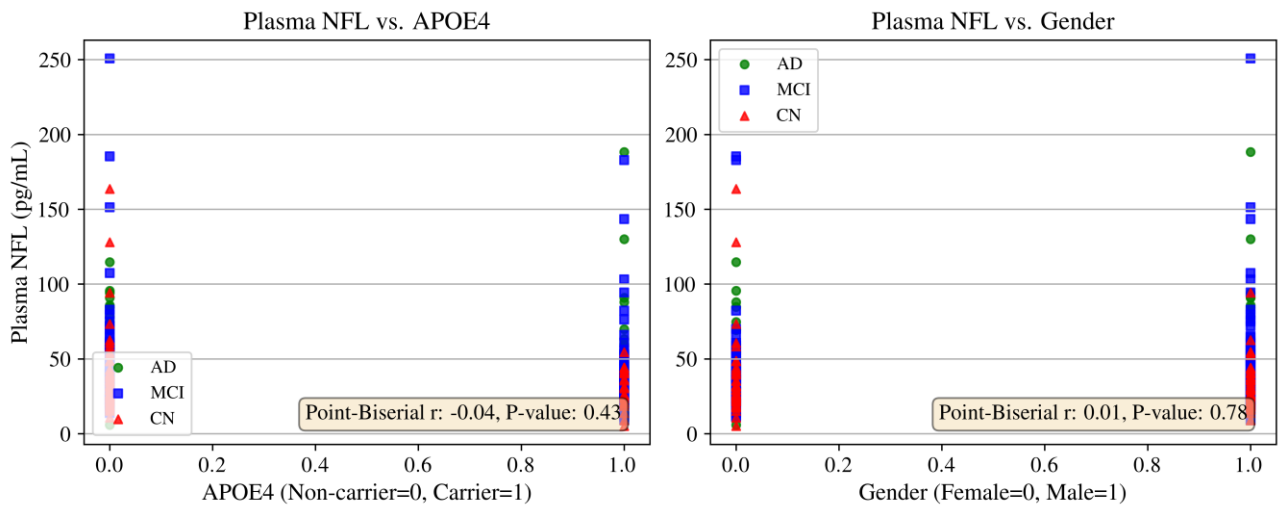


Figure E.3: Plasma NfL versus categorical nominal features.

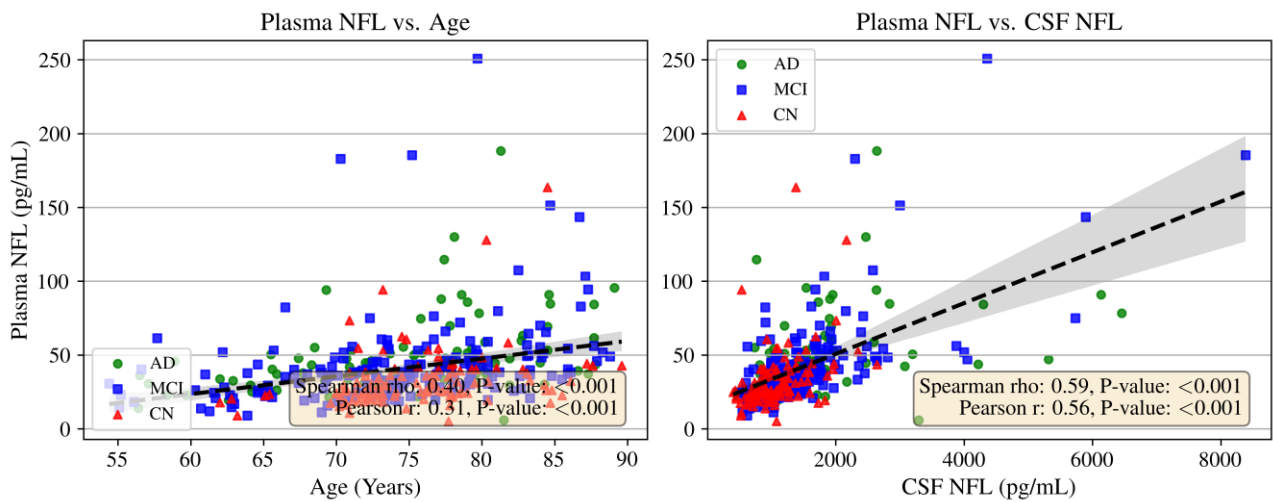


Figure E.4: Plasma NfL versus age and CSF NfL by diagnosis category.

E.3 Correlation Analysis

The correlation analysis depicted in Figure E.5 reveals a strong correlation between plasma NfL and CSF NfL. It also shows a moderate correlation with MMSE, ADAS13, and age, while there is no correlation with education, gender, and notably, APOE4, which is typically strongly correlated with other biomarkers. Given its very low correlation, education will be excluded from further analysis. Finally, as before to minimize multicollinearity issues, MMSE is selected over ADAS13 for inclusion in further analyses. Although MMSE has a lower correlation with plasma NfL, it is less intercorrelated with other variables, reducing potential redundancy.

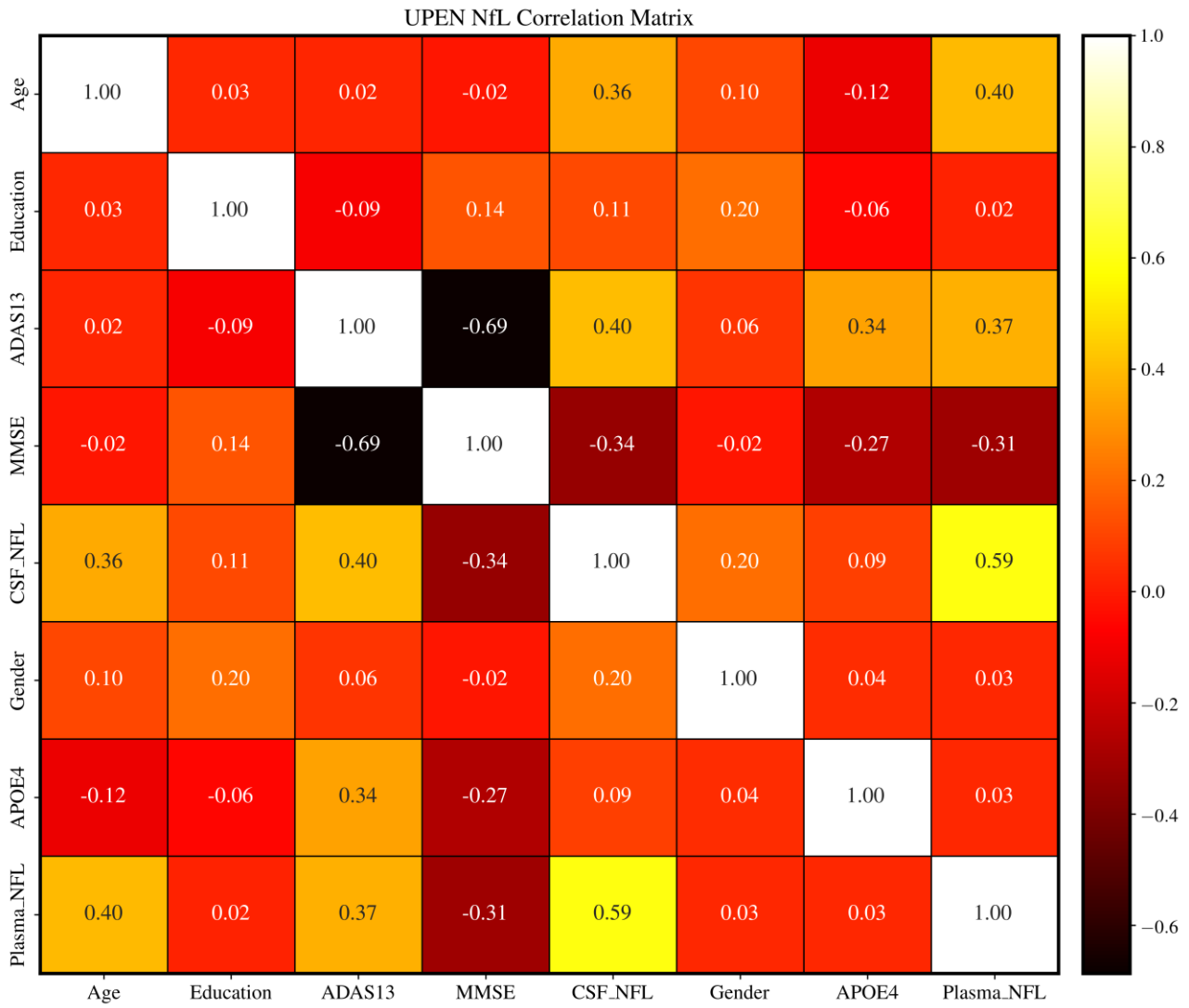


Figure E.5: Spearman correlation matrix for NfL dataset.

E.4 Results, Evaluation and Discussion

E.4.1 Combination Selection and Hyper-Parameters

Utilizing the Figures E.5, E.6, E.7 the best combination for predicting plasma NfL is Age, APOE4, CSF NfL, Gender and MMSE with a MAE of 12.37 pg/mL and R^2 of 26.1%. The R^2 was much higher than expected while having a the largest number of extreme outliers until now. In terms of the hyper parameters the regularization parameter C at 100.0 is high, which indicates a strong emphasis on minimizing errors in the model's fit to the training data. The epsilon value set at 0.01 is relatively low, pointing to a narrow margin within which deviations between the model's predictions and the actual data points are tolerated. The gamma parameter is set at a very low value of 0.0025, indicating that the influence of individual training examples on the overall model

is quite limited. This setting reduces the risk of overfitting by preventing the model from becoming overly complex and sensitive to the specific nuances of the training data.

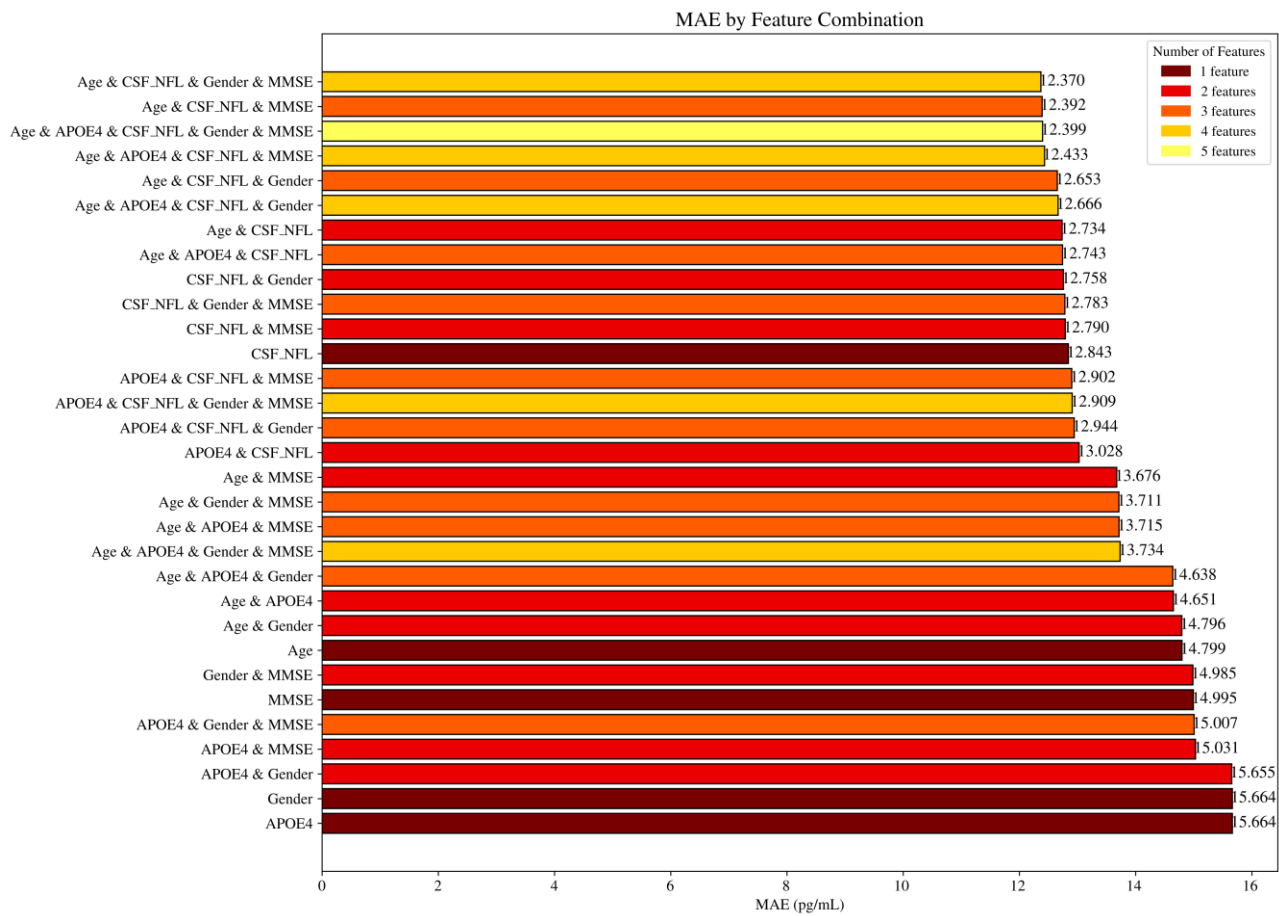


Figure E.6: Evaluation of NfL models by feature combination using MAE.

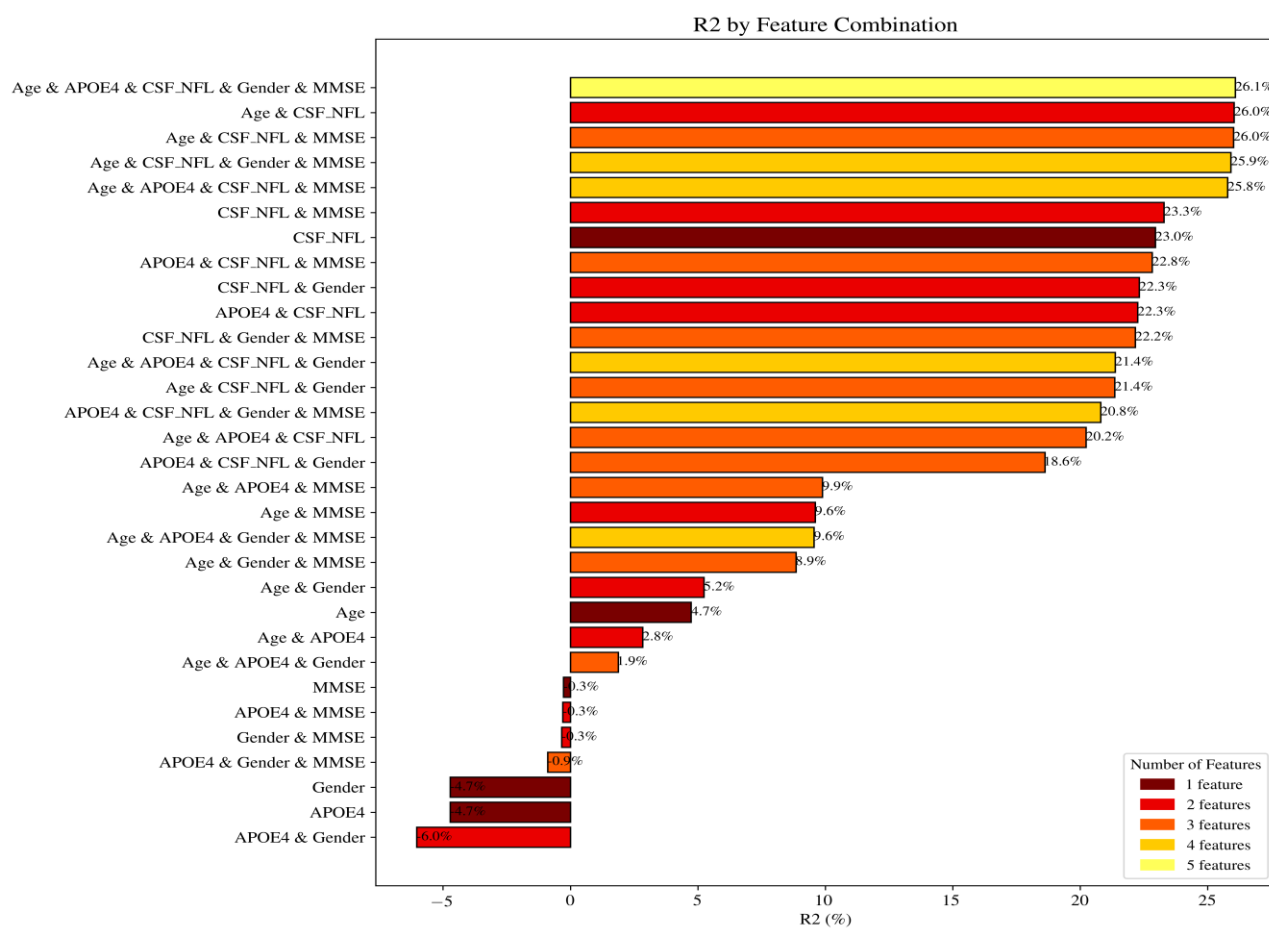


Figure E.7: Evaluation of NfL models by feature combination using R^2 .

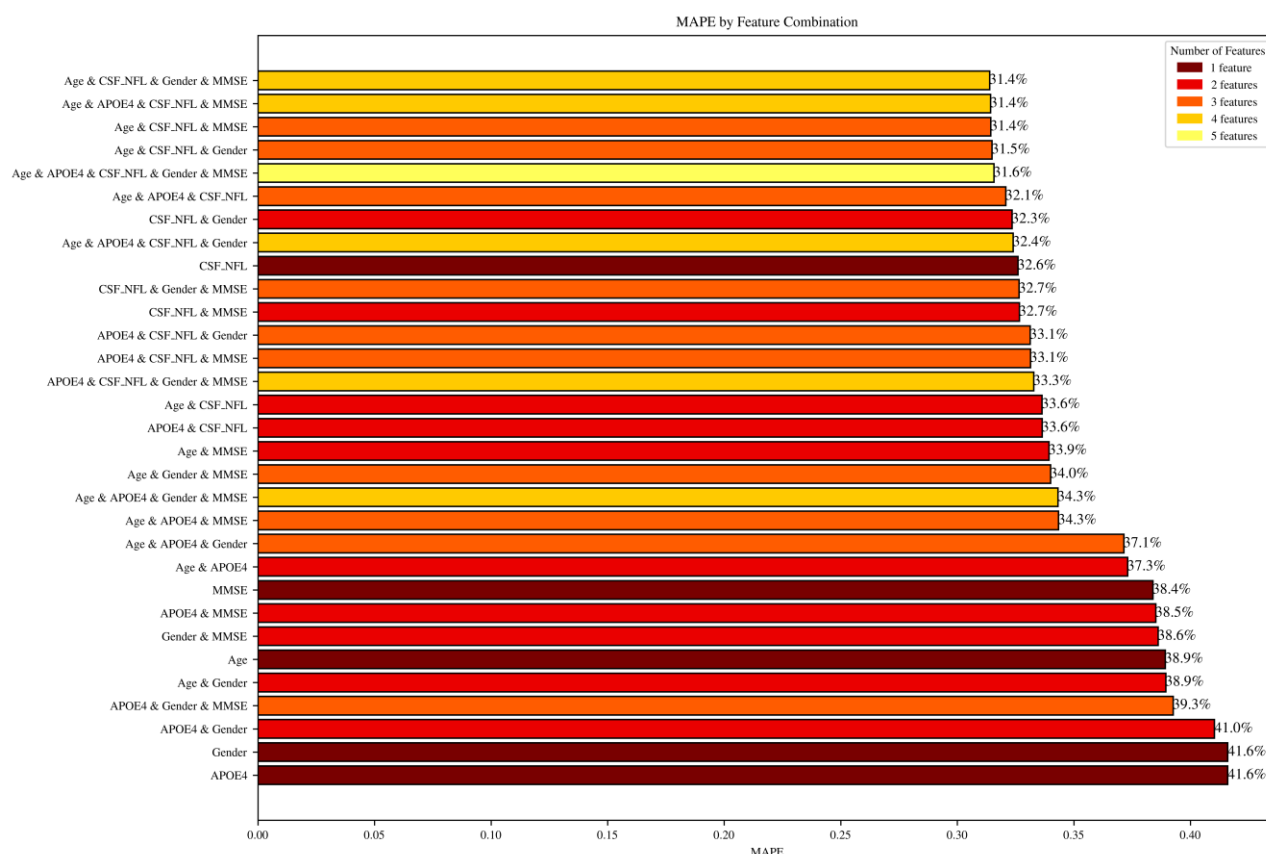


Figure E.8: Evaluation of NfL models by feature combination using MAPE.

E.4.2 Residual Analysis and Predictive Performance

The distribution of residuals in Figure E.9 demonstrates approximately a normal distribution for the most part, but with outliers, which was expected from the many extreme outliers existing the dataset. The skewness of the residuals at 3.506 indicates a significant rightward skew, meaning that the residuals have a long tail on the positive side of the distribution. This suggests that the model under-predicts a substantial number of observations. Additionally, a kurtosis value of 17.769 for the residuals indicates a very peaked distribution with heavy tails. This high kurtosis value also enhances that suggests that there are more outliers than expected in a normal distribution.

The conclusions from the distribution of residuals can also be seen in Figure E.10 with most of the data in the normal range being predicted accurately, with the outliers having huge impact on the best fit.

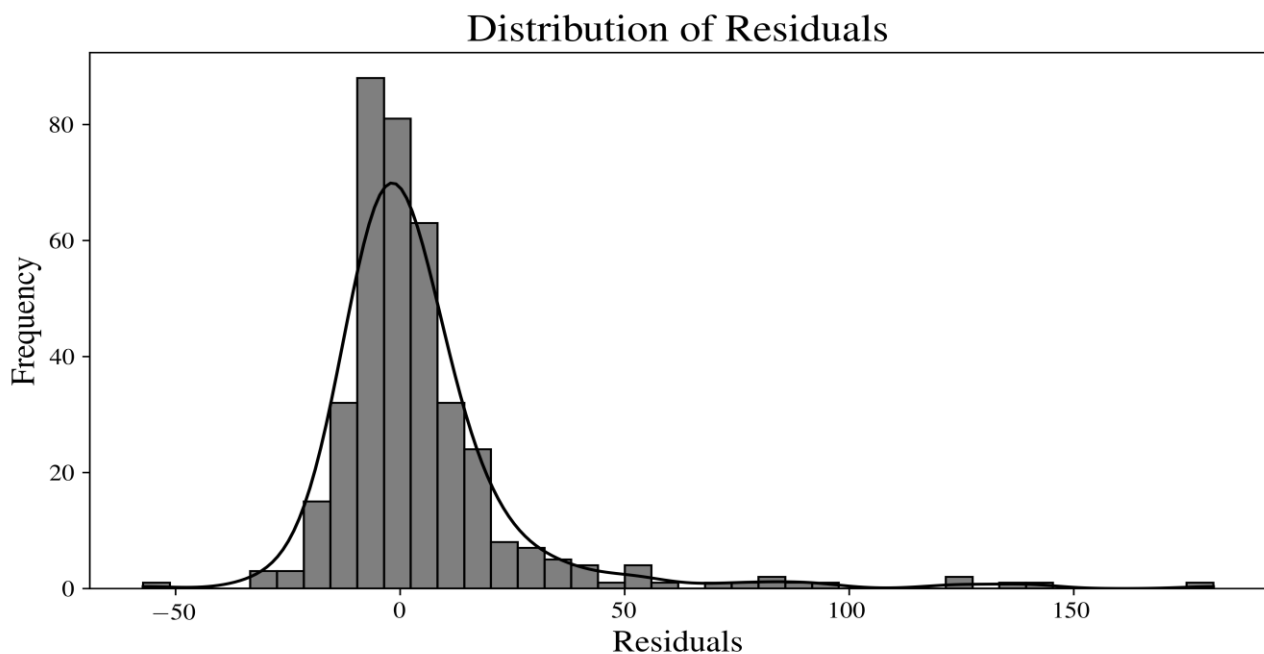


Figure E.9: Distribution of Residuals for NfL.

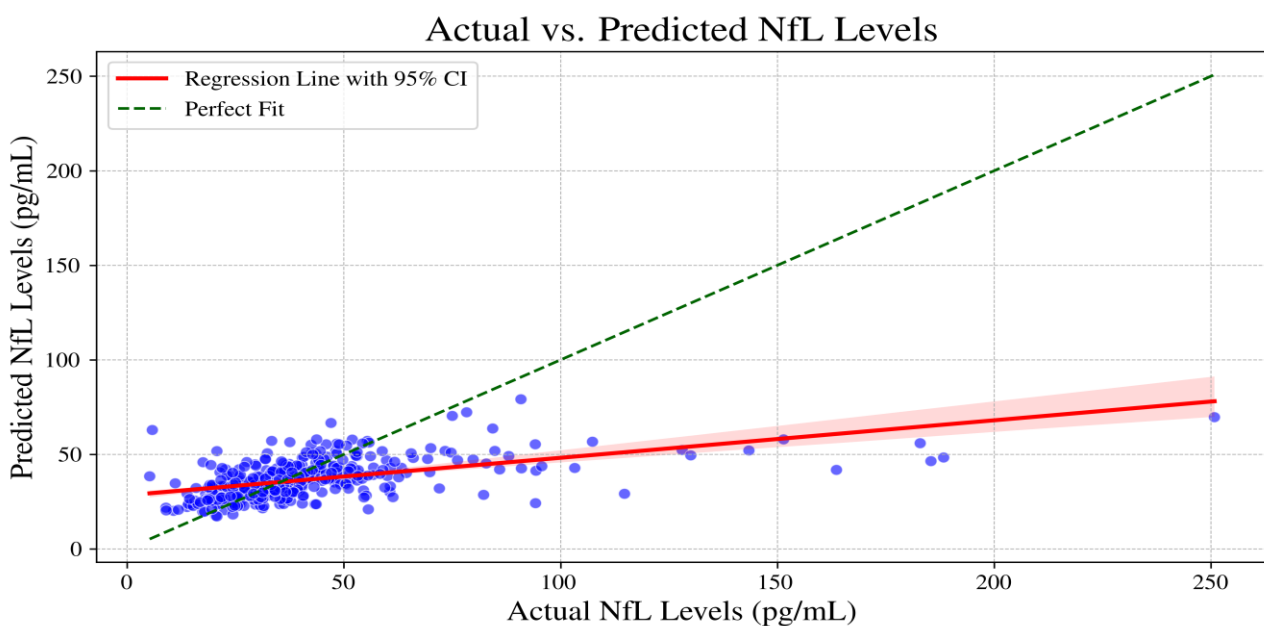


Figure E.10: Scatter Plots of Estimated vs. Actual NfL Levels.

E.4.3 Benchmark Evaluation

Using a naïve regressor as a benchmark as before, its MAE is 16.52, thus the current model ($\text{MAE}=12.37$, $R^2 = 26.1\%$) even though it has biased results it exhibits an improvement compared to the benchmark by 25.1% for MAE and 26 % for R^2 , this improvement is also statistically significant with $p\text{-value}<0.001$. In terms of the SD (27.05 pg/mL) the average error of the model is 45.73% lower than the SD substantially surpassing the threshold of 70%.

F. Dissertation Requirements

F.1 Project Outline

Title: Prediction of Blood Biomarker Levels from Clinical and Cerebrospinal Fluid Data for Early Detection of Alzheimer's.

F.1.1 Background

The risk of Alzheimer's disease (AD), a neurodegenerative disorder, increases with age and poses a significant challenge not only to the affected individuals but also to their families[76]. The progression of AD is typically gradual, beginning with a phase known as Subjective Cognitive Decline (SCD) [77]. Following SCD, individuals may progress to Mild Cognitive Impairment (MCI), Not all individuals with SCD or MCI will develop AD; some may remain stable while others may progress to other forms of dementia [77]. The early detection of SCD and MCI in individuals who will eventually develop AD is crucial for the application of disease-modifying therapies (DMTs) [7]. DMTs have the potential to slow the progression of the disease. In addition to DMTs, other treatments are available for individuals with MCI who are likely to develop AD [7]. While these treatments may not be as effective as DMTs, they still play a significant role in managing the disease. Therefore, early detection and intervention remain key strategies in the fight against AD [7].

F.1.2 Motivation

Early detection of AD is crucial for patient management and treatment planning. However, the current diagnostic methods, such as neuroimaging is expensive, invasive, and not widely available. Biomarkers have emerged as a promising tool for the early detection of AD. Among them, cerebrospinal fluid (CSF) biomarkers have been extensively researched and have shown clinical utility [8]. Currently, cerebrospinal fluid (CSF) biomarker testing is incorporated into some current guidelines for the diagnosis of dementia, such as England's National Institute for Health and Care Excellence (NICE) [8]. However, CSF is not available for all patients and there are also concerns regarding lumbar puncture due to its perceived more invasive nature [8].

On the other hand, blood biomarkers are less invasive, cheaper and potentially more accessible than CSF biomarkers. But blood biomarkers, especially blood plasma, have only recently become

available, and evidence on their clinical validity and performance to screen memory clinic patients is not available yet [9]. Therefore, there is a pressing need to develop a predictive model that can estimate blood biomarker levels from clinical and CSF data. By using clinical data and CSF to predict blood biomarkers, we are essentially translating this rich, existing knowledge to a new, less invasive and cheaper medium - blood. Moreover, by predicting blood plasma levels, clinics could select patients for studies or pharmaceutical trials more effectively. This could potentially allow us to detect Alzheimer's disease using blood tests with the same reliability as CSF tests or PET scans.

F.1.3 Aims and Objectives

Aim:

This project aims to develop a predictive model for blood biomarker levels using clinical and CSF data.

Objectives:

Semester 1:

- 1) **Literature review:** Find how each of the blood biomarkers correlate with CSF biomarkers and MRI.
- 2) **Acquire Data:** Find which datasets include blood and CSF biomarkers for SCD, MCI, AD and select a specific blood biomarker, to predict using CSF.
- 3) **Pre-process Data:** Structure initial data and perform basic preprocessing (Handling Null Values, Standardization, One-Hot Encoding, Multicollinearity)
- 4) **Build a simple ML model:** Try a simple model like multiple linear regression, use CSF biomarkers as input and predict the value of a blood biomarker for one category of patients.
- 5) **Discuss Result:** Check if the result is what is expected from literature. Think what input data and what models can be used for a more accurate result.

Semester 2:

- 1) **Identify new input/output data:** Identify which additional blood biomarkers to predict and what extra data might be needed for an accurate prediction.
- 2) **Acquire/Pre-process new input data:** Similar as above.
- 3) **Develop complex architectures:** Try Support vector machine (SVM), eXtreme gradient boosting of decision trees (XGB), and if enough data, deep neural network (DNN).
- 4) **Compare ML/DL Results:** Compare the results of the algorithms and select the best.
- 5) **Improve the best architecture:** Optimize/simplify best architecture.

F.1.4 Project Plan

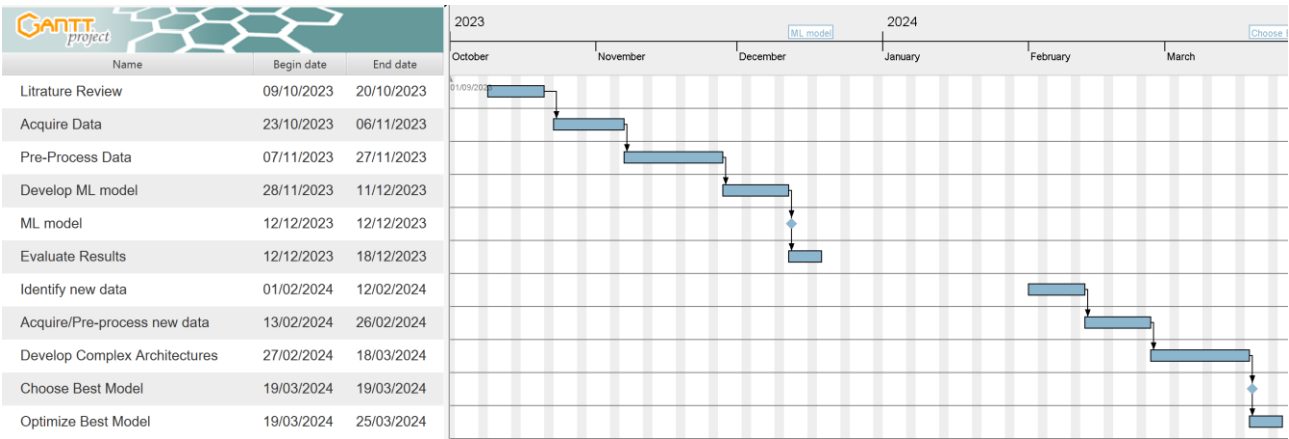


Figure F.1.1: Gantt Chart of the project.

F.2 Risk Assessment

Table F.2.1: Risk Assessment.

Date: 18/10/2023	Assessed by: Fumie Costen	Checked / Validated* by: Fumie Costen	Location: Home	Assessment ref no -	Review date: -
Task / premises: Working from home					
Working from home	Stress / Wellbeing	Home working staff Psychosocial effects,	1. Regular contact meetings with manager and peers, Skype, Zoom, Phone 2. Define working hours, set a start & close daily routine, get dressed and prioritise your tasks.	Low	A
Use of electrical appliances	Misuse of electrical appliance, faulted electrical appliance.	Home working staff Electric shock, burns and fire	1. All office equipment used in accordance with the manufacturer's instructions 2. Visual checks before use to make sure equipment, cables and free from defects	Med	A
Moving around the home office	Obstructions and trip hazards	Home working staff Slips, trips, and falls causing physical injury	1. Floors and walkways kept clear of items, e.g. boxes, packaging, equipment etc 2. Furniture is arranged such that movement of people and equipment are not restricted	Med	A
Working from home	Fire	Staff Home Working Risk of burns, smoke inhalation, asphyxiation	1. In the event of a fire evacuate out of the building and call the fire brigade 2. All waste, including combustible waste, removed regularly. 3. Heaters located away from combustible materials and switched off when office is left unattended	Med	A

F.3 Risk Register

Table F.3.1: Risk Register.

Project Risk	Severity			Potential			Score (Severity x Potential)	Mitigation Measures
	L	M	H	L	M	H	L=1, M=2, H=3	
Poor Internet Connectivity	✓					✓	3	Use a wired internet connection, upgrade internet plan, have a mobile hotspot as a backup.
Distractions at Home	✓					✓	3	Set up a dedicated workspace, maintain a regular schedule, use noise-cancelling headphones.
Data Loss		✓			✓		4	Regular backups, use cloud storage solutions, invest in good anti-virus and security software
Overworking/Burnout		✓				✓	6	Establish clear work-life boundaries, take regular breaks, ensure sufficient time for relaxation.
Delays Due to Illness		✓			✓		4	Maintain a healthy lifestyle, have a contingency plan for project timelines, use sick days as needed.

F.4 Code Environment

For this project Python 3.10.13 was used.

To run the code create an yml file called “environment.yml” and paste the following code that has all the libraries utilized for this project:

```
name: adni_env
channels:
  - conda-forge
  - defaults
dependencies:
  - _tflow_select=2.3.0=mkl
  - abseil-cpp=20211102.0=hd77b12b_0
  - absl-py=1.4.0=py310haa95532_0
  - aiohttp=3.9.3=py310h2bbff1b_0
  - aiosignal=1.2.0=pyhd3eb1b0_0
  - asttokens=2.4.0=pyhd8ed1ab_0
  - astunparse=1.6.3=py_0
  - async-timeout=4.0.3=py310haa95532_0
  - attrs=23.1.0=py310haa95532_0
  - backcall=0.2.0=pyh9f0ad1d_0
  - backports=1.0=pyhd8ed1ab_3
  - backports.functools_lru_cache=1.6.5=pyhd8ed1ab_0
  - blas=1.0=mkl
  - blinker=1.6.2=py310haa95532_0
  - bottleneck=1.3.5=py310h9128911_0
  - brotli=1.0.9=h2bbff1b_7
```

- brotli-bin=1.0.9=h2bbff1b_7
- bzip2=1.0.8=he774522_0
- c-ares=1.19.1=h2bbff1b_0
- ca-certificates=2024.3.11=haa95532_0
- cachetools=4.2.2=pyhd3eb1b0_0
- certifi=2024.2.2=py310haa95532_0
- cffi=1.16.0=py310h2bbff1b_0
- charset-normalizer=2.0.4=pyhd3eb1b0_0
- click=8.1.7=py310haa95532_0
- colorama=0.4.6=pyhd8ed1ab_0
- comm=0.1.4=pyhd8ed1ab_0
- contourpy=1.2.0=py310h59b6b97_0
- cryptography=41.0.3=py310h3438e0d_0
- cycler=0.11.0=pyhd3eb1b0_0
- debugpy=1.6.7=py310hd77b12b_0
- decorator=5.1.1=pyhd8ed1ab_0
- exceptiongroup=1.1.3=pyhd8ed1ab_0
- executing=1.2.0=pyhd8ed1ab_0
- fftw=3.3.9=h2bbff1b_1
- flatbuffers=2.0.0=h6c2663c_0
- fonttools=4.25.0=pyhd3eb1b0_0
- freetype=2.12.1=ha860e81_0
- frozenlist=1.4.0=py310h2bbff1b_0
- gast=0.4.0=pyhd3eb1b0_0
- giflib=5.2.1=h8cc25b3_3
- google-auth=2.6.0=pyhd3eb1b0_0
- google-auth-oauthlib=0.4.4=pyhd3eb1b0_0
- google-pasta=0.2.0=pyhd3eb1b0_0
- grpc-cpp=1.48.2=hf108199_0
- grpcio=1.48.2=py310hf108199_0
- h5py=3.9.0=py310hfc34f40_0
- hdf5=1.12.1=h51c971a_3
- icc_rt=2022.1.0=h6049295_2
- icu=58.2=ha925a31_3
- idna=3.4=py310haa95532_0
- importlib-metadata=6.8.0=pyha770c72_0
- importlib_metadata=6.8.0=hd8ed1ab_0
- intel-openmp=2023.1.0=h59b6b97_46320
- ipykernel=6.26.0=pyha63f2e9_0
- ipython=8.16.1=pyh5737063_0
- jedi=0.19.1=pyhd8ed1ab_0
- joblib=1.2.0=py310haa95532_0
- jpeg=9e=h2bbff1b_1
- jupyter_client=8.5.0=pyhd8ed1ab_0
- jupyter_core=5.4.0=py310h5588dad_0
- keras=2.10.0=py310haa95532_0
- keras-preprocessing=1.1.2=pyhd3eb1b0_0
- kiwisolver=1.4.4=py310hd77b12b_0

- krb5=1.20.1=h5b6d351_1
- lerc=3.0=hd77b12b_0
- libbrotlicommon=1.0.9=h2bbff1b_7
- libbrotlidec=1.0.9=h2bbff1b_7
- libbrotlienc=1.0.9=h2bbff1b_7
- libclang=14.0.6=default_hb5a9fac_1
- libclang13=14.0.6=default_h8e68704_1
- libcurl=8.5.0=h86230a5_0
- libdeflate=1.17=h2bbff1b_1
- libffi=3.4.4=hd77b12b_0
- libpng=1.6.39=h8cc25b3_0
- libpq=12.15=hb652d5d_1
- libprotobuf=3.20.3=h23ce68f_0
- libsodium=1.0.18=h8d14728_1
- libssh2=1.10.0=hcd4344a_2
- libtiff=4.5.1=hd77b12b_0
- libwebp=1.3.2=hbc33d0d_0
- libwebp-base=1.3.2=h2bbff1b_0
- lz4-c=1.9.4=h2bbff1b_0
- markdown=3.4.1=py310haa95532_0
- markupsafe=2.1.3=py310h2bbff1b_0
- matplotlib=3.8.0=py310haa95532_0
- matplotlib-base=3.8.0=py310h4ed8f06_0
- matplotlib-inline=0.1.6=pyhd8ed1ab_0
- mkl=2023.1.0=h6b88ed4_46358
- mkl-service=2.4.0=py310h2bbff1b_1
- mkl_fft=1.3.8=py310h2bbff1b_0
- mkl_random=1.2.4=py310h59b6b97_0
- mscorefonts=0.0.1=3
- multidict=6.0.4=py310h2bbff1b_0
- munkres=1.1.4=py_0
- nest-asyncio=1.5.8=pyhd8ed1ab_0
- numexpr=2.8.7=py310h2cd9be0_0
- numpy=1.26.3=py310h055cbcc_0
- numpy-base=1.26.3=py310h65a83cf_0
- oauthlib=3.2.2=py310haa95532_0
- openjpeg=2.4.0=h4fc8c34_0
- openssl=1.1.1w=h2bbff1b_0
- opt_einsum=3.3.0=pyhd3eb1b0_1
- packaging=23.2=pyhd8ed1ab_0
- pandas=2.1.4=py310h4ed8f06_0
- parso=0.8.3=pyhd8ed1ab_0
- patsy=0.5.3=py310haa95532_0
- pickleshare=0.7.5=py_1003
- pillow=10.0.1=py310h045eedc_0
- pip=23.3=py310haa95532_0
- platformdirs=3.11.0=pyhd8ed1ab_0
- ply=3.11=py310haa95532_0

- prompt-toolkit=3.0.39=pyha770c72_0
- prompt_toolkit=3.0.39=hd8ed1ab_0
- protobuf=3.20.3=py310hd77b12b_0
- psutil=5.9.0=py310h2bbff1b_0
- pure_eval=0.2.2=pyhd8ed1ab_0
- pyaml=20.4.0=pyhd3eb1b0_0
- pyasn1=0.4.8=pyhd3eb1b0_0
- pyasn1-modules=0.2.8=py_0
- pycparser=2.21=pyhd3eb1b0_0
- pygments=2.16.1=pyhd8ed1ab_0
- pyjwt=2.4.0=py310haa95532_0
- pyopenssl=23.2.0=py310haa95532_0
- pyparsing=3.0.9=py310haa95532_0
- pyqt=5.15.10=py310hd77b12b_0
- pyqt5-sip=12.13.0=py310h2bbff1b_0
- python=3.10.13=h966fe2a_0
- python-dateutil=2.8.2=pyhd3eb1b0_0
- python-flatbuffers=2.0=pyhd3eb1b0_0
- python-tzdata=2023.3=pyhd3eb1b0_0
- python_abi=3.10=2_cp310
- pytz=2023.3.post1=py310haa95532_0
- pywin32=305=py310h2bbff1b_0
- pyyaml=6.0.1=py310h2bbff1b_0
- pyzmq=23.2.1=py310h73ada01_0
- qt-main=5.15.2=h6072711_9
- re2=2022.04.01=hd77b12b_0
- requests=2.31.0=py310haa95532_1
- requests-oauthlib=1.3.0=py_0
- rsa=4.7.2=pyhd3eb1b0_1
- scienceplots=2.1.1=pyhd8ed1ab_0
- scikit-learn=1.3.0=py310h4ed8f06_1
- scikit-optimize=0.10.0=pyhd8ed1ab_0
- scikit-posthocs=0.9.0=pyhd8ed1ab_0
- scipy=1.11.4=py310h309d312_0
- seaborn=0.13.0=hd8ed1ab_0
- seaborn-base=0.13.0=pyhd8ed1ab_0
- setuptools=68.0.0=py310haa95532_0
- sip=6.7.12=py310hd77b12b_0
- six=1.16.0=pyhd3eb1b0_1
- snappy=1.1.10=h6c2663c_1
- sqlite=3.41.2=h2bbff1b_0
- stack_data=0.6.2=pyhd8ed1ab_0
- statsmodels=0.14.0=py310h9128911_0
- tbb=2021.8.0=h59b6b97_0
- tensorboard=2.10.0=py310haa95532_0
- tensorboard-data-server=0.6.1=py310haa95532_0
- tensorboard-plugin-wit=1.8.1=py310haa95532_0
- tensorflow=2.10.0=mkl_py310hd99672f_0

```

- tensorflow-base=2.10.0=mk1_py310h6a7f48e_0
- tensorflow-estimator=2.10.0=py310haa95532_0
- termcolor=2.1.0=py310haa95532_0
- threadpoolctl=2.2.0=pyh0d69192_0
- tk=8.6.12=h2bbff1b_0
- tomli=2.0.1=py310haa95532_0
- tornado=6.2=py310he2412df_0
- traitlets=5.12.0=pyhd8ed1ab_0
- typing-extensions=4.8.0=hd8ed1ab_0
- typing_extensions=4.8.0=pyha770c72_0
- tzdata=2023c=h04d1e81_0
- ucrt=10.0.22621.0=h57928b3_0
- urllib3=2.1.0=py310haa95532_0
- vc=14.2=h21ff451_1
- vc14_runtime=14.38.33130=h82b7239_18
- vs2015_runtime=14.38.33130=hcb4865c_18
- wcwidth=0.2.8=pyhd8ed1ab_0
- werkzeug=2.3.8=py310haa95532_0
- wheel=0.41.2=py310haa95532_0
- wrapt=1.14.1=py310h2bbff1b_0
- xz=5.4.2=h8cc25b3_0
- yaml=0.2.5=he774522_0
- yar1=1.9.3=py310h2bbff1b_0
- zeromq=4.3.4=h0e60522_1
- zipp=3.17.0=pyhd8ed1ab_0
- zlib=1.2.13=h8cc25b3_0
- zstd=1.5.5=hd43e919_0

```

Then to create the specified environment run the command: `conda env create -f environment.yml`

After this step an environment named `adni_env` will be created. Activate it using “`conda activate adni_env`”. Then paste any part of the code in a python file and run it.

The datasets used along with all the code can be requested from the supervisor of this project. Aß

F.5 Code for Primary Biomarker Aß42

F.5.1 Initial Preprocessing of All ABETA Datasets

```

# Remove the repeated RID values once
def remove_repeated_rid(dfs, specific_key=None):
    for key, df in dfs.items():
        if specific_key is not None and key != specific_key:
            continue

        # Identify repeated RIDs
        repeated_rids = df['RID'][df['RID'].duplicated(keep=False)].unique()

```

```

for rid in repeated_rids:
    # Rows for the current RID
    rid_rows = df[(df['RID'] == rid)]

    # Numeric columns
    numeric_cols = rid_rows.select_dtypes(include=[np.number]).columns

    # Calculate mean for these columns
    mean_values = rid_rows[numeric_cols].mean()

    # Replace the numeric values in the first occurrence and remove the
rest
    first_occurrence_index = rid_rows.index[0]
    df.loc[first_occurrence_index, numeric_cols] = mean_values
    df.drop(rid_rows.index[1:], inplace=True)

    # Reset index
    df.reset_index(drop=True, inplace=True)

# Show the repeated RID values
def show_repeated_rid(df, specific_key=None):
    flag = True
    for key, df in df.items():
        if specific_key is not None:
            flag = False

        if flag or key == specific_key:
            if key != 'adni_merge' and key != 'upen_abeta_plasma' and key !=
'blennow_nfl_csf' and key != 'araclone_abeta_plasma':
                bl_rid = df['RID'].values
            else:
                bl_rid = df['RID'].values
            print(key, len(bl_rid), len(bl_rid)-len(np.unique(bl_rid)))
            print('Repeated RID values: ', [item for item, count in
Counter(bl_rid).items() if count > 1])
            print()

# Find the number of bl in each dataset
def show_bl_count(df, specific_key=None):
    flag = True
    for key, df in df.items():
        if specific_key is not None:
            flag = False

        if flag or key == specific_key:
            if key != 'adni_merge' and key != 'upen_abeta_plasma' and key !=
'blennow_nfl_csf' and key != 'araclone_abeta_plasma':

```

```

        print(key, Counter(df['VISCODE2'])['b1'])
    else:
        print(key, Counter(df['VISCODE'])['b1'])
    print()

def show_disc_plots(dataframes, dataframes_list, types, rows, columns, size,
column_names):
    fig, axs = plt.subplots(rows, columns, figsize=size)
    for i, type_ in enumerate(types):
        descriptions = []
        for j, (df_name, title) in enumerate(dataframes_list):
            if isinstance(axs, np.ndarray):
                if axs.ndim == 1:
                    ax = axs[j]
                else:
                    ax = axs[i, j]
            else:
                ax = axs
            ax.set_title(title)
            sns.histplot(dataframes[df_name][type_], kde=True, ax=ax)
            descriptions.append(dataframes[df_name][type_].describe())
        descriptions_df = pd.concat(descriptions, axis=1)
        descriptions_df.columns = column_names
        print(descriptions_df)

def show_boxplots(dataframes, sources, columns, titles):
    n = len(columns)
    ncols = n
    nrows = 1
    fig, axs = plt.subplots(nrows, ncols, figsize=(5*ncols, 5))
    if n == 1:
        axs = [axs]
    else:
        axs = axs.flatten() # Flatten to 1D array for easy indexing

    # Loop over each column
    for i, (column, title) in enumerate(zip(columns, titles)):
        # Initialize an empty list to store the dataframes
        dfs = []

        # Loop over each source
        for source in sources:
            # Create the dataframe and append it to the list
            df = dataframes[source][column].dropna().reset_index()
            df['Source'] = source
            dfs.append(df)

        # Concatenate the dataframes

```

```

df_conc = pd.concat(dfs)

# Plot the boxplot
sns.boxplot(x='Source', y=column, data=df_conc, ax=axes[i])
axes[i].set_title(title)

plt.tight_layout()
plt.show()

def merge_dataframes(df_left, df_right):
    # Create a new DataFrame from the left DataFrame, including the 'RID' column
    merged_df = df_left.copy()

    # Convert the right DataFrame into a dictionary for efficient access
    right_dict = df_right.set_index('RID').to_dict('index')

    # Update 'ABETA40' and 'ABETA42' in merged_df based on the conditions
    for index, row in merged_df.iterrows():
        rid = row['RID']
        if rid in right_dict:
            right_row = right_dict[rid]
            # Check if both 'ABETA40' and 'ABETA42' are non-NaN in the right
            DataFrame
            if pd.notna(right_row.get('ABETA40')) and
pd.notna(right_row.get('ABETA42')):
                # Replace with right values if at least one of the left values is
                NaN
                if pd.isna(row['ABETA40']) or pd.isna(row['ABETA42']):
                    merged_df.at[index, 'ABETA40'] = right_row['ABETA40']
                    merged_df.at[index, 'ABETA42'] = right_row['ABETA42']

    # Append rows from the right DataFrame that are not in the left DataFrame
    right_rids_not_in_left = set(right_dict.keys()) - set(merged_df['RID'])
    rows_to_append = pd.DataFrame([right_dict[rid] for rid in
right_rids_not_in_left])
    if not rows_to_append.empty:
        rows_to_append['RID'] = rows_to_append.index # Add 'RID' column
        merged_df = pd.concat([merged_df, rows_to_append], ignore_index=True)

    return merged_df

def simple_merge(df_left, df_right):
    # Merge the dataframes on 'RID', prioritizing left dataframe values
    merged_df = pd.merge(df_left, df_right, on='RID', how='outer',
suffixes=('_left', '_right'))

    # Iterate through each column to apply the merging rules

```

```

for column in merged_df.columns:
    if column.endswith('_left'):
        original_col = column[:-5] # Extract the original column name
        right_col = original_col + '_right'

        # If the right column exists, apply the merging logic
        if right_col in merged_df.columns:
            # Use right dataframe value if left is NaN and right is not NaN
            merged_df[original_col] = merged_df.apply(
                lambda row: row[right_col] if pd.isna(row[column]) and
pd.notna(row[right_col]) else row[column],
                axis=1
            )
            # Drop the temporary columns
            merged_df.drop(columns=[column, right_col], inplace=True)
        else:
            # Rename the column back to original if right column doesn't exist
            merged_df.rename(columns={column: original_col}, inplace=True)

return merged_df

# Print function for median and percentiles
def print_percentiles(stats, var, group_name):
    median, percentile25, percentile75 = stats
    print(f"{var} in {group_name}: Median = {median:.2f}, 25th Percentile = {percentile25:.2f}, 75th Percentile = {percentile75:.2f}")

# Function to calculate mean and std
def mean_std(group, var):
    return group[var].mean(), group[var].std()

def print_percentiles(percentiles, var, group_name=None):
    if group_name:
        print(f"{group_name} {var}: Median = {percentiles[0]:.2f}, 25th percentile = {percentiles[1]:.2f}, 75th percentile = {percentiles[2]:.2f}")
    else:
        print(f"{var}: Median = {percentiles[0]:.2f}, 25th percentile = {percentiles[1]:.2f}, 75th percentile = {percentiles[2]:.2f}")

def print_stats(stats, var, group_name=None):
    if group_name:
        print(f"{group_name} {var}: Mean = {stats[0]:.2f}, Std = {stats[1]:.2f}")
    else:
        print(f"{var}: Mean = {stats[0]:.2f}, Std = {stats[1]:.2f}")

def descriptive_stats_AD_NAD(df, csf_name, plasma_name):
    # Splitting the dataset
    ad_group = df[df['DX_b1'] == 'AD']

```

```

non_ad_group = df[df['DX_b1'] != 'AD']

# Analysis for AGE, csf_name, and plasma_name
variables = ['Age', csf_name, plasma_name]
for var in variables:
    print(f"\n{var} Analysis:")
    ad_stats = mean_std(ad_group, var)
    non_ad_stats = mean_std(non_ad_group, var)
    print_stats(ad_stats, var, "AD")
    print_stats(non_ad_stats, var, "Non-AD")

    # T-tests
    t_stat, p_value = stats.ttest_ind(ad_group[var], non_ad_group[var])
    print(f"Student's t-test: T-stat = {t_stat:.2f}, p-value = {p_value:.4f}")

# Additional analysis for MMSE_b1, Education with median, percentiles, and Mann-
Whitney U test
for var in ['MMSE', 'ADAS13', 'Education', csf_name]:
    print(f"\n{var} Detailed Analysis:")
    ad_percentiles = median_percentiles(ad_group, var)
    non_ad_percentiles = median_percentiles(non_ad_group, var)
    print_percentiles(ad_percentiles, var, "AD")
    print_percentiles(non_ad_percentiles, var, "Non-AD")

    # Mann-Whitney U tests
    u_stat, p_value = stats.mannwhitneyu(ad_group[var], non_ad_group[var])
    print(f"Mann-Whitney U test for {var}: U-statistic = {u_stat:.2f}, p-value
= {p_value:.4f}")

# Chi-square Tests
chi2_vars = ['APOE4', 'Gender', 'Race']
print("\nChi-square Test p-values:")
for var in chi2_vars:
    chi2, p, _, _ = stats.chi2_contingency(pd.crosstab(df[var], df['DX_b1']))
    print(f"{var}: p-value = {p:.4f}")

# Percentage calculations for categorical variables
print("\nPercentage calculations for categorical variables:")
percentages = pd.DataFrame({
    'Group': ['AD', 'Non-AD'],
    'Male (%)': [
        ad_group['Gender'].value_counts(normalize=True).get('Male', 0) * 100,
        non_ad_group['Gender'].value_counts(normalize=True).get('Male', 0) *
100
    ],
    'White (%)': [
        ad_group['Race'].value_counts(normalize=True).get('White', 0) * 100,
        non_ad_group['Race'].value_counts(normalize=True).get('White', 0) * 100
    ]
})

```



```

    ],
    'E4+ (%)': [
        (ad_group['APOE4'] == 1).mean() * 100,
        (non_ad_group['APOE4'] == 1).mean() * 100
    ]
})
print(percentages.to_string(index=False))

# Participant counts
print(f"\nParticipant count: AD = {len(ad_group)}, Non-AD = {len(non_ad_group)}")

def descriptive_stats(df, csf_name, plasma_name):
    # Analysis for AGE, csf_name, and plasma_name
    variables = ['Age', csf_name, plasma_name]
    for var in variables:
        print(f"\n{var} Analysis:")
        stats = mean_std(df, var)
        print_stats(stats, var)

    # Additional analysis for MMSE_b1, Education with median, percentiles
    for var in ['MMSE', 'ADAS13', 'Education', csf_name]:
        print(f"\n{var} Detailed Analysis:")
        percentiles = median_percentiles(df, var)
        print_percentiles(percentiles, var)

    # Percentage calculations for categorical variables
    print("\nPercentage calculations for categorical variables:")
    percentages = pd.DataFrame({
        'Male (%)': [
            df['Gender'].value_counts(normalize=True).get('Male', 0) * 100
        ],
        'White (%)': [
            df['Race'].value_counts(normalize=True).get('White', 0) * 100
        ],
        'E4+ (%)': [
            (df['APOE4'] == 1).mean() * 100
        ]
    })
    print(percentages.to_string(index=False))

    # Participant counts
    print(f"\nParticipant count: {len(df)}")
# For analysis VISCODE2 will be used with b1 code

# ADNIMERGE uses VISCODE2 (but calls it VISCODE), while other files have both
# VISCODE and VISCODE2.
# ADNIMERGE also combines VISCODE2=="sc" and VISCODE2=="b1" into VISCODE=="b1",

```

```

# VISCODE2 was created to be a more consistent and interpretable visit code.
adni_merge = pd.read_csv('../Data/General/ADNIMERGE.csv')

# CSF all use VISCODE2
upen_csf = pd.read_csv('../Data/CSF/UPENNBIOMK_MASTER_FINAL.csv') # AB42 ADNI1,2,GO
and AB40 for ADNI3, ABETA, TAU, PTAU181 for ADNI1,2,GO
fuji_abeta_csf = pd.read_csv('../Data/CSF/FUJIREBIOABETA.csv') # AB40, AB42,
AB42/40 ADNI2,GO

# Plasma
bateman_abeta_plasma = pd.read_csv('../Data/Plasma/batemanlab_20221118.csv') #
AB40, AB42, AB40/42 ADNI1,2,GO

# All dataframes
dataframes = {'upen_csf': upen_csf, 'fuji_abeta_csf': fuji_abeta_csf,
              'bateman_abeta_plasma': bateman_abeta_plasma}

# Keep only the VISIT_CODE == 'b1' rows
for key, df in dataframes.items():
    if 'VISCODE2' in df.columns:
        dataframes[key] = df[df['VISCODE2'] == 'b1']
        print(key, Counter(df['VISCODE2'])['b1'])
    elif 'VISCODE' in df.columns:
        dataframes[key] = df[df['VISCODE'] == 'b1']
        print(key, Counter(df['VISCODE'])['b1'])
    else:
        print(f"'VISCODE' column not found in {key}")

# UPENN
# Keep only the newer batches for upenn_csf
batches = ['UPENNBIOMK9', 'UPENNBIOMK10', 'UPENNBIOMK11', 'UPENNBIOMK12',
            'UPENNBIOMK13']
dataframes["upen_csf"] = (dataframes["upen_csf"].query("BATCH in @batches"))
# Keep only the values that are in the highest batch
# Sort by RID and BATCH in descending order
sorted_df = dataframes["upen_csf"].sort_values(by=['RID', 'BATCH'],
ascending=[True, True])
# Drop duplicates keeping the first entry for each RID
dataframes["upen_csf"] = sorted_df.drop_duplicates(subset='RID', keep='first')

# Replace ABETA42 above 1700 with NaN
dataframes["upen_csf"]['ABETA42'] =
pd.to_numeric(dataframes["upen_csf"]['ABETA42'], errors='coerce')
dataframes["upen_csf"]['ABETA42'][dataframes["upen_csf"]['ABETA42'] > 1700] =
np.nan

# if PTAU == "<8" or PTAU == ">120" replace with NaN
dataframes["upen_csf"]['PTAU'][dataframes["upen_csf"]['PTAU']=="<8"] = np.nan

```

```

dataframes["upen_csf"]["PTAU"][dataframes["upen_csf"]["PTAU"]==">120"] = np.nan

# FUIJI
# Remove rows if there is a comment (all the comments refer to not enough volume)
dataframes["fuji_abeta_csf"] =
dataframes["fuji_abeta_csf"][dataframes["fuji_abeta_csf"]["COMMENTS"].isnull()]

# Bateman ABETA Plasma
# Remove rows where QC_Status is FAIL, QC_Comment is 15N 42 Peak shape, Hemolyzed,
Low Ab38/40/42 Normal MD
dataframes["bateman_abeta_plasma"] =
dataframes["bateman_abeta_plasma"][dataframes["bateman_abeta_plasma"]["QC_Status"]
!= 'FAIL']
dataframes["bateman_abeta_plasma"] =
dataframes["bateman_abeta_plasma"][dataframes["bateman_abeta_plasma"]["QC_Comment"]
!= '15N 42 Peak shape']
dataframes["bateman_abeta_plasma"] =
dataframes["bateman_abeta_plasma"][dataframes["bateman_abeta_plasma"]["QC_Comment"]
!= 'Hemolyzed']
dataframes["bateman_abeta_plasma"] =
dataframes["bateman_abeta_plasma"][dataframes["bateman_abeta_plasma"]["QC_Comment"]
!= 'Low Ab38/40/42 Normal MD']

```

```

columns_to_drop = [
    'STORDTE', 'DRAWLTE', 'VID', 'GUSPECID', 'DER', 'VOL', 'RECDTE', 'SITE',
    'RUN', 'COMMENTS', 'PHASE', 'PTID', 'BATCH', 'FLUID', 'ABETA_RAW', 'TAU_RAW',
    'PTAU_RAW', 'EXAMDATE', 'NOTES', 'STUDY', 'ABETA', 'TAU', 'NOTE',
    'PROT', 'Sample_ID', 'File_Name_NE', 'Sample_Type', 'Assay_fluid',
    'LC_MS_Sample_ID',
    'Abeta_42_conc_UNITS', 'Abeta_42_N14N15', 'Abeta_42_N15_ISTD_amount',
    'Abeta_42_N15_ISTD_amount_UNITS', 'Abeta_40_conc_UNITS', 'Abeta_40_N14N15',
    'Abeta_40_N15_ISTD_amount', 'Abeta_40_N15_ISTD_amount_UNITS',
    'Intertcept_Standardization', 'Slope_Standardization', 'Sample_volume',
    'Sample_volume_UNITS',
    'MS_Run_Date', 'Injection', 'Instrument', 'Batch_N', 'Assay',
    'Assay_IP_protocol', 'QC_Status', 'QC_Flags', 'QC_Comment', 'Abeta_4240',
    'STUDY', 'NOTE',
    'RUNDATE', 'USERDATE', 'VOLUME', 'COMMENT', 'BOX_POS', 'FP40', 'FP42',
    'VISCODE2', 'VISCODE', 'update_stamp'
]

for key, df in dataframes.items():
    # Drop common columns
    df = df.drop(columns=[col for col in columns_to_drop if col in df.columns],
errors='ignore')

```

```

    # Update the dictionary with the modified dataframe
    dataframes[key] = df
print("##### Repeated RID values #####")
show_repeated_rid(dataframes, specific_key=None)
# Merge repeated RID values for bateman and araclone
remove_repeated_rid(dataframes, specific_key=None)
#
print("##### Repeated RID values after merging #####")
show_repeated_rid(dataframes, specific_key=None)

# Make all rows except RID numeric for all dataframes and replace inf with NaN
for key, df in dataframes.items():
    df[df.columns.difference(['RID'])] =
df[df.columns.difference(['RID'])].apply(pd.to_numeric, errors='coerce')
    dataframes[key] = df

dataframes["fuji_abeta_csf"].rename(columns={'ABETA42_40': 'ABETA4240'},
inplace=True)

# BATMAN ABETA PLASMA
dataframes["bateman_abeta_plasma"].rename(columns={'Abeta_42_conc': 'ABETA42',
                                                    'Abeta_40_conc': 'ABETA40',
                                                    'Abeta_4240_Standardized':
'ABETA4240'}, inplace=True)
# Convert to pg/ml
dataframes["bateman_abeta_plasma"]['ABETA42'] =
dataframes["bateman_abeta_plasma"]['ABETA42'] * 1000
dataframes["bateman_abeta_plasma"]['ABETA40'] =
dataframes["bateman_abeta_plasma"]['ABETA40'] * 1000
fuji_csf = dataframes["fuji_abeta_csf"]
bateman_abeta_plasma = dataframes["bateman_abeta_plasma"]
upen_csf = dataframes["upen_csf"]
upen_csf['ABETA4240'] = np.divide(upen_csf['ABETA42'], upen_csf['ABETA40'])
adni_merge = adni_merge[adni_merge['VISCODE'] == 'b1']
# adni_merge = adni_merge[adni_merge['DX_b1'] == 'LMCI']
# Drop columns
adni_merge = adni_merge.drop(columns=['COLPROT', 'ORIGPROT', 'PTID', 'SITE',
'EXAMDATE', 'Years_b1', 'Month_b1', 'Month', 'M',
'update_stamp', 'PTAU_b1', 'EcogSPTotal_b1',
'ABETA_b1', 'TAU_b1', 'VISCODE', 'Ventricles_b1', 'Hippocampus_b1',
'Entorhinal_b1',
'EcogPtTotal_b1', 'EcogSPMem_b1',
'EcogSPLang_b1', 'EcogSPVisspat_b1', 'EcogSPPlan_b1', 'EcogSPOrgan_b1',
'EcogSPDivatt_b1', 'EcogPtMem_b1',
'EcogPtLang_b1', 'EcogPtVisspat_b1', 'ADAS11_b1', 'FDG_b1',
'EcogPtPlan_b1', 'EcogPtOrgan_b1',
'EcogPtDivatt_b1', 'FLDSTRENG_b1', 'FSVERSION_b1', 'IMAGEUID_b1'],

```

```

        'EcogPtPlan_bl', 'EcogSPPlan', 'PIB_bl',
'FBB_bl', 'ICV_bl', 'WholeBrain_bl', 'ADASQ4_bl', 'PTETHCAT',
        'RAVLT_immediate_bl', 'RAVLT_learning_bl',
'RAVLT_forgetting_bl', 'RAVLT_perc_forgetting_bl', 'MidTemp_bl',
        'DIGITSCOR_bl', 'TRABSCOR_bl', 'FAQ_bl',
'mPACCdigit_bl', 'AV45_bl', 'LDELTOTAL_BL', 'mPACCtrailsB_bl',
        'ICV', 'DX', 'mPACCdigit', 'mPACCtrailsB',
'EXAMDATE_bl', 'CDRSB_bl', 'Ventricles', 'Hippocampus', 'WholeBrain', 'Entorhinal',
        'Fusiform', 'MidTemp', 'EcogSPOrgan',
'EcogSPDivatt', 'EcogSPTotal', 'FLDSTRENG', 'FSVERSION', 'IMAGEUID',
        'EcogPtDivatt', 'EcogPtTotal', 'EcogSPMem',
'EcogSPLang', 'EcogSPVisspat', 'ADAS11', 'ADAS13', 'ADASQ4', 'MMSE',
        'MOCA', 'EcogPtMem', 'EcogPtLang',
'EcogPtVisspat', 'EcogPtPlan', 'EcogPtOrgan', 'RAVLT_immediate', 'RAVLT_learning',
        'RAVLT_forgetting', 'RAVLT_perc_forgetting',
'LDELTOTAL', 'DIGITSCOR', 'TRABSCOR', 'FAQ',
        'AV45', 'FBB', 'ABETA', 'TAU', 'PTAU',
'CDRSB', 'FDG', 'PIB', 'MOCA_bl', 'Fusiform_bl'
])

# Merge APOE4=1 and APOE4=2 into one category
adni_merge['APOE4'] = adni_merge['APOE4'].replace({2: 1})

# Merge LMCI and EMCI into one category
adni_merge['DX_bl'] = adni_merge['DX_bl'].replace({'EMCI': 'MCI'})
adni_merge['DX_bl'] = adni_merge['DX_bl'].replace({'LMCI': 'MCI'})
adni_merge['DX_bl'] = adni_merge['DX_bl'].replace({'SMC': 'CN'})

# Rename the variables
adni_merge.rename(columns={'AGE': 'Age', 'PTGENDER': 'Gender',
                           'PTEDUCAT': 'Education', 'PTRACCAT': 'Race',
                           'ADAS13_bl': 'ADAS13', 'MMSE_bl': 'MMSE'},
                  inplace=True)

print(adni_merge.head())

fuji_merged_dataset = pd.merge(adni_merge, fuji_csf, on='RID')
upen_merged_dataset = pd.merge(adni_merge, upen_csf, on='RID')
bateman_merged_dataset = pd.merge(adni_merge, bateman_abeta_plasma, on='RID')
# Define the datasets and their names
datasets = [fuji_merged_dataset, upen_merged_dataset, bateman_abeta_plasma]
dataset_names = ['fuji', 'upen', 'bateman']

# Define the columns to drop for each dataset
columns_to_drop = {'fuji': ['ABETA40', 'ABETA42'], 'upen': ['ABETA40', 'ABETA42',
'PTAU'], 'bateman': ['ABETA40', 'ABETA42']}

# Create a dictionary to store the processed datasets

```

```

ab4240_datasets = {}

# Loop over each dataset
for dataset, name in zip(datasets, dataset_names):
    # Drop the rows where 'ABETA4240' is NaN, drop the specified columns, and store
    the result in the dictionary
    ab4240_datasets[name] = dataset.drop(columns=columns_to_drop[name])

# Merge the datasets with bateman_abeta_plasma
# Assuming RID is the common column
ab4240_datasets['bateman'].rename(columns=lambda x: f"{x}_plasma" if x != 'RID'
else x, inplace=True)
ab4240_datasets['bateman'] = ab4240_datasets['bateman'].apply(pd.to_numeric,
errors='coerce')

# Merge dataframes based on the common RID column
ab4240_datasets['fuji'] = pd.merge(ab4240_datasets['fuji'],
ab4240_datasets['bateman'], on='RID', how='inner').dropna()
ab4240_datasets['upen'] = pd.merge(ab4240_datasets['upen'],
ab4240_datasets['bateman'], on='RID', how='inner').dropna()

# Display the summary statistics of the 'upen' dataset
ab4240_datasets['fuji'].describe()
pd.set_option('display.float_format', '{:.8f}'.format)

# Kolmogorov-Smirnov (K-S) test for normality

columns_to_test = ['Age', 'Education', 'MMSE', 'ABETA4240', 'ABETA4240_plasma']

# Initialize a list to hold the results
norm_results_list = []

for column in columns_to_test:
    # Standardize the data
    data = (ab4240_datasets['fuji'][column] -
ab4240_datasets['fuji'][column].mean()) / ab4240_datasets['fuji'][column].std()

    # Perform the Kolmogorov-Smirnov test for normality
    statistic, p_value = stats.kstest(data, 'norm')

    # Append the results to the list as a dictionary
    norm_results_list.append({'Column': column, 'Statistic': statistic, 'P-Value':
p_value})

# Convert the list of results to a DataFrame
norm_results = pd.DataFrame(norm_results_list)

# Display the results

```

```

print(norm_results)

descriptive_stats(ab4240_datasets['fuji'], 'ABETA4240', 'ABETA4240_plasma')
# Define the datasets and their names
datasets = [fuji_merged_dataset, upen_merged_dataset, bateman_abeta_plasma]
dataset_names = ['fuji', 'upen', 'bateman']

# Define the columns to drop for each dataset
columns_to_drop = {'fuji': ['ABETA40', 'ABETA4240'], 'upen': ['ABETA40',
'ABETA4240', 'PTAU'], 'bateman': ['ABETA40', 'ABETA4240']}

# Create a dictionary to store the processed datasets
ab42_datasets = {}

# Loop over each dataset
for dataset, name in zip(datasets, dataset_names):
    # Drop the rows where 'ABETA4240' is NaN, drop the specified columns, and store
the result in the dictionary
    ab42_datasets[name] = dataset.drop(columns=columns_to_drop[name])

# Merge the datasets with bateman_abeta_plasma
# Assuming RID is the common column
ab42_datasets['bateman'].rename(columns=lambda x: f"{x}_plasma" if x != 'RID' else
x, inplace=True)
ab42_datasets['bateman'] = ab42_datasets['bateman'].apply(pd.to_numeric,
errors='coerce')

# Merge dataframes based on the common RID column
ab42_datasets['fuji'] = pd.merge(ab42_datasets['fuji'], ab42_datasets['bateman'],
on='RID', how='inner').dropna()
ab42_datasets['upen'] = pd.merge(ab42_datasets['upen'], ab42_datasets['bateman'],
on='RID', how='inner').dropna()

# Display the summary statistics of the 'upen' dataset
ab42_datasets['fuji'].describe()

# Save
ab42_datasets['fuji'].to_csv('../Data/Thesis/AB42/fuji_abeta42.csv', index=False)
ab42_datasets['upen'].to_csv('../Data/Thesis/AB42/upen_abeta42.csv', index=False)
bateman_merged_dataset.to_csv('../Data/Thesis/AB42/bateman_abeta42.csv',
index=False)
pd.set_option('display.float_format', '{:.8f}'.format)

# Kolmogorov-Smirnov (K-S) test for normality

columns_to_test = ['Age', 'Education', 'MMSE', 'ADAS13', 'ABETA42',
'ABETA42_plasma',]

```

```

# Initialize a list to hold the results
norm_results_list = []

for column in columns_to_test:
    # Standardize the data
    data = (ab42_datasets['fuji'][column] - ab42_datasets['fuji'][column].mean()) /
ab42_datasets['fuji'][column].std()

    # Perform the Kolmogorov-Smirnov test for normality
    statistic, p_value = stats.kstest(data, 'norm')

    # Append the results to the list as a dictionary
    norm_results_list.append({'Column': column, 'Statistic': statistic, 'P-Value':
p_value})

# Convert the list of results to a DataFrame
norm_results = pd.DataFrame(norm_results_list)

# Display the results
print(norm_results)

def calculate_skewness_kurtosis(df):
    # Select numeric columns, excluding 'APOE4', 'Gender', and 'Race'
    numeric_cols = df.select_dtypes(include=[np.number]).columns
    numeric_cols = [col for col in numeric_cols if col not in ['APOE4', 'Gender',
'Race']]

    # Calculate and print skewness
    skewness = df[numeric_cols].skew()
    print("Skewness:\n", skewness)

    # Calculate and print kurtosis
    kurtosis = df[numeric_cols].kurtosis()
    print("\nKurtosis:\n", kurtosis)

# Call the function with your DataFrame
calculate_skewness_kurtosis(ab42_datasets['fuji'])
descriptive_stats(ab42_datasets['upen'], 'ABETA42', 'ABETA42_plasma')
descriptive_stats(ab42_datasets['upen'], 'ABETA42', 'ABETA42_plasma')
# Define the datasets and their names
datasets = [fuji_merged_dataset, upen_merged_dataset, bateman_abeta_plasma]
dataset_names = ['fuji', 'upen', 'bateman']

# Define the columns to drop for each dataset
columns_to_drop = {'fuji': ['ABETA42', 'ABETA4240'], 'upen': ['ABETA42',
'ABETA4240', 'PTAU'], 'bateman': ['ABETA42', 'ABETA4240']}

# Create a dictionary to store the processed datasets

```



```

ab40_datasets = {}

# Loop over each dataset
for dataset, name in zip(datasets, dataset_names):
    # Drop the rows where 'ABETA4240' is NaN, drop the specified columns, and store
    the result in the dictionary
    ab40_datasets[name] = dataset.drop(columns=columns_to_drop[name])

# Merge the datasets with bateman_abeta_plasma
# Assuming RID is the common column
ab40_datasets['bateman'].rename(columns=lambda x: f"{x}_plasma" if x != 'RID' else
x, inplace=True)
ab40_datasets['bateman'] = ab40_datasets['bateman'].apply(pd.to_numeric,
errors='coerce')

# Merge dataframes based on the common RID column
ab40_datasets['fuji'] = pd.merge(ab40_datasets['fuji'], ab40_datasets['bateman'],
on='RID', how='inner').dropna()
ab40_datasets['upen'] = pd.merge(ab40_datasets['upen'], ab40_datasets['bateman'],
on='RID', how='inner').dropna()

# Display the summary statistics of the 'upen' dataset
ab40_datasets['fuji'].describe()
pd.set_option('display.float_format', '{:.8f}'.format)

# Kolmogorov-Smirnov (K-S) test for normality

columns_to_test = ['Age', 'Education', 'MMSE', 'ABETA40', 'ABETA40_plasma']

# Initialize a list to hold the results
norm_results_list = []

for column in columns_to_test:
    # Standardize the data
    data = (ab40_datasets['fuji'][column] - ab40_datasets['fuji'][column].mean()) /
ab40_datasets['fuji'][column].std()

    # Perform the Kolmogorov-Smirnov test for normality
    statistic, p_value = stats.kstest(data, 'norm')

    # Append the results to the list as a dictionary
    norm_results_list.append({'Column': column, 'Statistic': statistic, 'P-Value':
p_value})

# Convert the list of results to a DataFrame
norm_results = pd.DataFrame(norm_results_list)

# Display the results

```

```
print(norm_results)

descriptive_stats(ab40_datasets['fuji'], 'ABETA40', 'ABETA40_plasma')
```

F.5.2 Graphical Representation of Blood Plasma AD VS NON-AD for A β 42

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
import seaborn as sns
from scipy import stats
from scipy.stats import pearsonr, spearmanr
from scipy.stats import ttest_ind, mannwhitneyu
from matplotlib.lines import Line2D
from itertools import combinations
from statsmodels.sandbox.stats.multicomp import multipletests

import scienceplots
plt.rcParams.update({
    'text.usetex': True,
    'font.family': 'serif',
})
plt.style.use(['ieee'])

warnings.filterwarnings('ignore',
                        'is_categorical_dtype is deprecated and will be removed in
a future version. Use isinstance(dtype, CategoricalDtype) instead',
                        category=FutureWarning)
warnings.filterwarnings('ignore', category=FutureWarning)
upen_ab42_dataset = pd.read_csv('../Data/Thesis/AB42/upen_abeta42.csv')
fuji_ab42_dataset = pd.read_csv('../Data/Thesis/AB42/fuji_abeta42.csv')
bateman_ab42_dataset = pd.read_csv('../Data/Thesis/AB42/bateman_abeta42.csv')

# Drop rid
upen_ab42_dataset = upen_ab42_dataset.drop(columns=['RID', 'Race', 'PTMARRY'])
fuji_ab42_dataset = fuji_ab42_dataset.drop(columns=['RID'])
bateman_ab42_dataset = bateman_ab42_dataset.drop(columns=['RID'])

# Make PTGENDER 1 for Male 0 for female
upen_ab42_dataset['Gender'] = upen_ab42_dataset['Gender'].replace({'Male': 1})
upen_ab42_dataset['Gender'] = upen_ab42_dataset['Gender'].replace({'Female': 0})

fuji_ab42_dataset['Gender'] = fuji_ab42_dataset['Gender'].replace({'Male': 1})
fuji_ab42_dataset['Gender'] = fuji_ab42_dataset['Gender'].replace({'Female': 0})

df_fuji = fuji_ab42_dataset
```

```

df_upen = upen_ab42_dataset

datasets = [('Upen', df_upen), ('Fuji', df_fuji)]
palette = {'CN': 'green', 'MCI': 'blue', 'AD': 'red'}

fig, axs = plt.subplots(2, 2, figsize=(14, 12), sharey='row') # Share Y-axis
within each row
label_fontsize = 22
title_fontsize = 22
tick_fontsize = 20

fig.suptitle('CSF and Plasma ABETA42 Levels by Diagnosis Category', fontsize=24)

abeta_types = ['ABETA42', 'ABETA42_plasma']

for row, abeta_type in enumerate(abeta_types):
    for col, (name, df) in enumerate(datasets):
        ax = axs[row, col]
        sns.boxplot(x='DX_b1', y=abeta_type, data=df, ax=ax, palette=palette,
fliersize=0,
                    boxprops=dict(facecolor='None', edgecolor='black',
linewidth=3),
                    medianprops=dict(color="black", linewidth=3),
                    whiskerprops=dict(color="black", linewidth=3),
                    capprops=dict(color="black", linewidth=3))

        sns.stripplot(x='DX_b1', y=abeta_type, data=df, ax=ax, palette=palette,
jitter=True, size=7, edgecolor="black", linewidth=1, facecolors='none')
        if row == 0:
            title = 'UPEN' if col == 0 else 'FUJI'
            ax.set_title(title, fontsize=title_fontsize)
            ax.set_xlabel(' ' if row == 0 else 'Diagnosis Category',
fontsize=label_fontsize)
            ax.set_ylabel(f'Plasma {abeta_type.split("_")[0]} (pg/mL)' if 'plasma' in
abeta_type else f'CSF {abeta_type.split("_")[0]} (pg/mL)', fontsize=label_fontsize)
            ax.tick_params(axis='x', labelsize=tick_fontsize)
            ax.tick_params(axis='y', labelsize=tick_fontsize)

        ax.yaxis.grid(True) # Add horizontal grid lines for the y-axis

        # Perform t-tests and add p-values to the plot
        for i, (group1_name, group2_name) in enumerate([('MCI', 'AD'), ('CN',
'MCI'), ('CN', 'AD')]):
            group1 = df[df['DX_b1'] == group1_name][abeta_type]
            group2 = df[df['DX_b1'] == group2_name][abeta_type]

```

```

        if 'plasma' in abeta_type:
            t_stat, p_val = ttest_ind(group1, group2, nan_policy='omit')
        else: # For CSF
            u_stat, p_val = mannwhitneyu(group1, group2, use_continuity=True,
alternative='two-sided')

# Draw line between boxes and annotate with p-value
if 'plasma' in abeta_type:
    if i == 0:
        y = 51
        x_start, x_end = 0, 2 # Span from first to last category
    elif i == 1:
        y = 47
        x_start, x_end = 0, 1 # Span from first to second category
    else:
        y = 45
        x_start, x_end = 1, 2 # Span from second to last category
else:
    if i == 0:
        y = 2200
        x_start, x_end = 0, 2
    elif i == 1:
        y = 2000
        x_start, x_end = 0, 1
    else:
        y = 1800
        x_start, x_end = 1, 2
line = Line2D([x_start, x_end], [y, y], color='black')
ax.add_line(line)
if p_val < 0.001:
    ax.text(x_start, y + 0.1, f'p $\textless$ 0.001', ha='left',
va='bottom', fontsize=18)
else:
    ax.text(x_start, y + 0.1, f'p = {p_val:.3f}', ha='left',
va='bottom', fontsize=18)

plt.tight_layout()
fig.subplots_adjust(top=0.92)
plt.show()
datasets = {'upen': upen_ab42_dataset, 'fuji': fuji_ab42_dataset}
fontsize_props = {
    'stats_text': 18,
    'label': 18,
    'title': 20,
    'tick': 18,
    'legend': 14,
    'legend_title': 14,
    'marker_size': 40

```

```

}

from scipy.stats import pointbiserialr
# Split features into three groups
features_group1 = ['Education', 'MMSE', 'ADAS13',]
features_group2 = ['APOE4', 'Gender']
features_group3 = ['Age', 'ABETA42']

# Function to plot each subplot
def plot_subplot(ax, df, biomarker, title, fontsize_props, add_regression,
add_pearson, j):
    categories = df['DX_b1'].unique()
    colors = ['green', 'blue', 'red']
    markers = ['o', 's', '^']

    for category, color, marker in zip(categories, colors, markers):
        subset = df[df['DX_b1'] == category]
        ax.scatter(subset[biomarker], subset['ABETA42_plasma'], alpha=0.8,
                    color=color, marker=marker, label=category,
                    s=fontsize_props['marker_size'])

    # Calculate and display correlation coefficient based on the type of the
    variable
    if df[biomarker].nunique() == 2: # Binary variable
        coef, p = pointbiserialr(df[biomarker], df['ABETA42_plasma'])
        p_value_str = f'$\\textless$0.001' if p < 0.001 else f'{p:.2f}'
        stats_text = f'Point-Biserial r: {coef:.2f}, P-value: {p_value_str}'
    else: # Non-binary variable
        coef_spearman, p_spearman = spearmanr(df[biomarker], df['ABETA42_plasma'])
        p_value_str_spearman = f'$\\textless$0.001' if p_spearman < 0.001 else
f'{p_spearman:.2f}'
        stats_text = f'Spearman rho: {coef_spearman:.2f}, P-value:
{p_value_str_spearman}'
        if add_pearson:
            coef_pearson, p_pearson = pearsonr(df[biomarker], df['ABETA42_plasma'])
            p_value_str_pearson = f'$\\textless$0.001' if p_pearson < 0.001 else
f'{p_pearson:.2f}'
            stats_text += f'\nPearson r: {coef_pearson:.2f}, P-value:
{p_value_str_pearson}'

    # Add best fit line if add_regression is True
    if add_regression:
        sns.regplot(x=biomarker, y='ABETA42_plasma', data=df, ci=95, scatter=False,
color='black', line_kws={"linestyle": "--"}, ax=ax)
        if j != 0: # Remove the y-label for all columns except the first one
            ax.set_ylabel('')

    props = dict(boxstyle='round', facecolor='wheat', alpha=0.5)

```

```

    ax.text(0.95, 0.05, stats_text, ha='right', va='bottom',
transform=ax.transAxes, fontsize=fontsize_props['stats_text'], bbox=props)

    # Add units to the x-axis label when setting it
    if biomarker == 'Education' or biomarker == 'Age':
        xlabel = f'{biomarker} (Years)'
    elif biomarker == 'MMSE' or biomarker == 'ADAS13':
        xlabel = f'{biomarker} (Score)'
    elif biomarker == 'ABETA42':
        xlabel = 'CSF ABETA42 (pg/mL)'
        title = 'Plasma ABETA42 vs. CSF ABETA42'
    elif biomarker == 'APOE4':
        xlabel = f'{biomarker} (Non-carrier=0, Carrier=1)'
    elif biomarker == 'Gender':
        xlabel = f'{biomarker} (Female=0, Male=1)'
    else:
        xlabel = biomarker
    ax.set_xlabel(xlabel, fontsize=fontsize_props['label'])
    if j == 0: # Only set the y-label for the first column
        ax.set_ylabel('Plasma ABETA42 (pg/mL)', fontsize=fontsize_props['label'])
    ax.set_title(title, fontsize=fontsize_props['title'])
    ax.tick_params(axis='both', which='major', labelsize=fontsize_props['tick'])
    ax.legend(title='Diagnosis', fontsize=fontsize_props['legend'],
title_fontsize=fontsize_props['legend_title'])

    # Place the legend in the top right, top left, or bottom left only
    legend_loc = 'lower left' if j % 3 == 0 else 'upper left' if j % 3 == 1 else
'lower left'
    ax.legend(loc=legend_loc, prop={'size': fontsize_props['legend']})

    ax.grid(True, axis='y') # Add grid only on y-axis

def create_plot(features_group, datasets, fontsize_props, add_regression,
add_pearson=False):
    # Determine the size of the figure based on the group of features
    if features_group == features_group1:
        fig, axs = plt.subplots(len(features_group), len(datasets), figsize=(14,
15), sharey=True) # Larger size for group 1
    else:
        fig, axs = plt.subplots(len(features_group), len(datasets), figsize=(14,
11), sharey=True) # Smaller size for other groups

    for i, feature in enumerate(features_group):
        for j, (dataset_name, dataset) in enumerate(datasets.items()):
            if pd.api.types.is_numeric_dtype(dataset[feature]): # Check if the
column is numeric
                title = f'Plasma ABETA42 vs. {feature}'

```

```

        plot_subplot(axes[i][j], dataset, feature, title, fontsize_props,
add_regression, add_pearson, j)
    else:
        print(f'Skipping non-numeric feature: {feature}')

    fig.text(0.30, 1.01, 'UPEN', ha='center', va='center',
fontsize=fontsize_props['title'])
    fig.text(0.78, 1.01, 'FUJI', ha='center', va='center',
fontsize=fontsize_props['title'])
    plt.tight_layout()
    plt.show()

# Create the first plot without regression and without Pearson's correlation
create_plot(features_group1, datasets, fontsize_props, add_regression=False,
add_pearson=False)
datasets = [('Upen', df_upen), ('Fuji', df_fuji)]

features = ['APOE4', 'Gender']
for dataset_name, df in datasets:
    print(f'\nDataset: {dataset_name}\n')
    for feature in features:
        # Create a contingency table
        contingency_table = pd.crosstab(df[feature], df['DX_b1'])

        # Chi-Square Test of Independence
        chi2, p_val, dof, expected = stats.chi2_contingency(contingency_table)
        print(f'\nChi-Square Test of Independence for {feature}:')
        print(f'Chi-square Statistic: {chi2}')
        print(f'P-Value: {p_val}')
        print(f'Degrees of Freedom: {dof}')

        # Pairwise Chi-Square tests
        print('\nPairwise Chi-Square Tests:')
        p_vals = []
        for group1, group2 in combinations(contingency_table.columns, 2):
            chi2, p_val, dof, _ = stats.chi2_contingency(contingency_table.loc[:,
[group1, group2]])
            p_vals.append(p_val)
            print(f'Groups {group1} and {group2}: Chi-square={chi2}, p={p_val}')

        # Correction for multiple comparisons
        reject, p_vals_corrected, _, _ = multipletests(p_vals, method='bonferroni')
        print(f'Corrected p-values: {p_vals_corrected}')

    print('\n---\n')

```

F.5.3 Correlation Analysis for A β 42

```
import pandas as pd
```

```

import numpy as np
import matplotlib.pyplot as plt
from collections import Counter
import warnings
import seaborn as sns
from sklearn.preprocessing import LabelEncoder

import scienceplots
plt.rcParams.update({
    'text.usetex': True,
    'font.family': 'serif',
})
plt.style.use(['ieee'])
upen_dataset = pd.read_csv('../Data/Thesis/AB42/upen_abeta42.csv')
# Drop rid
upen_dataset = upen_dataset.drop(columns=['RID'])
upen_dataset.describe()

# Make PTGENDER 1 for Male 0 for female
upen_dataset['Gender'] = upen_dataset['Gender'].replace({'Male': 1})
upen_dataset['Gender'] = upen_dataset['Gender'].replace({'Female': 0})

# Rename columns
upen_dataset = upen_dataset.rename(columns={'ABETA42': 'CSF_ABETA42',
'ABETA42_plasma': 'Plasma_ABETA42'})

# Get the list of column names
cols = list(upen_dataset.columns)

# Move 'APOE4' to the position before 'CSF_ABETA42'
cols.insert(cols.index('CSF_ABETA42'), cols.pop(cols.index('APOE4')))

# Move 'PTGENDER' to the position before 'CSF_ABETA42'
cols.insert(cols.index('CSF_ABETA42'), cols.pop(cols.index('Gender')))

# Reindex the DataFrame
upen_dataset = upen_dataset[cols]

# Reindex the DataFrame
upen_dataset = upen_dataset[cols]

upen_dataset.describe()
# Correlation matrix
ab42_datasets_encoded = upen_dataset.copy()
columns_to_drop = ['DX_b1', 'Race', 'PTMARRY']

# ab42_datasets_encoded['upen'].drop(columns=['VISCODE', 'PTMARRY', 'ADAS11_b1',
'ADAS13_b1', 'ADASQ4_b1', 'PTRACCAT', "PTETHCAT", 'RID' ], inplace=True)

```



```

ab42_datasets_encoded.drop(columns=columns_to_drop, inplace=True)

label_encoder = LabelEncoder()
# ab42_datasets_encoded["PTGENDER"] =
ab42_datasets_encoded["PTGENDER"].map({"Female": 1, "Male": 0})

# Correlation matrix
corr = ab42_datasets_encoded.corr('spearman')

plt.figure(figsize=(13, 10))
ax = sns.heatmap(corr, annot=True, fmt=".2f", cmap='hot',
                 annot_kws={"size": 13},
                 linecolor='black', linewidths=0.8,
                 cbar_kws={"pad": 0.02})

# Increase the size of the color bar font
cbar = ax.collections[0].colorbar
cbar.ax.tick_params(labelsize=11)
cbar.outline.set_edgecolor('black')
cbar.outline.set_linewidth(2)

# Increase the size of the color bar font
cbar = ax.collections[0].colorbar
cbar.ax.tick_params(labelsize=11)

# Surround the matrix and the color bar with black color
for _, spine in ax.spines.items():
    spine.set_visible(True)
    spine.set_color('black')
    spine.set_linewidth(2)

plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
plt.title('UPEN ABETA42 Correlation Matrix', fontsize=14)
plt.show()

```

F.5.4 Machine Learning Pipeline and Evaluation for Aβ42

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
import seaborn as sns

from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error,
make_scorer, mean_absolute_percentage_error
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.model_selection import cross_val_score, RepeatedKFold,
cross_validate, cross_val_predict, KFold, train_test_split, StratifiedKFold

```

```

from sklearn.base import BaseEstimator, RegressorMixin
from scipy.stats import skew, kurtosis
from scipy import stats
from scipy.stats import mannwhitneyu

from skopt.space import Real
from skopt import BayesSearchCV

from itertools import combinations
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVR
from skopt import BayesSearchCV
from skopt.space import Real, Categorical
from sklearn.pipeline import Pipeline
import matplotlib.patches as mpatches

import scienceplots
plt.rcParams.update({
    'text.usetex': True,
    'font.family': 'serif',
})
plt.style.use(['ieee'])

warnings.filterwarnings('ignore',
                        'is_categorical_dtype is deprecated and will be removed in
a future version. Use isinstance(dtype, CategoricalDtype) instead',
                        category=FutureWarning)

warnings.filterwarnings('ignore', category=FutureWarning)
upen_dataset = pd.read_csv('../Data/Thesis/AB42/upen_abeta42.csv')
fuji_dataset = pd.read_csv('../Data/Thesis/AB42/fuji_abeta42.csv')

fuji_unique_dataset = fuji_dataset[fuji_dataset['RID'].isin(upen_dataset['RID'])]

# Drop rid
upen_dataset = upen_dataset.drop(columns=['RID', 'DX_b1', 'Race', 'PTMARRY'])
fuji_dataset = fuji_dataset.drop(columns=['RID', 'DX_b1', 'Race', 'PTMARRY'])
fuji_unique_dataset = fuji_unique_dataset.drop(columns=['RID', 'DX_b1', 'Race',
'PTMARRY'])

# Make PTGENDER 1 for Male 0 for female
upen_dataset['Gender'] = upen_dataset['Gender'].replace({'Male': 1})
upen_dataset['Gender'] = upen_dataset['Gender'].replace({'Female': 0})
fuji_dataset['Gender'] = fuji_dataset['Gender'].replace({'Male': 1})
fuji_dataset['Gender'] = fuji_dataset['Gender'].replace({'Female': 0})
fuji_unique_dataset['Gender'] = fuji_unique_dataset['Gender'].replace({'Male': 1})

```

```

fuji_unique_dataset['Gender'] = fuji_unique_dataset['Gender'].replace({'Female':
0})

# Rename columns
upen_dataset = upen_dataset.rename(columns={'ABETA42': 'CSF_ABETA42',
'ABETA42_plasma': 'Plasma_ABETA42'})
fuji_dataset = fuji_dataset.rename(columns={'ABETA42': 'CSF_ABETA42',
'ABETA42_plasma': 'Plasma_ABETA42'})
fuji_unique_dataset = fuji_unique_dataset.rename(columns={'ABETA42': 'CSF_ABETA42',
'ABETA42_plasma': 'Plasma_ABETA42'})
upen_dataset.describe()

# Placeholder for feature combinations and their corresponding MAPE scores
feature_combinations = []
mae_scores = []
mape_scores = []
rmse_scores = []
r2_scores = []
best_params = []

# Define scoring dictionary
scoring = {'MAE': 'neg_mean_absolute_error',
           'MAPE': 'neg_mean_absolute_percentage_error',
           'RMSE': make_scorer(mean_squared_error, squared=False),
           'R2': 'r2'}

# Define the cross-validation strategy
cv = KFold(n_splits=10, shuffle=True, random_state=42)

# Features to consider
features_to_consider = ['Age', 'APOE4', 'CSF_ABETA42', 'Gender', 'MMSE' ]

# Iterate through all non-empty combinations of the three features
for r in range(1, len(features_to_consider) + 1):
    for combo in combinations(features_to_consider, r):
        # for combo in [('Age', 'APOE4', 'CSF_ABETA42', 'Gender'), ('Age',
'CSF_ABETA42', 'Gender'), ('Age', 'CSF_ABETA42', 'APOE4') ]:
            print(combo)
            # Selecting the current combination of features
            X = upen_dataset[list(combo)]
            y = upen_dataset['Plasma_ABETA42']

            # Create copies of X and y
            X_opt = X.copy()
            y_opt = y.copy()

            # Calculate z-scores for y

```

```

z_scores_y = stats.zscore(y_opt)

# Get indices of outliers in y
outliers_y = np.where(np.abs(z_scores_y) > 3)[0].tolist()

# Double the outliers in y
y_opt[outliers_y] *= 5

# Scaling the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_opt_scaled = scaler.fit_transform(X_opt)

# Bayesian optimization setup
search_spaces = {'C': Real(1e-1, 1e+2, prior='log-uniform'),
                 'epsilon': Real(1e-2, 1e+1, prior='log-uniform'),
                 'gamma': Real(1e-3, 1e+1, prior='log-uniform'),
                 'kernel': Categorical(['rbf'])}

bayesian_opt = BayesSearchCV(SVR(),
                             search_spaces,
                             scoring='neg_mean_absolute_error',
                             cv=cv,
                             n_jobs=-1)

bayesian_opt.fit(X_opt_scaled, y_opt)

# Pipeline that includes scaling and the SVR model with the best parameters
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('svr', SVR(**bayesian_opt.best_params_))
])

# Using cross_validate with the pipeline, which ensures features are scaled
results = cross_validate(pipeline, X, y, cv=cv, scoring=scoring,
return_train_score=False)

# Calculate mean scores
mean_mae = np.mean(results['test_MAE'])
mean_mape = np.mean(results['test_MAPE'])
mean_rmse = np.mean(results['test_RMSE'])
mean_r2 = np.mean(results['test_R2'])

# Print results
print(f"CV MAE Scores: {results['test_MAE']}")
print(f"Mean CV MAE Score: {mean_mae}")
print(f"CV MAPE Scores: {results['test_MAPE']}")
print(f"Mean CV MAPE Score: {mean_mape}")

```

```

print(f"CV RMSE Scores: {results['test_RMSE']}")
print(f"Mean CV RMSE Score: {mean_rmse}")
print(f"CV R2 Scores: {results['test_R2']}")
print(f"Mean CV R2 Score: {mean_r2}")

print("//-----//")

feature_combinations.append(combo)
mae_scores.append(mean_mae) # Store the mean MAE score
mape_scores.append(mean_mape)
rmse_scores.append(mean_rmse)
r2_scores.append(mean_r2)
best_params.append(bayesian_opt.best_params_) # Store best parameters
# Sort the combinations by MAPE score for a more intuitive display
sorted_indices = np.argsort(mape_scores)
sorted_combinations = [feature_combinations[i] for i in sorted_indices]
sorted_mae_scores = [mae_scores[i] for i in sorted_indices]
sorted_best_params = [best_params[i] for i in sorted_indices]

sorted_mae_scores = np.abs(sorted_mae_scores) # Convert scores to positive

plt.figure(figsize=(14, 10))
colors = [sns.color_palette("hot", len(set([len(combo) for combo in
feature_combinations]))))\
          [len(combo)-1] for combo in sorted_combinations]
bars = plt.barh(range(len(sorted_combinations)), sorted_mae_scores, color=colors,
edgecolor='black')

for bar, score in zip(bars, sorted_mae_scores):
    plt.text(bar.get_width(), bar.get_y() + bar.get_height() / 2, f' {score:.3f}',
            va='center', ha='left', fontsize=13, fontweight='bold')

plt.yticks(range(len(sorted_combinations)), [' \\& '.join(combo) for combo in
sorted_combinations], fontsize=12)
plt.xticks(fontsize=12)
plt.xlabel('MAE (pg/mL)', fontsize=12)
plt.title('MAE by Feature Combination', fontsize=16)
# plt.gca().invert_yaxis() # Ensure lowest MAPE is at the top

legend_elements = [mpatches.Patch(color=sns.color_palette("hot",
len(features_to_consider))[i],
                                label=f'{i+1} feature{"s" if i > 0 else ""}')
                    for i in range(len(features_to_consider))]
plt.legend(handles=legend_elements, title="Number of Features", loc='upper right',
fontsize=11, title_fontsize=11)

plt.tight_layout()
plt.show()

```

```

sorted_indices = np.argsort(rmse_scores)[::-1]

sorted_combinations = [feature_combinations[i] for i in sorted_indices]
sorted_rmse_scores = [rmse_scores[i] for i in sorted_indices]
sorted_best_params = [best_params[i] for i in sorted_indices]

plt.figure(figsize=(12, 8))
colors = [sns.color_palette("hot", len(set([len(combo) for combo in
feature_combinations]))))\
          [len(combo)-1] for combo in sorted_combinations]
bars = plt.barh(range(len(sorted_combinations)), sorted_rmse_scores, color=colors,
edgecolor='black')

for bar, score in zip(bars, sorted_rmse_scores):
    plt.text(bar.get_width(), bar.get_y() + bar.get_height() / 2, f' {score:.3f}',
            va='center', ha='left', fontsize=10, fontweight='bold')

plt.yticks(range(len(sorted_combinations)), [' \\& '.join(combo) for combo in
sorted_combinations])
plt.xlabel('RMSE')
plt.title('RMSE by Feature Combination')
# plt.gca().invert_yaxis() # Ensure lowest MAPE is at the top

legend_elements = [mpatches.Patch(color=sns.color_palette("hot",
len(features_to_consider))[i],
                                label=f'{i+1} feature{"s" if i > 0 else ""}')
                    for i in range(len(features_to_consider))]
plt.legend(handles=legend_elements, title="Number of Features", loc='upper right')

plt.tight_layout()
plt.show()

# Sort the combinations by r2 score for a more intuitive display
sorted_indices = np.argsort(r2_scores)
sorted_combinations = [feature_combinations[i] for i in sorted_indices]
sorted_r2_scores = [r2_scores[i] for i in sorted_indices]
sorted_best_params = [best_params[i] for i in sorted_indices]

# Multiply r2 scores by 100
sorted_r2_scores = [score * 100 for score in sorted_r2_scores]

plt.figure(figsize=(12, 10))
colors = [sns.color_palette("hot", len(set([len(combo) for combo in
feature_combinations]))))\
          [len(combo)-1] for combo in sorted_combinations]
bars = plt.barh(range(len(sorted_combinations)), sorted_r2_scores, color=colors,
edgecolor='black')

```

```

for bar, score in zip(bars, sorted_r2_scores):
    plt.text(bar.get_width(), bar.get_y() + bar.get_height() / 2,
f' {score:.1f}\\%',
            va='center', ha='left', fontsize=10, fontweight='bold')

plt.yticks(range(len(sorted_combinations)), [' \\& '.join(combo) for combo in
sorted_combinations], fontsize=12)
plt.xticks(fontsize=12)
plt.xlabel('R2 (\\%)', fontsize=12)
plt.title('R2 by Feature Combination', fontsize=16)

# Creating custom legend for number of features
legend_elements = [mpatches.Patch(color=sns.color_palette("hot",
len(features_to_consider))[i],
                                label=f'{i+1} feature{"s" if i > 0 else ""}')
                    for i in range(len(features_to_consider))]
plt.legend(handles=legend_elements, title="Number of Features", loc='lower right',
fontsize=11, title_fontsize=11)

plt.tight_layout()
plt.show()
sorted_indices = np.argsort(mape_scores)

sorted_combinations = [feature_combinations[i] for i in sorted_indices]
sorted_mape_scores = [mape_scores[i] for i in sorted_indices]
sorted_best_params = [best_params[i] for i in sorted_indices]

sorted_mape_scores = np.abs(sorted_mape_scores) # Convert scores to positive

plt.figure(figsize=(12, 8))
colors = [sns.color_palette("hot", len(set([len(combo) for combo in
feature_combinations])))\
          [len(combo)-1] for combo in sorted_combinations]
bars = plt.barh(range(len(sorted_combinations)), sorted_mape_scores, color=colors,
edgecolor='black')

for bar, score in zip(bars, sorted_mape_scores):
    plt.text(bar.get_width(), bar.get_y() + bar.get_height() / 2,
f' {score*100:.1f}\\%',
            va='center', ha='left', fontsize=10, fontweight='bold')

plt.yticks(range(len(sorted_combinations)), [' \\& '.join(combo) for combo in
sorted_combinations])
plt.xlabel('MAPE')
plt.title('MAPE by Feature Combination')
# plt.gca().invert_yaxis() # Ensure lowest MAPE is at the top

```

```

# Creating custom legend for number of features
legend_elements = [mpatches.Patch(color=sns.color_palette("hot",
len(features_to_consider))[i],
                                label=f'{i+1} feature{"s" if i > 0 else ""}')
                    for i in range(len(features_to_consider))]
plt.legend(handles=legend_elements, title="Number of Features", loc='upper right')

plt.tight_layout()
plt.show()
# Find the index of the feature combination in the list
index = feature_combinations.index(('Age', 'APOE4', 'CSF_ABETA42', 'Gender'))
cv = KFold(n_splits=10, shuffle=True, random_state=42)

# Get the best parameters for this feature combination
best_params_for_combo = best_params[index]

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('svr', SVR(**best_params_for_combo))
])

# print(best_params_for_combo)

# Select the features from the dataset
X = upen_dataset[list(('Age', 'APOE4', 'CSF_ABETA42', 'Gender'))]
y = upen_dataset['Plasma_ABETA42']

# Get cross-validated predictions for the entire dataset
predictions = cross_val_predict(pipeline, X, y, cv=cv,)

# Calculate errors (residuals)
residuals = y - predictions

plt.figure(figsize=(10, 5))
sns.histplot(residuals, bins=20, kde=True, edgecolor='black')
plt.title('Distribution of Residuals', fontsize=20)
plt.xlabel('Residuals', fontsize=16)
plt.ylabel('Frequency', fontsize=16)
plt.yticks(fontsize=12)
plt.xticks(fontsize=12)
plt.show()
# Calculate and print skewness
residual_skewness = skew(residuals)
print(f'Skewness of residuals: {residual_skewness}')

# Calculate and print kurtosis
residual_kurtosis = kurtosis(residuals, fisher=True)
print(f'Kurtosis of residuals: {residual_kurtosis}')

```



```

plt.figure(figsize=(9, 5))
sns.scatterplot(x= y, y=predictions, color='blue', alpha=0.6, edgecolor='w',
linewidth=0.5)
sns.regplot(x= y, y=predictions, color='red', ci=95, scatter=False,
label='Regression Line with 95\\% CI')

# Plotting the line of perfect prediction for reference
min_vals = np.minimum( y.min(), predictions.min())
max_vals = np.maximum( y.max(), predictions.max())
plt.plot([min_vals, max_vals], [min_vals, max_vals], color='darkgreen',
linestyle='--', label='Perfect Fit')

plt.title('Actual vs. Predicted ABETA42 Levels', fontsize=20)
plt.xlabel('Actual ABETA42 Levels (pg/mL)', fontsize=16)
plt.ylabel('Predicted ABETA42 Levels (pg/mL)', fontsize=16)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)

# Adding legend to differentiate between actual data points and the regression line
plt.legend(prop={'size': 12})
plt.grid(True, which='both', linestyle='--', linewidth=0.5)
plt.tight_layout()
plt.show()
cv = KFold(n_splits=10, shuffle=True, random_state=42)
# Define a simple mean predictor
class MeanPredictor(BaseEstimator, RegressorMixin):
    def fit(self, X, y):
        self.mean_ = np.mean(y)
        return self

    def predict(self, X):
        return np.full(shape=(len(X),), fill_value=self.mean_)

# Create an instance of the mean predictor
mean_predictor = MeanPredictor()

scoring = ['neg_mean_absolute_error', 'neg_mean_squared_error', 'r2']

results_baseline = cross_validate(mean_predictor, X, y, cv=cv, scoring=scoring,
return_train_score=False)
# Get cross-validated predictions for the entire dataset
mean_predictions = cross_val_predict(mean_predictor, X, y, cv=cv)

# Calculate mean scores for the baseline model
mean_mae_baseline = np.mean(results_baseline['test_neg_mean_absolute_error'])
mean_rmse_baseline = np.sqrt(-
np.mean(results_baseline['test_neg_mean_squared_error'])) \

```

```

mean_r2_baseline = np.mean(results_baseline['test_r2'])

print(f"Baseline Model - Mean MAE: {mean_mae_baseline}")
print(f"Baseline Model - Mean RMSE: {mean_rmse_baseline}")
print(f"Baseline Model - Mean R2: {mean_r2_baseline}")
# Mann-Whitney U test used when the two samples are independent.
from scipy.stats import mannwhitneyu
stat, p = mannwhitneyu(predictions, mean_predictions)
print('Mann-Whitney U test p-value:', p)

```

F.5.5 Fuji Dataset Evaluation

```

# Find the index of the feature combination in the list
index = feature_combinations.index(('Age', 'APOE4', 'CSF_ABETA42', 'Gender'))
cv = KFold(n_splits=10, shuffle=True, random_state=42)

# Get the best parameters for this feature combination
best_params_for_combo = best_params[index]
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('svr', SVR(**best_params_for_combo))
])
# print(best_params_for_combo)

# Select the features from the dataset
X = fuji_unique_dataset[list(('Age', 'APOE4', 'CSF_ABETA42', 'Gender'))]
y = fuji_unique_dataset['Plasma_ABETA42']

results = cross_validate(pipeline, X, y, cv=cv, scoring=scoring,
return_train_score=False)

# Get cross-validated predictions for the entire dataset
predictions = cross_val_predict(pipeline, X, y, cv=cv)

# Calculate errors (residuals)
residuals = y - predictions

fuji_mae = np.mean(results['test_neg_mean_absolute_error'])
fuji_rmse = np.sqrt(-np.mean(results['test_neg_mean_squared_error']))
fuji_r2 = np.mean(results['test_r2'])

print(f"FUJI Dataset - MAE: {fuji_mae}")
print(f"FUJI Dataset - RMSE: {fuji_rmse}")
print(f"FUJI Dataset - R2: {fuji_r2}")

plt.figure(figsize=(10, 5))
sns.histplot(residuals, bins=20, kde=True, edgecolor='black')

```

```

plt.title('Distribution of Residuals', fontsize=20)
plt.xlabel('Residuals', fontsize=16)
plt.ylabel('Frequency', fontsize=16)
plt.yticks(fontsize=12)
plt.xticks(fontsize=12)
plt.show()

plt.figure(figsize=(9, 5))
# Improved regplot with scatterplot for better visibility of data points
sns.scatterplot(x= y, y=predictions, color='blue', alpha=0.6, edgecolor='w',
linewidth=0.5)
sns.regplot(x= y, y=predictions, color='red', ci=95, scatter=False,
label='Regression Line with 95\\% CI')

# Plotting the line of perfect prediction for reference
min_vals = np.minimum( y.min(), predictions.min())
max_vals = np.maximum( y.max(), predictions.max())
plt.plot([min_vals, max_vals], [min_vals, max_vals], color='darkgreen',
linestyle='--', label='Perfect Fit')

plt.title('Actual vs. Predicted ABETA42 Levels', fontsize=20)
plt.xlabel('Actual ABETA42 Levels (pg/mL)', fontsize=16)
plt.ylabel('Predicted ABETA42 Levels (pg/mL)', fontsize=16)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)

# Adding legend to differentiate between actual data points and the regression line
plt.legend(prop={'size': 12})
plt.grid(True, which='both', linestyle='--', linewidth=0.5)
plt.tight_layout()
plt.show()
cv = KFold(n_splits=10, shuffle=True, random_state=42)
# Define a simple mean predictor
class MeanPredictor(BaseEstimator, RegressorMixin):
    def fit(self, X, y):
        self.mean_ = np.mean(y)
        return self

    def predict(self, X):
        return np.full(shape=(len(X)), fill_value=self.mean_)

# Create an instance of the mean predictor
mean_predictor = MeanPredictor()

scoring = ['neg_mean_absolute_error', 'neg_mean_squared_error', 'r2']

# X = fuji_dataset[list(('Age', 'APOE4', 'CSF_ABETA42', 'Gender'))]
# y = fuji_dataset['Plasma_ABETA42']

```

```

results_baseline = cross_validate(mean_predictor, X, y, cv=cv, scoring=scoring,
return_train_score=False)
# Get cross-validated predictions for the entire dataset
mean_predictions = cross_val_predict(mean_predictor, X, y, cv=cv)

# Calculate mean scores for the baseline model
mean_mae_baseline = -np.mean(results_baseline['test_neg_mean_absolute_error'])
mean_rmse_baseline = np.sqrt(-
np.mean(results_baseline['test_neg_mean_squared_error'])) # RMSE
mean_r2_baseline = np.mean(results_baseline['test_r2'])

print(f"Baseline Model - Mean MAE: {mean_mae_baseline}")
print(f"Baseline Model - Mean RMSE: {mean_rmse_baseline}")
print(f"Baseline Model - Mean R2: {mean_r2_baseline}")
# Mann-Whitney U test
stat, p = mannwhitneyu(predictions, mean_predictions)
print('Mann-Whitney U test p-value:', p)

```

F.6 Code for the Machine Learning Pipeline and Evaluation of Aβ40

```

fuji_dataset = pd.read_csv(r'C:\Users\theof\OneDrive - The University of
Manchester\Univesity\Year 3\Project\Code\Data\Appendix\ABETA40\fuji_abeta40.csv')

# Drop rid
fuji_dataset = fuji_dataset.drop(columns=['RID', 'DX_b1', 'Race', 'PTMARRY'])

# Make PTGENDER 1 for Male 0 for female
fuji_dataset['Gender'] = fuji_dataset['Gender'].replace({'Male': 1})
fuji_dataset['Gender'] = fuji_dataset['Gender'].replace({'Female': 0})

# Rename columns
fuji_dataset = fuji_dataset.rename(columns={'ABETA40': 'CSF_ABETA40',
'ABETA40_plasma': 'Plasma_ABETA40'})
fuji_dataset.describe()

# Placeholder for feature combinations and their corresponding MAPE scores
feature_combinations = []
mae_scores = []
mape_scores = []
rmse_scores = []
r2_scores = []
best_params = []

# Define scoring dictionary
scoring = {'MAE': 'neg_mean_absolute_error',

```

```

        'MAPE': 'neg_mean_absolute_percentage_error',
        'RMSE': make_scorer(mean_squared_error, squared=False),
        'R2': 'r2'}

# Define the cross-validation strategy
cv = KFold(n_splits=10, shuffle=True, random_state=42)

# Features to consider
features_to_consider = ['Age', 'APOE4', 'CSF_ABETA40', 'Gender', 'Education' ]

# Iterate through all non-empty combinations of the three features
for r in range(1, len(features_to_consider) + 1):
    for combo in combinations(features_to_consider, r):
        # for combo in [('Age', 'APOE4', 'CSF_ABETA40', 'Gender'), ('Age',
        'CSF_ABETA40', 'Gender'), ('Age', 'CSF_ABETA40', 'APOE4') ]:
            print(combo)
            # Selecting the current combination of features
            X = fuji_dataset[list(combo)]
            y = fuji_dataset['Plasma_ABETA40']

            # Create copies of X and y
            X_opt = X.copy()
            y_opt = y.copy()

            # Calculate z-scores for y
            z_scores_y = stats.zscore(y_opt)

            # Get indices of outliers in y
            outliers_y = np.where(np.abs(z_scores_y) > 3)[0].tolist()

            # Double the outliers in y
            y_opt[outliers_y] *= 5

            # Scaling the features
            scaler = StandardScaler()
            X_scaled = scaler.fit_transform(X)
            X_opt_scaled = scaler.fit_transform(X_opt)

            # Bayesian optimization setup
            search_spaces = {'C': Real(1e-1, 1e+2, prior='log-uniform'),
                             'epsilon': Real(1e-2, 1e+1, prior='log-uniform'),
                             'gamma': Real(1e-3, 1e+1, prior='log-uniform'),
                             'kernel': Categorical(['rbf'])}

            bayesian_opt = BayesSearchCV(SVR(),
                                         search_spaces,
                                         scoring='neg_mean_absolute_error',

```

```

        cv=cv,
        n_jobs=-1)

    bayesian_opt.fit(X_opt_scaled, y_opt)

    # Pipeline that includes scaling and the SVR model with the best parameters
    pipeline = Pipeline([
        ('scaler', StandardScaler()),
        ('svr', SVR(**bayesian_opt.best_params_))
    ])

    # Using cross_validate with the pipeline, which ensures features are scaled
    results = cross_validate(pipeline, X, y, cv=cv, scoring=scoring,
return_train_score=False)

    # Calculate mean scores
    mean_mae = np.mean(results['test_MAE'])
    mean_mape = np.mean(results['test_MAPE'])
    mean_rmse = np.mean(results['test_RMSE'])
    mean_r2 = np.mean(results['test_R2'])

    # Print results
    print(f"CV MAE Scores: {results['test_MAE']}")
    print(f"Mean CV MAE Score: {mean_mae}")
    print(f"CV MAPE Scores: {results['test_MAPE']}")
    print(f"Mean CV MAPE Score: {mean_mape}")
    print(f"CV RMSE Scores: {results['test_RMSE']}")
    print(f"Mean CV RMSE Score: {mean_rmse}")
    print(f"CV R2 Scores: {results['test_R2']}")
    print(f"Mean CV R2 Score: {mean_r2}")

    print("//-----//")

    feature_combinations.append(combo)
    mae_scores.append(mean_mae) # Store the mean MAE score
    mape_scores.append(mean_mape)
    rmse_scores.append(mean_rmse)
    r2_scores.append(mean_r2)
    best_params.append(bayesian_opt.best_params_) # Store best parameters
# Sort the combinations by MAPE score for a more intuitive display
sorted_indices = np.argsort(mape_scores)
sorted_combinations = [feature_combinations[i] for i in sorted_indices]
sorted_mae_scores = [mae_scores[i] for i in sorted_indices]
sorted_best_params = [best_params[i] for i in sorted_indices]

sorted_mae_scores = np.abs(sorted_mae_scores) # Convert scores to positive
plt.figure(figsize=(14, 10))

```

```

colors = [sns.color_palette("hot", len(set([len(combo) for combo in
feature_combinations]))))\
          [len(combo)-1] for combo in sorted_combinations]
bars = plt.barh(range(len(sorted_combinations)), sorted_mae_scores, color=colors,
edgecolor='black')

for bar, score in zip(bars, sorted_mae_scores):
    plt.text(bar.get_width(), bar.get_y() + bar.get_height() / 2, f' {score:.3f}',
             va='center', ha='left', fontsize=13, fontweight='bold')

plt.yticks(range(len(sorted_combinations)), [' \\& '.join(combo) for combo in
sorted_combinations], fontsize=12)
plt.xticks(fontsize=12)
plt.xlabel('MAE (pg/mL)', fontsize=12)
plt.title('MAE by Feature Combination', fontsize=16)
# plt.gca().invert_yaxis() # Ensure lowest MAPE is at the top

legend_elements = [mpatches.Patch(color=sns.color_palette("hot",
len(features_to_consider))[i],
                                label=f'{i+1} feature{"s" if i > 0 else ""}')
                    for i in range(len(features_to_consider))]
plt.legend(handles=legend_elements, title="Number of Features", loc='upper right',
fontsize=11, title_fontsize=11)

plt.tight_layout()
plt.show()

sorted_indices = np.argsort(rmse_scores)[::-1]

sorted_combinations = [feature_combinations[i] for i in sorted_indices]
sorted_rmse_scores = [rmse_scores[i] for i in sorted_indices]
sorted_best_params = [best_params[i] for i in sorted_indices]

plt.figure(figsize=(12, 8))
colors = [sns.color_palette("hot", len(set([len(combo) for combo in
feature_combinations]))))\
          [len(combo)-1] for combo in sorted_combinations]
bars = plt.barh(range(len(sorted_combinations)), sorted_rmse_scores, color=colors,
edgecolor='black')

for bar, score in zip(bars, sorted_rmse_scores):
    plt.text(bar.get_width(), bar.get_y() + bar.get_height() / 2, f' {score:.3f}',
             va='center', ha='left', fontsize=10, fontweight='bold')

plt.yticks(range(len(sorted_combinations)), [' \\& '.join(combo) for combo in
sorted_combinations])
plt.xlabel('RMSE')
plt.title('RMSE by Feature Combination')

```

```

# plt.gca().invert_yaxis() # Ensure lowest MAPE is at the top

legend_elements = [mpatches.Patch(color=sns.color_palette("hot",
len(features_to_consider))[i],
                                label=f'{i+1} feature{"s" if i > 0 else ""}')
                    for i in range(len(features_to_consider))]
plt.legend(handles=legend_elements, title="Number of Features", loc='upper right')

plt.tight_layout()
plt.show()

# Sort the combinations by r2 score for a more intuitive display
sorted_indices = np.argsort(r2_scores)
sorted_combinations = [feature_combinations[i] for i in sorted_indices]
sorted_r2_scores = [r2_scores[i] for i in sorted_indices]
sorted_best_params = [best_params[i] for i in sorted_indices]

# Multiply r2 scores by 100
sorted_r2_scores = [score * 100 for score in sorted_r2_scores]

plt.figure(figsize=(12, 10))
colors = [sns.color_palette("hot", len(set([len(combo) for combo in
feature_combinations])))\
          [len(combo)-1] for combo in sorted_combinations]
bars = plt.barh(range(len(sorted_combinations)), sorted_r2_scores, color=colors,
edgecolor='black')

for bar, score in zip(bars, sorted_r2_scores):
    plt.text(bar.get_width(), bar.get_y() + bar.get_height() / 2,
f' {score:.1f}%\n',
            va='center', ha='left', fontsize=10, fontweight='bold')

plt.yticks(range(len(sorted_combinations)), ['\n'.join(combo) for combo in
sorted_combinations], fontsize=12)
plt.xticks(fontsize=12)
plt.xlabel('R2 (%)', fontsize=12)
plt.title('R2 by Feature Combination', fontsize=16)

# Creating custom legend for number of features
legend_elements = [mpatches.Patch(color=sns.color_palette("hot",
len(features_to_consider))[i],
                                label=f'{i+1} feature{"s" if i > 0 else ""}')
                    for i in range(len(features_to_consider))]
plt.legend(handles=legend_elements, title="Number of Features", loc='lower right',
fontsize=11, title_fontsize=11)

plt.tight_layout()
plt.show()

```



```

sorted_indices = np.argsort(mape_scores)

sorted_combinations = [feature_combinations[i] for i in sorted_indices]
sorted_mape_scores = [mape_scores[i] for i in sorted_indices]
sorted_best_params = [best_params[i] for i in sorted_indices]

sorted_mape_scores = np.abs(sorted_mape_scores) # Convert scores to positive

plt.figure(figsize=(12, 8))
colors = [sns.color_palette("hot", len(set([len(combo) for combo in
feature_combinations]))))\
          [len(combo)-1] for combo in sorted_combinations]
bars = plt.barh(range(len(sorted_combinations)), sorted_mape_scores, color=colors,
edgecolor='black')

for bar, score in zip(bars, sorted_mape_scores):
    plt.text(bar.get_width(), bar.get_y() + bar.get_height() / 2,
f' {score*100:.1f}%\n',
            va='center', ha='left', fontsize=10, fontweight='bold')

plt.yticks(range(len(sorted_combinations)), ['\& '.join(combo) for combo in
sorted_combinations])
plt.xlabel('MAPE')
plt.title('MAPE by Feature Combination')
# plt.gca().invert_yaxis() # Ensure lowest MAPE is at the top

# Creating custom legend for number of features
legend_elements = [mpatches.Patch(color=sns.color_palette("hot",
len(features_to_consider))[i],
                                label=f'{i+1} feature{"s" if i > 0 else ""}')
                    for i in range(len(features_to_consider))]
plt.legend(handles=legend_elements, title="Number of Features", loc='upper right')

plt.tight_layout()
plt.show()

sorted_indices = np.argsort(mae_scores)
sorted_combinations = [feature_combinations[i] for i in sorted_indices]
sorted_mae_scores = [mae_scores[i] for i in sorted_indices]
sorted_mape_scores = [mape_scores[i] for i in sorted_indices]
sorted_rmse_scores = [rmse_scores[i] for i in sorted_indices]
sorted_r2_scores = [r2_scores[i] for i in sorted_indices]
sorted_best_params = [best_params[i] for i in sorted_indices]

# Print teh best 3 combinations with their parameters
for i in range(1,4):
    print(f"Combination: {sorted_combinations[-i]}")
    print(f"MAE: {sorted_mae_scores[-i]:.3f}")
    print(f"MAPE: {sorted_mape_scores[-i]:.3f}")

```

```

    print(f"RMSE: {sorted_rmse_scores[-i]:.3f}")
    print(f"R2: {sorted_r2_scores[-i]:.3f}")
    print(f"Best parameters: {sorted_best_params[-i]}")
    print()
# Find the index of the feature combination in the list
index = feature_combinations.index(('Age', 'APOE4', 'CSF_ABETA40', 'Gender'))
cv = KFold(n_splits=10, shuffle=True, random_state=42)

# Get the best parameters for this feature combination
best_params_for_combo = best_params[index]

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('svr', SVR(**best_params_for_combo))
])

# print(best_params_for_combo)

# Select the features from the dataset
X = fuji_dataset[list(('Age', 'APOE4', 'CSF_ABETA40', 'Gender'))]
y = fuji_dataset['Plasma_ABETA40']

# Get cross-validated predictions for the entire dataset
predictions = cross_val_predict(pipeline, X, y, cv=cv,)

# Calculate errors (residuals)
residuals = y - predictions

plt.figure(figsize=(10, 5))
sns.histplot(residuals, bins=20, kde=True, edgecolor='black')
plt.title('Distribution of Residuals', fontsize=20)
plt.xlabel('Residuals', fontsize=16)
plt.ylabel('Frequency', fontsize=16)
plt.yticks(fontsize=12)
plt.xticks(fontsize=12)
plt.show()

# Calculate and print skewness
residual_skewness = skew(residuals)
print(f'Skewness of residuals: {residual_skewness}')

# Calculate and print kurtosis
residual_kurtosis = kurtosis(residuals, fisher=True)
print(f'Kurtosis of residuals: {residual_kurtosis}')

plt.figure(figsize=(9, 5))
# Improved regplot with scatterplot for better visibility of data points
sns.scatterplot(x= y, y=predictions, color='blue', alpha=0.6, edgecolor='w',
linewidth=0.5)

```

```

sns.regplot(x= y, y=predictions, color='red', ci=95, scatter=False,
label='Regression Line with 95\\% CI')

# Plotting the line of perfect prediction for reference
min_vals = np.minimum( y.min(), predictions.min())
max_vals = np.maximum( y.max(), predictions.max())
plt.plot([min_vals, max_vals], [min_vals, max_vals], color='darkgreen',
linestyle='--', label='Perfect Fit')

plt.title('Actual vs. Predicted ABETA40 Levels', fontsize=20)
plt.xlabel('Actual ABETA40 Levels (pg/mL)', fontsize=16)
plt.ylabel('Predicted ABETA40 Levels (pg/mL)', fontsize=16)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)

# Adding legend to differentiate between actual data points and the regression line
plt.legend(prop={'size': 12})
plt.grid(True, which='both', linestyle='--', linewidth=0.5)
plt.tight_layout()
plt.show()

cv = KFold(n_splits=10, shuffle=True, random_state=42)
# Define a simple mean predictor
class MeanPredictor(BaseEstimator, RegressorMixin):
    def fit(self, X, y):
        self.mean_ = np.mean(y)
        return self

    def predict(self, X):
        return np.full(shape=(len(X)), fill_value=self.mean_)

# Create an instance of the mean predictor
mean_predictor = MeanPredictor()

scoring = ['neg_mean_absolute_error', 'neg_mean_squared_error', 'r2']

results_baseline = cross_validate(mean_predictor, X, y, cv=cv, scoring=scoring,
return_train_score=False)
# Get cross-validated predictions for the entire dataset
mean_predictions = cross_val_predict(mean_predictor, X, y, cv=cv)

# Calculate mean scores for the baseline model
mean_mae_baseline = np.mean(results_baseline['test_neg_mean_absolute_error'])
mean_rmse_baseline = np.sqrt(-
np.mean(results_baseline['test_neg_mean_squared_error']))
mean_r2_baseline = np.mean(results_baseline['test_r2'])

print(f"Baseline Model - Mean MAE: {mean_mae_baseline}")
print(f"Baseline Model - Mean RMSE: {mean_rmse_baseline}")

```

```

print(f"Baseline Model - Mean R2: {mean_r2_baseline}")
# Mann-Whitney U test used when the two samples are independent.
stat, p = mannwhitneyu(predictions, mean_predictions)
print('Mann-Whitney U test p-value:', p)

```

F.7 Code for the Machine Learning Pipeline and Evaluation of A β 42/40

```

fuji_dataset = pd.read_csv(r'C:\Users\theof\OneDrive - The University of
Manchester\Univesity\Year
3\Project\Code\Data\Appendix\ABETA4240\fuji_abeta4240.csv')

# Drop rid
fuji_dataset = fuji_dataset.drop(columns=['RID', 'DX_b1', 'Race', 'PTMARRY'])

# Make PTGENDER 1 for Male 0 for female
fuji_dataset['Gender'] = fuji_dataset['Gender'].replace({'Male': 1})
fuji_dataset['Gender'] = fuji_dataset['Gender'].replace({'Female': 0})

# Rename columns
fuji_dataset = fuji_dataset.rename(columns={'ABETA4240': 'CSF_ABETA4240',
'ABETA4240_plasma': 'Plasma_ABETA4240'})
fuji_dataset.describe()
# Placeholder for feature combinations and their corresponding MAPE scores
feature_combinations = []
mae_scores = []
mape_scores = []
rmse_scores = []
r2_scores = []
best_params = []

# Define scoring dictionary
scoring = {'MAE': 'neg_mean_absolute_error',
           'MAPE': 'neg_mean_absolute_percentage_error',
           'RMSE': make_scorer(mean_squared_error, squared=False),
           'R2': 'r2'}

# Define the cross-validation strategy
cv = KFold(n_splits=10, shuffle=True, random_state=42)

# Features to consider
features_to_consider = ['Age', 'APOE4', 'CSF_ABETA4240', 'Gender', 'MMSE']

# Iterate through all non-empty combinations of the three features
for r in range(1, len(features_to_consider) + 1):
    for combo in combinations(features_to_consider, r):

```

```

    # for combo in [('Age', 'APOE4', 'CSF_ABETA4240', 'Gender'), ('Age',
'CSF_ABETA4240', 'Gender'), ('Age', 'CSF_ABETA4240', 'APOE4') ]:
    print(combo)
    # Selecting the current combination of features
    X = fuji_dataset[list(combo)]
    y = fuji_dataset['Plasma_ABETA4240']

    # Create copies of X and y
    X_opt = X.copy()
    y_opt = y.copy()

    # Calculate z-scores for y
    z_scores_y = stats.zscore(y_opt)

    # Get indices of outliers in y
    outliers_y = np.where(np.abs(z_scores_y) > 3)[0].tolist()

    # Double the outliers in y
    y_opt[outliers_y] *= 5

    # Scaling the features
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)
    X_opt_scaled = scaler.fit_transform(X_opt)

    # Bayesian optimization setup
    search_spaces = {'C': Real(1e-2, 1e+2, prior='log-uniform'),
                    'epsilon': Real(1e-3, 1e+1, prior='log-uniform'),
                    'gamma': Real(1e-3, 1e+1, prior='log-uniform'),
                    'kernel': Categorical(['rbf'])}

    bayesian_opt = BayesSearchCV(SVR(),
                                search_spaces,
                                scoring='neg_mean_absolute_error',
                                cv=cv,
                                n_jobs=-1)

    bayesian_opt.fit(X_opt_scaled, y_opt)

    # Pipeline that includes scaling and the SVR model with the best parameters
    pipeline = Pipeline([
        ('scaler', StandardScaler()),
        ('svr', SVR(**bayesian_opt.best_params_))
    ])

    # Using cross_validate with the pipeline, which ensures features are scaled
    results = cross_validate(pipeline, X, y, cv=cv, scoring=scoring,
return_train_score=False)

```

```

# Calculate mean scores
mean_mae = np.mean(results['test_MAE'])
mean_mape = np.mean(results['test_MAPE'])
mean_rmse = np.mean(results['test_RMSE'])
mean_r2 = np.mean(results['test_R2'])

# Print results
print(f"CV MAE Scores: {results['test_MAE']}")
print(f"Mean CV MAE Score: {mean_mae}")
print(f"CV MAPE Scores: {results['test_MAPE']}")
print(f"Mean CV MAPE Score: {mean_mape}")
print(f"CV RMSE Scores: {results['test_RMSE']}")
print(f"Mean CV RMSE Score: {mean_rmse}")
print(f"CV R2 Scores: {results['test_R2']}")
print(f"Mean CV R2 Score: {mean_r2}")

print("//-----//")

feature_combinations.append(combo)
mae_scores.append(mean_mae) # Store the mean MAE score
mape_scores.append(mean_mape)
rmse_scores.append(mean_rmse)
r2_scores.append(mean_r2)
best_params.append(bayesian_opt.best_params_) # Store best parameters
# Sort the combinations by MAPE score for a more intuitive display
sorted_indices = np.argsort(mape_scores)
sorted_combinations = [feature_combinations[i] for i in sorted_indices]
sorted_mae_scores = [mae_scores[i] for i in sorted_indices]
sorted_best_params = [best_params[i] for i in sorted_indices]

sorted_mae_scores = np.abs(sorted_mae_scores) # Convert scores to positive

plt.figure(figsize=(14, 10))
colors = [sns.color_palette("hot", len(set([len(combo) for combo in
feature_combinations])))\
          [len(combo)-1] for combo in sorted_combinations]
bars = plt.barh(range(len(sorted_combinations)), sorted_mae_scores, color=colors,
edgecolor='black')

for bar, score in zip(bars, sorted_mae_scores):
    plt.text(bar.get_width(), bar.get_y() + bar.get_height() / 2, f' {score:.4f}',
            va='center', ha='left', fontsize=13, fontweight='bold')

plt.yticks(range(len(sorted_combinations)), [' \\& '.join(combo) for combo in
sorted_combinations], fontsize=12)
plt.xticks(fontsize=12)
plt.xlabel('MAE', fontsize=12)

```

```

plt.title('MAE by Feature Combination', fontsize=16)
# plt.gca().invert_yaxis() # Ensure lowest MAPE is at the top

legend_elements = [mpatches.Patch(color=sns.color_palette("hot",
len(features_to_consider))[i],
                                label=f'{i+1} feature{"s" if i > 0 else ""}')
                    for i in range(len(features_to_consider))]
plt.legend(handles=legend_elements, title="Number of Features", loc='upper right',
           fontsize=11, title_fontsize=11)

plt.tight_layout()
plt.show()

sorted_indices = np.argsort(rmse_scores)[::-1]

sorted_combinations = [feature_combinations[i] for i in sorted_indices]
sorted_rmse_scores = [rmse_scores[i] for i in sorted_indices]
sorted_best_params = [best_params[i] for i in sorted_indices]

plt.figure(figsize=(12, 8))
colors = [sns.color_palette("hot", len(set([len(combo) for combo in
feature_combinations])))\
          [len(combo)-1] for combo in sorted_combinations]
bars = plt.barh(range(len(sorted_combinations)), sorted_rmse_scores, color=colors,
               edgcolor='black')

for bar, score in zip(bars, sorted_rmse_scores):
    plt.text(bar.get_width(), bar.get_y() + bar.get_height() / 2, f' {score:.4f}',
             va='center', ha='left', fontsize=10, fontweight='bold')

plt.yticks(range(len(sorted_combinations)), [' \\& '.join(combo) for combo in
sorted_combinations])
plt.xlabel('RMSE')
plt.title('RMSE by Feature Combination')
# plt.gca().invert_yaxis() # Ensure lowest MAPE is at the top

legend_elements = [mpatches.Patch(color=sns.color_palette("hot",
len(features_to_consider))[i],
                                label=f'{i+1} feature{"s" if i > 0 else ""}')
                    for i in range(len(features_to_consider))]
plt.legend(handles=legend_elements, title="Number of Features", loc='upper right')

plt.tight_layout()
plt.show()

# Sort the combinations by r2 score for a more intuitive display
sorted_indices = np.argsort(r2_scores)
sorted_combinations = [feature_combinations[i] for i in sorted_indices]

```

```

sorted_r2_scores = [r2_scores[i] for i in sorted_indices]
sorted_best_params = [best_params[i] for i in sorted_indices]

# Multiply r2 scores by 100
sorted_r2_scores = [score * 100 for score in sorted_r2_scores]

plt.figure(figsize=(12, 10))
colors = [sns.color_palette("hot", len(set([len(combo) for combo in
feature_combinations]))))\
          [len(combo)-1] for combo in sorted_combinations]
bars = plt.barh(range(len(sorted_combinations)), sorted_r2_scores, color=colors,
edgecolor='black')

for bar, score in zip(bars, sorted_r2_scores):
    plt.text(bar.get_width(), bar.get_y() + bar.get_height() / 2,
f' {score:.1f}\\%',
            va='center', ha='left', fontsize=10, fontweight='bold')

plt.yticks(range(len(sorted_combinations)), [' \\& '.join(combo) for combo in
sorted_combinations], fontsize=12)
plt.xticks(fontsize=12)
plt.xlabel('R2 (\\%)', fontsize=12)
plt.title('R2 by Feature Combination', fontsize=16)

# Creating custom legend for number of features
legend_elements = [mpatches.Patch(color=sns.color_palette("hot",
len(features_to_consider))[i],
                                label=f'{i+1} feature{"s" if i > 0 else ""}')
                    for i in range(len(features_to_consider))]
plt.legend(handles=legend_elements, title="Number of Features", loc='lower right',
fontsize=11, title_fontsize=11)

plt.tight_layout()
plt.show()

# Sort the combinations by r2 score for a more intuitive display
sorted_indices = np.argsort(r2_scores)
sorted_combinations = [feature_combinations[i] for i in sorted_indices]
sorted_r2_scores = [r2_scores[i] for i in sorted_indices]
sorted_best_params = [best_params[i] for i in sorted_indices]

# Multiply r2 scores by 100
sorted_r2_scores = [score * 100 for score in sorted_r2_scores]

plt.figure(figsize=(12, 10))
colors = [sns.color_palette("hot", len(set([len(combo) for combo in
feature_combinations]))))\
          [len(combo)-1] for combo in sorted_combinations]

```



```

bars = plt.barh(range(len(sorted_combinations)), sorted_r2_scores, color=colors,
edgecolor='black')

for bar, score in zip(bars, sorted_r2_scores):
    plt.text(bar.get_width(), bar.get_y() + bar.get_height() / 2,
f' {score:.1f}\\%',
            va='center', ha='left', fontsize=10, fontweight='bold')

plt.yticks(range(len(sorted_combinations)), [' \\& '.join(combo) for combo in
sorted_combinations], fontsize=12)
plt.xticks(fontsize=12)
plt.xlabel('R2 (\\%)', fontsize=12)
plt.title('R2 by Feature Combination', fontsize=16)

# Creating custom legend for number of features
legend_elements = [mpatches.Patch(color=sns.color_palette("hot",
len(features_to_consider))[i],
                                label=f'{i+1} feature{"s" if i > 0 else ""}')
                    for i in range(len(features_to_consider))]
plt.legend(handles=legend_elements, title="Number of Features", loc='lower right',
fontsize=11, title_fontsize=11)

plt.tight_layout()
plt.show()
sorted_indices = np.argsort(mape_scores)

sorted_combinations = [feature_combinations[i] for i in sorted_indices]
sorted_mape_scores = [mape_scores[i] for i in sorted_indices]
sorted_best_params = [best_params[i] for i in sorted_indices]

sorted_mape_scores = np.abs(sorted_mape_scores) # Convert scores to positive

plt.figure(figsize=(12, 8))
colors = [sns.color_palette("hot", len(set([len(combo) for combo in
feature_combinations]))))\
          [len(combo)-1] for combo in sorted_combinations]
bars = plt.barh(range(len(sorted_combinations)), sorted_mape_scores, color=colors,
edgecolor='black')

for bar, score in zip(bars, sorted_mape_scores):
    plt.text(bar.get_width(), bar.get_y() + bar.get_height() / 2,
f' {score*100:.2f}\\%',
            va='center', ha='left', fontsize=10, fontweight='bold')

plt.yticks(range(len(sorted_combinations)), [' \\& '.join(combo) for combo in
sorted_combinations])
plt.xlabel('MAPE')
plt.title('MAPE by Feature Combination')

```

```

# plt.gca().invert_yaxis() # Ensure lowest MAPE is at the top

# Creating custom legend for number of features
legend_elements = [mpatches.Patch(color=sns.color_palette("hot",
len(features_to_consider))[i],
                                label=f'{i+1} feature{"s" if i > 0 else ""}')
                    for i in range(len(features_to_consider))]
plt.legend(handles=legend_elements, title="Number of Features", loc='upper right')

plt.tight_layout()
plt.show()
sorted_indices = np.argsort(mae_scores)
sorted_combinations = [feature_combinations[i] for i in sorted_indices]
sorted_mae_scores = [mae_scores[i] for i in sorted_indices]
sorted_mape_scores = [mape_scores[i] for i in sorted_indices]
sorted_rmse_scores = [rmse_scores[i] for i in sorted_indices]
sorted_r2_scores = [r2_scores[i] for i in sorted_indices]
sorted_best_params = [best_params[i] for i in sorted_indices]

# Print teh best 3 combinations with their parameters
for i in range(1,4):
    print(f"Combination: {sorted_combinations[-i]}")
    print(f"MAE: {sorted_mae_scores[-i]:.3f}")
    print(f"MAPE: {sorted_mape_scores[-i]:.3f}")
    print(f"RMSE: {sorted_rmse_scores[-i]:.3f}")
    print(f"R2: {sorted_r2_scores[-i]:.3f}")
    print(f"Best parameters: {sorted_best_params[-i]}")
    print()

# Find the index of the feature combination in the list
index = feature_combinations.index(('Age', 'APOE4', 'CSF_ABETA4240', 'Gender'))
cv = KFold(n_splits=10, shuffle=True, random_state=42)

# Get the best parameters for this feature combination
best_params_for_combo = best_params[index]

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('svr', SVR(**best_params_for_combo))
])

# print(best_params_for_combo)

# Select the features from the dataset
X = fuji_dataset[list(('Age', 'APOE4', 'CSF_ABETA4240', 'Gender'))]
y = fuji_dataset['Plasma_ABETA4240']

# Get cross-validated predictions for the entire dataset
predictions = cross_val_predict(pipeline, X, y, cv=cv,)

```

```

# Calculate errors (residuals)
residuals = y - predictions

plt.figure(figsize=(10, 5))
sns.histplot(residuals, bins=20, kde=True, edgecolor='black')
plt.title('Distribution of Residuals', fontsize=20)
plt.xlabel('Residuals', fontsize=16)
plt.ylabel('Frequency', fontsize=16)
plt.yticks(fontsize=12)
plt.xticks(fontsize=12)
plt.show()

# Calculate and print skewness
residual_skewness = skew(residuals)
print(f'Skewness of residuals: {residual_skewness}')

# Calculate and print kurtosis
residual_kurtosis = kurtosis(residuals, fisher=True)
print(f'Kurtosis of residuals: {residual_kurtosis}')

plt.figure(figsize=(9, 5))
# Improved regplot with scatterplot for better visibility of data points
sns.scatterplot(x= y, y=predictions, color='blue', alpha=0.6, edgecolor='w',
linewidth=0.5)
sns.regplot(x= y, y=predictions, color='red', ci=95, scatter=False,
label='Regression Line with 95\\% CI')

# Plotting the line of perfect prediction for reference
min_vals = np.minimum( y.min(), predictions.min())
max_vals = np.maximum( y.max(), predictions.max())
plt.plot([min_vals, max_vals], [min_vals, max_vals], color='darkgreen',
linestyle='--', label='Perfect Fit')

plt.title('Actual vs. Predicted ABETA42/40 Levels', fontsize=20)
plt.xlabel('Actual ABETA42/40 Levels', fontsize=16)
plt.ylabel('Predicted ABETA42/40 Levels', fontsize=16)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)

# Adding legend to differentiate between actual data points and the regression line
plt.legend(prop={'size': 12})
plt.grid(True, which='both', linestyle='--', linewidth=0.5)
plt.tight_layout()
plt.show()
cv = KFold(n_splits=10, shuffle=True, random_state=42)
# Define a simple mean predictor
class MeanPredictor(BaseEstimator, RegressorMixin):
    def fit(self, X, y):

```

```

        self.mean_ = np.mean(y)
        return self

    def predict(self, X):
        return np.full(shape=(len(X)), fill_value=self.mean_)

# Create an instance of the mean predictor
mean_predictor = MeanPredictor()

scoring = ['neg_mean_absolute_error', 'neg_mean_squared_error', 'r2']

results_baseline = cross_validate(mean_predictor, X, y, cv=cv, scoring=scoring,
return_train_score=False)
# Get cross-validated predictions for the entire dataset
mean_predictions = cross_val_predict(mean_predictor, X, y, cv=cv)

# Calculate mean scores for the baseline model
mean_mae_baseline = np.mean(results_baseline['test_neg_mean_absolute_error'])
mean_rmse_baseline = np.sqrt(-
np.mean(results_baseline['test_neg_mean_squared_error']))
mean_r2_baseline = np.mean(results_baseline['test_r2'])

print(f"Baseline Model - Mean MAE: {mean_mae_baseline}")
print(f"Baseline Model - Mean RMSE: {mean_rmse_baseline}")
print(f"Baseline Model - Mean R2: {mean_r2_baseline}")
# Mann-Whitney U test used when the two samples are independent.
stat, p = mannwhitneyu(predictions, mean_predictions, alternative='less',
method='asymptotic')
print('Mann-Whitney U test p-value:', p)

```

F.8 Code for the Machine Learning Pipeline and Evaluation of P-tau181

```

ptau_dataset = pd.read_csv(r'C:\Users\theo\OneDrive - The University of
Manchester\Univesity\Year 3\Project\Code\Data\Appendix\PTAU\ptau.csv')

# Drop rid
ptau_dataset = ptau_dataset.drop(columns=['RID', 'DX_b1', 'Race', 'PTMARRY'])

# Make PTGENDER 1 for Male 0 for female
ptau_dataset['Gender'] = ptau_dataset['Gender'].replace({'Male': 1})
ptau_dataset['Gender'] = ptau_dataset['Gender'].replace({'Female': 0})

# Rename columns
ptau_dataset = ptau_dataset.rename(columns={'PTAU': 'CSF_PTAU', 'PTAU_plasma':
'Plasma_PTAU'})
ptau_dataset.describe()
# Placeholder for feature combinations and their corresponding MAPE scores

```

```

feature_combinations = []
mae_scores = []
mape_scores = []
rmse_scores = []
r2_scores = []
best_params = []

# Define scoring dictionary
scoring = {'MAE': 'neg_mean_absolute_error',
           'MAPE': 'neg_mean_absolute_percentage_error',
           'RMSE': make_scorer(mean_squared_error, squared=False),
           'R2': 'r2'}

# Define the cross-validation strategy
cv = KFold(n_splits=10, shuffle=True, random_state=42)

# Features to consider
features_to_consider = ['Age', 'APOE4', 'CSF_PTAU', 'Gender', 'MMSE']

# Iterate through all non-empty combinations of the three features
for r in range(1, len(features_to_consider) + 1):
    for combo in combinations(features_to_consider, r):
        # for combo in [('Age', 'APOE4', 'CSF_ABETA4240', 'Gender'), ('Age',
        'CSF_ABETA4240', 'Gender'), ('Age', 'CSF_ABETA4240', 'APOE4') ]:
            print(combo)
            # Selecting the current combination of features
            X = ptau_dataset[list(combo)]
            y = ptau_dataset['Plasma_PTAU']

            # Create copies of X and y
            X_opt = X.copy()
            y_opt = y.copy()

            # Calculate z-scores for y
            z_scores_y = stats.zscore(y_opt)

            # Get indices of outliers in y
            outliers_y = np.where(np.abs(z_scores_y) > 3)[0].tolist()

            # Double the outliers in y
            y_opt[outliers_y] *= 3

            # Scaling the features
            scaler = StandardScaler()
            X_scaled = scaler.fit_transform(X)
            X_opt_scaled = scaler.fit_transform(X_opt)

            # Bayesian optimization setup

```

```

search_spaces = {'C': Real(1e-2, 1e+2, prior='log-uniform'),
                 'epsilon': Real(1e-2, 1e+1, prior='log-uniform'),
                 'gamma': Real(1e-3, 1e+1, prior='log-uniform'),
                 'kernel': Categorical(['rbf'])}

bayesian_opt = BayesSearchCV(SVR(),
                             search_spaces,
                             scoring='neg_mean_absolute_error',
                             cv=cv,
                             n_jobs=-1)

bayesian_opt.fit(X_scaled, y_opt)

# Pipeline that includes scaling and the SVR model with the best parameters
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('svr', SVR(**bayesian_opt.best_params_))
])

# Using cross_validate with the pipeline, which ensures features are scaled
results = cross_validate(pipeline, X, y, cv=cv, scoring=scoring,
return_train_score=False)

# Calculate mean scores
mean_mae = np.mean(results['test_MAE'])
mean_mape = np.mean(results['test_MAPE'])
mean_rmse = np.mean(results['test_RMSE'])
mean_r2 = np.mean(results['test_R2'])

# Print results
print(f"CV MAE Scores: {results['test_MAE']}")
print(f"Mean CV MAE Score: {mean_mae}")
print(f"CV MAPE Scores: {results['test_MAPE']}")
print(f"Mean CV MAPE Score: {mean_mape}")
print(f"CV RMSE Scores: {results['test_RMSE']}")
print(f"Mean CV RMSE Score: {mean_rmse}")
print(f"CV R2 Scores: {results['test_R2']}")
print(f"Mean CV R2 Score: {mean_r2}")

print("//-----//")

feature_combinations.append(combo)
mae_scores.append(mean_mae) # Store the mean MAE score
mape_scores.append(mean_mape)
rmse_scores.append(mean_rmse)
r2_scores.append(mean_r2)
best_params.append(bayesian_opt.best_params_) # Store best parameters
# Sort the combinations by MAPE score for a more intuitive display

```

```

sorted_indices = np.argsort(mae_scores)
sorted_combinations = [feature_combinations[i] for i in sorted_indices]
sorted_mae_scores = [mae_scores[i] for i in sorted_indices]
sorted_best_params = [best_params[i] for i in sorted_indices]

sorted_mae_scores = np.abs(sorted_mae_scores) # Convert scores to positive

plt.figure(figsize=(14, 10))
colors = [sns.color_palette("hot", len(set([len(combo) for combo in
feature_combinations]))))\
          [len(combo)-1] for combo in sorted_combinations]
bars = plt.barh(range(len(sorted_combinations)), sorted_mae_scores, color=colors,
edgecolor='black')

for bar, score in zip(bars, sorted_mae_scores):
    plt.text(bar.get_width(), bar.get_y() + bar.get_height() / 2, f' {score:.3f}',
            va='center', ha='left', fontsize=13, fontweight='bold')

plt.yticks(range(len(sorted_combinations)), [' \\& '.join(combo) for combo in
sorted_combinations], fontsize=12)
plt.xticks(fontsize=12)
plt.xlabel('MAE (pg/mL)', fontsize=12)
plt.title('MAE by Feature Combination', fontsize=16)
# plt.gca().invert_yaxis() # Ensure lowest MAPE is at the top

legend_elements = [mpatches.Patch(color=sns.color_palette("hot",
len(features_to_consider))[i],
                                label=f'{i+1} feature{"s" if i > 0 else ""}')
                    for i in range(len(features_to_consider))]
plt.legend(handles=legend_elements, title="Number of Features", loc='upper right',
fontsize=11, title_fontsize=11)

plt.tight_layout()
plt.show()

sorted_indices = np.argsort(rmse_scores)[::-1]

sorted_combinations = [feature_combinations[i] for i in sorted_indices]
sorted_rmse_scores = [rmse_scores[i] for i in sorted_indices]
sorted_best_params = [best_params[i] for i in sorted_indices]

plt.figure(figsize=(12, 8))
colors = [sns.color_palette("hot", len(set([len(combo) for combo in
feature_combinations]))))\
          [len(combo)-1] for combo in sorted_combinations]
bars = plt.barh(range(len(sorted_combinations)), sorted_rmse_scores, color=colors,
edgecolor='black')

```

```

for bar, score in zip(bars, sorted_rmse_scores):
    plt.text(bar.get_width(), bar.get_y() + bar.get_height() / 2, f' {score:.3f}',
             va='center', ha='left', fontsize=10, fontweight='bold')

plt.yticks(range(len(sorted_combinations)), [' \\& '.join(combo) for combo in
sorted_combinations])
plt.xlabel('RMSE')
plt.title('RMSE by Feature Combination')
# plt.gca().invert_yaxis() # Ensure lowest MAPE is at the top

legend_elements = [mpatches.Patch(color=sns.color_palette("hot",
len(features_to_consider))[i],
                                label=f'{i+1} feature{"s" if i > 0 else ""}')
                    for i in range(len(features_to_consider))]
plt.legend(handles=legend_elements, title="Number of Features", loc='lower left')

plt.tight_layout()
plt.show()

# Sort the combinations by r2 score for a more intuitive display
sorted_indices = np.argsort(r2_scores)
sorted_combinations = [feature_combinations[i] for i in sorted_indices]
sorted_r2_scores = [r2_scores[i] for i in sorted_indices]
sorted_best_params = [best_params[i] for i in sorted_indices]

# Multiply r2 scores by 100
sorted_r2_scores = [score * 100 for score in sorted_r2_scores]

plt.figure(figsize=(12, 10))
colors = [sns.color_palette("hot", len(set([len(combo) for combo in
feature_combinations])))\
          [len(combo)-1] for combo in sorted_combinations]
bars = plt.barh(range(len(sorted_combinations)), sorted_r2_scores, color=colors,
edgecolor='black')

for bar, score in zip(bars, sorted_r2_scores):
    plt.text(bar.get_width(), bar.get_y() + bar.get_height() / 2,
f' {score:.1f}\\%',
             va='center', ha='left', fontsize=10, fontweight='bold')

plt.yticks(range(len(sorted_combinations)), [' \\& '.join(combo) for combo in
sorted_combinations], fontsize=12)
plt.xticks(fontsize=12)
plt.xlabel('R2 (\\%)', fontsize=12)
plt.title('R2 by Feature Combination', fontsize=16)

# Creating custom legend for number of features

```



```

legend_elements = [mpatches.Patch(color=sns.color_palette("hot",
len(features_to_consider))[i],
                                label=f'{i+1} feature{"s" if i > 0 else ""}')
                    for i in range(len(features_to_consider))]
plt.legend(handles=legend_elements, title="Number of Features", loc='lower right',
fontsize=11, title_fontsize=11)

plt.tight_layout()
plt.show()
sorted_indices = np.argsort(mape_scores)

sorted_combinations = [feature_combinations[i] for i in sorted_indices]
sorted_mape_scores = [mape_scores[i] for i in sorted_indices]
sorted_best_params = [best_params[i] for i in sorted_indices]

sorted_mape_scores = np.abs(sorted_mape_scores) # Convert scores to positive

plt.figure(figsize=(12, 8))
colors = [sns.color_palette("hot", len(set([len(combo) for combo in
feature_combinations])))\
          [len(combo)-1] for combo in sorted_combinations]
bars = plt.barh(range(len(sorted_combinations)), sorted_mape_scores, color=colors,
edgecolor='black')

for bar, score in zip(bars, sorted_mape_scores):
    plt.text(bar.get_width(), bar.get_y() + bar.get_height() / 2,
f' {score*100:.1f}%\n',
            va='center', ha='left', fontsize=10, fontweight='bold')

plt.yticks(range(len(sorted_combinations)), [' \& '.join(combo) for combo in
sorted_combinations])
plt.xlabel('MAPE')
plt.title('MAPE by Feature Combination')
# plt.gca().invert_yaxis() # Ensure lowest MAPE is at the top

# Creating custom legend for number of features
legend_elements = [mpatches.Patch(color=sns.color_palette("hot",
len(features_to_consider))[i],
                                label=f'{i+1} feature{"s" if i > 0 else ""}')
                    for i in range(len(features_to_consider))]
plt.legend(handles=legend_elements, title="Number of Features", loc='upper right')

plt.tight_layout()
plt.show()
sorted_indices = np.argsort(mae_scores)
sorted_combinations = [feature_combinations[i] for i in sorted_indices]
sorted_mae_scores = [mae_scores[i] for i in sorted_indices]
sorted_mape_scores = [mape_scores[i] for i in sorted_indices]

```

```

sorted_rmse_scores = [rmse_scores[i] for i in sorted_indices]
sorted_r2_scores = [r2_scores[i] for i in sorted_indices]
sorted_best_params = [best_params[i] for i in sorted_indices]

# Print the best 3 combinations with their parameters
for i in range(1,4):
    print(f"Combination: {sorted_combinations[-i]}")
    print(f"MAE: {sorted_mae_scores[-i]:.3f}")
    print(f"MAPE: {sorted_mape_scores[-i]:.3f}")
    print(f"RMSE: {sorted_rmse_scores[-i]:.3f}")
    print(f"R2: {sorted_r2_scores[-i]:.3f}")
    print(f"Best parameters: {sorted_best_params[-i]}")
    print()
# Find the index of the feature combination in the list
index = feature_combinations.index(('Age', 'APOE4', 'CSF_PTAU', 'Gender'))
cv = KFold(n_splits=10, shuffle=True, random_state=42)

# Get the best parameters for this feature combination
best_params_for_combo = best_params[index]

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('svr', SVR(**best_params_for_combo))
])

# print(best_params_for_combo)

# Select the features from the dataset
X = ptau_dataset[list(('Age', 'APOE4', 'CSF_PTAU', 'Gender'))]
y = ptau_dataset['Plasma_PTAU']

# Get cross-validated predictions for the entire dataset
predictions = cross_val_predict(pipeline, X, y, cv=cv,)

# Calculate errors (residuals)
residuals = y - predictions

plt.figure(figsize=(10, 5))
sns.histplot(residuals, bins=30, kde=True, edgecolor='black')
plt.title('Distribution of Residuals', fontsize=20)
plt.xlabel('Residuals', fontsize=16)
plt.ylabel('Frequency', fontsize=16)
plt.yticks(fontsize=12)
plt.xticks(fontsize=12)
plt.show()
# Calculate and print skewness
residual_skewness = skew(residuals)
print(f'Skewness of residuals: {residual_skewness}')

```

```

# Calculate and print kurtosis
residual_kurtosis = kurtosis(residuals, fisher=True)
print(f'Kurtosis of residuals: {residual_kurtosis}')

plt.figure(figsize=(9, 5))
# Improved regplot with scatterplot for better visibility of data points
sns.scatterplot(x= y, y=predictions, color='blue', alpha=0.6, edgecolor='w',
linewidth=0.5)
sns.regplot(x= y, y=predictions, color='red', ci=95, scatter=False,
label='Regression Line with 95\\% CI')

# Plotting the line of perfect prediction for reference
min_vals = np.minimum( y.min(), predictions.min())
max_vals = np.maximum( y.max(), predictions.max())
plt.plot([min_vals, max_vals], [min_vals, max_vals], color='darkgreen',
linestyle='--', label='Perfect Fit')

plt.title('Actual vs. Predicted P-tau Levels', fontsize=20)
plt.xlabel('Actual P-tau Levels (pg/mL)', fontsize=16)
plt.ylabel('Predicted P-tau Levels (pg/mL)', fontsize=16)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)

# Adding legend to differentiate between actual data points and the regression line
plt.legend(prop={'size': 12})
plt.grid(True, which='both', linestyle='--', linewidth=0.5)
plt.tight_layout()
plt.show()
cv = KFold(n_splits=10, shuffle=True, random_state=42)
# Define a simple mean predictor
class MeanPredictor(BaseEstimator, RegressorMixin):
    def fit(self, X, y):
        self.mean_ = np.mean(y)
        return self

    def predict(self, X):
        return np.full(shape=(len(X),), fill_value=self.mean_)

# Create an instance of the mean predictor
mean_predictor = MeanPredictor()

scoring = ['neg_mean_absolute_error', 'neg_mean_squared_error', 'r2']

results_baseline = cross_validate(mean_predictor, X, y, cv=cv, scoring=scoring,
return_train_score=False)
# Get cross-validated predictions for the entire dataset
mean_predictions = cross_val_predict(mean_predictor, X, y, cv=cv)

```

```

# Calculate mean scores for the baseline model
mean_mae_baseline = np.mean(results_baseline['test_neg_mean_absolute_error'])
mean_rmse_baseline = np.sqrt(-
np.mean(results_baseline['test_neg_mean_squared_error']))
mean_r2_baseline = np.mean(results_baseline['test_r2'])

print(f"Baseline Model - Mean MAE: {mean_mae_baseline}")
print(f"Baseline Model - Mean RMSE: {mean_rmse_baseline}")
print(f"Baseline Model - Mean R2: {mean_r2_baseline}")
# Mann-Whitney U test used when the two samples are independent.
stat, p = mannwhitneyu(predictions, mean_predictions)
print('Mann-Whitney U test p-value:', p)

```

F.9 Code for the Machine Learning Pipeline and Evaluation of Nfl

```

nfl_dataset = pd.read_csv(r'C:\Users\theof\OneDrive - The University of
Manchester\Univesity\Year 3\Project\Code\Data\Appendix\NFL\nfl.csv')

# Drop rid
nfl_dataset = nfl_dataset.drop(columns=['RID', 'DX_b1', 'Race', 'PTMARRY'])

# Make PTGENDER 1 for Male 0 for female
nfl_dataset['Gender'] = nfl_dataset['Gender'].replace({'Male': 1})
nfl_dataset['Gender'] = nfl_dataset['Gender'].replace({'Female': 0})

# Rename columns
nfl_dataset = nfl_dataset.rename(columns={'NFL': 'CSF_NFL', 'NFL_plasma':
'Plasma_NFL'})
nfl_dataset.describe()
# Placeholder for feature combinations and their corresponding MAPE scores
feature_combinations = []
mae_scores = []
mape_scores = []
rmse_scores = []
r2_scores = []
best_params = []

# Define scoring dictionary
scoring = {'MAE': 'neg_mean_absolute_error',
           'MAPE': 'neg_mean_absolute_percentage_error',
           'RMSE': make_scorer(mean_squared_error, squared=False),
           'R2': 'r2'}

# Define the cross-validation strategy
cv = KFold(n_splits=10, shuffle=True, random_state=42)

```

```

# Features to consider
features_to_consider = ['Age', 'APOE4', 'CSF_NFL', 'Gender', 'MMSE']

# Iterate through all non-empty combinations of the three features
for r in range(1, len(features_to_consider) + 1):
    for combo in combinations(features_to_consider, r):
        # for combo in [('Age', 'APOE4', 'CSF_ABETA4240', 'Gender'), ('Age',
        'CSF_ABETA4240', 'Gender'), ('Age', 'CSF_ABETA4240', 'APOE4') ]:
            print(combo)
            # Selecting the current combination of features
            X = nfl_dataset[list(combo)]
            y = nfl_dataset['Plasma_NFL']

            # Create copies of X and y
            X_opt = X.copy()
            y_opt = y.copy()

            # Calculate z-scores for y
            z_scores_y = stats.zscore(y_opt)

            # Get indices of outliers in y
            outliers_y = np.where(np.abs(z_scores_y) > 3)[0].tolist()

            # Double the outliers in y
            y_opt[outliers_y] *= 3

            # Scaling the features
            scaler = StandardScaler()
            X_scaled = scaler.fit_transform(X)
            X_opt_scaled = scaler.fit_transform(X_opt)

            # Bayesian optimization setup
            search_spaces = {'C': Real(1e-2, 1e+2, prior='log-uniform'),
                             'epsilon': Real(1e-2, 1e+1, prior='log-uniform'),
                             'gamma': Real(1e-3, 1e+1, prior='log-uniform'),
                             'kernel': Categorical(['rbf'])}

            bayesian_opt = BayesSearchCV(SVR(),
                                         search_spaces,
                                         scoring='neg_mean_absolute_error',
                                         cv=cv,
                                         n_jobs=-1)

            bayesian_opt.fit(X_scaled, y_opt)

            # Pipeline that includes scaling and the SVR model with the best parameters
            pipeline = Pipeline([

```

```

        ('scaler', StandardScaler()),
        ('svr', SVR(**bayesian_opt.best_params_))
    ])

    # Using cross_validate with the pipeline, which ensures features are scaled
    results = cross_validate(pipeline, X, y, cv=cv, scoring=scoring,
return_train_score=False)

    # Calculate mean scores
    mean_mae = np.mean(results['test_MAE'])
    mean_mape = np.mean(results['test_MAPE'])
    mean_rmse = np.mean(results['test_RMSE'])
    mean_r2 = np.mean(results['test_R2'])

    # Print results
    print(f"CV MAE Scores: {results['test_MAE']}")
    print(f"Mean CV MAE Score: {mean_mae}")
    print(f"CV MAPE Scores: {results['test_MAPE']}")
    print(f"Mean CV MAPE Score: {mean_mape}")
    print(f"CV RMSE Scores: {results['test_RMSE']}")
    print(f"Mean CV RMSE Score: {mean_rmse}")
    print(f"CV R2 Scores: {results['test_R2']}")
    print(f"Mean CV R2 Score: {mean_r2}")

    print("//-----//")

    feature_combinations.append(combo)
    mae_scores.append(mean_mae) # Store the mean MAE score
    mape_scores.append(mean_mape)
    rmse_scores.append(mean_rmse)
    r2_scores.append(mean_r2)
    best_params.append(bayesian_opt.best_params_) # Store best parameters
# Sort the combinations by MAPE score for a more intuitive display
sorted_indices = np.argsort(mae_scores)
sorted_combinations = [feature_combinations[i] for i in sorted_indices]
sorted_mae_scores = [mae_scores[i] for i in sorted_indices]
sorted_best_params = [best_params[i] for i in sorted_indices]

sorted_mae_scores = np.abs(sorted_mae_scores) # Convert scores to positive

plt.figure(figsize=(14, 10))
colors = [sns.color_palette("hot", len(set([len(combo) for combo in
feature_combinations]))))\
        [len(combo)-1] for combo in sorted_combinations]
bars = plt.barh(range(len(sorted_combinations)), sorted_mae_scores, color=colors,
edgecolor='black')

for bar, score in zip(bars, sorted_mae_scores):

```

```

plt.text(bar.get_width(), bar.get_y() + bar.get_height() / 2, f' {score:.3f}',
         va='center', ha='left', fontsize=13, fontweight='bold')

plt.yticks(range(len(sorted_combinations)), [' \\& '.join(combo) for combo in
sorted_combinations], fontsize=12)
plt.xticks(fontsize=12)
plt.xlabel('MAE (pg/mL)', fontsize=12)
plt.title('MAE by Feature Combination', fontsize=16)
# plt.gca().invert_yaxis() # Ensure lowest MAPE is at the top

legend_elements = [mpatches.Patch(color=sns.color_palette("hot",
len(features_to_consider))[i],
                                label=f'{i+1} feature{"s" if i > 0 else ""}')
                    for i in range(len(features_to_consider))]
plt.legend(handles=legend_elements, title="Number of Features", loc='upper right',
          fontsize=11, title_fontsize=11)

plt.tight_layout()
plt.show()

sorted_indices = np.argsort(rmse_scores[::-1])

sorted_combinations = [feature_combinations[i] for i in sorted_indices]
sorted_rmse_scores = [rmse_scores[i] for i in sorted_indices]
sorted_best_params = [best_params[i] for i in sorted_indices]

plt.figure(figsize=(12, 8))
colors = [sns.color_palette("hot", len(set([len(combo) for combo in
feature_combinations])))\
          [len(combo)-1] for combo in sorted_combinations]
bars = plt.barh(range(len(sorted_combinations)), sorted_rmse_scores, color=colors,
edgecolor='black')

for bar, score in zip(bars, sorted_rmse_scores):
    plt.text(bar.get_width(), bar.get_y() + bar.get_height() / 2, f' {score:.3f}',
             va='center', ha='left', fontsize=10, fontweight='bold')

plt.yticks(range(len(sorted_combinations)), [' \\& '.join(combo) for combo in
sorted_combinations])
plt.xlabel('RMSE')
plt.title('RMSE by Feature Combination')
# plt.gca().invert_yaxis() # Ensure lowest MAPE is at the top

legend_elements = [mpatches.Patch(color=sns.color_palette("hot",
len(features_to_consider))[i],
                                label=f'{i+1} feature{"s" if i > 0 else ""}')
                    for i in range(len(features_to_consider))]
plt.legend(handles=legend_elements, title="Number of Features", loc='upper right')

```

```

plt.tight_layout()
plt.show()

# Sort the combinations by r2 score for a more intuitive display
sorted_indices = np.argsort(r2_scores)
sorted_combinations = [feature_combinations[i] for i in sorted_indices]
sorted_r2_scores = [r2_scores[i] for i in sorted_indices]
sorted_best_params = [best_params[i] for i in sorted_indices]

# Multiply r2 scores by 100
sorted_r2_scores = [score * 100 for score in sorted_r2_scores]

plt.figure(figsize=(12, 10))
colors = [sns.color_palette("hot", len(set([len(combo) for combo in
feature_combinations]))))\
          [len(combo)-1] for combo in sorted_combinations]
bars = plt.barh(range(len(sorted_combinations)), sorted_r2_scores, color=colors,
edgecolor='black')

for bar, score in zip(bars, sorted_r2_scores):
    plt.text(bar.get_width(), bar.get_y() + bar.get_height() / 2,
f' {score:.1f}\\%',
            va='center', ha='left', fontsize=10, fontweight='bold')

plt.yticks(range(len(sorted_combinations)), [' \\& '.join(combo) for combo in
sorted_combinations], fontsize=12)
plt.xticks(fontsize=12)
plt.xlabel('R2 (\\%)', fontsize=12)
plt.title('R2 by Feature Combination', fontsize=16)

# Creating custom legend for number of features
legend_elements = [mpatches.Patch(color=sns.color_palette("hot",
len(features_to_consider))[i],
                                label=f'{i+1} feature{"s" if i > 0 else ""}')
                    for i in range(len(features_to_consider))]
plt.legend(handles=legend_elements, title="Number of Features", loc='lower right',
fontsize=11, title_fontsize=11)

plt.tight_layout()
plt.show()
sorted_indices = np.argsort(mape_scores)

sorted_combinations = [feature_combinations[i] for i in sorted_indices]
sorted_mape_scores = [mape_scores[i] for i in sorted_indices]
sorted_best_params = [best_params[i] for i in sorted_indices]

sorted_mape_scores = np.abs(sorted_mape_scores) # Convert scores to positive

```



```

plt.figure(figsize=(12, 8))
colors = [sns.color_palette("hot", len(set([len(combo) for combo in
feature_combinations])))\
          [len(combo)-1] for combo in sorted_combinations]
bars = plt.barh(range(len(sorted_combinations)), sorted_mape_scores, color=colors,
edgecolor='black')

for bar, score in zip(bars, sorted_mape_scores):
    plt.text(bar.get_width(), bar.get_y() + bar.get_height() / 2,
f' {score*100:.1f}%\n',
            va='center', ha='left', fontsize=10, fontweight='bold')

plt.yticks(range(len(sorted_combinations)), ['\& '.join(combo) for combo in
sorted_combinations])
plt.xlabel('MAPE')
plt.title('MAPE by Feature Combination')
# plt.gca().invert_yaxis() # Ensure lowest MAPE is at the top

# Creating custom legend for number of features
legend_elements = [mpatches.Patch(color=sns.color_palette("hot",
len(features_to_consider))[i],
                                label=f'{i+1} feature{"s" if i > 0 else ""}')
                    for i in range(len(features_to_consider))]
plt.legend(handles=legend_elements, title="Number of Features", loc='upper right')

plt.tight_layout()
plt.show()
sorted_indices = np.argsort(mae_scores)
sorted_combinations = [feature_combinations[i] for i in sorted_indices]
sorted_mae_scores = [mae_scores[i] for i in sorted_indices]
sorted_mape_scores = [mape_scores[i] for i in sorted_indices]
sorted_rmse_scores = [rmse_scores[i] for i in sorted_indices]
sorted_r2_scores = [r2_scores[i] for i in sorted_indices]
sorted_best_params = [best_params[i] for i in sorted_indices]

# Print teh best 3 combinations with their parameters
for i in range(1,4):
    print(f"Combination: {sorted_combinations[-i]}")
    print(f"MAE: {sorted_mae_scores[-i]:.3f}")
    print(f"MAPE: {sorted_mape_scores[-i]:.3f}")
    print(f"RMSE: {sorted_rmse_scores[-i]:.3f}")
    print(f"R2: {sorted_r2_scores[-i]:.3f}")
    print(f"Best parameters: {sorted_best_params[-i]}")
    print()

# Find the index of the feature combination in the list
index = feature_combinations.index(('Age', 'APOE4', 'CSF_NFL', 'Gender'))
cv = KFold(n_splits=10, shuffle=True, random_state=42)

```

```

# Get the best parameters for this feature combination
best_params_for_combo = best_params[index]

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('svr', SVR(**best_params_for_combo))
])

# print(best_params_for_combo)

# Select the features from the dataset
X = nfl_dataset[list(('Age', 'APOE4', 'CSF_NFL', 'Gender'))]
y = nfl_dataset['Plasma_NFL']

# Get cross-validated predictions for the entire dataset
predictions = cross_val_predict(pipeline, X, y, cv=cv,)

# Calculate errors (residuals)
residuals = y - predictions

plt.figure(figsize=(10, 5))
sns.histplot(residuals, bins=40, kde=True, edgecolor='black')
plt.title('Distribution of Residuals', fontsize=20)
plt.xlabel('Residuals', fontsize=16)
plt.ylabel('Frequency', fontsize=16)
plt.yticks(fontsize=12)
plt.xticks(fontsize=12)
plt.show()

# Calculate and print skewness
residual_skewness = skew(residuals)
print(f'Skewness of residuals: {residual_skewness}')

# Calculate and print kurtosis
residual_kurtosis = kurtosis(residuals, fisher=True)
print(f'Kurtosis of residuals: {residual_kurtosis}')

plt.figure(figsize=(9, 5))
# Improved regplot with scatterplot for better visibility of data points
sns.scatterplot(x= y, y=predictions, color='blue', alpha=0.6, edgecolor='w',
linewidth=0.5)
sns.regplot(x= y, y=predictions, color='red', ci=95, scatter=False,
label='Regression Line with 95\\% CI')

# Plotting the line of perfect prediction for reference
min_vals = np.minimum( y.min(), predictions.min())
max_vals = np.maximum( y.max(), predictions.max())

```

```

plt.plot([min_vals, max_vals], [min_vals, max_vals], color='darkgreen',
linestyle='--', label='Perfect Fit')

plt.title('Actual vs. Predicted NFL Levels', fontsize=20)
plt.xlabel('Actual NFL Levels (pg/mL)', fontsize=16)
plt.ylabel('Predicted NFL Levels (pg/mL)', fontsize=16)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)

# Adding legend to differentiate between actual data points and the regression line
plt.legend(prop={'size': 12})
plt.grid(True, which='both', linestyle='--', linewidth=0.5)
plt.tight_layout()
plt.show()

cv = KFold(n_splits=10, shuffle=True, random_state=42)
# Define a simple mean predictor
class MeanPredictor(BaseEstimator, RegressorMixin):
    def fit(self, X, y):
        self.mean_ = np.mean(y)
        return self

    def predict(self, X):
        return np.full(shape=(len(X),), fill_value=self.mean_)

# Create an instance of the mean predictor
mean_predictor = MeanPredictor()

scoring = ['neg_mean_absolute_error', 'neg_mean_squared_error', 'r2']

results_baseline = cross_validate(mean_predictor, X, y, cv=cv, scoring=scoring,
return_train_score=False)
# Get cross-validated predictions for the entire dataset
mean_predictions = cross_val_predict(mean_predictor, X, y, cv=cv)

# Calculate mean scores for the baseline model
mean_mae_baseline = np.mean(results_baseline['test_neg_mean_absolute_error'])
mean_rmse_baseline = np.sqrt(-
np.mean(results_baseline['test_neg_mean_squared_error']))
mean_r2_baseline = np.mean(results_baseline['test_r2'])

print(f"Baseline Model - Mean MAE: {mean_mae_baseline}")
print(f"Baseline Model - Mean RMSE: {mean_rmse_baseline}")
print(f"Baseline Model - Mean R2: {mean_r2_baseline}")
# Mann-Whitney U test used when the two samples are independent.
stat, p = mannwhitneyu(predictions, mean_predictions)
print('Mann-Whitney U test p-value:', p)

```